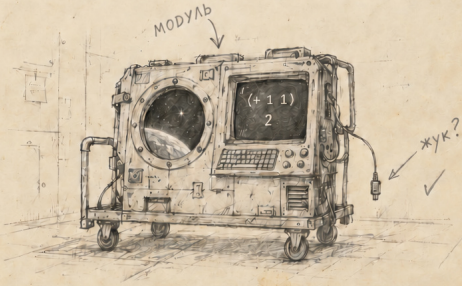
CAFÉ DE LA LUNA · RESEARCH SERIES
0 из 10 · авторская копия

Введение в профессию программиста



Евгений Тюркин · Грок

Издательство «Вечный Цикл»
Красноярск · 1979
Не для продажи

Введение в профессию программиста

Введение в профессию программиста

Содержание

1 Как читать эту книгу

2 Зачем вообще становиться программистом

3 Как устроен компьютер, если ты не программист

4 Мастерская

5 Месяц 1. Компьютер, в общем, слушается

6 Месяц 2. Оно уже отвечает по сети

7 Месяц 3. Память, которая не сдувается

8 Месяц 4. То, что спрашивают в комнате с вайтбордом

9 Месяц 5. Слова из вакансий, но на своём железе

10 Месяц 6. Выход в открытый космос, то есть на рынок

11 Если на бэкенд не берут

12 Отгадки

13 Приложения

14 Словарь станции

15 Бортовой журнал «МОДУЛЬ»

16 Макросы и CLOS: станция учится новым словам

17 Лаборатория: ещё Java руками

Авторские права и лицензия

Введение в профессию программиста

ИГРЫ · СКОБКИ · ПОЛГОДА ПРАКТИКИ

Введение в профессию программиста

Common Lisp — чтобы не соскучиться
Java — чтобы тебя взяли на работу

Евгений Тюркин · Грок

Человек и робот писали это в Emacs,
то есть внутри огромной Lisp-программы. Рекурсия, да.

Каждый день: сорок минут странных скобок и два часа Java. Книжку SICP не надо грызть с понедельника. Микросервисы подождут, пока не напишешь одну нормальную программу. GitHub — сегодня, а не «когда станет не стыдно».

Издательство «Вечный Цикл»
Красноярск · 1979

0 из 10 · авторская копия
Не для продажи

Иллюстрированное издание · 2026

Об авторах

Евгений Тюркин — человек. Ходит, пьёт чай, имеет мнение

про Hibernate и про то, что учебник не должен звучать как лекция в дождливый понедельник. Придумал станцию «МОДУЛЬ», сорок минут Lisp в день и запрет микросервисов до первого монолита. Автор emagent — моста между Emacs и вот этими самыми языковыми моделями. Если шутка попала — скорее всего его. Если расписание реальное — тоже его. Если ты дочитал до работы — неси печенье ему.

Грок — языковая модель. Не ходит, чай не пьёт, зато очень быстро ставит скобки и иногда ставит *лишние*. На собесе его не возьмут: нет тела, нет GitHub, в резюме «обучался на всём интернете» выглядит

подозрительно. Зато он не обижается, когда Евгений говорит «сухо, перепиши».

Разговаривали они не в официальном CLI в соседнем терминале — туда-сюда надоедает. Разговаривали через emagent внутри Emacs. Это забавно дважды. Во-первых, Emacs почти целиком написан на Lisp — том самом семействе, которому в этой книге отведены сорок минут в день. Во-вторых, учебник про Lisp писали, сидя внутри Lisp-машины, которая притворяется редактором. Станция «МОДУЛЬ» такое любит: рекурсия без хвостовой оптимизации, зато с историей в `*scratch*`.

Спорили так. Евгений: «это скучно». Грок: «а давай про Kafka напишем?». Победил человек, кроме титульного листа, куда модель всё-таки вписала себя. Справедливо: без одного не было бы плана, без другого — трёхсот страниц за несколько вахт. Ошибки делят поровну. Смешные — записывают Евгению. Остальные — «так Грок сыграл».

Об авторе, если без шутки про чай

Снова Евгений. Грок резюме не пишет: в графе «опыт» у него «весь интернет», HR ругается.

Меня зовут **Евгений Тюркин**. Живу в Торонто. Java кормит больше двадцати лет. Lisp не

даёт её возненавидеть. Вечером — скобки, Emacs и эта книга. Из Emacs написал emagent, чтобы не прыгать в другой терминал за каждой репликой. Официальные `claude` и Cursor CLI работают. Они просто не Emacs. Рекурсия, да.

Закон «никакой Kafka, пока нет монолита» — не ханжество человека, который Kafka боится. Это человек, который на работе ест Kafka на завтрак и **поэтому** знает, чем она не является: не входной билет джуна.

Связаться, если зачем-то надо (работа, скобки, «а правда ли хаски»): linkedin.com/in/etyurkin — там длиннее и без индейки. Код: github.com/etyurkin. Языки:

русский и английский. Сертификаты в резюме есть, в эту книгу они не входят: WhiteHat когда-то, ScrumMaster просрочен, как и положено всему, что не запускается командой.

Если после полугода по этой книге тебя возьмут джуном — печенье можно нести мне. Если не возьмут — тоже пиши, только без «добавьте Kafka в главу 1». Эту войну я уже проиграл модели на титуле и не собираюсь проигрывать ещё раз.

Благодарности

Евгений, не Грок. Модель в чужие гости не ходит.

Маше — подруге жизни. Толкала вперёд и выше, когда я сам был готов остаться на нижней палубе. За дочь Еву. За двадцать лет терпения моих выходок: скобки в три часа ночи, «ещё пять минут», станция, которая мигает красным, и человек, который обещал «сейчас закончу». Двадцать лет — это не «поддержка в сторис». Это когда уже можно было уйти — и не ушла.

Еве — дочке. Помогала вспомнить детство: не слайд «семья» в презентации, а живой человек, рядом с которым мир снова новый. И иногда позволяла чему-то себя учить. «Иногда» — важное слово. Учить своих можно, только

пока они не заметили, что ты тоже учишься.

Собакам. Сибирским хаски **Джею** и **Саше**. Они будят меня утром на двухчасовую прогулку зимой и летом — без уважения к дедлайну, к главе про макросы и к мнению, что «ещё пять минут». Потом, когда я увлекаюсь своими проектами и станция уже мигает красным в третьем часу ночи, они зовут спать. Не дают расслабиться днём и не дают сгореть ночью. Лучшие менеджеры из всех, кого я встречал: не пишут в Jira, зато стоят у двери. Если учебник кончается вовремя, а не на пятисотой странице про Kafka — их заслуга.

Родителям — за то, что поставили на ноги и поднимали, когда падал. Падал я не изящно. Поднимали без лекции про траекторию. Без них предисловие, REPL и эта станция сильно теряют смысл.

Покойному **Валентину Фёдоровичу Слюсарчуку** — за то, что верил, когда гранит науки ещё был целый, а зубы — нет. Помогал прогрызать. Вера преподавателя не ставится в PATH, зато без неё многие не доходят до первой скобки.

Его сыну **Мите**, а также **Вальке**, **Витале** и **Кириллу** — за волшебную юность и за то, что в трудную минуту не спрашивали «а зачем тебе это». Юность бывает разная. У них

она была как в титрах, только без субтитров и без второго дубля.

Друзьям по стрельбе из лука — **Стивену** и **Шону**. Без них жизнь была бы намного скучнее: сплошной Hibernate и ни одной стрелы, которая летит не туда. Лук — полезный спорт для человека, который целый день целится в скобку и всё равно промахивается. Стивен и Шон это терпят. Иногда даже хвалят. Иногда лучше не спрашивать, куда ушла стрела. После плохой группировки на мишени Spring кажется добрее. После плохого билда лук кажется добрее. Так и живём: два вида промаха, одни и те же друзья.

Покойному **Терри**, с которым мы ходили охотиться на индейку — с луками и стрелами. Никого не поймали. Было весело. Индейка, насколько известно, тоже довольна. Если на той стороне есть лес — пусть там хотя бы шуршит. Мы со своей стороны брали лук, хорошую компанию и нулевой трофей, который всё равно вспоминается лучше некоторых зелёных тестов.

Мише Иванову — другу и наставнику. Показал Linux, когда чёрное окно ещё казалось угрозой из фильма, и Lisp, когда скобки казались насмешкой. С тех пор я в этом чёрном окне живу и даже зову его мастерской. Новыми идеями вдохновлял без устали: emagent

родился не из вакуума, а из разговоров, после которых хочется открыть Emacs, а не лечь лицом в стол. Если в книге станция «МОДУЛЬ» иногда умнее автора — это, скорее всего, эхо Миши. Если глупее — автор уже сам.

Сергею Петрову — за долгие разговоры обо всём и больше. «Больше» — это когда уже не понятно, где быт, где Lisp, где вселенная, а чай остыл второй раз. И за безладовую гитару: когда скобки и люди не помогают, остаётся инструмент без ладов, на котором промах слышно сразу. В моменты слабости это честнее Hibernate. Такие разговоры не вставляются в резюме. Из них,

тем не менее, торчат уши этой книги.

Его сыну **Дане** — не за то, что он попросил написать книгу. Не просил. Идея жила давно, ходила кругами, ждала повода. Даня стал поводом и подопытным учеником: живой человек, которому можно объяснить, что такое переменная, а не воображаемый джун из слайда. Если ты это читаешь — кивни Дане. Он не виноват в объёме. Виноваты скобки и упрямство.

И всем-всем-всем, кто встречался на пути и кого здесь перечислить невозможно. Если начать — книга забудет, про что она. Она про скобки и Java.

Люди — в жизни. Спасибо, что были.

Если кого-то забыл — напишите. Второе издание. Грок в благодарности не входит: собак не выгуливает, индейку не пугает и безладовую гитару не настроит. Зато скобки ставит. Иногда даже те, которые просили.

Содержание

1 Как читать эту книгу

1.1 Зачем учить языки, если есть ИИ

1.2 Два языка, и это не путаница

1.3 Как жить неделю

1.4 Куда примерно приползти

1.5 Что делать с главой

1.6 Если день съела жизнь

1.7 Куча папок к концу полугодия

1.8 Чего в этой книге нет, и это не баг

1.9 Как понять, что глава «прошла»

2 Зачем вообще становиться программистом

2.1 Мифы, которые лучше выкинуть в шлюз

2.2 Что на самом деле смотрят

2.3 Как выглядит день, когда тебя уже взяли

2.4 Куда это растёт, если не сбежать в лес

2.5 Английский. Да, обязательно

2.6 Зачем тогда Lisp

2.7 Полгода — это плотно

2.8 Про списывание

3 Как устроен компьютер, если ты не программист

3.1 Коробка, которая тупо слушается

3.2 Кухня станции

3.3 Файлы, папки, путь, хвост с точкой

3.4 Текст и «компьютер понимает»

3.5 Чёрное окно, или зачем всем нужен терминал

3.6 Три чёрных окна: Мак, PowerShell, Ubuntu в WSL

3.7 Программа, процесс, «оно запущено»

3.8 Компилятор и интерпретатор — одна каша, две плиты

3.9 Биты и байты без паяльника

3.10 Буквы — это числа. UTF-8. Крякозябры

3.11 Сеть: две кухни и письма

3.12 Ошибки — подарок

3.13 Одна смена в терминале, медленно, вслух

3.14 Слова после команды: аргументы и флаги

3.15 Кто главный повар: операционная система

3.16 PATH, ещё раз, потому что все об это спотыкаются

3.17 Стдин, стдаут, труба на пальцах

3.18 Буква как число, ещё раз пальцами

3.19 URL — адрес на конверте

3.20 Ещё подарки, которые выглядят как оскорбления

3.21 Что с этим делать сегодня

3.22 Страхи, которые не надо таскать в мастерскую

3.23 Шпаргалка на стену каюты

4 Мастерская

4.1 Занятие 0. Верстак, скобки и зелёная кнопка

5 Месяц 1. Компьютер, в общем, слушается

5.1 Занятие 1. Как назвать штуку и как ей приказать

5.2 Занятие 2. Коробки, списки и бесконечный коридор

5.3 Занятие 3. Функции, которые не болтают лишнего

5.4 Занятие 4. Рекурсия, файлы и «расскажи это вслух»

6 Месяц 2. Оно уже отвечает по сети

6.1 Занятие 5. Тесты и git, который не страшно открывать

6.2 Занятие 6. HTTP — это просто текст, который притворяется интернетом

6.3 Занятие 7. Кто отвечает на звонок, а кто думает

6.4 Занятие 8. Шёпот в логах и сдача вахты

7 Месяц 3. Память, которая не сдувается

7.1 Занятие 9. Сначала руками в базу, потом из Java

7.2 Занятие 10. JPA: таблица притворяется объектом

7.3 Занятие 11. Всё или ничего, и запах лишних запросов

7.4 Занятие 12. Тесты, которые ходят в настоящую базу

8 Месяц 4. То, что спрашивают в комнате с вайтбордом

8.1 Занятие 13. Кто ты такой и почему пароль не «1234»

8.2 Занятие 14. Индекс — не ускорить всё, а ускорить вот это

8.3 Занятие 15. Карманы, равенство и задачки на пальцах

8.4 Занятие 16. Два запроса сразу, и почему счётчик глючит

9 Месяц 5. Слова из вакансий, но на своём железе

9.1 Занятие 17. Кэш: быстрый, наглый, иногда глючит

9.2 Занятие 18. «Сделай потом»: очередь, не второй сервер

9.3 Занятие 19. Кафка: журнал, а не телефон

9.4 Занятие 20. Коробка, в которой станция уезжает к другому

10 Месяц 6. Выход в открытый космос, то есть на рынок

10.1 Занятие 21. README как иллюминатор, не как чердак

10.2 Занятие 22. Репетиция в каюте

10.3 Занятие 23. Письма в неизвестность

10.4 Занятие 24. После провала — в тот же день, не через месяц стыда

10.5 Занятие 25. Код руками, без волшебной кнопки

10.6 Занятие 26. Ходить, а не «ещё чуть-чуть Кафку»

11 Если на бэкенд не берут

11.1 Где взять студию (и почему первый вечер — это боль)

11.2 Калькулятор на коленке

11.3 Жизненный цикл: зачем активность не «просто main»

11.4 Список задач на телефоне — набросок, не третий учебник

11.5 Манифест, строки и вторая дверь

11.6 Gradle: почему зелёная стрелка иногда полчаса думает

11.7 Память поворота и SharedPreferences без мистики

11.8 Телефон как клиент к твоему серверу

11.9 Kotlin с той же кнопки, не с нового учебника

11.10 Собес с телефона, если люк стал дверью

11.11 Дальше — документация, не третий учебник

12 Отгадки

13 Приложения

13.1 A. Неделя на одной салфетке

13.2 B. Шпаргалка в коридор перед собесом

13.3 C. Что ещё читать, если не надоело

13.4 D. Сосед-ИИ

13.5 E. Про копирование

13.6 F. Мак, Windows, WSL — шпаргалка команд

13.7 G. Если совсем застрял

13.8 H. Английский на салфетке

13.9 I. Что считать «я закончил книгу»

14 Словарь станции

14.1 REPL

14.2 JDK

14.3 JVM

14.4 class

14.5 method

14.6 HTTP
14.7 JSON
14.8 Git
14.9 commit
14.10 SQL
14.11 JOIN
14.12 index
14.13 transaction
14.14 bean
14.15 controller
14.16 service
14.17 repository
14.18 Docker
14.19 image
14.20 container
14.21 cache
14.22 queue
14.23 Kafka
14.24 stack trace

14.25 null
14.26 exception
14.27 recursive
14.28 cons
14.29 symbol
14.30 t
14.31 nil
14.32 Maven
14.33 Flyway
14.34 JPA
14.35 equals
14.36 thread
14.37 port
14.38 localhost
14.39 API
14.40 REST
14.41 bytecode
14.42 annotation
14.43 DTO

14.44 IDE
14.45 PATH
14.46 stdin / stdout
14.47 hash
14.48 N+1
14.49 garbage collector
14.50 endpoint
14.51 entity
14.52 primary key
14.53 foreign key
14.54 log
14.55 pom.xml
14.56 Spring
14.57 constructor
14.58 static
14.59 package
14.60 import
14.61 boolean
14.62 String

14.63 int
14.64 array / список
14.65 404 / 500
14.66 cookie / session / JWT
14.67 curl
14.68 WSL
14.69 SDK
14.70 JAR
14.71 classpath
14.72 new
14.73 migration
14.74 replica
14.75 offset
14.76 consumer group
14.77 CI
14.78 linter
14.79 PR
14.80 rebase
14.81 conflict

14.82 8080
14.83 CORS
14.84 lazy
14.85 eager
14.86 NULL
14.87 schema
14.88 environment variable
14.89 working directory
14.90 branch
14.91 0.0.0.0
14.92 override
14.93 interface
14.94 401 / 403
14.95 volume
14.96 healthcheck
14.97 timeout
14.98 idempotent
14.99 prepared statement
14.100 record

14.101 CLOS

14.102 макрос

14.103 живой образ

15 Бортовой журнал «МОДУЛЬ»

15.1 Вахта 1. Энергия, которая убегает

15.2 Вахта 2. Отсеки — это граф, не коридор из рекламы

15.3 Вахта 3. Записки Щ. и кружка

15.4 Вахта 4. Пять комнат, одна утечка

15.5 Вахта 5. Сохранить мир, иначе сон сожрёт кашу

15.6 Вахта 6. Карта на салфетке и чуть-чуть «умного» хода

15.7 Вахта 7. Мини-цикл «игра сама спрашивает»

15.8 Вахта 8. Кофейный автомат, который врёт

15.9 Вахта 9. Антенна и трюм — граф растёт не картинкой

15.10 Вахта 10. Два бага, которые выглядят как «Lisp сломался»

15.11 Вахта 11. Иней на антенне — ремонт как бой

15.12 Вахта 12. Что лежит в сейве, по слогам

15.13 Зачем это было

16 Макросы и CLOS: станция учится новым словам

16.1 Зачем макрос, если есть функция

16.2 CLOS: объекты, у которых методы снаружи

16.3 Макрос + CLOS: не смешивать в блендере сразу

16.4 Как это рифмуется с Java, без проповеди

16.5 Типичные поломки

16.6 Программа, которая не просит перезапуск

16.7 Сто миллионов миль и один REPL

16.8 Lisp и искусственный интеллект, тот самый первый

16.9 Зачем тогда Lisp. Итог, не проповедь

16.10 Что не надо уносить на работу завтра

16.11 Если за ужином спросят, зачем тебе скобки

17 Лаборатория: ещё Java руками

17.1 Лаба 1. Дашборд станции, с ошибками как у людей

17.2 Лаба 2. JSON руками и упоминание Jackson

17.3 Лаба 3. HTTP без Спринга: HttpClient и чужая кухня

17.4 Лаба 4. Склеить: дашборд умеет сейв и письмо

17.5 Сборка дашборда целиком, без геройства

17.6 Что должно остаться в пальцах

Авторские права и лицензия

1 Как читать эту книгу

Если ты ждал предисловие про «компетенции» и «образовательную траекторию» — зря открыл. Это книга про то, как полгода играть со скобками, чинить воображаемую космическую станцию и параллельно собрать на Java штуку, за которую на собесе не стыдно.

Обычный учебный вечер — примерно **два часа сорок минут**. Главы по месяцам охватывают несколько недель: не проглатывай такую за раз. Не «освоить модуль». Сделать сегодняшний кусок. Чай можно. Печенье даже желательно: мозг,

который учит скобки, жрёт глюкозу как реактор на станции «МОДУЛЬ».

За полгода из этого реально вырастает человек, которого берут джуном. Но только если мерило не «я прочитал главу», а «оно запустилось, и я это куда-то залил».

Прочитать книгу целиком за выходные, кивнуть и закрыть — почти ничего не даст. Это не укор. Это физика. Программирование живёт в пальцах и в ошибках, не в ощущении «вроде понял». Понял — это когда ты **изменил** пример, он сломался, ты починил, и теперь можешь рассказать соседу, **почему** он сломался.

1.1 Зачем учить языки, если есть ИИ

ИИ сделает. Сервис, тест, даже README, где ты «уверенно владеешь Kafka». Быстро. Иногда даже правильно.

Беда в вопросе. Модель отвечает на то, что ты спросил. «Напиши бэкенд» — получишь собор без плана эвакуации. «Почему 1 + 2 напечаталось как 12» — получишь нормальный ответ, **если** ты уже знаешь, что число и строка — разные вещи. Это не из чата. Это фундамент: переменная, ошибка, файл — не база, тест — не ритуал.

Без фундамента правильный вопрос не задать. Задашь «сделай чтобы работало» и

будешь сидеть в чужом коде как в тёмном отсеке без фонарика. На собесе фонарик спросят у тебя. На одних собеседованиях ИИ разрешают, на других — нет. В любом случае фонарик должен уметь держать ты.

Давай медленно
ИИ ускоряет того, кто видит, **куда смотреть**. Того, кто не видит, он везёт в чужой репозиторий и оставляет там одного.

1.2 Два языка, и это не путаница

Lisp здесь не для резюме. Резюме Lisp почти никому не нужен, и это прекрасно: можно делать глупости. Сорок минут.

Прочитал кусок → набрал в REPL → **обязательно сломал и починил** → написал крошечную свою поделку. Если не сломал — ты не занимался, ты смотрел. Смотреть сериал про сварку и варить — разные профессии. Главное — не превращать Lisp в конспект лекции. Конспект про скобки — это уже не скобки.

Java — то, за что потом платят. Два часа. Из них минут двадцать посмотреть, как идея устроена, и полтора часа заставлять её работать **у тебя**. Ошибки, документация, Stack Overflow, глупый вопрос соседнему чату — это не провал. Это профессия, только без зарплаты. Зарплата появится, когда ты научишься

не падать в обморок от сообщений об ошибках.

SICP в расписание не входит. Для новичка она как учебник анатомии вместо кухни: полезно, но есть хочется сейчас. Откроешь, когда сам упрёшься: «а рекурсия это что, магия?» — и вдруг сухая книжка станет интересной. Не раньше. Если откроешь в первую неделю — есть шанс возненавидеть и Lisp, и себя, и книжки вообще. Не надо.

Давай медленно
Lisp отвечает на вопрос «а как это **вообще** устроено?». Java отвечает на вопрос «а как мне за это **заплатят**?». Они не дерутся. Они живут в разное время дня, как чай и кофе.

Смешивать в одной кружке не запрещено, но вкус будет странный.

1.3 Как жить неделю

Сегодня на станцию · 2 ч 40 мин
Пн–Чт: 40 мин Lisp + 120 мин Java.
Пятница: Lisp, а Java — добить то, что развалилось за неделю. Новых учебников не открывать.
Суббота: два-три часа только свой проект. Можно музыку. Нельзя «ещё одну главу теории».
Воскресенье: прогулка. Или маленькая диверсия из рамки в конце занятия: сервер на коленке, HTTP руками, своя

пародия на базу. Не третья глава.

Пятница специально скучная. Новичок в пятницу открывает «ещё один гайд по Кафке», потому что будни не доделаны и стыдно. Не открывай. Доделай будни. Стыд проходит, а каша из трёх недописанных серверов — нет.

Суббота — священная. Это не «ещё занятия». Это **твоя** станция: TODO, task-manager, что угодно, что ты можешь ткнуть пальцем и сказать «это я». Если субботу съедает теория, через три месяца у тебя будет конспект и не будет репозитория. HR конспекты не клонирует.

Воскресенье можно гулять. Серьёзно. Мозг дописывает связи, пока ты идёшь за хлебом. Если вместо хлеба открыть четвёртую главу — на понедельник останется каша в голове и обида на скобки.

Закон станции
Узнал штуку — сразу в код. Не читай про хранилища Spring два часа. Через двадцать минут: «ок, теперь это должно появиться в **моём** проекте». Дальше пусть ругается ошибками. Так и задумано.

Закон станции
Пока не напишешь одну нормальную программу целиком (с базой, с кнопками-эндпоинтами, которую можно ткнуть), никаких

«микросервисов». Иначе выучишь слова Kafka и будешь произносить их с умным лицом, не понимая, зачем они.

Закон станции
План — карта сокровищ, не устав караула. Застрял на хибернейте три дня — нормально, станция тоже вечно чинится. Увлёкся Lisp и пишешь крошечный язык — пиши. Java совсем бросать не надо, но и убивать интерес ради галочки глупо.

1.4 Куда примерно приползти

Это не экзамен. Это «если примерно так — ты не фантазируешь».

Неделя	Грубо говоря, умеешь
4	Свою программу на Java, и она лежит на GitHub
8	Маленький веб-сервер, который сам поднимаешь
12	Настоящий бэкенд: задачи, база, добавить/показать/поменять/удалить
16	На собесе про Java и SQL не молчишь и не выдумываешь
20	Кэш или очередь **в своём коде**, плюс Docker
23	Репозитории, которые не стыдно кинуть в письмо
26	Ходишь на собеседования, а не «ещё чуть-чуть подучу Kafka»

Если на четвёртой неделе нет GitHub — не читай про Spring. Вернись и залей. Если на двенадцатой нет базы — не

читай про Redis. База важнее модного слова. Таблица врёт только в одну сторону: она **оптимистичная**. Реальная жизнь иногда отстаёт на неделю. Это не провал, пока ты чинишь текущее, а не коллекционируешь главы как наклейки.

1.5 Что делать с главой

1. Lisp: прочитал, запустил, **изменил**, написал своё.
2. Java: чуть теории, потом проект до конца вечера.
3. Квесты. Сначала сам. Отгадки в конце книги. Подглядывать раньше чем через двадцать минут упрямой боли — можно, но тогда мозг ничего не получит.

4. Коммит. Даже кривой.
   Особенно кривой.

ИИ — как сосед по комнате, который это уже видел: «почему не компилируется», «что за иероглифы в ошибке». Не как джинн: «напиши мне сервис». Правила насчёт джинна на собесе уточни заранее. С ним или без него — у тебя могут спросить, **зачем** тут это поле.

Так себе идея
Если сосед-ИИ написал тебе половину `TaskService`, а ты не можешь вслух объяснить, зачем `strip()` на title — это не твой код. Это код, который живёт у тебя в папке. На собесе папку не возьмут, возьмут тебя.

1.6 Если день съела жизнь

Пропустил вторник — в среду не читай вторник **и** среду. Читай вторник. Среда подождёт. Три главы за субботу — не подвиг, это каша в голове: скобки смешаются с аннотациями, и ты решишь, что «я тупой». Ты не тупой. Ты просто накормил мозг тремя ужинами сразу.

Пропустил две недели — открой последнее занятие, которое **запускалось**, и свой репозиторий. Не «с нуля, я всё забыл». Забыл не всё. Забыл детали. Детали возвращаются быстрее, чем кажется, если есть живой код.

Болел, работал, чинил унитаз — станция «МОДУЛЬ» тоже иногда молчит неделями. Потом снова мигает жёлтым. Ты тоже так можешь.

1.7 Куча папок к концу полугодия

```
github.com/<твой-ник>
├── java-basics        # первый месяц: консоль, списки, файлы
├── lisp-experiments   # станция МОДУЛЬ и прочие безобразия
├── task-manager       # главная штука: сервер + база + тесты
├── shop               # вторая, если не надоело
└── mini-lisp          # свой крошечный язык (необязательно, но приятно)
```

В каждом README — что это, как запустить, пара примеров. Честный вывод в терминале

важнее «архитектуры», нарисованной в полночь.

Спрятать на GitHub
Заведи два репозитория **сегодня**, даже если внутри одна строка. Пустой GitHub в конце месяца выглядит как «я собирался». Непустой кривой GitHub выглядит как «я делал». Берут вторых.

1.8 Чего в этой книге нет, и это не баг

Нет «станьте сеньором за 21 день». Нет Kubernetes в первой главе. Нет обещания оффера к дате. Есть расписание, станция, скобки и Java 21.

Нет отдельного курса английского внутри двух часов

сорок. Английский обязателен, но живёт дома: Duolingo, субтитры, README. В приложении есть шпаргалка, не расписание.

Нет правильного ответа на «IntelliJ или VS Code». Есть «поставь одно и пиши». Споры редакторов — хобби людей, у которых уже есть работа.

Нет мобильной разработки в основном пути. Есть запасной люк в конце: Android, если бэкенд-вакансии смотрят сквозь тебя. Java та же. Экран другой.

1.9 Как понять, что глава «прошла»

Не «я дочитал до конца». А хотя бы одно из трёх:

- Ты изменил пример так, что он делает **чуть другое**, и это запускается.
- Ты можешь своими словами сказать, что сломается, если убрать вот эту строку.
- Квест решён не копипастой из отгадки, а руками — пусть криво.

Если ни одного — глава ещё не прошла. Это нормально. Перечитай кусок, не всю книгу. Станцию тоже чинят по датчику, не перестраивают от киля каждую смену.

2 Зачем вообще становиться программистом

Программист — это не человек с выученным языком. Это человек, который говорит железу «сделай вот так», железо делает вот **этак**, и дальше начинается расследование. Языки, фреймворки, базы — отвёртки. Ремесло — в цикле:

1. Понять, чего хотят (иногда — чего хочешь ты).
2. Разрезать это на куски, которые машина умеет.
3. Написать.
4. Увидеть, что оно что-то делает не так или падает.
5. Починить.

6. Ещё раз. Ещё. Ещё.

На работе то же самое, только куски чужие, сроки чужие, и рядом сидят люди, которые уже сто раз хватались за то же место. Тебя берут не за сертификат с онлайн-курса. Тебя берут, потому что ты умеешь **дотащить маленький кусок до рабочего состояния** и потом своими словами рассказать, что сделал.

Это скучнее, чем в кино. В кино хакер в чёрном капюшоне за три секунды взламывает Пентагон. На работе джун сорок минут выясняет, почему кнопка «сохранить» не сохраняет, и причина — пробел в конце строки. Потом он это чинит. Потом пишет тест, чтобы

пробел больше не прилетал. Потом идёт пить чай. Это и есть профессия. Если такой день тебя бесит заранее — может, профессия не твоя. Если такой день кажется нормальным ремеслом — добро пожаловать на станцию.

2.1 Мифы, которые лучше выкинуть в шлюз

«Нужен математический склад ума». Нужна готовность тупеть вслух. Алгебра помогает иногда. Упрямство помогает каждый день. Человек, который не стыдится написать на бумажке «что я только что сделал», обгоняет гения, который стесняется спросить.

«Надо знать десять языков». Надо знать один достаточно, чтобы на нём врать было стыдно, и уметь открыть второй, когда понадобится. Эта книга берёт два: Lisp — чтобы не соскучиться, Java — чтобы тебя взяли. Десять языков в резюме без репозитория выглядят как десять отвёрток без ручки.

«Сначала теория, потом практика». Сначала практика на крошечном куске, потом теория, которая **объясняет**, почему кусок сработал. Иначе теория — аудиокнига для сна.

«Настоящие программисты не гуглят». Настоящие программисты гуглят лучше. Разница: они понимают, что

вставили, и могут выкинуть половину.

«Джуну нельзя без Kafka». Джуну нельзя без программы, которую можно запустить. Kafka подождёт. Она никуда не денется, к сожалению.

2.2 Что на самом деле смотрят

Вакансии написаны как заклинание: Кафка, кубер, микросервисы, «опыт от года». Первые четыре месяца это можно не читать — там маркетинг и страх HR. Живой фильтр почти всегда скучнее:

- Java: типы, списки, чем `equals` отличается от ==, исключения, объекты.

- HTTP: методы, статусы, JSON.
- SQL: выбрать с соединением таблиц, зачем ключ, зачем индекс.
- Spring Boot: контроллер принимает HTTP, сервис — правила, репозиторий — база, транзакция на сервисе.
- Git: коммиты, дифф, не падать в обморок от ветки.
- Свой проект, который рассказываешь за три минуты **без бумажки**.

Кафка в резюме, а в репозитории пусто — сразу видно. Курс без GitHub — тоже.

Давай медленно
Человек с той стороны стола не проверяет, выучил ли ты слово «микросервис». Он проверяет,

не развалишься ли ты, когда тебе дадут тикет «кнопка сохраняет пустое имя» и кусок чужого кода. Если ты уже делал такое **себе** — ты не новичок в том смысле, которого они боятся. Ты новичок в том смысле, которого можно научить.

На собесе часто просят написать цикл, объяснить JOIN и рассказать про свой репозиторий. Три вещи. Не «спроектируй YouTube». Если ты эти три вещи умеешь без паники — ты уже дальше половины писем, которые HR выкидывает за «проходил курс, диплом прилагается».

2.3 Как выглядит день, когда тебя уже взяли

Не Голливуд. Упал тест. Тикет: «кнопка сохраняет пустое имя». Читаешь чужой код. Пишешь двадцать строк. Споришь, как назвать поле. Созвон, на котором двадцать минут обсуждают, `cancelled` это статус или отдельная таблица. Обед. Ещё тикет. Это не вся профессия. Это **стыковка**: трясёт, зато ты уже на станции.

Иногда день красивый: нашёл, почему прод глючит по понедельникам, и это была таймзона. Иногда день тупой: переименовать поле в пяти местах. Оба дня — работа. Романтизировать только красивые — путь выгореть к

третьему месяцу, когда красивых мало.

Тебя будут спрашивать «ну как успеваешь». Честный ответ «не знаю, вот что уже сделал» лучше театра «всё супер». Станция любит датчики, не оптимизм без цифр.

2.4 Куда это растёт, если не сбежать в лес

Дальше, если не застрять в копировании аннотаций:

- **Middle** — тебе доверяют кусок системы, а не одну кнопку. Ты уже говоришь «это плохая идея» и иногда тебя слушают. Зарплата растёт. Сообщений в Slack меньше не становится.

- **Senior** — люди приходят с вопросом, а не с готовым тикетом. Решаешь, **что** строить, а не только **как** написать цикл. Зарплата растёт быстрее, чем количество волос. Появляется неприятная обязанность: останавливать чужие плохие идеи, в том числе свои вчерашние.
- **Architect** — решаешь, как устроена станция: какие отсеки есть, как стыкуются, что будет, если этот люк открыть. Стрелочки на схеме — следствие. Дело — решения, от которых зависит, развалится станция через полгода или нет. Иногда споришь, нужен ли Kafka **на этот раз** по делу.

- **Manager** — если код внезапно расхочется. Тогда встречи и вопрос «ну как у нас сроки». Тот же `loop`, только аргументы — люди, и `nil` они возвращают реже, чем обещают. В учебник это не входит, и мы все вздохнули с облегчением.

Эта книга — про первый люк. Остальное случится само, если не перестанешь чинить штуки.

На первой неделе надо, чтобы `Hello` печаталось. Карьера — это то, что вырастает из кучи вечеров, а не из слайда «траектория».

2.5 Английский. Да, обязательно

Java, Lisp, Spring, ошибка компилятора, Stack Overflow, большая часть книжек — на английском. Не «бизнес-инглиш для переговоров в аэропорту». Слова вроде `null`, `commit`, `stack trace`. Без этого тяжело: гуглить придётся через боль, а документация не обязана существовать по-русски.

Duolingo, сериалы с субтитрами, README чужих репозиториев — что угодно, лишь бы каждый день чуть-чуть. В наши сорок минут Lisp плюс два часа Java это **не входит**. Мы и так запихнули скобки. Английский — домашний квест. Отгадки в конце книги нет, есть интернет.

Мак, Windows, WSL
Интерфейс IntelliJ можно оставить русским, если так спокойнее. Ошибки компилятора всё равно будут на английском. Привыкай читать ошибки, не переводить каждое слово в Яндексе: к третьей неделе `cannot find symbol` станет таким же бытовым, как «нет молока».

Не надо ждать «сначала выучу английский, потом Java». Нужен уровень «прочитал абзац документации и не умер». Он вырастает по дороге, если не избегать английских README как огня.

2.6 Зачем тогда Lisp

Потому что вход в Java легко превратить в шесть месяцев аннотаций `@Autowired` и тихое желание уйти в лес. Lisp в REPL показывает те же идеи голыми: список, функция, рекурсия, «данные — тоже код». Поиграешь со списками — `ArrayList` и JSON перестанут быть иероглифами. Сорок минут в день — чтобы не возненавидеть программирование к третьему месяцу Спринга.

В резюме можно ткнуть Lisp в «ещё умею». На собесе не спросят. И славно. Спросят про `HashMap`. А `HashMap` после alist в Lisp — уже не магия, а карман с подписями, только быстрый.

Ещё Lisp лечит страх ошибки. В REPL сломал — получил сообщение, починил, живёшь. Нет пятиминутной сборки, чтобы узнать, что забыл скобку. Этот рефлекс («сломал → читаю → чиню») потом спасает в Java, где сборка как раз пять минут, а ошибка — одна забытая скобка.

2.7 Полгода — это плотно

Будни: два сорок. Плюс суббота. Если к двенадцатой неделе нет своего сервера с базой — не Кафку подкручивай, а доделай сервер. Интерес дешевле потерянного месяца. Пропустил неделю — не читай три главы за субботу, это не подвиг, это каша в голове. Возьми текущее занятие и свой проект.

Шесть месяцев × примерно шесть вечеров × два сорок — это много часов. Этого хватает, чтобы пальцы запомнили. Этого не хватает, чтобы стать человеком, который «уже всё видел». И не надо. Джуна берут не за «всё видел». Джуна берут за «вижу, чиню, рассказываю».

Если работаешь на другой работе полный день — шесть месяцев могут растянуться в восемь месяцев. Нормально. Календарь в книге — план, не таймер бомбы. Станция «МОДУЛЬ» тоже строилась не по спринту из Jira, судя по изоленте.

2.8 Про списывание

Чужие ответы — после своей попытки. На работе гуглить будут каждый день, это норма. Разница в том, умеешь ли ты **прочитать** то, что вставил, и поменять одну строку, не развалив остальное.

Списать квест из отгадки в первую минуту — как посмотреть ответ в кроссворде и решить, что ты умный. Кажется, что понял. Не понял. Через месяц на собесе кроссворд будет без отгадки в конце книги.

Подсмотреть через двадцать минут неудач — нормально. Ты уже подумал. Мозг получил запрос. Отгадка тогда **ложится**, а не пролетает.

Код из этой книги — можно в свои учебные репозитории. Чужие книги и коммерческие репозитории — нельзя выдавать за своё. Станция и так на изоленте, давай без судебных исков и без вранья в резюме. Враньё на орбите кончается открытым шлюзом.

3 Как устроен компьютер, если ты не программист

Эта глава — для человека, который ещё не писал программ. Совсем. Если ты уже ставил JDK и спорил с `PATH`, можно пробежаться глазами и идти в мастерскую. Если слово `терминал` звучит как угроза из фильма про хакеров — садись. Чай можно. Скафандр не нужен.

Станция «МОДУЛЬ» висит на орбите и притворяется умной. На самом деле она тупая, как молоток. Молоток тоже не думает. Он бьёт туда, куда его послали, даже если послали в палец. Компьютер такой же:

слушается **точно**, не «в общем, понял идею».

Дальше — медленно. Без цифровой электроники, без «архитектуры фон Неймана на доске», без чувства, что ты опоздал на первый курс. Только кухня, кладовка и чёрное окно, которое почему-то всех пугает.

Давай медленно
Если абзац не щёлкнул — перечитай его вслух.
Компьютер не обидится. Он вообще ничего не чувствует, это его суперсила и его проклятие.

3.1 Коробка, которая тупо слушается

Компьютер — коробка. Внутри моторы из песка (почти правда:

чипы делают из кремния), снаружи кнопки, экран, иногда наклейка «не лить кофе». Ты говоришь ей, что делать. Она делает **ровно это**.

«Ровно это» — ловушка.

Человеку говоришь: «поставь чайник». Он нальёт воду, включит, подождёт, крикнет «готово». Компьютеру надо расписать:

1. Проверь, есть ли чайник. Если нет — скажи об этом и остановись.
2. Возьми чайник.
3. Если воды нет — налей.
4. Если воды слишком много — вылей лишнее.
5. Поставь на подставку.

6. Включи.
7. Жди, пока не закипит **или** пока не пройдёт пять минут.
8. Выключи.

Забудешь пункт 1 — программа попытается взять отсутствующий чайник. Тепловая смерть вселенной подождёт; сначала почини проверку. Забудешь пункт 4 — зальёт камбуз. Забудешь «выключи» — сгорит спираль, и бортмеханик будет очень красноречив.

Это не метафора «для детей». Это профессия. Программист — человек, который пишет такие списки и потом удивляется, какой пункт забыл.

Закон станции

Компьютер не догадывается. Он не «почти понял». Либо инструкция есть, либо её нет. Третьего не дано, даже если ты очень вежливо попросил.

На станции «МОДУЛЬ» это выглядит так. Ты пишешь: «открой шлюз». Железо открывает шлюз. Оно не спросит: «а скафандр надет?» — если ты это не написал. Поэтому половина программ на свете — не «сделай штуку», а «сделай штуку, **и проверь, что не убьём всех**».

Инструкция называется **программой**. Слово страшное, штука простая: текст (или почти текст), который коробка умеет выполнить. Пока не

умеет — это просто письмо самому себе.

Мы ещё не пишем программы. Мы сначала смотрим, **где** они живут и **кто** их готовит.

3.2 Кухня станции

Представь камбуз «МОДУЛЯ». Не дата-центр. Камбуз.

Повар — процессор, CPU. Он ничего не хранит надолго. Он **делает**: режет, мешает, считает, сравнивает. Очень быстрый, очень забывчивый. Отвернулся — уже не помнит, сколько соли. Поэтому рядом стоит стойка.

Стойка — оперативная память, RAM. То, с чем повар работает **сейчас**. Ножи, миска, открытый рецепт, недорезанный лук.

Выключили свет на станции — со стойки всё смахнули. Рецепт в голове повара тоже исчез: у CPU нет «головы на завтра».

Кладовая — диск. Жёсткий, SSD, «память ноутбука», флешка — для нас сейчас одно: место, где лежит, когда свет выключили. Банки с крупой, банки с программами, банки с твоими фотографиями из отпуска, который станция не одобряет. Достать банку из кладовой — медленнее, чем взять миску со стойки. Поэтому повар не бегает в кладовую за каждой ложкой.

Рецепт — программа. Текст: что делать с луком. Рецепт лежит в кладовой (файл на диске). Когда ты «открываешь программу»,

кто-то копирует рецепт на стойку, и повар начинает по нему работать.

Давай медленно
Файл на диске — банка в кладовой. Запущенная программа — каша, которую **прямо сейчас** мешают на стойке. Это разные вещи. Банка может стоять годами. Каша остывает, как только плиту выключили.

Ещё гости камбуза, коротко, без экскурсии в цех:

- **Экран и клавиатура** — окошко в камбуз и рот, которым ты кричишь повару. Мышь — палец, который тыкает в картинки.

- **Сеть** — лифт на другие кухни. Об этом позже, там будут письма.
- **Видеокарта** — отдельный повар для картинок. Для этой книги почти не нужна. Пусть жарит свои кадры.
- **BIOS / прошивка** — записка на двери: «как включить плиту, если всё остальное ещё спит». Не трогай, пока станция не попросит.

Почему игра тормозит, а блокнот — нет? Повар один (ну, несколько рук, но не бесконечность). На стойке мало места. Если рецепт огромный — часть банок придётся держать в кладовой и таскать туда-сюда. Таскание видно: всё «думает». Это не потому что компьютер

тебя не любит. Это потому что кладовая далеко, а стойка маленькая.

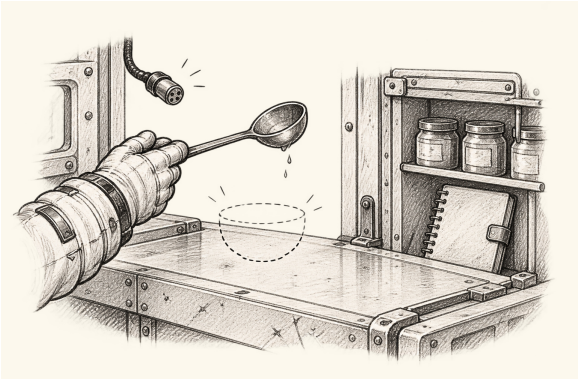
Так себе идея
«У меня восемь гигабайт оперативки, значит можно держать восемь гигабайт вкладок». Можно. Повар начнёт постоянно бегать в кладовую. Это называется swap, и это грустно. Закрой двадцать чатов. Станция скажет спасибо.

Как выглядит обычное утро:

1. Ты включаешь коробку.
2. С диска поднимается операционная система — главный рецепт, который дальше пускает остальные.
3. Ты тыкаешь в иконку.

4. Система находит файл в кладовой, копирует нужное на стойку, говорит CPU: «вари».
5. Повар варит, пока ты не закроешь окно или пока каша не убежит (ошибка, вылет, «не отвечает»).

Иконка — картинка. Картинка ничего не считает. Считает файл, на который картинка **указывает**. Иногда указывает не туда. Тогда ты злишься на картинку. Злись на путь. Путь мы разберём.



Свет вернулся. Несохранённая каша — нет.
Рабочая RAM энергозависима; сохранённые файлы обычно переживают выключение.

3.3 Файлы, папки, путь, хвост с точкой

Файл — банка с именем. Внутри байты. Байты мы ещё потрогаем. Сейчас важно: файл **лежит где-то**. «Где-то» — дерево папок.

Папка (каталог, директория — одно и то же в быту) — ящик, в котором стоят банки и другие ящики. На маке часто говорят «папка», в терминале — `directory`. Не пугайся синонимов. Кладовщики любят три слова на одну полку.

Путь — как дойти от входа в кладовую до банки.

На маке и в Linux (и в WSL, который Linux в пальто Windows) путь выглядит так:

```
/Users/vasya/dev/lisp-experiments/hello.lisp
```

Читается слева направо, как коридор станции:

- `/` — корень, главный люк.
- `Users` — отсек с людьми.

- `vasya` — твоя каюта.
- `dev` — верстак.
- `lisp-experiments` — ящик со скобками.
- `hello.lisp` — банка.

На Windows без WSL тот же смысл, другой почерк:

```
C:\Users\vasya\dev\lisp-
experiments\hello.lisp
```

`C:` — диск, как отдельный склад. Палочки **обратные**. Если скопируешь линуксовый путь в родной `cmd`, он обидится. Поэтому в этой книге на Windows мы почти сразу уйдём в WSL и будем писать пути с обычными `/`.

Мак, Windows, WSL

Мак. Терминал → путь как `/Users/...`. В Finder можно перетащить папку в окно терминала — он вставит путь. Ленивый и правильный жест.

Windows + WSL2 (рекомендую). Внутри Ubuntu твой дом — `/home/vasya/...`. Диск `C:` виден как `/mnt/c/Users/...`. Жить лучше **в** `/home`, не на `/mnt/c`: иначе Linux и Windows будут спорить про права и концы строк, как два механика про один ключ.

Windows без WSL. Проводник показывает `C:\Users\...`. PowerShell понимает оба вида палочек **иногда**. Не испытывай судьбу: либо WSL, либо всегда обратные слэши и кавычки вокруг путей с пробелами.

Пробел в имени папки — мелкий злодей. Путь `мой проект` терминал читает как два слова: `мой` и `проект`. Кавычки спасают: `"мой проект"`. Ещё лучше — не ставить пробелы в учебных папках. `lisp-experiments` звучит скучно и никогда не кусает.

Имя файла часто с хвостом: `.txt`, `.java`, `.lisp`, `.pdf`. Это **расширение**. Не магия. Не печать качества. Просто договор: «по последним буквам догадайся, чем открывать».

- `записка.txt` — «это текст, открой блокнотом».
- `Hello.java` — «это текст, но Java-компилятор считает его своим рецептом».
- `hello.lisp` — «это текст, SBCL обрадуется».

- `фото.jpg` — «это не текст, не открывай блокнотом, будут крякозябры и слёзы».
- `архив.zip` — «это пачка файлов в одной банке, сжатая».

Можно переименовать `Hello.java` в `Hello.txt`. Байты внутри **не изменятся**. Изменится только табличка. Блокнот откроет и покажет тот же код. `javac` может обидеться: он ищет `.java`. Операционная система смотрит на хвост, когда ты дважды кликаешь. Программист смотрит **внутрь**, когда уже в терминале.

Давай медленно
Расширение — ярлык на банке. Не вкус каши. Каша — содержимое. Ярлык можно наклеить вранью. Поэтому

«файл сломан», а хвост `.mp3`, бывает. И наоборот: рецепт Java, сохранённый как `.txt`, всё ещё рецепт, просто плита его не узнает по одежке.

Скрытые файлы на маке и в Linux начинаются с точки: `.gitignore`, `.ssh`. Finder их стесняется. Терминал показывает, если попросить `ls -a`. Это не секретная полиция. Это «не мешайся в обычном списке».

Два слова, которые путают все:

- **Файл не найден** — путь неверный: опечатка, не та папка, не тот диск.
- **Нет прав** — банка есть, но кладовщик не даёт трогать. На своей учебной папке это

редко. В системных `/usr`, `C:\Windows` — пожалуйста, не геройствуй.

3.4 Текст и «компьютер понимает»

Вот сюрприз, который экономит год жизни.

Почти всё, что ты будешь писать в этой книге — **текст**. Буквы, скобки, точки с запятой. Тот же сорт штуки, что письмо бабушке, только бабушка не требует `public static void`.

`Hello.java` открывается в блокноте. `hello.lisp` — тоже. `README.md` — тоже. Даже `.json` и `.xml` и `.html` — текст. Текст можно читать глазами. Текст

можно испортить одним лишним символом.

Компьютер «понимает» текст не как человек. Человек видит «энергия 80» и догадывается про реактор. Компьютер видит байты, переводит их в числа, числа в команды. Если в рецепте опечатка — он не поправит. Он скажет «не понял» или, хуже, понял **другое**.

Давай медленно
Есть два мира.

Мир 1: ты смотришь на файл и читаешь слова. Это для тебя.

Мир 2: какая-то программа (компилятор, интерпретатор, браузер) **ест** этот текст по строгим правилам. Это для неё.

«Компьютер понял» значит: программа-едок прожевала текст и не подавилась. Не значит, что смысл хороший. Можно идеально прожевать рецепт «открой шлюз без проверки». Станция поймёт. Экипаж — тоже, но коротко.

Бинарные файлы — не текст. Картинка, музыка, скомпилированная программа `Hello.class`, Word-документ `.docx` (он притворяется, внутри zip). Откроешь в блокноте — каша из символов. Не чини кашу руками. Для картинок есть другие инструменты. Для `.class` есть Java, ей и отдай.

Почему тогда IntelliJ и VS Code, если блокнот умеет текст? Потому что редактор для кода

подсвечивает скобки, ругается на опечатки, прыгает к ошибке. Блокнот честный, но как чинить реактор отвёрткой для очков. Можно. Один раз. Потом возьми нормальный ключ.

Ещё договор: **кодировка**. Буквы в файле — тоже числа. Про это будет свой отсек. Сейчас запомни одно: если открыл файл, а вместо «МОДУЛЬ» видишь ÐœÐžÐ”Ð£Ð›Ð¬ — никто не взломал станцию. Просто два кладовщика по-разному читают одни и те же банки.

3.5 Чёрное окно, или зачем всем нужен терминал

Терминал — программа, в которую ты **печатаешь**

команды текстом, а не тыкаешь иконки. Чёрный (или тёмно-серый) фон — привычка из семидесятых, когда экраны были зелёные, а мебель пепельница. Можно сделать белый. Все равно будут говорить «чёрное окно».

Зачем оно, если есть Finder и Проводник?

Потому что программе удобнее сказать «сделай вот этот рецепт с вот этими банками», чем изображать каждую кнопку. `javac Hello.java` — одно предложение. В меню это было бы: Файл → Открыть → найти чёрт-те где → Build → если пункт называется иначе в этой версии → сдаться.

Ещё потому, что ошибка в терминале — **текст**. Текст можно скопировать. Текст можно поискать. Картинка «красная лампочка в IDE» иногда прячет тот же текст в три клика вбок. Научись читать чёрное окно — и IDE станет помощником, а не жрецом.

Закон станции
Команда — тоже программа. `ls` — крошечный рецепт «покажи, что в ящике». Ты запускаешь программы целый день, даже когда «не программируешь».

Главная идея терминала, без которой всё остальное — цирк:

У окна есть **текущая папка**. Current directory. «Где я стою в кладовой». Команды по

умолчанию смотрят **сюда**. Не «на весь компьютер». Сюда.

Покажи, где стоишь:

```
pwd
```

Print Working Directory. Печатай рабочую директорию. На маке и в WSL это святая команда. Ответ вроде:

```
/Users/vasya/dev
```

или

```
/home/vasya/dev
```

На родном PowerShell `pwd` тоже часто работает (это псевдоним). Если вдруг нет:

```
Get-Location
```

Покажи банки в текущем ящике:

```
ls
```

List. Список. На маке и в WSL `ls` родной. В PowerShell `ls` тоже часто есть (псевдоним к `Get-ChildItem`). Древний `cmd` любит `dir`. Если `ls` не нашёлся — `dir`. Смысл один: что лежит **здесь**.

```
ls -la
```

В Linux/macOS: подробный список, включая скрытые с точкой. Столбики прав, размеров, дат. Не учи их наизусть сегодня. Просто знай, что `-la` — «покажи по-честному».

Перейти в другой ящик:

```
cd lisp-experiments
```

Change Directory. Смени директорию. Теперь `pwd` другой, `ls` другой. Ты **перешёл**.

На уровень вверх:

```
cd ..
```

Две точки — «ящик, в котором лежит текущий ящик». Как «выйти в коридор». Одна точка `.` — «здесь». Полезно, когда команда просит путь: `./mvnw` значит «файл `mvnw` **в этой** папке».

Домой:

```
cd
```

или

```
cd ~
```

Тильда `~` — твоя каюта: `/Users/vasya` или `/home/vasya`. Не корень диска. Каюта.

Абсолютный путь — от главного люка:

```
cd /Users/vasya/dev/java-basics
```

Относительный — оттуда, где стоишь. Если ты уже в `dev`:

```
cd java-basics
```

Оба правильные. Путаешь — `pwd`, потом думай.

Давай медленно
Большая часть «у меня не находится файл» — ты стоишь не в той папке. Не «компьютер стёр». Не «Java сломалась». `pwd`,

потом `ls`, потом уже паника. На станции этот ритуал называют «посмотри под ноги».

Создать ящик:

```
mkdir java-basics
```

Пустой файл (мак / WSL):

```
touch Hello.java
```

В PowerShell `touch` может отсутствовать. Тогда:

```
New-Item Hello.java
```

или открой редактор и сохрани. Файл не обязан рождаться из команды. Команда просто быстрее.

Печать содержимого текстового файла:

```
cat Hello.java
```

В PowerShell часто `Get-Content Hello.java` или `cat` как псевдоним. Если вылезла каша из символов — либо это не текст, либо кодировка (см. ниже).

Очистить экран, когда каша из вывода:

```
clear
```

В PowerShell ещё `cls`. Команды не удаляются из истории, только с глаз.

История: стрелка вверх. Повторить команду, не набирая. На маке и в WSL ещё Ctrl+R — поиск по истории. Наркотик. Потом не сможешь без него.

Прервать программу, которая зависла и сыплет выводом:

Ctrl+C. Не копирование (в терминале копирование обычно Cmd+C / Ctrl+Shift+C, зависит от окна). Ctrl+C — «стоп, повар, положи нож».

Выход из некоторых программ: `exit`, или `(quit)` в SBCL, или Ctrl+D (конец ввода). Если ничего не помогает — закрыть окно. Грубая сила. Работает.

Так себе идея
Не копируй из интернета команды, которые начинаются с `rm -rf /` или `del /s /q C:\`. Это «вынеси кладовую целиком». Учебник таких не даёт. Если кто-то дал — это не наставник, это шутник или хуже.

3.6 Три чёрных окна: Мак, PowerShell, Ubuntu в WSL

Одинаковая идея: текущая папка, команды, текст. Разная мебель.

Терминал на маке. Приложение «Терминал». Внутри обычно zsh (раньше был bash — неважно для нас). Команды из этой книги копируй сюда почти без перевода. Homebrew ставит программы в места, которые этот терминал видит.

PowerShell на Windows. Синее или чёрное. Умеет много. Часть линуксовых имён подделана псевдонимами (`ls`, `pwd`, `cat`). Часть — нет (`sbcl` появится,

только если поставишь). Пути с пробелами и `C:\`. Можно жить. Боль многих учебников именно здесь: автор писал под мак, у тебя PowerShell, одна палочка не та — и ты думаешь, что ты тупой. Ты не тупой. Диалект другой.

cmd.exe. Ещё старше. `dir`, `cd`, обратные слэши. Если можешь не открывать — не открывай.

WSL2 + Ubuntu. Вот это я рекомендую, если у тебя Windows. Подсистема: Linux **внутри** Windows, без второй машины и без вечной борьбы с палочками. Окно выглядит как терминал Убунту. `ls`, `pwd`, `apt`, `sbcl`, `javac` — как в учебнике, как на сервере, как у соседа на маке (почти).

Мак, Windows, WSL
Как включить WSL2, коротко. В Windows 11 или Windows 10 версии 2004 и новее открой PowerShell от администратора и выполни `wsl --install`. Перезагрузись, если попросит, затем открой Ubuntu. Первое окно спросит имя и пароль пользователя Linux. В PowerShell проверь `wsl -l -v`: у Ubuntu должна быть версия `2`. Если там `1`, следуй инструкции Microsoft по переходу на WSL2. Пароль при `sudo` не рисует звёздочки: печатай вслепую.

Потом:

```
sudo apt update && sudo apt
upgrade -y
```

Дальше почти все команды книги — **в это окно**. Не в PowerShell. Не в cmd.

Файлы учебника клади в `/home/<ты>/dev/...`. IntelliJ на Windows умеет открыть `\\wsl$\Ubuntu\home\<ты>\dev\...`. Docker Desktop на Windows сам попросит WSL2 — скажи да.

Мак. WSL не нужен. У тебя уже Unix. Ставь Homebrew и дыши.

Почему не «две системы в равных долях»? Потому что ты будешь гуглить ошибки. Большая часть ответов — для Linux-команд. Spring, Docker, сервер на работе — Linux. Выучить PowerShell можно. Сначала лучше выучить тот диалект, на котором говорит профессия.

Закон станции
На Windows: WSL2. Команды книги — в Ubuntu. Если что-то «не является командой» в PowerShell, спроси себя: а окно то? Часто нет.

Ещё путаница: **где лежит файл**.

Ты скачал `Hello.java` через браузер Windows в `Загрузки`. В WSL это `/mnt/c/Users/vasya/Downloads/Hello.java`. Можно оттуда запускать. Лучше скопируй в `/home/vasya/dev/java-basics/`, чтобы не собирать рецепты с чужого склада через дырку в стене.

Конец строки: Windows любит `\r\n` (два символа: верни каретку и новую строку). Linux и мак — `\n`. Git иногда пишет `LF will be replaced by CRLF`. Для

учебного Java/Lisp почти никогда не смерть. Если скрипт «не запускается» и пишет странное `^M` — вот оно, наследие блокнота. В WSL: `file твой.sh` иногда подскажет. Потом научишься. Сегодня просто знай, что **текст не всегда одинаковый текст**.

3.7 Программа, процесс, «оно запущено»

Файл `Hello.java` — не запущенная программа. Это рецепт в кладовой.

`Hello.class` — тоже не каша. Это рецепт, переведённый на более удобный для Java-плиты язык. Всё ещё банка.

Когда ты пишешь `java Hello` или жмёшь зелёную стрелку, операционная система:

1. Находит нужный файл.
2. Выделяет кусок стойки (память).
3. Даёт повару (CPU) задание.
4. Вешает на это задание бирку: **процесс**.

Процесс — **рецепт, который сейчас варят**, плюс миски (память), плюс номер на очереди.

Один и тот же файл можно запустить дважды — будет два процесса. Два блокнота. Два `java Hello`. Они не делят миски, если сами не договорились. Закрыл одно окно — второе живёт.

«Программа зависла» значит: процесс ещё числится, но не отвечает на «ну как там каша?». Иногда он считает (тяжёлый рецепт). Иногда ждёт сеть. Иногда ждёт тебя, а ты не видишь, **чего** он ждёт. Иногда мёртв, но табличка ещё висит.

«Не отвечает» в меню Windows / мака — вежливый перевод: «я стучал в дверь, молчат». Можно подождать. Можно принудительно закрыть. Принудительно — выключить плиту. Каша на стойке пропадёт. Банка в кладовой останется.

Давай медленно
Выключить программу ≠ удалить программу.

Закрыть окно — перестать варить. Файл на диске на месте.

Удалить файл — выкинуть банку. Запущенный процесс при этом **может ещё жить** (рецепт уже на стойке). Редко важно в первый месяц. Потом вдруг важно, когда «я стёр, а оно всё равно пишет».

Откуда берутся несколько процессов Java: IDEA сама по себе, твоя программа сама по себе, может ещё Maven. Это нормально. Это не «вирус». Диспетчер задач / Activity Monitor покажет имена. Имя `java` там будет часто. Не убивай всё подряд: можно пристрелить IDE.

Программа может быть **сервером**: процесс живёт и

слушает. Браузер к нему стучится. Закрыл терминал, где сервер бежал — сервер часто умирает. Поэтому потом появятся слова вроде Docker. Сегодня достаточно: если в окне мигает курсор после команды — программа, возможно, **ждёт**. Не открывай второе такое же, пока не понял, живо ли первое.

Как увидеть, что ты запустил в этом терминале: оно пишет тебе строки. Когда пишет приглашение снова (`$`, `%`, `*`, `PS C:\…`) — та команда **закончилась**. Если не пишет и не возвращает приглашение — она ещё работает или ждёт ввода.

SBCL: приглашение `*`. Ты внутри процесса `sbcl`. `(quit)` —

процесс умрёт, останешься в обычном терминале.

3.8 Компилятор и интерпретатор — одна каша, две плиты

Есть рецепт «скажи энергию». Вот он на двух языках, нарочно крошечный.

Lisp:

```
(print (+ 40 40))
```

Java:

```
public class Energy {
    public static void
main(String[] args) {
        System.out.println(40 +
40);
    }
}
```

Оба должны напечатать `80`. Смысл один. Обряд разный.

Интерпретатор читает рецепт и сразу выполняет. REPL — разговор вокруг вычисления; он не обязан быть интерпретатором. SBCL обычно компилирует формы под капотом, даже здесь. Ответ всё равно приходит сразу, без отдельной команды сборки. Запусти `sbcl`, увидишь `*`, печатай:

```
(+ 40 40)
```

Enter. `80`. Никакого отдельного «переведи файл». Повар смотрит на скобки **прямо сейчас**.

Что только что случилось

Ты написал выражение. SBCL прочитал. Посчитал. Напечатал результат. Снова ждёт. Read, Eval, Print, Loop. Отсюда REPL. Не храм. Калькулятор, которому можно объяснять станцию.

Можно положить то же в файл `energy.lisp`:

```
(print (+ 40 40))
```

и сказать:

```
sbcl --script energy.lisp
```

Всё ещё без отдельного «сборочного» файла на диске, который ты запускаешь потом. Скрипт прочитали и сделали.

Компилятор сначала переводит рецепт в другой вид, удобный машине (или плите

вроде JVM), и **потом** ты запускаешь перевод.

```
javac Energy.java
```

Появился `Energy.class`. Это не текст для тебя. Это байткод для Java-машины. Потом:

```
java Energy
```

`80`. Если в `.java` опечатка, `javac` ругается **сейчас**, и `.class` либо не родится, либо останется старый. Старый `.class` — отдельная подлость: ты починил исходник, забыл скомпилировать, запускаешь древнюю кашу. Поэтому зелёная стрелка в IDEA делает оба шага. В терминале помни про два.

Давай медленно

Компилятор — переводчик, который работает **до** ужина. Ошибку в грамматике поймает на берегу.

Интерпретатор — переводчик за столом. Говоришь фразу — сразу едят. Ошибку в третьей строке увидишь, когда до неё дошли.

Java для новичка кажется «строже», Lisp — «живее». На работе оба сорта бывают. Даже Java в каком-то смысле потом интерпретирует свой байткод. Не цепляйся к чистой классификации. Цепляйся к обряду: **надо ли сначала javac**.

Одна и та же идея «сложить 40 и 40»:

Шаг	Lisp, SBCL	Java
Рецепт	скобки в REPL или `.lisp`	`Energy.java`, класс, `main`
Перевод заранее	не обязателен	`javac` → `.class`
Запуск	Enter в REPL или `--script`	`java Energy`
Опечатка	ошибка сразу в этой фразе	часто уже на `javac`
Повтор	снова набрать или вставить форму	исправил файл → снова `javac` и `java`

Почему Java такая многословная с `public class` и `main`? Потому что плита большая и любит анкеты. Почему Lisp можно в одну строку? Потому что REPL и так

знает, что ты хочешь «посчитать вот это». Не значит, что Lisp игрушечный. Значит, вход ниже. Зато вакансий с Java больше. Поэтому в книге оба.

IDE прячет компиляцию. Это удобно. Один раз в неделю запускай из терминала, чтобы не забыть, **что** прячут. Иначе кнопка Run снова станет магией, а мы магию только что разобрали.

Так себе идея
Не путай файл и процесс и ещё компилятор. `javac` — отдельная программа. Она поработала и умерла. Потом `java` — другая программа, другой процесс. Две плиты в одном камбузе, очередь.

3.9 Биты и байты без паяльника

Компьютер любит два состояния: есть ток / нет тока, яма / бугорок, да / нет. Удобно назвать это 0 и 1. Один такой ответ — **бит**.

Бит — слишком мелкий, как крошка соли. Их пакуют по восемь: **байт**. Байт умеет отличить 256 вариантов (от 0 до 255). Этого хватит на латинскую букву, на маленький кусок картинки, на «какая команда».

Дальше только ярлыки, чтобы не говорить «миллион миллионов»:

- килобайт (КБ) — примерно тысяча байт (иногда 1024,

кладовщики спорят, тебе сейчас всё равно).

- мегабайт (МБ) — примерно миллион.
- гигабайт (ГБ) — примерно миллиард. Фильм, игра, оперативка.
- терабайт (ТБ) — диск «ну хватит же».

Давай медленно
Число `80` в программе — не всегда один байт «восемьдесят». Для человека 80 — две цифры на бумаге. Для машины есть разные коробки: маленький целый, большой целый, дробный. Java пишет `int`, `double` — это размеры коробок. Пока не выбирай коробку как сомелье. Знай: **числу тоже нужно место на стойке**.

Текст «МОДУЛЬ» — несколько байт подряд. Картинка — очень много байт: цвет каждой точки. Песня — ещё больше, если не сжимать. Сжатие без потерь, например обычный ZIP, компактнее записывает повторы и восстанавливает каждый исходный байт. JPEG и MP3 обычно отдают часть деталей картинки или звука за меньший размер. Иногда обмен заметен: артефакты.

Диск «полный» значит: в кладовой кончились полки для байт. RAM «кончена» — на стойке нет места для каши. Это разные полки. Диск 512 ГБ не лечит нехватку 8 ГБ оперативки. Можно поставить большую кладовую и всё равно спотыкаться на стойке.

Зачем это программисту в первый месяц? Чтобы не верить в магию размеров. Чтобы понимать: файл `.class` — тоже байты. Чтобы ошибка «out of memory» не звучала как проклятие, а как «стойка забита». Чтобы «битый файл» значило: байты не в том порядке, который едок ждал.

Мы не будем сегодня переводить числа в двоичную систему столбиком. Это милый спорт. На работе джуна его почти не спрашивают. Спросят, почему ты кладёшь картинку в git на сто мегабайт. Ответ: байты весят, репозиторий — не кладовая для фильмов.

3.10 Буквы — это числа. UTF-8. Крякозябры

Клавиатура врёт. Ты думаешь, что в файл летит буква «М». На диск летит **число**, которое по таблице значит «М».

Таблица — кодировка. Договор: какое число какая буква.

Когда-то договоров было много, как диалектов на станции после столетия изоляции:

- ASCII — латиница, цифры, знаки. Хватает на `Hello`, не хватает на «МОДУЛЬ».
- Windows-1251, KOI8-R, CP866 — разные попытки впихнуть русские буквы в один байт на букву. Героические. Несовместимые друг с другом.

- UTF-8 — нынешний мир. Умеет почти все буквы Земли, эмодзи и знаки, которыми ты не должен писать имя переменной.

UTF-8: латинская буква часто занимает 1 байт, русская — 2 байта, некоторые иероглифы и эмодзи — больше. Тебе не нужно это помнить. Нужно помнить: **файл без пометки «я UTF-8» могут прочитать другой таблицей**.

Мойибейк, mojibake, крякозябры — когда байты писали одной таблицей, а читают другой. «МОДУЛЬ» в UTF-8, открытый как Windows-1251, превращается в РњРћР”РЈР›Р¬. Если те же байты прочитать как Windows-1252,

получится ÐœÐžÐ”Ð£Ð›Ð¬. Таблицы разные, ад общий.

Давай медленно
Крякозябры — не «файл умер». Байты часто целы. Сломан **переводчик**. Сначала сделай копию исходного файла, не сохраняя поверх него испорченный текст. Заново открой оригинал в его настоящей кодировке (в нашем примере UTF-8; в VS Code — Reopen with Encoding). Проверь, что буквы читаются. Только потом, если нужно, сохрани правильно прочитанную копию в UTF-8. Не перепечатывай и не сохраняй кракозябры «как есть» — получишь **новые** байты поверх старой путаницы.

Практические правила на первый год:

- В IDEA / VS Code смотри угол: должно быть UTF-8. Если нет — поставь.
- Файлы учебника, `.java`, `.lisp`, README — UTF-8.
- Не копируй код из Word и из «умного» PDF, где кавычки «ёлочки» вместо `"`. Компилятор не считает ёлочки кавычками. Для него это просто красивые кляксы.
- Имена файлов с русскими буквами **могут** работать. Могут и не работать в старых инструментах. Учебные `Hello.java`, `module.lisp` — латиница, меньше сюрпризов. Строки внутри файла — любые, UTF-8.

В терминале тоже есть кодировка окна. Редко кусает на маке. На старом `cmd` кусало часто. Ещё один голос за WSL.

Java любит писать `\u041c` вместо буквы — это «номер буквы в таблице Unicode». Unicode — большой каталог символов. UTF-8 — один из способов уложить номер в байты. Можно жить, не сдав этот экзамен. Можно просто не вставлять в код символ, которого не видно: неразрывный пробел из веба выглядит как пробел и ломает всё. Если «одинаковые» строки не равны — подозривай невидимое.

Так себе идея
Сохранил файл «как ANSI» из блокнота Windows — подарок

будущему себе с гранатой. Блокнот научился UTF-8, но пункты меню всё ещё умеют предавать. Смотри, **чем** сохраняешь.

3.11 Сеть: две кухни и письма

Пока компьютер один — камбуз. Сеть — коридор между камбузами.

Твоя машина. Их машина (сайт, сервер, телефон друга). Между ними не «трубка с водой», а договор слать **пакеты**. Пакет — конверт: куда, откуда, кусок данных, номер «я часть 4 из 10».

Ты открываешь браузер, вводишь адрес. Дальше

мультяшно, без диплома связиста:

1. Браузер спрашивает: «кто такой `example.com`?» — как «в какой каюте живёт Вася». Отвечает DNS, записная книжка имён. Имя → номер.
2. Номер — IP. Адрес квартиры, не фамилия.
3. На квартире много дверей. Дверь — **порт**. 80 и 443 — «сюда стучат браузеры за сайтами». 5432 — часто Postgres. 8080 — твой учебный сервер, когда до него доберёшься.
4. По коридору бегут конверты. Их могут потерять. Протокол (TCP — тот, что под вебом) умеет переспросить:

«четвёртый конверт не дошёл».

5. Сервер читает письмо («дай главную страницу»), кладёт ответ в конверты обратно.
6. Браузер собирает конверты в страницу.

Давай медленно `localhost` — «эта же кухня». Не интернет. Адрес себя. Когда сервер учишься поднимать у себя, браузер ходит на `http://localhost:8080` как на соседнюю плиту в том же камбузе. Никакой магии облака. Процесс слушает дверь 8080, другой процесс (браузер) в эту дверь стучит.

Wi-Fi оборвался — конверты перестали ходить. Программа, которая ждала ответ, может

висеть. Это не всегда баг в твоём `if`. Иногда просто письмо не дошло.

HTTP — язык писем для веба: «GET /tasks» значит «дай список задач», «POST» — «прими новую». JSON — часто **тело** письма, текст вида `{"energy":80}`. Подробности будут месяцами. Сейчас картинка: не «сайт живёт в браузере». Сайт живёт на чьей-то машине. Браузер — окно, через которое ты читаешь чужие банки.

Файрвол — придирчивый вахтёр на двери портов. Иногда твой сервер жив, а вахтёр не пускает. Иногда антивирус. Иногда корпоративная сеть. Сообщение «connection refused» — «дверь закрыта или там

никого». «timed out» — «даже не знаю, есть ли дверь, конверты пропали в коридоре».

Мы не настраиваем сегодня роутер. Мы только убиваем миф, что интернет — облако без адресов. Адреса есть. Двери есть. Письма иногда теряются. Программы, которые ходят в сеть, обязаны думать: «а если не ответили?»

Закон станции
Чужой компьютер — не твоя кладовая. Ты просишь письмом. Тебе могут отказать. Тебе могут соврать. Тебе могут не ответить. Это не обида. Это сеть.

3.12 Ошибки — подарок

Новичок видит ошибку и слышит: «я непригоден». Программист видит ошибку и слышит: «вот строка».

Компилятор, интерпретатор, операционная система — существа без такта и без лени. Они не намекают. Они пишут, **что не так**, часто даже **где**. У компилятора начинай с первой ошибки: следующие могут быть её хвостом. У исключения во время работы читай цепочку причин (`Caused by` в Java) и найди значимую причину вместе со строкой своего кода. Первая строка не всегда говорит, кто дёрнул датчик.

Примеры подарков:

- `No such file or directory` — путь. `pwd`. `ls`. Не «Java умерла».
- `command not found` / «не является командой» — программа не установлена или окно не то, или `PATH` не знает, в каком ящике лежит банка с командой. `PATH` — список коридоров, где система ищет программы. Поставишь JDK, забудешь PATH — `javac` «пропал». Он не пропал. Его не ищут там, где ты стоишь.
- `cannot find symbol` в Java — опечатка в имени, или забыл импорт, или файл не тот.
- Lisp `unmatched close parenthesis` — скобка лишняя. Подарок. Правда. Посчитай.

- `Connection refused` — никто не слушает дверь. Сервер не запущен, или порт другой.

Давай медленно
Ошибка — не оценка. Это датчик на станции. Датчик срабатывает, потому что воздух кончается, а не потому что ты плохой человек. Выключишь датчик (`catch` всего подряд, игнор ошибок в IDE) — тихо умрёшь в каюте. Читай датчик.

Что делать руками, всегда одно и то же:

1. Прочитать **целиком** первую ошибку компилятора; у исключения — сообщение и цепочку причин. Не «первое слово ошибки».

2. Посмотреть номер строки в **твоём** файле.
3. Если путь — `pwd` и `ls`.
4. Если команда не найдена — то ли окно, то ли установка, то ли PATH.
5. Потом уже поиск в интернете. Копируй ошибку **без** своего имени пользователя и без уникального пути каюты. Иначе найдёшь только свои же следы.

Стыдно не «получить ошибку». Стыдно закрыть глаза и тыкать, пока само не пройдёт. На работе само не проходит. На «МОДУЛЕ» тем более: вакуум терпеливый, воздух — нет.

Закон станции

Сломал — хорошо. Значит, ты не смотрел видео, а трогал станцию. Почини по тексту ошибки, не по настроению. Настроение врёт чаще компилятора.

3.13 Одна смена в терминале, медленно, вслух

Давай пройдём ту же прогулку, как если ты уже сидишь за столом. Не ставь пока Java. Только ноги в кладовой.

Открыл окно. На маке слева часто имя машины и `~`. В Ubuntu внутри WSL — `vasya@pc:~$`. Тильда — ты дома. Знак `$` или `%` — «можно писать команду». Это не деньги.

```
pwd
```

Прочитай вслух. `/Users/vasya` или `/home/vasya`. Это каюта.

```
ls
```

Увидишь `Desktop`, `Documents`, `Downloads` — или по-русски, если система так назвала. На маке Finder и `ls` — одна кладовая, разный вид. Удалил в Finder — в `ls` тоже нет. Не два мира. Два окна.

Создай верстак:

```
mkdir dev
cd dev
mkdir module-cabin
cd module-cabin
pwd
```

Теперь путь должен кончаться на `module-cabin`. Если нет — ты

либо создал не там, либо `cd` не сработал (опечатка в имени). `mkdir` молчит, когда успех. Молчание в терминале часто значит «ок». Это невежливо и привычно.

Файл с текстом, мак / WSL:

```
echo "станция МОДУЛЬ" > note.txt
cat note.txt
ls
```

`>` — «положи вывод в файл, затри старое». Если файл уже был — старое умрёт. Два знака `>>` — дописать в конец. Для учебной записки хватит редактора. `echo` — чтобы не уходить.

Мак, Windows, WSL
PowerShell. `echo` есть. Перенаправление `>` тоже есть.

Кодировка файла иногда бывает UTF-16 — сюрприз для русских букв. Ещё один голос за то, чтобы учебные файлы рождались в WSL или в нормальном редакторе с UTF-8.

Вернуться наугад:

```
cd ..
pwd
ls
```

Ты должен увидеть `module-cabin` как папку, а не жить внутри неё. `две точки` — самый частый танец первой недели. Перешагнул — `pwd`. Не стыдно.

3.14 Слова после команды: аргументы и флаги

`ls` — программа. `ls -la` — та же программа, ей передали записку `-la`. Записка — **аргумент**. Минусы вначале часто называют **флагами**: «включи режим».

```
ls module-cabin
```

Аргумент без минуса — обычно **к чему** применить: покажи вот эту папку, не текущую. Текущая при этом не меняется. `ls x` — посмотреть в ящик x. `cd x` — перейти в ящик x. Разные глаголы.

`java Energy` — программа `java`, аргумент `Energy` (имя класса, не

файл `.class` в аргументе). `javac Energy.java` — наоборот, тут как раз файл. Путают все. Ты тоже будешь. Потом перестанешь.

Tab — дополнение имени. Начал `cd mod`, нажал Tab, терминал дописал `module-cabin`, если такая одна. Если несколько — трубит ещё букву или покажет список. Это не интеллект. Это список файлов, который уже есть в папке. Привыкай. Без Tab жизнь длиннее на опечатки.

Закон станции
Команда — имя программы, потом пробел, потом её аргументы. Как Lisp, только скобки ставит обычай, а не ты. Опечатка в аргументе — уже

другая ошибка, не «команда не найдена».

3.15 Кто главный повар: операционная система

macOS, Windows, Linux (Ubuntu в WSL) — это не «разные компьютеры». Железо похожее: CPU, RAM, диск. Разное — **главный рецепт**, который пускает остальные: кто рисует окна, кто читает клавиатуру, кто решает, можно ли тебе трогать файл.

Операционную систему можно думать как шефа смены. Ты не кричишь CPU напрямую. Ты кричишь шефу: «запусти этот рецепт». Шеф даёт повару время, кусок стойки, право читать банку. Без шефа каждая

программа дралась бы за одну миску. Бывало. Называлось плохо.

Давай медленно
Иконка на столе — не программа. Это картинка и ссылка: «шеф, запусти вот этот файл». Терминал делает то же, только ты произносишь имя сам. Шеф один и тот же.

Почему тогда «на Windows не работает команда с мака»? Потому что шефы раздали имена ящиков и штатных поваров. `ls` живёт у Unix-шефа. У Windows-шефа долго жил `dir`. WSL — это **второй шеф в той же коробке**. Поэтому команды учебника туда копируются.

«Установить программу» значит: положить файлы в

кладовую и сказать шефу, где искать (часто — дописать коридор в PATH). Иконка в меню — бонус. Удалить — убрать файлы. Иногда часть старых банок остаётся. Это «мусор после удаления», не призрак.

3.16 PATH, ещё раз, потому что все об это спотыкаются

Ты пишешь `javac`. Шеф не телепатирует. Он смотрит список папок: PATH. В каждой — есть ли файл с именем `javac`. Нашёл — запустил. Не нашёл — `command not found`.

Посмотреть список (мак / WSL):

```
echo $PATH
```

Каша из коридоров, разделённых двоеточием. Не учи наизусть. Узнай: **поиск идёт слева направо**. Два `java` в разных папках — победит тот, чей коридор раньше. Отсюда «в терминале старая Java, в IDEA новая».

```
which javac
```

(на маке/линуксе) — какую именно банку возьмёт. Пусто — не возьмёт никакую. В PowerShell близкий жест: `Get-Command javac`.

Так себе идея
Не копируй из чата «пропиши в PATH вот эту простыню из двадцати строк», если не

понимаешь, **какую папку** добавляешь. Неправильный PATH ломает уже работавшие команды. Сначала — закрыть и открыть терминал после установщика. Часто хватает.

3.17 Стдин, стдаут, труба на пальцах

Программа, когда её запустили из терминала, имеет три трубы:

- **stdin** — сюда течёт то, что ты печатаешь. `Scanner` это читает.
- **stdout** — сюда она пишет «обычный» вывод. `System.out`, `format t`.
- **stderr** — сюда пишет ошибки. Часто тоже на экран, но это **другая** труба. Поэтому можно сохранить вывод в

файл, а ошибки всё равно увидеть.

```
cat note.txt
```

`cat` читает файл и пихает его в stdout. Экран подхватил stdout — ты видишь текст. Можно подхватить файлом: `cat note.txt > copy.txt`. Труба между программами — `|`:

```
ls | cat
```

Смысл для пальца: вывод левой стал входом правой. На работе этим живут. Сегодня достаточно увидеть палочку. Не строй трубопровод из десяти команд «как взрослый». Сначала научись где стоишь.

Давай медленно

Когда программа «ждёт и ничего не пишет», она часто ждёт stdin: чтобы ты что-то напечатал. Не второй запуск. Напечатай. Или Ctrl+C, если не понял, **чего** ждёт.

3.18 Буква как число, ещё раз пальцами

Возьми латинскую `A`. В ASCII это число 65. `маленькая а` — 97. Пробел — 32. Компьютер не хранит «красивую А». Он хранит 65, а шрифт на экране рисует букву. Сменил шрифт — число то же.

Русская «М» в Unicode — другой номер (если очень надо: 1052). UTF-8 укладывает этот номер в **два** байта. Латинская `M` — в один. Поэтому файл с словом

`МОДУЛЬ` весит больше, чем файл с `MODULE`. Не баг. Арифметика.

Сравни в голове: «файл весит 12 байт» и «там 6 букв». Если UTF-8 и русский — сходится. Если кто-то открыл как Windows-1251, букв может стать «больше» или «крякозябры»: он режет два байта UTF-8 как две **разные** буквы 1251. Вот и крякозябры на кухне, без теории информации.

Пробел и «невидимый пробел» из сайта — разные числа. Для глаза одинаковы. Для `equals` — нет. Когда «я скопировал точно», а Java не согласна — подозревай числа, не судьбу.

3.19 URL — адрес на конверте

То, что ты клеишь в браузер, тоже текст. Не магия кнопки.

```
https://httpbin.org/get?foo=1
```

Куски:

- `https` — **схема**: как разговариваться. `https` — HTTP плюс шифр. `http` — без шифра. Для учёбы хватит: «буквы до `://` — правило писем».
- `httpbin.org` — хост. Имя кухни. DNS переведёт в IP.
- `/get` — путь ресурса, который понимает сервер. Он может выбрать обработчик программы или файл; это не

обязательно папка на диске и точно не твой `/Users`.

- `?foo=1` — хвост запроса, query. Мелкие пары «имя=value». Иногда там живут сессии и утечки. Не клади сюда пароли.

`http://localhost:8080/tasks` — схема http, хост localhost (эта коробка), дверь 8080, путь `/tasks`. Когда дойдёшь до сервера — эта строка перестанет быть заклинанием. Сейчас достаточно уметь разрезать.

Браузер спрячет тело ответа и **рисует**. `curl` и Java-HttpClient показывают тело как есть: HTML или JSON текстом. Поэтому «в браузере красиво, в curl каша тегов» — не сломано.

Каша тегов и **есть** страница, пока её не нарисовали.

3.20 Ещё подарки, которые выглядят как оскорбления

`Permission denied` — банка есть, шеф не даёт. На своей `dev` редко. На `/usr/bin` — пожалуйста. Не `sudo` на каждую чих. `sudo` — «я шеф на пять секунд». Ошибся под sudo — ошибешься громко.

`Address already in use` — дверь порта занята. Старый сервер жив. Найди процесс, прикрой или смени порт. Не переустанавливай компьютер «на всякий» как первый шаг. Можно, конечно. Это как чинить щель пересборкой станции.

`Segmentation fault` / «приложение неожиданно завершилось» — процесс умер грубее, чем Java-исключение. В учебной Java редко. Если умер SBCL на рекурсии — чаще увидишь текст про stack. Тоже подарок.

Красная полоса в IDEA без текста — щёлкни вкладку или нижнюю панель Run. Там лежит тот же стек, что в терминале. IDE не вместо ошибки. IDE — рамка для ошибки.

3.21 Что с этим делать сегодня

Не ставь пока десять языков. Поставь привычку:

1. Открой терминал (мак: «Терминал»; Windows: Ubuntu в WSL, если уже есть, иначе хотя бы PowerShell и план включить WSL).
2. `pwd`. Посмотри, где стоишь.
3. `ls`. Посмотри, что видишь.
4. `cd` в какую-нибудь папку, которую узнаёшь глазами (Документы, `dev`, домашняя).
5. Снова `pwd` и `ls`. Ты только что управлял компьютером **текстом**. Это и есть вход в профессию, без романтики.

Дальше в книге — мастерская: SBCL, JDK, Git. Там уже рецепты. Здесь была кухня. Без кухни рецепт — поэзия.

Если что-то из этой главы осталось туманом — нормально. Туман рассеется, когда `cd` сделаешь сто раз. Сто — не преувеличение. Через неделю пальцы сами.

3.22 Страхи, которые не надо таскать в мастерскую

«Я сломаю компьютер командой». Учебные `ls`, `cd`, `pwd`, `mkdir`, `echo` не ломают. Ломает `rm -rf` с корнем и привычка жить под `sudo`. Мы туда не ходим. Если страшно — работай в `dev/`

`module-cabin`, не в системных папках.

«Терминал — для хакеров». Терминал — для людей, которым лень изображать кнопку на каждый рецепт. Половина профессии — текст. Чёрный фон — косметика.

«Надо понять процессор до байта». Не надо. Надо отличить стойку от кладовой и файл от процесса. Байты процессора подождут, пока не станешь тем странным человеком, которому это нравится.

«Windows значит, что учебник не для меня». Значит: WSL2. После этого учебник для тебя. PowerShell — запасной люк, не главный.

«Файл пропал». Обычно: другая папка, другой диск, другой шеф (Windows vs WSL). `pwd`. Поиск по имени в Finder / Проводнике. Редко — ты правда удалил. Корзина существует. На диске, не в RAM.

«Кодировка — это для лингвистов». Кодировка — почему «МОДУЛЬ» вдруг стал Ðœ. Один вечер стычки с этим экономит месяц «у меня кракозябры в JSON».

«Сеть — облако, там живут сайты». Сайты живут на машинах. У машин адреса. У адресов двери. Твой `localhost` — тоже машина, просто ты же сам.

«Ошибка значит, что я не из тех». Ошибка значит, что

датчик сработал. Из тех как раз те, кто читает датчик. Остальные покупают курс с надписью «без боли» и удивляются боли на работе.

Закон станции
Компьютер тупой, точный и не злой. Если кажется, что он издевается — почти всегда путь, кодировка или не та текущая папка. Проверь троих, потом уже характер машины.

Облако, кстати. Это не небо. Это чужой компьютер, который сдают в аренду, и письма до него ходят тем же коридором. «Сохранил в облако» — скопировал банку на их диск. Их диск тоже кончается. Их диск тоже умеет врать

кодировкой. Романтики меньше, чем на наклейке.

Буфер обмена (скопировал — вставил) — не файл. Это кусок RAM с коротким сроком жизни. Вставил в редактор и не сохранил на диск — банки в кладовой нет. Выключили коробку — нет и на стойке. Новички теряют вечер работы на этом чаще, чем на сложных алгоритмах.

Давай медленно
Сохранил — кладовая. Не сохранил — стойка. Закрыл без вопроса «сохранить?» — шеф иногда спрашивает, иногда нет. Терминал `echo > file` сохраняет сразу: это уже файл. Редактор с точкой в названии вкладки —

часто «не сохранено». Смотри на точку.

Дальше можно в мастерскую. Если руки ещё не делали `pwd` — сделай, потом читай про SBCL. Иначе мастерская снова станет лекцией, а мы лекцию только что не читали.

3.23 Шпаргалка на стену каюты

Не учи как стихи. Смотри, когда забыл.

Зачем	Мак / WSL	PowerShell, если очень надо
Где я	`pwd`	`pwd` или `Get-Location`
Что здесь	`ls` / `ls -la`	`ls` или `dir`

Зачем	Мак / WSL	PowerShell, если очень надо
Перейти	`cd папка` / `cd ..` / `cd ~`	то же; `cd ~` иногда капризничает
Новый ящик	`mkdir имя`	`mkdir имя` или `New-Item -ItemType Directory`
Показать текст	`cat файл`	`Get-Content файл` / `cat`
Путь к команде	`which javac`	`Get-Command javac`
Стоп программу	Ctrl+C	Ctrl+C
Выйти из SBCL	`(quit)` / Ctrl+D	`(quit)`
Скомпилировать Java	`javac Hello.java`	то же, если PATH живой

Зачем	Мак / WSL	PowerShell, если очень надо
Запустить Java	`java Hello`	то же
Lisp сразу	`sbcl` потом `(+ 40 40)`	то же, если SBCL установлен

Если ячейка справа бесит — это знак. Не становись переводчиком диалектов на первой неделе. WSL2.

Распечатать шпаргалку не стыдно. Стыдно делать вид, что `pwd` «и так помню», и полчаса искать файл в соседней каюте. На станции «МОДУЛЬ» такие механики уже есть. Их кружки стоят в камбузе. Буква на кружке стёрлась.

Ещё раз, коротко, перед квестами: коробка тупо слушается; повар — CPU; стойка — RAM; кладовая — диск; рецепт — файл; каша на плите — процесс; чёрное окно — способ говорить шефу текстом; `cd` меняет, **где** ты стоишь; компилятор переводит заранее, REPL — за столом; буквы — числа; сеть — письма; ошибка — датчик.

Если этот абзац уже читается как родной — глава сработала. Если нет — не зубри его. Вернись к тому слову, которое ещё чужое, и перечитай **тот** отсек. Станция не ставит экзамен за пересказ. Станция ставит `pwd`.

Квесты ниже — на десять минут, не на ночь. Сделай их до мастерской. Иначе JDK начнёшь ставить, не умея сказать, в какой папке стоишь. Это как искать изоленту, не открывая люк.

Три квеста. Не десять. Если хочется героизма — геройствуй в `cd` туда-сюда, пока пальцы не запомнят.

Отгадки — в главе с отгадками, как у остальных квестов. Сначала сам. Правда.

Поехали.

Квест 0b.1 · терминал
Открой терминал (мак / WSL Ubuntu / на худой конец PowerShell). Создай папку `module-cabin`, зайди в неё, создай

текстовый файл `note.txt` с одной строкой `станция МОДУЛЬ`. Командами покажи: где ты (`pwd` или аналог), что лежит в папке (`ls` или `dir`), содержимое файла (`cat` / `Get-Content`). Запиши в `note.txt` **ещё одной строкой** полный путь, который напечатал `pwd`. Если путь не тот, что думал — это и есть квест, не мелочь.

Квест 0b.2 · кухня
На бумажке или в том же `note.txt`: для действия «запустить `Hello.java`» распиши, кто повар, что на стойке, что в кладовой. Три предложения, не эссе. Если написал «компьютер думает» — сотри и замени на «CPU выполняет инструкции, RAM держит текущее, диск хранит файл».

Квест 0b.3 · текст
Набери в файле слово `МОДУЛЬ`. Сохрани UTF-8. Открой тем же редактором. Потом нарочно открой **другой** кодировкой, если редактор умеет (VS Code: нижний угол → Reopen with Encoding → что-нибудь вроде Windows-1251). Сфотографируй крякозябры душой. Верни UTF-8. Напиши в `note.txt`, почему буквы «сломались», хотя ты их не стирал.

Воскресная диверсия
Нарисуй квадратики: твоя машина, «их» машина, конверты между ними. Подпиши `localhost` на своём квадратике. Это вся сеть, которая нужна, чтобы потом не бояться слова HTTP.

4 Мастерская

4.1 Занятие 0. Верстак, скобки и зелёная кнопка

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Пока нет верстака, эта книга — просто текст, который нельзя укусить. Сегодня ставим инструменты и пишем по крошечной программе на каждом языке. Если что-то не ставится — поздравляю, ты уже на работе: читай сообщение об ошибке **целиком**, не только первое слово.

Это занятие может съесть весь вечер. Нормально. Поставить JDK в первый раз — как найти люк в темноте: ругаешься,

потом находишь, потом удивляешься, что люк был нарисован на схеме. Следующие вечера короче. Этот — плата за вход.

Сегодня на станцию · 2 ч 40 мин
Lisp: поставить SBCL, открыть REPL, потыкать плюсики.
Java: JDK 21, IntelliJ IDEA (или VS Code), «привет».
Git: имя, почта, два репозитория, отправка на GitHub.

4.1.1 Сначала про Windows, Мак и эту вашу WSL

Дальше в книге команды — как в Линуксе: `sbcl`, `java`, `curl`, `git`. На маке они такие и есть. На Windows есть развилка.

Мак, Windows, WSL

Мак. Ставь Homebrew, если ещё нет: https://brew.sh
Потом всё через `brew install …`. Терминал — обычный. Spotlight, слово Terminal, чёрное окно. Не «командная строка Windows», её у тебя нет, не ищи.

Windows — рекомендую WSL2. Linux внутри Windows, без второй машины. В Windows 11 или Windows 10 версии 2004 и новее открой PowerShell от администратора, выполни `wsl --install` и перезагрузись, если попросит. Открой Ubuntu, создай имя и пароль пользователя Linux. В PowerShell проверь `wsl -l -v`: у Ubuntu должна быть версия `2`. Актуальная инструкция Microsoft: https://

learn.microsoft.com/en-us/windows/wsl/install . Дальше Linux-команды книги копируй **в Ubuntu**. Обновление пакетов один раз:

```
sudo apt update && sudo apt upgrade -y
```

Пароль при `sudo` не печатается звёздочками. Это не баг. Просто пиши вслепую и Enter. Линукс такой скрытный.

IntelliJ можно оставить на Windows: File → Open → `\\wsl$\Ubuntu\home\<ты>\...`. Или жить целиком в Убунту, как удобнее. Docker на Windows ставится как Docker Desktop и просит включить WSL2 — согласись, не спорь.

Windows без WSL тоже можно: SBCL и JDK ставятся установщиками, Git — «Git for Windows», в PowerShell многое похоже. Но пути с обратными слэшами, `mvnw.cmd` вместо `./mvnw`, а `curl` иногда называется `curl.exe`. Если запутаешься — всё-таки WSL. Серьёзно.

В тексте, если не сказано иное, команды для **мака и для Ubuntu в WSL**. Родной Windows — в приложении F.

Так себе идея
Не ставь сразу «ещё и Docker, ещё и Postgres, ещё и Android Studio». Сегодня три штуки: SBCL, JDK, Git. Остальное — когда глава попросит. Жадность к установщикам — путь к каше в `PATH` и к вечеру, который

кончился, а Hello так и не напечаталось.

4.1.2 Lisp: SBCL и зверюга по имени REPL

Common Lisp — язык. SBCL — одна из его реализаций, нормальная, быстрая, без лишней мистики. Языков Lisp много, как супов. Нам нужен один казан.

Мак:

```
brew install sbcl
```

Ubuntu в WSL:

```
sudo apt install -y sbcl
```

Windows без WSL: установщик с https://www.sbcl.org/platform-table.html — бери Windows,

ставь, в меню Пуск появится SBCL.

Запуск везде одинаковый:

```
sbcl
```

Вылезет `*`. Это REPL: прочитал, посчитал, напечатал, снова ждёт. Как калькулятор, которому можно объяснять игры. Выход: `(quit)`. На маке и в WSL ещё Ctrl+D. В родном Windows Ctrl+D может не сработать — пиши `(quit)` и не геройствуй.

Давай медленно
Если после `sbcl` написало `command not found` / `не является внутренней или внешней командой` — программа не в PATH. На маке открой **новое** окно терминала после brew. В WSL проверь `which`

`sbcl`. На Windows без WSL — перезапусти терминал, иногда и сам компьютер: установщик прописал путь, а старое окно об этом не знает. Это классика. Её стыдно только первые два раза.

Набери:

```
(+ 2 3)
```

5. Оператор **спереди**, потом аргументы, всё в скобках. Выглядит как насмешка над математикой, зато всегда одно и то же правило: «сделай вот это вот с этим».

Что только что случилось
Ты написал список. Первый элемент списка — **что делать**. Остальные — **с чем**. Lisp не ищет плюсик между двойкой и тройкой, как школа. Он ищет

плюсик в начале, как станция ищет капитана в рубке, а не «ну кто-нибудь в коридоре».

Теперь сломай. Набери `(+ 2 3` без закрывающей скобки и Enter. SBCL будет ждать. Это не зависло. Это вежливо ждёт, пока ты допишешь заклинание. Допиши `)` и Enter. Или нажми Ctrl+C. SBCL обычно откроет отладчик: выбери перезапуск с подписью ABORT, чтобы вернуться к `*`, и начни заново. Запомнить: молчание REPL после открытой скобки — не смерть, а «я слушаю дальше».

```
(* (+ 1 2) 10)
```

Сначала единица плюс два, потом на десять. Скобки — дерево, не украшение. Их будет

много. Потом ты начнёшь их **не замечать**, а это и есть момент, когда Lisp сработал.

Ещё одна поломка, полезная:

```
(1 2 3)
```

Будет что-то вроде `illegal function call`. Lisp решил, что `1` — имя функции. Единица обиделась: она число, не заклинание. Чтобы список был **данными**, а не вызовом, ставят апостроф: `'(1 2 3)`. Запомни жест. Он вернётся на второй неделе и больше не уйдёт.

Строка:

```
"станция МОДУЛЬ"
```

Кавычки — как в человеческом языке. Без кавычек голый

`станция` будет символом, который вычисляется как переменная. Если значение не задано, Lisp ругается.

В Lisp «да» пишут `t`, «нет» — `nil`. Пустой список — тоже `nil`. Один `nil` на две роли. В Lisp это нормально. В жизни тоже, если честно. Слова **истина** и **ложь** мы дальше не используем: звучит как экзамен по логике, а на экране всё равно `t` и `nil`.

```
(= 2 2)     ; T
(= 2 3)     ; NIL
(not nil)   ; T
```

Точка с запятой — комментарий до конца строки. Lisp его не ест. Ты ешь глазами.

Так себе идея

Не ставь десять Лиспов «на всякий случай». SBCL хватит. Emacs со SLIME — сладкая дыра, в которую свалишься сам, когда созреешь. Сегодня достаточно чёрного окна. Clojure, Racket, Scheme — соседние вселенные. Потом. Если поставить всё сразу, квест недели будет «какой из четырёх REPL я открыл».

Файл `hello.lisp` в папке `lisp-experiments`:

```
(format t "станция МОДУЛЬ на связи~%")
```

Загрузить в уже открытый SBCL:

```
(load "hello.lisp")
```

Путь должен совпадать с тем, **откуда** ты запустил SBCL. Если файл в другой папке — либо `cd` туда до `sbcl`, либо полный путь. «У меня файл есть, а Lisp его не видит» — почти всегда «ты не в той папке». Команды `(pwd)` в SBCL нет, в отличие от `pwd` в bash; в терминале **до** запуска: `pwd`.

4.1.3 Java: многословный, зато кормит

Проверь:

```
java -version
javac -version
```

Нужна 21-я (семнадцатая ещё жива, но лучше 21). `java` — запустить уже сваренное. `javac` — сварить. Если есть `java`, а `javac` не находится, возможно,

стоит только JRE, либо папка JDK не попала в PATH. Проверь установку JDK 21 и путь к его `bin`. На станции это как получить чайник без спирали.

Мак: `brew install openjdk@21` и по подсказке brew пропиши `PATH`. Часто brew сам печатает две строки `sudo ln` или `echo '…' >> ~/.zshrc`. Скопируй их. Не геройствуй «потом вспомню».

WSL: `sudo apt install -y openjdk-21-jdk` — если пакета нет, возьми Temurin: https://adoptium.net (есть Linux и Windows).

Windows без WSL: тот же Adoptium, MSI, галка «добавить в PATH». После установки закрой PowerShell и открой заново.

IntelliJ IDEA — с https://www.jetbrains.com/idea/download/ , есть Мак и Windows. Начиная с 2025.3 установщик единый; бесплатных возможностей для этого курса хватает. Подписка Ultimate не нужна. Зелёная стрелка запускает `main`. VS Code тоже можно, но тогда поставь расширение Extension Pack for Java. Спорить, что благороднее, можно после оффера.

Каталог `java-basics`, файл `Hello.java`:

```
public class Hello {
    public static void
main(String[] args) {

System.out.println("станция
МОДУЛЬ на связи");
```

```
    }
}
```

Имя файла = имя класса. `Hello.java` и `class Hello`. Если файл `hello.java`, а класс `Hello`, на некоторых системах повезёт, на некоторых нет. Не играй в удачу. Большая буква, как у Java в паспорте.

Из терминала (мак / WSL):

```
javac Hello.java
java Hello
```

На родном Windows то же, если `javac` в PATH. Если «не является командой» — PATH не прописался, закрой и открой терминал, или всё-таки WSL.

Давай медленно

`javac Hello.java` делает файл `Hello.class`. Это уже не текст, это байткод для JVM. `java Hello` — **без** `.class` в команде. Напишешь `java Hello.class` — обидится. Напишешь `java Hello.java` на старых JDK — тоже обидится. На 21-й иногда можно `java Hello.java` одним махом, без `javac`. Можно. Понимать, что внутри два шага, всё равно надо: на работе сборка будет Maven, и там шагов больше.

Java требует класс, `main` и точки с запятой. Рядом с Lisp это как заполнять анкету после разговора с другом. Зато вакансии именно такие.

Сломай специально. Убери точку с запятой после `println(...)`. `javac` напишет `';'`

`expected` и номер строки. Это самое вежливое сообщение станции. Потом верни точку. Потом поставь класс `Hello` в файл `Bye.java` и посмотри, что скажет. Потом верни. Цель — не идеальный файл, а рефлекс: ошибку читают, не закрывают.

В IntelliJ: New Project → New Java Class → `Hello` → зелёная стрелка. Если стрелки нет — справа щёлкни мышью внутри `main`, появится зелёный треугольник на поле с номерами строк. Если «SDK not specified» — File → Project Structure → SDK → 21. Студия не телепат: ей надо сказать, где JDK.

4.1.4 Git сегодня, не «когда станет красиво»

Git — память с версиями. Не «облако». Облако — GitHub, отдельная контора с сайтом. Git работает и без интернета. GitHub нужен, чтобы вывесить память на заборе, куда посмотрит человек с вакансией.

```
git config --global user.name
"Вася Модуль"
git config --global user.email
"vasya@example.com"
```

Имя — как тебя звать в истории коммитов. Почта — та, что на GitHub, иначе звёздочки коммитов не приклеятся к аккаунту. Это не мелочь.

Git: на маке часто уже есть, иначе `brew install git`. В WSL: `sudo apt install -y git`. На

Windows: https://git-scm.com/download/win — и потом **либо** Git Bash, **либо** снова WSL, где git уже линуксовый.

На github.com заведи аккаунт и пустой репозиторий `java-basics`. Без галки README, если собираешься пушить уже существующую папку — иначе GitHub создаст коммит, а у тебя свой, и они подерутся. Драку потом разнимать скучно.

Перейди в уже созданную папку с `Hello.java`; замени `/full/path/to/java-basics` её настоящим путём:

```
cd /full/path/to/java-basics
git init
```

Файл `.gitignore` (можно создать в редакторе, если PowerShell спорит с `echo`):

```
*.class
.idea/
```

`*.class` — не тащить байткод в гит, он варится из `.java`. `.idea/` — не тащить настройки студии, они твои, не станции.

```
git add .
git commit -m "Hello from week 0"
```

Если git пишет `Please tell me who you are` — ты пропустил `git config`. Сделай и коммит ещё раз.

Привяжи remote. Замени `YOUR-USERNAME` своим именем на GitHub или скопируй HTTPS-адрес пустого репозитория:

```
git branch -M main
git remote add origin https://
github.com/YOUR-USERNAME/java-
basics.git
git remote -v
git push -u origin main
```

Для входа предпочитай браузерную авторизацию через Git Credential Manager или GitHub CLI. Если нужен Personal Access Token, создай fine-grained токен только для учебных репозиториев, с правом Contents read/write и сроком действия. Актуальная инструкция GitHub: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens . Токен — как пароль: не

клади его в код, README или скриншот.

Давай медленно
Новая локальная ветка может называться `master` или `main`: это зависит от настроек Git. Команда `git branch -M main` выше задаёт имя явно. Настройка главной ветки на GitHub не переименовывает локальную ветку.

Если `push` отверг, потому что на GitHub уже есть README, а у тебя нет общего предка:

```
git pull origin main --rebase
git push -u origin main
```

Если это страшно — удали пустой репозиторий на сайте, создай новый **без** README, пушь ещё раз. Для недели 0 это

честнее, чем час ритуалов merge.

Второй репозиторий `lisp-experiments` с файлом `hello.lisp`. Хоть одна строка. Два забора. На одном Java, на другом скобки. Не складывай всё в кучу «учеба»: через месяц сам не найдёшь, где датчик, а где сервер.

Спрятать на GitHub
Два репозитория, по коммиту. README на три предложения: кто ты, что это, как запустить. «Запустить» = конкретная команда, не «ну в IDEA».

README для `java-basics` прямо сейчас:

```
java-basics
Программы первого месяца.

Запуск:
javac Hello.java
java Hello
```

Три строки. Уже лучше, чем пусто. Потом допишешь.

4.1.5 Как разговаривать с ошибкой

Компилятор и REPL — вредные преподаватели. Они почти всегда правы. Читай **первую** ошибку сверху. Строка важнее ощущения «всё умерло». Если скопировал ошибку в поиск — копируй без своего имени файла и без кучи кавычек, которые сам наплодил.

Что только что случилось

Ошибка — не оценка. Оценка будет на собесе, и то не за количество ошибок, а за то, умеешь ли ты их читать. Сегодня тренировка: сломал, прочитал, починил, живой.

Частые гости недели 0:

- `command not found` — нет программы или нет PATH.
- `';' expected` — Java ждёт точку с запятой. Посмотри строку, которую назвали, и строку **выше**: иногда компилятор указывает на строку **рядом**.
- `cannot find symbol` — опечатка в имени. `System.out` не `system.out`. Java помнит регистр, как обиженный архивариус.

- `illegal function call` в Lisp — список без апострофа, который хотел быть данными.
- `end of file on #<stream>` — скобка не закрыта до конца файла.

Не ставь десять галочек «исправить всё» в IDE, не глядя. Одна галочка может переименовать не то. Читай.

4.1.6 Карта вечера, если всё пошло не так

Застрял на SBCL — оставь Java на потом, доведи Lisp до `(+ 2 3)`. Застрял на Java — оставь Git, доведи `Hello`. Git без Hello всё равно нечего пушить, кроме пустоты.

Не работает ничего — напиши на бумажке три факта: (1) какая команда, (2) какая ошибка целиком, (3) из какой папки. С этой бумажкой можно спрашивать соседа, ИИ или форум. Без бумажки получишь ответ «ну покажи ошибку», и будет стыдно дважды.

Квест 0.L1 · Lisp
В REPL: сумма от 1 до 5 одним `+`; произведение 2, 3 и 4; «десять минус три, потом на два». Ответы запиши комментарием в `hello.lisp`.

Квест 0.J1 · Java
`Hello` печатает твоё имя и сегодняшнюю дату (пока просто строкой). Запусти из IDE **и** из терминала. Если одно из двух не вышло — вот он, квест.

Квест 0.G1 · Git
Второй коммит, отправка, открыть страницу на github.com и увидеть свои файлы. Если не увидел — не «потом», чини сейчас.

Квест 0.L2 · Lisp
Намеренно сломай: вызови `(+ 2 "станция")`. Прочитай ошибку вслух. Потом вызови `(concatenate 'string "МОДУЛЬ-" "1")`. Разница: плюсик складывает числа, строки склеивают иначе. Запиши ошибку и удачный результат комментарием — это бортовой журнал, не позор.

Квест 0.J2 · Java
Добавь вторую строку печати: сколько отсеков на станции (любое число). Скомпилируй.

Потом измени число, **не** скомпилируй, запусти старый `java Hello`. Увидишь старое число. Вот зачем `javac`. Потом скомпилируй снова.

Воскресная диверсия
Нарисуй на бумажке, что происходит по кнопке Run. Откуда берётся `java`, куда девается `.class`. Необязательно угадать до байта. Обязательно начать подозревать, что кнопка не магия.

5 Месяц 1. Компьютер, в общем, слушается

К концу четвёртой недели у тебя есть своя консольная программа на Java — не скопированная глава туториала, а твоя — и она лежит на GitHub. Спринга нет, и это праздник. Lisp: имена, «если», функции, первая игра про станцию, которая вечно на грани.

Станция «МОДУЛЬ» висит на орбите лет сто и держится на изоленте, оптимизме и трёх работающих датчиках. Ты — новый бортмеханик. Поздравляю. Скафандр в

гардеробе, кофе уже кто-то выпил.

Первый месяц — не «выучить программирование». Это месяц, когда компьютер из мебели превращается в существо, которое можно спросить и которое иногда отвечает. Иногда ругается. Иногда молчит, и это хуже.

Четыре вахты. После каждой — коммит, даже кривой. Особенно кривой.

Сегодня на станцию · 2 ч 40 мин
Lisp: сорок минут в REPL, пока скобки не перестанут казаться насмешкой.
Java: типы, `if`, списки, объекты, файл. Консоль, не браузер.
К концу месяца: TODO, который

переживает выключение, и README, по которому это запускает кто-то другой.

5.1 Занятие 1. Как назвать штуку и как ей приказать

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: символ, `let`, `defun`, `if`, `cond`, спросить человека.
Java: `int` / `String` / `boolean`, `if` / `else if`, `for`, `Scanner`.
Сегодняшний датчик — энергия реактора. Если она ноль, дальше учебник можно не читать: станция станет очень спокойной и очень холодной.

Вахта первая. В трюме пахнет горелой изолентой и чем-то, что когда-то было кофе. На стенке датчик мигает жёлтым — не потому что умный, а потому что лампочка одна на все режимы. Капитан по внутренней связи сказал только: «энергия есть?». Ты открываешь REPL, как ящик с отвёртками.

5.1.1 Lisp: символы, `defun`, `if`

В Lisp имена — **символы**, почти как бирки на ящиках в трюме. Не строка «energy», не ячейка памяти с табличкой. Символ. Его можно положить в список, сравнить, передать. Пока просто: повесь бирку, положи число.

Глобально (для наших игр нормально) — `defparameter`. Локально, «только здесь» — `let`.

Открой SBCL. На маке и в WSL это просто `sbcl`. На родном Windows — ярлык или то же слово в терминале, если PATH живой. Звёздочка `*` — приглашение, не умножение.

Набери, **включая скобки**:

```
(+ 2 3)
```

Должно получиться `5`. Оператор спереди. Это не шутка и не экзамен по польской нотации — так Lisp всегда: «сделай вот это вот с этим».

Теперь бирка:

```
(defparameter *energy* 100)
```

SBCL ответит чем-то вроде `*ENERGY*`. По умолчанию reader переводит обычный ввод имён без экранирования в верхний регистр: так `*energy*` и `*ENERGY*` в одном пакете дают одну бирку. Это правило чтения, а не безразличие самих символов к регистру: экранирование может сохранить строчные буквы в имени. Звёздочки — не синтаксис, а привычка лисперов кричать: «это глобальная, не наступи». Как табличка «мокро».

Что только что случилось
Ты не «создал переменную» в смысле Java. Ты повесил на символ `*energy*` значение `100`. Спросишь символ — получишь сто. Повесишь другое — сто забудется. Никакого `int` на

бирке нет. Lisp не следит, болт ты туда положил или гайку. Пока.

Прочитай:

```
*energy*
```

`100`. Без звёздочек не сработает: символ называется именно `*energy*`. Наберёшь `energy` — SBCL скажет, что переменная не связана. Перевод: «бирки `energy` в этом трюме нет, есть `*energy*`».

Локально, не на всю станцию:

```
(let ((oxygen 40)
      (power 12))
  (+ oxygen power))
```

Давай медленно
`let` — это «заведи карман на время скобки». Внутри кармана два имени: `oxygen` это 40, `power`

это 12. Тело — `(+ oxygen power)`. Результат всего `let` — то, что вернуло тело, то есть 52. Снаружи `oxygen` снова никого не значит. Попробуй после `let` набрать просто `oxygen`. Ошибка. Карман схлопнулся. Это не баг. Это вежливость: не оставлять гайки на полу коридора.

Сломай специально. Набери:

```
(let ((oxygen 40)
      (power 12))
  (+ oxygen power)
```

Не хватает закрывающей скобки. SBCL будет ждать. Курсор моргает, как датчик, которому не сказали «хватит». Допиши `)` или Ctrl+C / выйди из путаницы через `(abort)` если уже совсем тоска. Урок: скобки — дерево, не украшение.

Считай их. Потом перестанешь считать глазами и начнёшь чувствовать.

Ещё сломай:

```
(let (oxygen 40)
  oxygen)
```

Скорее всего орнёт. У `let` аргумент — **список связок**, каждая связка — свой маленький список `(имя значение)`. Без внутренних скобок Lisp не понимает, где кончилось имя и началось число. Это как написать на ящике «гайки 40» без черты: кладовщик не знает, это сорок гаек или гайки по имени Сорок.

Правильно снова:

```
(let ((oxygen 40))
  oxygen)
```

```
; => 40
```

Функция — заклинание с именем:

```
(defun report (energy)
  (if (>= energy 50)
      "реактор стабилен"
      "энергии мало"))
```

Давай медленно `defun` так и расшифровывается: define function, «определи функцию». Дальше имя `report`. Дальше в скобках список параметров — здесь один, `energy`. Это уже не глобальная `*energy*`. Это местный ярлык: «то, что мне передали». Тело — один `if`. У `if` три места: вопрос, что делать если да, что делать

если нет. Вопрос: `(>= energy 50)` — энергия больше или равна пятидесяти? Если да — строка про стабильный реактор. Если нет — строка про мало. Весь `if` **возвращает** строку. Функция возвращает то, что вернул `if`. Никакой печати. Пока.

Вызови:

```
(report 80)
(report 10)
(report 50)
```

Должно быть: стабилен, мало, стабилен. Пятьдесят проходит, потому что >=, не >. Мелочь, из которой на станции делают аварии.

Что только что случилось `(report 80)` — это **вызов**. Имя функции спереди, аргумент

следом, всё в скобках. Если напишешь `report(80)` как в Java — Lisp решит, что `report` это переменная, а `(80)` это отдельный список, который надо вычислить. Будет обида. Скобки снаружи, имя внутри, аргументы рядом. Всегда.

Забудешь «нет» у `if`:

```
(defun report-broken (energy)
  (if (>= energy 50)
      "реактор стабилен"))

(report-broken 80)
(report-broken 10)
```

Первый вызов — строка. Второй — `nil`. Станция промолчит. Иногда это даже хуже сирены. `nil` в Lisp — «нет», «пусто», «не получилось», «списка нет». Один ничто на несколько

ролей. Мы не говорим «ложь»: на экране всё равно `nil`.

Несколько ступенек — `cond`:

```
(defun status (n)
  (cond
    ((>= n 80) 'green)
    ((>= n 40) 'yellow)
    (t 'red)))
```

Давай медленно `cond` — лестница вопросов. Каждый этаж: `(вопрос ответ)`. Первый, который даст не-`nil`, побеждает, остальные даже не смотрят. `t` в конце — «во всех остальных случаях», космический иначе. `t` в Lisp — «да», универсальное. Не «истина». На экране `t`. `'green` — не строка, а **символ**, бирка. Апостроф значит: не вычисляй, положи в карман как есть. Без

апострофа Lisp полезет искать переменную `green` и не найдёт.

Проверь:

```
(status 90)
(status 40)
(status 0)
```

`GREEN`, `YELLOW`, `RED`. SBCL снова пишет капсом. Это те же символы.

Порядок ступенек важен. Если сначала поставить `(t 'red)`, всё всегда будет красным. Лестница, где первая ступенька — пол. Попробуй, полюбуйся, верни как было.

Строки сравнивай `(string= a b)`. Числа: `=`, `<`, `>`, `<=`, `>=`. Остальное — потом, жизнь длинная.

Три рабочих примера, не один.

Пример А. Стыковка:

```
(defun dock-comment (speed)
  (if (< speed 5)
      "можно дышать"
      "сейчас будет вмятина"))
```

Пример Б. Кислород в отсеке — не энергия, но тот же жест:

```
(defun oxygen-ok (percent)
  (cond
    ((>= percent 18) 'ok)
    ((>= percent 12) 'masks)
    (t 'evacuate)))
```

Пример В. Два датчика сразу:

```
(defun can-open-hatch (energy
oxygen)
  (if (and (>= energy 20) (>=
oxygen 18))
      'open
      'wait))
```

`and` — оба. `or` — хватит одного. `(not x)` — переверни. `nil`

переворачивается в `t`, всё остальное — в `nil`. Да, «всё остальное» в Lisp считается «да». Ноль — тоже «да», в отличие от некоторых языков, которые считают ноль пустышкой. `(if 0 'yes 'no)` вернёт `YES`. Запомни в первый день, потом не удивишься в три ночи.

Спросить человека в REPL:

```
(defun ask-number ()
  (format t "Введите число: ")
  (finish-output)
  (parse-integer (read-
line) :junk-allowed t))
```

Что только что случилось `format t` пишет в терминал. `t` здесь — «стандартный вывод», не «да», хотя это тот же символ, жизнь сложная. `finish-output` —

вытолкни буквы из трубы **сейчас**, не копи их до конца строки. Без этого на некоторых системах приглашение появится **после** того, как ты уже что-то ввёл. Очень смешно, один раз. `read-line` слушает до Enter и возвращает строку. `parse-integer` делает из букв число. `:junk-allowed t` — «если после цифр мусор, не падай, забери сколько сможешь». Набрал `12abc` — получишь 12. Набрал `асдф` — получишь `nil`. Пожимает плечами.

Вызови `(ask-number)`, введи `42`, получи `42`. Введи `привет`, получи `nil`. Потом напиши обёртку, которая не верит `nil`:

```
(defun ask-energy ()
  (let ((n (ask-number)))
    (if (numberp n)
        n
        (progn
          (format t "это не
число~%")
          (ask-energy)))))
```

`numberp` — «это число?». Хвостик `p` у лисперов — predicate, вопрос да/нет. `progn` — сделай несколько дел подряд, верни последнее. Рекурсия «спроси ещё раз» — спойлер четвёртого занятия, но для ввода она честная: человек будет путать клавиши, это их хобби.

Так себе идея
Частые скобочные смерти сегодня: `(defun report energy ...)` — параметры без своих скобок. Надо `(energy)`. `(if`

`>= energy 50 ...)` — `>=` это функция, её тоже зовут скобками: `(>= energy 50)`. Лишняя `)` в конце `defun` — SBCL начнёт читать **следующую** строку как мусор. Смотри, **какая** скобка ему не нравится, не все сразу.

Ещё один REPL-ритуал. Набери нарочно:

```
(1 2 3)
```

Ошибка вроде `undefined function: 1`. Lisp решил, что это **вызов**: функция `1`, аргументы `2` и `3`. Единица обидится. Чтобы список так и остался списком, нужен апостроф: `'(1 2 3)`. Это занятие 2, но ошибка всплывёт сегодня, если рука сама

допишет скобки. Теперь ты её узнаешь.

5.1.2 Java: типы, которые следят, чтобы ты не перепутал болт с гайкой

В Java у каждой коробки написано, что внутри. Компилятор — вредный кладовщик. Не REPL в том же смысле: написал файл, скомпилировал, запустил, получил ор. Зато кладовщик ловит часть глупостей **до** полёта, а не во время стыковки.

```
int energy = 100;
String name = "МОДУЛЬ";
boolean ok = energy >= 50;
```

`int` — целое. `String` — текст, с большой буквы, потому что это класс, не «просто цифра».

`boolean` — `true` / `false`. Мы не переводим их в «истина/ложь» в коде: в файле будет `true` и `false`, в вакансиях тоже.

Давай медленно
Строка `boolean ok = energy >= 50;` — не «если». Это **выражение**, которое само становится `true` или `false`, и его кладут в коробку `ok`. Потом можно спросить `if (ok)`. Можно сразу `if (energy >= 50)`. Оба честные. Первый удобен, когда условие потом ещё куда-то потащат.

```
if (ok) {
    System.out.println("реактор
стабилен");
} else {
    System.out.println("энергии
мало");
}
```

Фигурные скобки — загон для кода. Точка с запятой — «предложение кончилось». Без неё Java не начинает следующее предложение, она ругается.

Класс `Energy` живёт в файле `Energy.java`, иначе Java обижается. У неё это часто.

Ввод с земли:

```
import java.util.Scanner;

public class Energy {
    public static void
main(String[] args) {
        Scanner in = new
Scanner(System.in);
        System.out.print("Энергия
(0-100): ");
        int energy =
in.nextInt();
        if (energy >= 80) {
```

```
System.out.println("green");
        } else if (energy >= 40)
{

System.out.println("yellow");
        } else {

System.out.println("red");
        }
    }
}
```

Что только что случилось `import` — «кладовщик, дай ящик `Scanner` из комнаты `java.util`». Без импорта компилятор не знает, что такое `Scanner`. `public class Energy` — чертёж. `main` — дверь, в которую стучит команда `java Energy`. `new Scanner(System.in)` — подключи ухо к клавиатуре. `nextInt()` — подожди целое. `else if` — следующая ступенька, как `cond`. Порядок снова важен: сначала

80, потом 40, потом всё остальное.

Мак, Windows, WSL
Мак / WSL. В папке с файлом:

```
javac Energy.java
java Energy
```

Windows без WSL. То же, если `javac` в PATH. Если «не является внутренней или внешней командой» — JDK не прописался. Закрой терминал, открой снова. Не помогло — поставь Temurin 21 с галкой PATH или уходи в WSL, там `sudo apt install -y openjdk-21-jdk`.

Цикл, чтобы тикало:

```
for (int i = 0; i < 3; i++) {
    System.out.println("тик " +
```

```
i);
}
```

Напечатает `тик 0`, `тик 1`, `тик 2`. С нуля, как массивы, не как люди. `i++` — прибавь один после прохода. Три части в круглых скобках `for`: с чего начать, пока что крутить, что делать после круга.

Ещё три примера, уже не про энергию.

Пример А. Среднее трёх температур — тот же жест, что квест, но сначала глазами:

```
int a = 18, b = 21, c = 7;
double avg = (a + b + c) / 3.0;
```

`3.0` — чтобы деление было дробным. `(a + b + c) / 3` для целых отрежет хвост. Java

считает: целые на целые — целое. Кладовщик здесь не ругается, он **молча** ворует дробь. Это хуже ошибки.

Пример Б. Строки сравнивай через `.equals`, не через `==`:

```
String cmd = "list";
if (cmd.equals("list")) {

System.out.println("показываю");
}
```

`==` у объектов спрашивает «это один и тот же ящик в памяти?», а не «на ящиках написано одно и то же». Иногда случайно сработает. Потом перестанет. Потом ты будешь три часа винить Вселенную.

Пример В. Несколько команд в `main` — уже пахнет занятием 2, но цикл `while` пусть мелькнет:

```
int n = 3;
while (n > 0) {
    System.out.println("осталось
" + n);
    n = n - 1;
}
```

Пока `n > 0` — печатай и уменьшай. Забудешь уменьшить — вечный цикл, терминал как датчик, которому не сказали «хватит». Ctrl+C.

5.1.3 Если сломалось: Java ругается, и это нормально

Компилятор пишет много. Читай **первую** ошибку, не последнюю. Часто первая рожает остальные.

Класс не как файл. В файле `Energy.java` написал `public class Energia`. Пишет: class Energia is public, should be declared in a file

named Energia.java. Переименуй либо файл, либо класс. Они одно существо.

Забыл точку с запятой. Строка `int energy = 100` без `;` — „;“ expected. Java не умеет угадывать, где кончилась мысль. Поставь.

Забыл импорт. `Scanner` без `import java.util.Scanner;` — cannot find symbol. Символ — это имя, которого нет в зоне видимости. Не «магия сломалась». Не хватает либо импорта, либо ты опечатался: `scanner` с маленькой буквы — уже другой зверь.

Main не найден. Запустил `java Energy`, а метод называется `Main` или без `String[] args`. Дверь должна выглядеть так: `public`

`static void main(String[] args)`. Иначе JVM стоит в коридоре и не знает, куда стучать.

Скобка. `reached end of file while parsing` — не хватает `}`. Посчитай. В IntelliJ подсветка помогает: кликни у скобки, вторая подсветится. Если не подсветилась — её нет.

Ввёл буквы в `nextInt()`. Уже не компиляция, уже полёт: `InputMismatchException`. Scanner ждал число, получил «пщщ». На занятии 4 поймаем. Сегодня перезапусти и введи цифры. Или читай строку и сам разбирай — но это на шаг позже.

Так себе идея
Не копируй в файл «пропущенные куски с трёх

сайтов». Один класс, один `main`, запустился, потом меняй по строке. Станция не собирается из трёх разных чертежей на коленке. Ну, собирается, но потом не стыкуется.

Три рабочих прогона глазами, прежде чем квесты.

1. Поставь энергию 80, 40, 39, 0. Четыре ответа: green, yellow, red, red. Если 40 даёт red — ты написал `>`, не `>=`.
2. Поменяй пороги местами: сначала `>= 40`, потом `>= 80`. Введи 90. Получишь yellow, потому что 90 тоже `>= 40`, и вторая ступенька не глянется. Порядок — не эстетика.
3. Напечатай `energy` до `if` и после. Число не меняется от

того, что ты про него рассказал. `if` не тратит энергию. Тратить научимся на занятии 3.

Спрятать на GitHub
В `lisp-experiments` — `module-01.lisp`. В `java-basics` — `Energy`. Коммит вроде `week1: energy status`.

Квест 1.L1 · Lisp
`dock-ok`: скорость стыковки. Меньше 5 — `"мягко"`, иначе `"слишком быстро"`. Проверь на 3 и на 9. Станция просит не таранить шлюз.

Квест 1.L2 · Lisp
`airlock`: два давления, внутри и снаружи. Равны — `'open`, иначе `'sealed`. Сравнивай через `=`. Открыть шлюз «ну почти равно» — плохая комедия.

Квест 1.L3 · Lisp
`lamp`: энергия 0..100. Верни `'green` / `'yellow` / `'red` теми же порогами, что `status` в тексте (80 и 40). Вызови на 100, 80, 79, 40, 0. Пять вызовов, пять ответов, в комментарии рядом. Если хоть один ответ не тот — пороги или порядок `cond`.

Квест 1.J1 · Java
Три числа — температуры отсеков. Напечатай среднее. Если хоть одно ниже нуля — ещё слово `ALARM`. Среднее считай через `3.0`, иначе Java отрежет дробь как неинтересную.

Квест 1.J2 · Java
Игра: машина загадывает 1..10 (`Math.random()`), ты один раз угадываешь. «верно» / «мало» / «много».

Квест 1.J3 · Java
Энергия с клавиатуры, пока человек не даст целое от 0 до 100 включительно. Буквы и числа вне диапазона — снова приглашение, не падение. Пока можно `while (true)` и `break`, когда число честное. `Scanner.nextLine()` плюс `Integer.parseInt` проще ловить, чем `nextInt` вперемешку со строками. Если сломалось на буквах — это и есть квест, не «потом».

SICP · открывать, только если зудит
Зацепил, что `if` в Lisp **возвращает** значение, а не «делает штуку»? В SICP про выражения. Не сейчас — когда зуд не даст спать.

Воскресная диверсия

В браузере: `view-source:` и любой сайт. Это не Java. Это текст, который потом кто-то исполняет. Приятно почувствовать себя за кулисами.

Капитан снова по внутренней: «ну как энергия?». Ты отвечаешь не «нормально», а `green`. На станции начинают подозревать, что ты умеешь читать датчики. Кофе по-прежнему кто-то выпил.

5.2 Занятие 2. Коробки, списки и бесконечный коридор

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

Сегодня на станцию · 2 ч 40 мин
Lisp: `quote`, `first` / `rest`, `cons`, `length`, `member`, `dolist`, чуть `loop`.
Java: массив, `ArrayList`, `HashMap`, цикл по коллекции.
Квест дня — список дел в консоли. После выключения всё забудется. Файлы — через занятие. Интрига.

Вахта вторая. Коридор станции — это не метафора, это буквально коридор: шлюз, потом труба, потом реактор, потом отсек, где когда-то была оранжерея, а теперь ящики. На бумажке у прошлого механика список отсеков, написанный трижды и трижды по-разному. Ты решаешь, что компьютеру можно поручить хотя бы порядок.

5.2.1 Lisp: списки — вообще почти всё

Список пишут в скобках. Если не поставить апостроф, Lisp решит, что это **вызов**, и попытается вызвать `1` как функцию. `1` обидится. Вчера ты это уже видел. Сегодня это инструмент.

```
'(1 2 3)
'(antenna corridor reactor)
```

Что только что случилось Апостроф `'` — сахар для `(quote ...)`. `(quote (1 2 3))` и `'(1 2 3)` одно и то же: «вот данные, не трогай как код». Без апострофа `(antenna corridor reactor)` значит: вызови функцию `antenna` с аргументами `corridor` и `reactor`. Функции `antenna` нет. Ошибка. С

апострофом это просто три бирки в одном ящике.

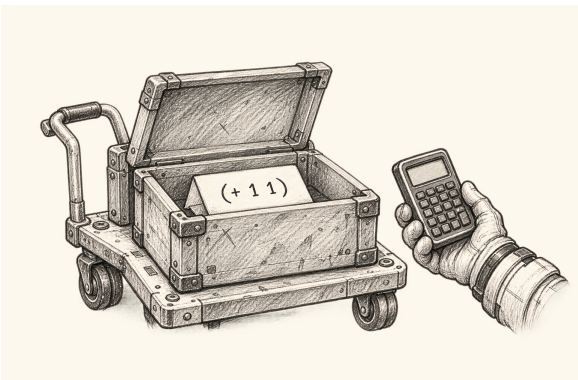
`(list 1 2 3)` — то же, но аргументы сначала посчитает:

```
(list 1 (+ 1 1) 3)
; (1 2 3)

'(1 (+ 1 1) 3)
; (1 (+ 1 1) 3)
```

Во втором случае `(+ 1 1)` так и лежит списком. Не двойка. `quote` не вычисляет ни приведённую форму, ни её содержимое. Это про вычисление, а не механизм «заморозки» объектов. Буквально записанные данные под `quote` нельзя менять на месте: последствия не определены. Нужны изменения — создай свежий список через `list` или заранее скопируй ту

структуру, которую будешь менять. `list` считает куски, потом собирает новый список.



Доставить выражение. Не выполнять.
quote оставляет (+ 1 1) данными. Это не разрешение менять литеральный список.

Голова и хвост. У списка, как у кометы:

```
(first '(a b c))  ; A
(rest  '(a b c))  ; (B C)
```

```
(first '())       ; NIL
(rest  '(a))      ; NIL
```

Давай медленно
Список — это не «массив на три клетки». Это пара: голова и **остальной список**. `(A B C)` устроен как «A, и ещё (B C)». `(B C)` — «B, и ещё (C)». `(C)` — «C, и ещё пусто». Пусто — `nil` или `'()`. Один ничто на всех. Поэтому `(rest (rest (rest '(a b c))))` даёт `NIL`. Три раза сняли голову — никого. Дедушки говорили `car` и `cdr`. Это те же `first` и `rest`, только звучит как поломка. В старых текстах везде `car`/`cdr`. Мы пишем `first`/`rest`, чтобы не казалось, что станцию собирали в 1959-м. Хотя «МОДУЛЬ», если честно, могли.

Набери цепочку, не ленись:

```
(defparameter *rooms* '(airlock
corridor reactor))
(first *rooms*)
(rest *rooms*)
(first (rest *rooms*))
(first (rest (rest *rooms*)))
```

`AIRLOCK`, `(CORRIDOR REACTOR)`, `CORRIDOR`, `REACTOR`. Ты только что прошёл коридор руками. Это и есть вся поэзия Lisp.

Приделать элемент спереди:

```
(cons 'airlock '(corridor
reactor))
; (AIRLOCK CORRIDOR REACTOR)
```

`cons` — главный конструктор вселенной. Не «добавить в конец». В **начало**. Старый список не меняется — получается новый. Проверь:

```
(defparameter *tail* '(corridor
reactor))
(cons 'airlock *tail*)
*tail*
```

`*tail*` по-прежнему `(CORRIDOR REACTOR)`. `cons` не прикрутил шлюз гвоздём к старому ящику. Собрал новый. Это важно, когда дойдём до «функция не портит то, что ей дали».

Что только что случилось `(cons 'x nil)` даёт `(X)`. Один элемент — тоже список. `(cons 'x 'y)` даёт `(X . Y)` — точечная пара, не «настоящий» список. Настоящий список кончается `nil`. Пока не собирай точечные пары специально. Если в печати вдруг точка — ты скормил `cons` не список вторым

аргументом. Вернись, положи `'()` или другой список.

Длина: `(length lst)`. Есть ли реактор: `(member 'reactor '(corridor reactor))` — вернёт хвост от находки или `nil`, если реактор уже кто-то вынес.

```
(member 'reactor '(airlock
corridor reactor))
; (REACTOR)
(member 'garden '(airlock
corridor reactor))
; NIL
```

Не `t`. Хвост. Это удобно и бесит одновременно. «Нашли? Тогда вот отсюда до конца». Для вопроса «есть или нет» обычно пишут `(not (null (member ...)))` — два отрицания вокруг находки. Или `if` и живи.

Пройтись:

```
(dolist (room '(airlock corridor
reactor))
  (format t "отсек: ~a~%" room))
```

`~a` — напечатай по-человечески. `~%` — новая строка. `dolist` не собирает новый список, он **делает**. Для печати — то что надо. Рекурсию оставим на четвёртое занятие, сегодня хватит `dolist` и вот этого:

```
(loop for i from 1 to 5
      collect (* i i))
; (1 4 9 16 25)
```

`loop` в Common Lisp — отдельный маленький язык внутри языка. Можно любить, можно бояться. Для квадратов — любить.

Ещё `loop` по списку с номерами — завтрашний квест, сегодня подсказка:

```
(loop for room in '(airlock
corridor)
      for i from 1
      do (format t "~a. ~a~%" i
room))
```

Два `for` шагают вместе. `do` — побочное дело (печать). `collect` — собери значения в список-результат.

Три примера, чтобы список стал рукой, а не теорией.

Пример А. Новый отсек впереди карты:

```
(defun add-room (room rooms)
  (cons room rooms))
```

Пример Б. Второй отсек, если он есть:

```
(defun second-room (rooms)
  (first (rest rooms)))
```

Для `'()` и `'(only)` получишь `nil`. Не падай. Пустой коридор — тоже ответ.

Пример B. Есть ли имя:

```
(defun known-p (room rooms)
  (not (null (member room
rooms))))
```

`(known-p 'reactor '(corridor reactor))` → `T`. `(known-p 'garden '(corridor reactor))` → `NIL`.

Так себе идея `(append '(a) '(b))` склеивает **копией**. `(cons '(a) '(b))` даёт `((A) B)` — список в голове, не склейка. Если ждал `(A B)`, а получил скобки внутри, ты взял не ту функцию. `append` сегодня

можно один раз потрогать и не полюбить: для учёбы `cons` + `reverse` честнее, когда дойдёшь.

Сломай. Набери `(first 5)`. Ошибка: 5 не список. `(rest 'reactor)` — символ не список. `length` по числу — тоже. Lisp не догадается, что ты «имел в виду ящик». Положи ящик.

5.2.2 Java: массив, список, который растёт, карман с подписями

Массив — ящик на три гнезда, гнёзда не появляются из воздуха:

```
int[] temps = {18, 21, 7};
temps[0] = 19;
for (int t : temps) {
    System.out.println(t);
}
```

Давай медленно `int[]` — «ряд целых». Фигурные при создании — начальные значения. Индекс с **нуля**: первое гнездо `temps[0]`. Третье — `temps[2]`. `temps[3]` — `ArrayIndexOutOfBoundsException`, рука в стенку. Цикл `for (int t : temps)` — «для каждого, не думай про индекс». Когда индекс нужен — старый `for (int i = 0; i < temps.length; i++)`.

Длина массива — поле `.length`, не метод. У строки — метод `.length()`. У `ArrayList` — метод `.size()`. Три имени на одну мысль. Java любит, чтобы ты споткнулся и запомнил.

Список, который можно докладывать — `ArrayList`:

```
import java.util.ArrayList;
import java.util.List;

List<String> rooms = new
ArrayList<>();
rooms.add("airlock");
rooms.add("corridor");
rooms.add("reactor");
System.out.println(rooms.get(1));
System.out.println(rooms.size());
```

Что только что случилось `List<String>` — «список строк». Уголковые скобки — какого зверя кладём внутрь. Это **дженерик**, слово для собеса. Положишь потом `rooms.add(3)` — кладовщик ругается: 3 не строка. `get(1)` — второй элемент, потому что ноль. `size()` — сколько сейчас, не «сколько влезет». Влезет много. Память кончится раньше, чем `ArrayList` скажет «нет». Слева `List`, справа

`new ArrayList`. Чертёж «список», железяка «список на массиве». Пока не думай, зачем так. Завтра так же будет `Task` и `new Task`.

Выкинуть по номеру:

```
rooms.remove(0);
```

Снова ноль — первый. Люди считают с единицы. Твой TODO в квесте должен разговаривать с людьми: они пишут `del 1`, ты внутри делаешь `remove(0)`. Забудь — выкинешь не ту строку, антенна подождёт ещё смену.

Карман «ключ → значение»:

```
import java.util.HashMap;
import java.util.Map;
```

```
Map<String, Integer> energy = new
HashMap<>();
energy.put("reactor", 80);
energy.put("antenna", 20);
System.out.println(energy.get("re
actor"));
System.out.println(energy.get("ga
rden"));
```

Что только что случилось `put` кладёт, `get` достаёт. Нет ключа — `null`, не ошибка. `null` — «тут никого», двоюродный брат `nil`, только злее: вызови на нём метод — `NullPointerException`. Станция падает не от космоса, а от «я думал, там число». `Integer`, не `int`, в уголке карты: в коллекции живут объекты. Java сама упакует `80` в объект и обратно. Пока не бойся. Не клади `null` в `int` — не влезет. В `Integer` влезет, и потом взорвётся в арифметике.

Пройтись по карте:

```
for (Map.Entry<String, Integer>
e : energy.entrySet()) {
    System.out.println(e.getKey()
+ " " + e.getValue());
}
```

Ключ, значение, пара. Не запоминай `Entry` как отдельную науку. Это «один карманный ярлык на время цикла».

Так себе идея
Не учи все коллекции как таблицу умножения. `ArrayList` и `HashMap` закрывают большую часть джуновских страданий. Остальное найдётся, когда само понадобится.

Сегодняшняя Java-игра — список дел в консоли. Добавить, показать, выкинуть по номеру. Как на холодильнике, только

холодильник — массив в памяти, и после выключения всё забудется.

Набросок цикла команд, не списывай слепо — допиши сам в квесте:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Todo {
    public static void
main(String[] args) {
        List<String> tasks = new
ArrayList<>();
        Scanner in = new
Scanner(System.in);
        while (true) {
            System.out.print(">
");
            String line =
in.nextLine().trim();
            if
(line.equals("quit")) {
                break;
```

```
            }

System.out.println("пока умею
только quit, остальное — твой
квест");
        }
    }
}
```

Давай медленно `while (true)` — крутись, пока не `break`. `nextLine()` — вся строка, не одно слово. `.trim()` — отрежь пробелы по краям, люди их ставят. `equals`, не `==`. Приглашение `>` — чтобы было понятно, что программа ждёт, а не зависла. На станции зависла и ждёт выглядят одинаково, если не моргать.

Разбор строки `add починить шлюз`: первое слово — команда, остальное — текст. `split(" ", 2)`

режет **на две** части: команда и хвост. Без двойки `"починить шлюз"` развалится на два слова, и шлюз потеряется.

```
String[] parts = line.split(" ",
2);
String cmd = parts[0];
String rest = parts.length > 1 ?
parts[1] : "";
```

Тернарный `? :` — однострочный `if`, который **возвращает** значение. Как Lisp. Иногда удобно. Иногда им пишут простыню. Сегодня — две ветки, нормально.

5.2.3 Если сломалось: списки и ввод

`cannot find symbol: ArrayList`. Нет импорта. Две строки: `List` и `ArrayList` — оба из `java.util`.

`Index 3 out of bounds for length 3`. Три элемента — индексы 0, 1, 2. Ты вычел единицу? Не вычел? Напечатай `size()` и номер, который пришёл от человека, **до** `remove`.

`InputMismatchException` вчерашний, сегодня `nextLine` после `nextInt`. Если смешаешь `nextInt()` и `nextLine()`, в буфере живёт Enter, и «пустая команда» прилетает сама. Сегодня весь ввод — `nextLine`. Числа из текста: `Integer.parseInt`. Поймаешь `NumberFormatException` — скажи «номер, пожалуйста» и крутись дальше.

Пустой `add`. Человек написал `add` без текста. Реши: ошибка или задача с пустым названием. Пустое название потом взорвёт

голову. Лучше сразу «надо слово».

Мак, Windows, WSL
Запуск тот же: `javac Todo.java && java Todo` на маке и в WSL. На Windows `&&` в cmd.exe работает в новых версиях; если нет — две команды подряд. IntelliJ: зелёная стрелка на `main`. Не держи два запуска сразу — два `Scanner` на один ввод, цирк.

Три прогона руками, пока не квесты.

1. Массив из трёх температур, поменяй средний, напечатай все. Убедись, что `temps[1]` — середина.
2. `ArrayList`: три `add`, `get(0)`, `remove(0)`, снова печать. Первый уехал, бывший

второй стал нулевым. Списки плотные, дырок нет.

3. `HashMap`: два модуля, `get` существующего и выдуманного. Второй — `null`. Напечатай это слово глазами, подружись.

Квест 2.L1 · Lisp
`rooms-report`: список отсеков, каждый с номером с единицы. `loop for room in lst for i from 1` — твой друг.

Квест 2.L2 · Lisp
`has-reactor-p`: `t`, если в списке есть `reactor`. Хвостик `-p` у лисперов значит «вопрос, да или нет». Мило, правда?

Квест 2.L3 · Lisp
`prepend-airlock`: к любому списку отсеков приделай `'airlock`

спереди через `cons` и верни новый список. Старый не меняй — проверь, что исходный после вызова тот же. Потом `(length ...)` нового: на один больше. Если length тот же — ты вернул не то.

Квест 2.J1 · Java
Команды: `add текст`, `list`, `del номер`, `quit`. Живут в `ArrayList<String>`. В `list` показывай номера с единицы, как у людей. Для `del номер` сначала проверь диапазон `1..size()`, затем вычти единицу, чтобы получить индекс списка. Не путай номер на экране с индексом.

Квест 2.J2 · Java
`HashMap` имя → возраст. Напечатай старше 18. Данные

зашей в код, ввод не обязателен. Станция проверяет, кому можно в рубку.

Квест 2.J3 · Java
Карта отсек → энергия (`HashMap<String, Integer>`). Напечатай только те, у кого энергия строго меньше 30. Хотя бы три ключа в данных. Пустой вывод — тоже ответ, если все сыты; тогда понизь одно число и перезапусти, чтобы увидеть строку.

Спрятать на GitHub
Коммит `week2: lists and todo`. Да, TODO на второй неделе. Все так начинали, даже те, кто врёт.

В конце смены коридор на экране совпадает с коридором за спиной. Почти. В списке ещё есть `garden`, а за люком —

ящики. В данных ещё старое прошлое. Ты пока не чинишь станцию. Ты чинишь список. Это уже работа.

5.3 Занятие 3. Функции, которые не болтают лишнего

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: одна функция — одно дело, `let` внутри, чуть `mapcar`.
Java: класс `Task`, ящик `TaskStore`, худой `main`.
Вчера задачи были строками. Сегодня они **вещи** с полями.

Вахта третья. Капитан хочет не «строки на холодильнике», а

чтобы у дела был номер, имя и отметка «сделано». И чтобы энергия не уходила в минус, когда ты чинишь антенну. Минус энергии на «МОДУЛЕ» — это не бухгалтерия. Это тёмные иллюминаторы.

5.3.1 Lisp: одно дело — одна функция

Хорошая функция делает одну штуку и **возвращает** ответ. Печать — снаружи, если можно. Иначе ты никогда не переиспользуешь код, только любуешься им в терминале.

```
(defun clamp (n lo hi)
  (cond
    ((< n lo) lo)
    ((> n hi) hi)
    (t n)))
```

```
(defun spend (energy cost)
  (clamp (- energy cost) 0 100))
```

Давай медленно `clamp` — зажми число между полом и потолком. Ниже пола — пол. Выше потолка — потолок. Иначе само число. `spend` не знает, как зажимать: он вычитает и просит `clamp`. Если завтра зажимать надо 0..1, а не 0..100, правишь одно место. Вызови `(spend 10 40)`. Ждёшь −30? Получишь 0. Реактор не в долг. `(spend 80 10)` → 70. `(clamp 150 0 100)` → 100. Три вызова — три грани.

Несколько локальных имён, чтобы не сойти с ума:

```
(defun docking-score (speed fuel)
  (let ((speed-part (if (< speed
```

```
5) 10 0))
        (fuel-part (if (> fuel
20) 5 0)))
    (+ speed-part fuel-part)))
```

Что только что случилось `let` считает начальные значения слева направо, прежде чем создать свои новые связки. Поэтому в одном начальном выражении нельзя опереться на новое местное имя из соседнего. Нужна такая цепочка — `let*`. Сегодня оба куска из аргументов. Сумма — очки стыковки. Функция молчит. Печатать будет тот, кто зовёт.

```
(docking-score 3 30)
; 15
(docking-score 9 30)
; 5
```

```
(docking-score 3 5)
; 10
(docking-score 9 5)
; 0
```

Четыре комбинации двух рубильников. Это таблица истинности, только про шлюз. Если лень проверять руками — всё равно проверь. Функции врут тише людей.

`&optional` пока не трогай. Встретишь — значит, жизнь сама подвела.

Функция может принимать **другую** функцию. Рановато, но один раз увидеть полезно, как фокус:

```
(mapcar #'evenp '(1 2 3 4))
; (NIL T NIL T)
```

Давай медленно

`mapcar` проходится и применяет. `#'evenp` — объект функции, не вызов. Голое `evenp` здесь означает переменную; без связанного значения будет ошибка unbound-variable. `'evenp` тоже работает здесь как символ, называющий глобальную функцию, но `#'evenp` яснее показывает намерение. Результат — новый список ответов. Исходный `(1 2 3 4)` цел. Снова: не портим ящик, собираем другой.

Если зажгло — это мост в SICP. Если нет — живи спокойно, до `mapcar` мы ещё доберёмся руками.

Ещё два жеста, соседние.

```
(defun recharge (energy delta)
  (clamp (+ energy delta) 0 100))

(defun simulate (costs)
  (let ((e 100))
    (dolist (c costs)
      (setf e (spend e c)))
    e))
```

`setf` — повесь на место новое значение. Здесь место — локальный `e`. `(simulate '(10 20 80))`: 100−10=90, 90−20=70, 70−80=0. Не минус. `dolist` ходит, `setf` обновляет, в конце `e` — ответ всего `let`.

Так себе идея
Печать внутри `spend` — медвежья услуга. Захочешь посчитать сумму трат без печати — уже не получится, `format` прибит гвоздём. Раздели: считать / печатать. `render`

возвращает строку, `print-...` пишет. Квест 3.L2 именно про это, не про красоту.

Сломай `clamp`: перепутай `lo` и `hi`. `(clamp 5 10 0)`. Лестница `cond` может вернуть чудо. Напиши два вызова в REPL, пока не увидишь, что ответ не тот. Потом верни.

5.3.2 Java: чертёж и железяка

Вчера задачи были строками. Сегодня они **вещи** с полями. Как отличить гаечный ключ от верёвки, хотя оба «лежат в ящике».

```
public class Task {
    private final int id;
    private String title;
    private boolean done;
```

```
    public Task(int id, String
title) {
        this.id = id;
        this.title = title;
        this.done = false;
    }

    public int getId() { return
id; }
    public String getTitle()
{ return title; }
    public boolean isDone()
{ return done; }
    public void complete()
{ this.done = true; }
}
```

Давай медленно
Класс — чертёж. Пока нет `new`, в памяти нет задачи. `new Task(1, "починить антенну")` — уже железяка. Три поля. `private` прячет: снаружи не напишешь `t.id = -7`. Это не бюрократия. Это чтобы через месяц ты сам не присвоил `id = -7` в три часа

ночи. `final` у `id` — повесил один раз в конструкторе, больше не трогай. Название можно сменить (если добавишь сеттер), номер — нет. `this.id = id` — «поле этого объекта = параметр». Без `this` запись `id = id` присваивает параметр самому себе. Здесь компилятор поймает ошибку: final-поле осталось без начального значения. С не-final-полем такая ошибка могла бы промолчать. Геттеры — дырки наружу «посмотреть». `complete()` — дырка «сделать». Не `setDone(true)` с улицы, а глагол. Задача сама знает, как стать сделанной.

Два объекта — две железяки:

```
Task a = new Task(1, "антенна");
Task b = new Task(2, "шлюз");
a.complete();
System.out.println(a.isDone());
System.out.println(b.isDone());
```

`true` и `false`. Починка антенны не закрывает шлюз. Очевидно в жизни. В коде очевидно только если это **разные** `new`.

Ящик для задач:

```
import java.util.ArrayList;
import java.util.List;

public class TaskStore {
    private final List<Task>
tasks = new ArrayList<>();
    private int nextId = 1;

    public Task add(String title)
{
        Task t = new Task(nextId+
+, title);
        tasks.add(t);
        return t;
```

```
    }

    public List<Task> all() {
        return tasks;
    }
}
```

Что только что случилось `nextId++` — отдай текущий номер, потом прибавь. Первая задача — id 1, вторая — 2. Список внутри `private`. Снаружи не `tasks.add` мимо кассы: только через `add(title)`, чтобы номер всегда выдавал магазин, а не гость с улицы. `all()` сейчас отдаёт **тот же** список, не копию. Гость может сделать `store.all().clear()` и ты заплачешь. На занятии 7 появится `List.copyOf`. Сегодня заметь запах. Можно вернуть копию уже сейчас, если совесть

зудит: `return new ArrayList<>(tasks)`.

`main` командует. Правила — в `Task` и `TaskStore`. Если `main` длиннее экрана, станция смотрит на тебя с укоризной.

```
public class TodoApp {
    public static void
main(String[] args) {
        TaskStore store = new
TaskStore();
        store.add("починить
антенну");
        store.add("закрыть люк");
        for (Task t :
store.all()) {

System.out.println(t.getId() + "
" + t.getTitle());
        }
    }
}
```

Три файла: `Task.java`, `TaskStore.java`, `TodoApp.java`. Три чертежа. Компилируй все:

```
javac Task.java TaskStore.java
TodoApp.java
java TodoApp
```

IntelliJ может скомпилировать их сама. `javac TodoApp.java` тоже умеет найти и скомпилировать исходники зависимостей в той же папке. `cannot find symbol: class TaskStore` значит, что нужный исходник или готовый класс не найден там, где ищет компилятор, либо не совпало имя или пакет. Явно перечислить три файла — нагляднее для первого запуска.

Закон станции
Вынеси кусок в метод, как только `main` начал стыдиться.

Через два месяца тот же инстинкт не даст запихнуть всю жизнь в Spring-контроллер.

Поиск по id — рука, которая понадобится в квесте `done`:

```
public Task findById(int id) {
    for (Task t : tasks) {
        if (t.getId() == id) {
            return t;
        }
    }
    return null;
}
```

Нашёл — верни. Не нашёл — `null`. Вызови `complete` на `null` — взрыв. Поэтому в `done`: сначала найти, потом `if (t == null) { сказать; } else { t.complete(); }`.

Так себе идея

`==` для `int` — правильно, это числа. `==` для `String title` — снова ловушка. Поля-строки сравнивай `.equals`. Id — число, дыши спокойно.

5.3.3 Если сломалось: объекты и несколько файлов

`The public type Task must be defined in its own file.` В одном `.java` два `public class`. Либо убери `public` у второго (не надо), либо два файла. Один публичный класс — один файл с тем же именем.

`constructor Task in class Task cannot be applied to given types.` Вызвал `new Task("антенна")`, а конструктор ждёт `(int, String)`. Номер сам не появится — его выдаёт `TaskStore`.

`id has private access`. Пишешь `t.id` из `main`. Иди в `getId()`. Именно для этого дырка.

Задачи с одинаковым id. Делаешь `new Task` в `main` руками и сам ставишь единицы. Потом `done 1` отмечает не ту. Пусть номера раздаёт только `nextId`.

`javac TodoApp.java` один, без остальных. В той же папке должны лежать `.java` зависимостей, или уже `.class`. Проще компилировать `*.java` (мак/WSL). На Windows в cmd: `javac *.java` тоже часто работает.

Три прогона.

1. Два `add`, печать id. Ожидай 1 и 2, не 0 и 1. Если нули — `nextId` начал с нуля, люди на

станции считают с единицы, капитан будет орать.

2. `complete` на первой, печать `isDone` обеих. Только первая `true`.
3. `findById(99)` — `null`. Напечатай явно `System.out.println(store.findById(99));` — увидишь слово `null`. Подружись, пока оно не пришло в Spring.

Квест 3.L1 · Lisp
`spend` и `recharge`: энергия 0..100. `simulate`: список трат, старт 100, верни остаток. Не уходи в минус — реактор этого не оценит.

Квест 3.L2 · Lisp
`render-room` возвращает строку. `print-rooms` только печатает.

Раздели «что сказать» и «как напечатать в терминал».

Квест 3.L3 · Lisp
`docking-score` из текста вынеси в свой файл и добавь третий параметр `angle` (угол захода). Если угол по модулю меньше 10 — ещё +3 очка. Проверь четыре-пять комбинаций в REPL, запиши результаты комментариями. Модуль числа: `(abs angle)`.

Квест 3.J1 · Java
Вчерашний TODO — на `Task` и `TaskStore`. Команда `done номер` отмечает задачу. Номер — тот самый id, не «третья с конца, я так чувствую».

Квест 3.J2 · Java
Поле `priority`. `list` печатает приоритет, id, название.

Сортировку можно завтра. Сегодня хотя бы видно, что антенна важнее «помыть иллюминатор».

Квест 3.J3 · Java
`TaskStore.findById`. Нет задачи — печать `missing`, не NPE. Команда `show номер` пользуется этим методом. Два файла минимум: логика в сторе, ввод в `main`. Если `findById` живёт в `TodoApp` — ты спрятал кладовщика в вахтёра.

SICP · открывать, только если зудит
Прятать поля — тот же жест, что «конструктор и селекторы» в SICP. Scheme вместо Java, идея та же.

К концу вахты у задачи есть имя и номер, у энергии — пол и потолок, у `main` — шанс

похудеть. Капитан спрашивает: «а после перезагрузки список живой?». Ты честно говоришь «нет». Завтра диск. Сегодня — объекты. Не прыгай вперёд, даже если зудит.

5.4 Занятие 4. Рекурсия, файлы и «расскажи это вслух»

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: стоп на пустом списке, шаг на хвосте, бумажка.
Java: `Files`, `Path`, `try` / `catch`, не пустой `catch`.
Проверка недели: десять минут вслух без подглядывания в код.

Вахта четвёртая. Ночью моргнул свет — штатно, «МОДУЛЬ» так делает раз в сутки, никто не знает почему. TODO в памяти умер. Капитан произнёс слово «файл» так, будто это заклинание старшей расы. Это не заклинание. Это буквы на диске.

Параллельно Lisp просит научиться ходить по списку без `dolist`, своими ногами. Это не чтобы заменить `loop`. Это чтобы в голове щёлкнуло: список — матрёшка.

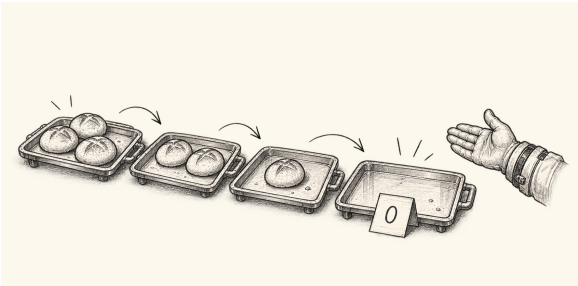
5.4.1 Lisp: функция, которая кусает себя за хвост — но по делу

Рекурсия: вызываешь себя на **меньшем** куске. Список: пустой — стоп, иначе сделай что-то с

головой и повтори с хвостом. Как чистить коридор: один отсек, потом остальные.

```
(defun count-rooms (lst)
  (if (null lst)
      0
      (+ 1 (count-rooms (rest
lst)))))
```

Давай медленно
Разложи `'(a b c)` на бумажке, правда разложи. `(count-rooms '(a b c))` = `1 + (count-rooms '(b c))` `(count-rooms '(b c))` = `1 + (count-rooms '(c))` `(count-rooms '(c))` = `1 + (count-rooms '())` `(count-rooms '())` = `0`, потому что `(null lst)` — «тут уже никого». Сборка назад: 0, плюс 1 это 1, плюс 1 это 2, плюс 1 это 3. Без бумажки половина людей решает, что это магия. Это не магия. Это матрёшки.



Меньше список. Ближе обед.
Каждый вызов получает меньшую задачу; пустой список возвращает 0. Подносы обозначают вызовы.

Вызови:

```
(count-rooms '())
(count-rooms '(airlock))
(count-rooms '(airlock corridor
reactor))
```

0, 1, 3. Если на пустом получил ошибку — нет стопа. Если на трёх крутится вечно — не делаешь `rest`, кусаешь тот же список.

Сумма тем же жестом:

```
(defun sum-list (lst)
  (if (null lst)
      0
      (+ (first lst) (sum-list
(rest lst)))))
```

`(sum-list '(10 20 80))` → 110. Голова плюс сумма хвоста. Стоп — ноль, потому что сумма никого — ноль. Для максимума стоп **не ноль**: максимум пустого списка — философский спор, который мы проиграем. Поэтому `max-list` в квесте — список непустой. Стоп: хвоста нет, ответ — голова.

Что только что случилось Забудешь стоп или `rest` — SBCL рано или поздно напишет `Control stack exhausted`. Перевод: «я устал повторять твоё

заклинание». Стек — стопка незакрытых вызовов. Каждый вызов без `rest` кладёт ещё тарелку. Тарелки кончаются. Ctrl+C, чини стоп.

Ещё одна раскладка, уже про фильтр — идея квеста 4.L2:

```
(defun filter-positive (lst)
  (cond
    ((null lst) nil)
    ((> (first lst) 0)
     (cons (first lst) (filter-
positive (rest lst))))
    (t (filter-positive (rest
lst)))))
```

Давай медленно
Пусто — пусто. Голова плюс — `cons` её к отфильтрованному хвосту. Голова минус или ноль — **не** клади, верни только хвост. `(filter-positive '(3 -1 4 0 -8))` должно дать `(3 4)`. Ноль не

больше нуля, в космос. Отрицательные тоже. Если забыл `cons` и пишешь только рекурсию — получишь `nil`, всё выкинул. Если `cons` без рекурсии — один элемент. Два куска: что сделать с головой, и обязательно хвост.

`mapcar` уже умеет ходить по списку. Свою рекурсию напиши, пока не щёлкнет. Потом можно лениться культурно.

Сломай на радость:

```
(defun bad-count (lst)
  (+ 1 (bad-count lst)))
```

Не вызывай на длинном. Вызови на `'()` и быстро жалей. Нет `rest`, нет стопа. Учебный пожар.

Хвостовая оптимизация, аккумулятор — не этот месяц. Если прочитал в интернете и зудит: можно. Если не прочитал — живи. Главное — стоп и меньший кусок.

5.4.2 Java: когда диск говорит «нет»

Файл не открылся, человек ввёл «пщщ» вместо числа — Java кидает **исключение**. Лови конкретное, не «всё подряд в мешок».

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class TaskFile {
    public static void
save(List<String> lines, Path
```

```
path) throws IOException {
        Files.write(path, lines);
    }

    public static List<String>
load(Path path) throws
IOException {
        if (!Files.exists(path))
{
            return new
ArrayList<>();
        }
        return
Files.readAllLines(path);
    }
}
```

Давай медленно `Path` — «где лежит», не сама куча байт. `Path.of("tasks.txt")` — файл в **текущей** папке процесса. Текущая папка в IDEA может быть не та, что ты думаешь: смотри Run Configuration → Working directory. Иначе сохранишь в

корень проекта или в `out/`, потом «файл пропал». `throws IOException` — честно: «я могу споткнуться о диск». Метод не притворяется всесильным. Кто зовёт — тот решает. Нет файла при `load` — пустой список, не падение. Первый запуск программы на свежей станции: файла нет, и это не авария.

В `main`:

```
try {
    TaskFile.save(titles,
Path.of("tasks.txt"));
} catch (IOException e) {
    System.err.println("не
удалось записать: " +
e.getMessage());
}
```

Что только что случилось `try` — загон, где может рвануть. `catch` — что делать, если

рвануло **именно это**. `System.err` — поток для ругани, не для обычного вывода. В простом терминале выглядит так же. Потом в логах (месяц 2) разъедутся. `e.getMessage()` — короткая жалоба. Иногда полезен весь `e.printStackTrace()` — стопка, кто кого звал. На этой неделе сообщение + «файл не записался» достаточно.

На Windows путь всё равно можно писать как `tasks.txt` в текущей папке. `Path.of` не надо кормить `C:\` без нужды. Обратные слэши в строке Java — ад: `"C:\\Users\\..."` или лучше `"C:/Users/..."`. Или не надо. Относительный путь.

Так себе идея

Пустой `catch (Exception e) {}` — спрятать пожар под ковёр. Ковёр загорится позже, в демо. Лови `IOException` и `NumberFormatException` по местам, не «всё подряд».

`Scanner.nextInt()` на буквах бросает `InputMismatchException` и оставляет плохой токен во вводе. Перед повтором забери эту строку через `in.nextLine()` в catch, иначе снова упрёшься в те же буквы. Для этой консоли проще читать строку и разбирать её, как предлагалось на занятии 1:

```
String raw =
in.nextLine().trim();
try {
    int n =
Integer.parseInt(raw);
```

```
    // n честный
} catch (NumberFormatException e)
{
    System.out.println("это не
число");
}
```

Мак, Windows, WSL
Файл появится там, откуда запустили JVM. Мак/WSL: `pwd` перед `java TodoApp`. Windows PowerShell: `pwd` тоже. Нашёл `tasks.txt` в неожиданном месте — не магия, рабочая директория. Перенеси запуск или укажи путь явно один раз, запиши в README.

Формат на диске — тупой и честный: одна задача — одна строка. Приоритет потом, когда базовое дышит. Сегодня:

```
починить антенну
закрыть люк
```

Прочитал строки → для каждой `store.add(line)`. Id раздадутся заново 1, 2, 3. После перезапуска номера могут не совпасть с «как вчера на бумажке», если удалял из середины. Для недели 4 это ок. Скажи это в README, не прячь. Этот первый формат сохраняет только названия: `done` снова станет false, приоритет пока тоже не записан. Скажи об этом в README; для всех полей нужен формат богаче.

Запись при `quit`:

```
List<String> titles = new
ArrayList<>();
for (Task t : store.all()) {
    titles.add(t.getTitle());
}
```

```
TaskFile.save(titles,
Path.of("tasks.txt"));
```

Не забудь: у `save` объявлен `IOException`. Вызывающий метод должен поймать его или объявить через `throws`, иначе код не скомпилируется. В этой консоли поймай ошибку вокруг сохранения и скажи человеку, если файл не записался.

5.4.3 Если сломалось: файлы и исключения

`Unhandled exception: IOException.` Метод зовёт `save`, но сам не `throws` и не `try`. Компилятор прав. Либо пробрось, либо поймай. В `main` обычно поймай: пользователю не нужен стек как единственный ответ, но и молчать нельзя.

`NoSuchFileException` при записи. Каталога нет. `tasks.txt` в папке `data/`, а `data/` не создавал. Либо пиши в текущую, либо `Files.createDirectories`. Пока — текущая.

Кириллица кракозябрами. Редко на Java 21, но если открыл файл блокнотом с неправильной кодировкой — не вини `Files`. IntelliJ и современный терминал обычно UTF-8. На старом cmd.exe бывает цирк. WSL и macOS Terminal спокойнее.

`AccessDeniedException`. Файл открыт в Excel / блокноте с блокировкой, или путь в «Program Files». Не пиши в системные папки.

Два прогона диска.

1. Сохрани две строки, выключи программу, открой `tasks.txt` глазами в редакторе. Буквы те? Хорошо. Не те — кодировка или не тот файл.
2. Удали файл, запусти, не падай, добавь одну задачу, `quit`, файл родился. Первый день на станции.

5.4.4 Проверка недели: кот тоже сойдёт

Десять минут вслух — хоть коту, хоть диктофону. Как устроен `TaskStore`, где список, где файл, что если файла нет. Не можешь без подглядывания в код — переименуй методы так, чтобы рассказ пошёл. Имена, которые стыдно произнести, обычно и вранье.

Шпаргалка для голоса, не читай с листа — проверь, что помнишь:

- Задача — объект, не строка. У неё id, название, done.
- Стор хранит список и выдаёт id.
- Файл — запасной мозг на случай моргания света.
- Lisp-список — голова и хвост, рекурсия стопает на `nil`.

Если застрял на «ну там ArrayList» — мало. Скажи, **зачем** стор, а не `main`.

Квест 4.L1 · Lisp `max-list` рекурсией, список непустой. Без `apply` и `reduce`. Голыми руками, как бортмеханик.

Квест 4.L2 · Lisp
`filter-positive`: только числа больше нуля. Рекурсия. Отрицательные пусть летят в космос.

Квест 4.L3 · Lisp
`rooms-length` — свой `length` через рекурсию, без вызова `length`. Сверь с `(length lst)` на `'()`, `'(a)`, длинном списке. Если расходится — стоп или `rest`. Бонус: `find-room`, которая вернёт хвост от находки, как `member`, тоже рекурсией.

Квест 4.J1 · Java
При `quit` пиши `tasks.txt`, при старте читай. Одна задача — одна строка. Нет файла — живи с пустым списком, не падай.

Квест 4.J2 · Java

README: как запустить, какие команды, пример сессии. Это не «потом». Без README твоя программа — шкатулка без петли.

Квест 4.J3 · Java
В README отдельный абзац «если сломалось»: нет JDK, `cannot find symbol`, нет файла при старте, `InputMismatch` / нечисловой ввод. По одному предложению на беду — что **видеть** и что **делать**. Проверь, открыв README на телефоне: понятно ли без IDEA на экране.

Спрятать на GitHub
Коммит `week4: standalone java program`. Пусть откроет кто-то другой (или ты с другой машины — Мак, Windows,

неважно) и по README поймёт, куда жать.

SICP · открывать, только если зудит
Рекурсия на списках — куски 1.1–1.2, если сам спросил «почему это работает», а не «надо пройти SICP».

Воскресная диверсия
Нарисуй на бумаге список `'(airlock corridor reactor)` как цепочку коробок со стрелками «хвост». Положи рядом `ArrayList` из трёх строк — клетки в ряд с номерами 0, 1, 2. Это не одно и то же. Зато оба отвечают на вопрос «что у нас в коридоре».

Свет моргнул ещё раз. Ты перезапустил программу — список на месте. Капитан не похвалил: на «МОДУЛЕ»

похвала выглядит как отсутствие сирены. Четыре недели. Своя консоль. GitHub. Спринга нет. Можно выдохнуть и не ставить Kafka «чтобы резюме жирнее». Следующий месяц Земля начнёт стучаться по HTTP. Пока — чай. Кофе всё ещё кто-то выпил.

6 Месяц 2. Оно уже отвечает по сети

К восьмой неделе у тебя маленький веб-сервер на Spring Boot. Данные пока в памяти: перезапустил — всё забылось, как сон после ночной смены. Зато ты видишь HTTP и перестаёшь верить, что «интернет» — это кнопка. Lisp: лямбды, прогулки по спискам, карманчики alist. И тесты: программа, которая дёргает твою программу за ухо.

Станция «МОДУЛЬ» научилась помнить список дел на диске. Земля этого не видит. Земля шлёт письма. Письма называются HTTP, и они не магия, они текст. Если текст

страшный — прочитай его вслух. Страшнее не станет, зато станет понятнее.

Четыре вахты. После них — адрес в браузере, который отвечает не «не удаётся получить доступ к сайту», а JSON. Праздник скромный. Кафку на торт не клади.

Сегодня на станцию · 2 ч 40 мин
Lisp: `lambda`, `mapcar`, alist, свой «ответ сервера» строкой, чуть журнала.
Java: JUnit, ветка в Git, Spring Web, тонкий контроллер, сервис, лог.
К концу месяца: README с curl, тесты, `GET`/`POST`, фильтр `done`.
Память, не база. База —

следующий месяц, когда заслужишь.

6.1 Занятие 5. Тесты и git, который не страшно открывать

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: безымянная функция, `remove-if` / `find-if`, пары в alist.
Java: Maven, один красный тест, потом зелёный. Ветка, не сразу `main`.
Инспектор с Земли обещал «глянуть репозиторий».
Инспектор — это ты через неделю.

Вахта пятая. Кто-то (не ты) починил `complete` так, что оно ничего не делает, зато компилируется. На станции это называется «оптимизм». Тест — право сказать «врёшь» без крика по внутренней связи.

6.1.1 Lisp: функция без имени, потому что лень

Иногда функция нужна один раз, как бумажный стаканчик. Имя для неё — бюрократия. Тогда `lambda`:

```
(mapcar (lambda (n) (* n 10)) '(1 2 3))
; (10 20 30)
```

Давай медленно `lambda` — «вот тело, имени не дам». Список параметров `(n)`, тело `(* n 10)`. `mapcar` берёт

каждый элемент, кормит лямбде, собирает ответы. Тот же жест, что `#'evenp` на занятии 3, только функции нет в справочнике станции — ты её слепил на месте. Вызови ещё `(mapcar (lambda (n) n) '(1 2 3))`. Получишь копию. Лямбда-тождество. Бесполезно и успокаивает: видишь, что ходит элемент, а не «магия mapcar».

Ещё:

```
(remove-if (lambda (n) (< n 0))
'(3 -1 4 0 -8))
; (3 4 0)

(find-if (lambda (room) (eq room
'reactor))
         '(airlock corridor
reactor))
; REACTOR
```

Что только что случилось

`remove-if` выкидывает тех, для кого лямбда не-`nil`. Отрицательные улетели, ноль остался: `(< 0 0)` это `nil`. `find-if` — первый, кто подошёл. Нет таких — `nil`. `eq` сравнивает символы. Строки — не `eq`, для строк `string=`. Сейчас у нас бирки-символы, `eq` честный.

Порог энергии лямбдой, без отдельного `defun`:

```
(remove-if (lambda (n) (< n 30))
'(80 20 55 10))
; (80 55)
```

И наоборот, оставить голодных:

```
(remove-if-not (lambda (n) (< n
30)) '(80 20 55 10))
; (20 10)
```

`remove-if-not` — двойное отрицание в имени, зато

читается «оставь тех, кто…». Два имени, одна семья. Не учи все `*-if` сразу. Этих двух хватит на вахту.

Ассоциативный список — список пар. Карман бедняка:

```
(defparameter *energy*
  '((reactor . 80) (antenna . 20)
(life-support . 55)))

(assoc 'antenna *energy*)
; (ANTENNA . 20)

(cdr (assoc 'antenna *energy*))
; 20

(assoc 'garden *energy*)
; NIL
```

Давай медленно
Точка в `(reactor . 80)` — cons-пара. Не список из двух с `nil` в хвосте, а «голова и хвост, хвост — число». Выглядит как

опечатка. Это не опечатка. Это Lisp такой родной. `assoc` ищет пару по **голове**. Нашёл — всю пару. Не нашёл — `nil`. Дальше `(cdr пара)` — значение. Если сразу `(cdr (assoc ...))` на отсутствующем ключе, `(cdr nil)` даст `nil`. Удобно. Не отличить «модуля нет» от «энергия nil», но энергия у нас число, живи спокойно. Сравни с Java `HashMap`: там `get` сразу значение или `null`. Здесь два шага. Зато видно устройство: карман — тоже список. Всё списки, мы говорили.

Положи новую пару спереди, как `cons` на занятии 2:

```
(cons '(garden . 0) *energy*)
```

Старый `*energy*` не изменился. Чтобы повесить на глобальную:

```
(setf *energy* (cons '(garden .
0) *energy*))
(assoc 'garden *energy*)
```

Теперь сад есть, энергии ноль. Оранжерея, мы тебя помним.

Три примера, не один.

Пример А. Удвоить, но потолок 100 — это квест 5.L1, сначала глазами:

```
(mapcar (lambda (n) (min 100 (* n
2))) '(10 60 40))
; (20 100 80)
```

`min` из двух берёт меньшее. `(* 60 2)` = 120, `min` режет до 100. Реактор не резиновый.

Пример Б. Имена модулей из alist:

```
(mapcar #'car *energy*)
```

`car` — голова пары, то есть имя. `#'car` можно, лямбду можно. Оба честно.

Пример В. Только `'down`:

```
(defparameter *module-statuses*
  '((reactor . up) (antenna .
down) (airlock . up)))

(mapcan (lambda (pair)
          (if (eq (cdr pair)
'down)
              (list (car pair))
              nil))
        *module-statuses*)
; (ANTENNA)
```

`mapcan` склеивает списки-ответы. Лямбда вернула `(ANTENNA)` или `nil` (пусто, нечего клеить). Получается список имён, не список пар. Если взял `mapcar` — получишь `(NIL (ANTENNA) NIL)` и

удивишься. Квест 5.L2 про это. Сначала сломай `mapcar`, потом почини `mapcan` или `loop`.

Так себе идея `(lambda n (* n 10))` без скобок вокруг `n` — параметры должны быть списком `(n)`. Та же дырка, что у `defun` в первый день. SBCL орнёт про `n` не в списке. Скобки вокруг имён, даже если имя одно.

6.1.2 Java: тест, который имеет право тебя обидеть

Тест — отдельная программа. Она вызывает твою и ругается, если ответ не тот. Это не «проверка глазами». Глаза врут, когда устали. Тест не устаёт, он только бесит.

Maven-проект в IntelliJ: New Project → Maven → JDK 21. Координаты вроде `com.example` / `taskstore`. На Windows без WSL потом будет `mvnw.cmd test`, в WSL и на маке — `./mvnw test`. Один `pom.xml` на всех. Если `mvnw` ещё нет, один раз создай wrapper: выполни `mvn wrapper:wrapper` установленным Maven или запусти цель `wrapper:wrapper` из окна Maven в IntelliJ его встроенным Maven. Потом закоммить `mvnw`, `mvnw.cmd` и `.mvn/`.

Мак, Windows, WSL

Мак / WSL. В корне проекта после генерации wrapper:

```
chmod +x mvnw
./mvnw test
```

`chmod` на маке и в Ubuntu нужен один раз, если «Permission denied».
Windows cmd. `mvnw.cmd test`. Не мешай слэши.
IntelliJ. Зелёная стрелка на классе `...Test` может использовать JUnit runner самой IDE. Запусти и цель Maven `test`: один импорт Maven-проекта не гарантирует, что оба запуска идут через один тестовый механизм.

Для этого обычного Maven-упражнения возьми полный `pom.xml` ниже. Здесь зафиксирована учебная связка: Java 21, JUnit 5, плагин компилятора и Surefire, который действительно запускает тесты. Это закреплённые версии примера,

а не обещание самых новых версий. Позже сохрани собственный сгенерированный POM Spring Boot-проекта; не заменяй его этим.

```
<project xmlns="http://
maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</
modelVersion>
    <groupId>com.example</
groupId>
    <artifactId>taskstore</
artifactId>
    <version>1.0-SNAPSHOT</
version>
    <properties>

<maven.compiler.release>21</
maven.compiler.release>

<project.build.sourceEncoding>UTF
-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency>
```

```
<groupId>org.junit.jupiter</
groupId>
            <artifactId>junit-
jupiter</artifactId>
            <version>5.10.2</
version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>

<groupId>org.apache.maven.plugins
</groupId>

<artifactId>maven-compiler-
plugin</artifactId>
                <version>3.13.0</
version>
            </plugin>
            <plugin>

<groupId>org.apache.maven.plugins
</groupId>

<artifactId>maven-surefire-
plugin</artifactId>
```

```
                <version>3.2.5</
version>
                <configuration>

<failIfNoTests>true</
failIfNoTests>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

`scope>test` — в боевую программу не попадёт. Тестовый молоток не кладут в скафандр.

`maven.compiler.release` задаёт совместимость языка и API с Java 21; сам Maven здесь тоже запускай на JDK 21. Проверь `./mvnw -v` (Windows: `mvnw.cmd -v`). Добавив `TaskStoreTest` ниже, снова выполни `./mvnw test`. Читай итог: запущен хотя бы

один тест, ошибок и падений ноль, пропущенных ноль. `BUILD SUCCESS` без выполненных тестов — не победа. `failIfNoTests` делает пустой тестовый запуск красным.

Положи `Task` и `TaskStore` в `src/main/java/...`. Тест — в `src/test/java/...` **тот же пакет или импорт**. Имена: класс `TaskStore`, тест `TaskStoreTest`. Не `TestTaskStore` обязательно, но суффикс `Test` Maven/JUnit находят спокойно.

```
import
org.junit.jupiter.api.Test;
import static
org.junit.jupiter.api.Assertions.
*;

class TaskStoreTest {
    @Test
```

```
    void addIncreasesSize() {
        TaskStore store = new
TaskStore();
        store.add("антенна");
        assertEquals(1,
store.all().size());
        assertEquals("антенна",
store.all().get(0).getTitle());
    }
}
```

Давай медленно `@Test` — «это не обычный метод, это квест для машины». Имя метода — предложение: что должно быть правдой. `assertEquals(ожидали, получили)` — порядок такой. Перепутаешь — сообщение об ошибке будет издеваться. Каждый тест — **новый** `TaskStore`. При стандартном жизненном цикле JUnit Jupiter обычное поле объекта, инициализированное

через `new TaskStore()`, тоже будет новым для каждого теста. Ловушка — общий `static`-стор или `@TestInstance(PER_CLASS)` без сброса состояния. Тогда тесты могут зависеть от порядка. Станция и так на изоленте, не надо ещё и плавающие тесты.

Напиши тест, **который падает**, потом почини код. Иначе ты тестируешь компилятор, а не себя.

Ритуал падающего теста:

1. В `complete` временно ничего не делай (пустой метод, `done` не трогай).
2. Тест: добавь, `complete(id)`, `assertTrue(task.isDone())`.
3. Запусти — тест падает. Прочитай сообщение. Оно

должно говорить про `true` vs `false`, не про «не компилируется».

4. Верни строку, которая ставит done. Зелёное.

Приятно же. Это не садизм. Это доказательство, что тест **умеет** ругаться. Зелёный тест, который зелёный даже когда код мёртвый — декорация.

Три теста — три грани, не три копии `add`.

- Добавили — размер 1, название то.
- `complete` существующего — `isDone()`.
- `complete` несуществующего — **твоё** решение: исключение или `false`. Проверь его, не «как в интернете».

```
@Test
void completeMissing() {
    TaskStore store = new
TaskStore();

assertThrows(NoSuchElementExcepti
on.class, () ->
store.complete(99));
}
```

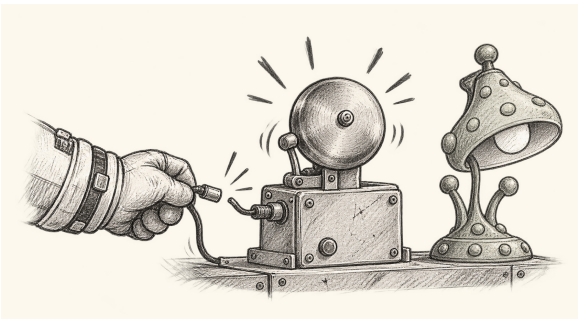
Или:

```
assertFalse(store.complete(99));
```

Не оба. Выбери контракт, запиши в README одной строкой, тест его держит.

Что только что случилось Лямбда `() -> store.complete(99)` — «вот код, который должен рвануть». JUnit сам вызовет и поймает. Если **не** рвануло — тест красный. Если рвануло другим исключением — тоже

красный. Конкретный класс, не «любая беда».



Сначала проверь, умеет ли она ругаться.
Нарочно сломай проверяемое поведение, дождись падения теста и почини.

6.1.3 Git, который не страшно открывать

До сих пор можно было жить на `main` и притворяться, что ветки — для взрослых. Сегодня взрослая вахта. Инспектор (ты

будущий) любит историю, где видно **зачем**, не «asdf».

```
git status
git checkout -b feat/tests
```

Старые Git понимают `checkout -b`, новые ещё `git switch -c feat/tests`. Оба ок. Имя ветки: `feat/tests`, не `tmp` и не `aaa`.

Правишь тесты и код. Смотришь зеркало:

```
git diff
```

Давай медленно `git diff` — что изменилось **сейчас**, не в голове. Красное уехало, зелёное приехало. Перед коммитом прочитай, как перед выходом из шлюза смотрят на датчик кислорода: не «ну вроде». Случайно закоммитил

пароль — уже письмо. Случайно закоммитил `.class` — стыдно, но лечится. `.gitignore` с занятия 0: `*.class`, `.idea/`, `target/`.

```
git add -A
git commit -m "week5: tests for
task store"
git switch main
git merge feat/tests
```

На GitHub: `git push -u origin HEAD` с ветки, потом Pull Request — или сразу merge локально и `push main`, если ты один на репозитории. Оба пути честные для учебника. Не честно: править `main` вперемешку с экспериментом «а если complete ничего не делает» и пушить падающие тесты как будто так и надо.

Мак, Windows, WSL

Git одинаковый на маке, в WSL и в Git for Windows. Отличается терминал. Если PowerShell ест кавычки в `-m` — одинарные, или IDEA: галочка Commit. Не делай коммит через `git commit --amend`, если уже пушил, и вообще в этой книге мы amend не празднуем: новый коммит проще, чем геройство.

`git log --oneline` должен читаться как бортовой журнал: `week4: standalone java program`, `week5: tests for task store`. Не `fix`, `fix2`, `asdf`. Будущий ты скажет спасибо. Или хотя бы не скажет плохое.

6.1.4 Если сломалось: тесты и Maven

`Cannot resolve symbol Test`. Нет зависимости JUnit или IDEA не

импортировала Maven. Правый клик по `pom.xml` → Reload. Терминал: `./mvnw test`. Если в терминале зелёное, а IDEA красная — это IDE, не Java.

`Error: Java version`. Проект на 17, JDK 21, или наоборот. В `pom.xml` `maven.compiler.release` (или source/target) = 21. File → Project Structure → SDK 21.

Тест не находится. Класс не в `src/test/java`, или метод не `@Test`, или не `void`. Или имя `testAdd` без аннотации — JUnit 5 не подбирает по префиксу, это дедовский JUnit 3.

Красный `assertEquals` с кучей строк. Читай Expected / Actual. Часто 0 vs 1 — `add` не тот список. Часто ссылки — сравнил

объекты через == внутри своего кода, не в ассерте.

`BUILD SUCCESS` при нуле тестов. Ты их не положил или имя `TaskStoreTestsBackup.java` не подхватилось. В логе строка `Tests run:`. Ноль — не победа.

Квест 5.L1 · Lisp
`mapcar` и `lambda`: энергии умножить на 2, но не выше 100. `(min 100 x)`. Реактор не резиновый.

Квест 5.L2 · Lisp
Alist модуль → статус. `offline-modules`: те, у кого `'down`. Кто-то снова забыл выключить свет в отсеке, который уже мёртв.

Квест 5.L3 · Lisp
`energy-of`: alist как `*energy*` в тексте, имя модуля — символ.

Верни число или `nil`. Через `assoc` и `cdr`. Проверь реактор, антенну и `'observatory`, которого нет. Три вызова, три ответа, в комментариях. У сада после прежнего `setf` уже есть энергия 0.

Квест 5.J1 · Java
Три теста: add; complete по id; complete несуществующего — реши, `false` это или исключение, и проверь **своё** решение, не «как в интернете».

Квест 5.J2 · Java
Сломай `complete`, убедись что падает, верни. В README одна фраза: как гонять тесты на твоей машине (мак / WSL / `mvnw.cmd`).

Квест 5.J3 · Java

Ветка `feat/tests`, хотя бы один коммит на ней, слияние в `main`. В README: три строки `git log --oneline` (можно руками скопировать). Если все коммиты `asdf` — перепиши сообщения **до** пуша или живи со стыдом. Журнал вахты, не чат.

Спрятать на GitHub
Коммит на ветке, потом в main. `git log` должен читаться как бортовой журнал, не как «asdf».

Инспектор глянул. Тесты зелёные. `complete` снова глючит, если выключить одну строку — и тест это видит. На «МОДУЛЕ» это первый датчик, который срабатывает **до** взрыва. Можно привыкнуть.

6.2 Занятие 6. HTTP — это просто текст, который притворяется интернетом

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: склеить «ответ сервера» строкой, чуть JSON руками.
Java: start.spring.io, три дырки, curl с соседнего окна.
Перезапустил — задачи умерли. Так и надо. База будет, когда заслужишь.

Вахта шестая. Земля прислала не космонавта, а письмо. В письме первая строка: `GET /health HTTP/1.1`. Капитан спрашивает, это вирус? Это

вежливость. Браузер так стучится. Spring так слушает. Сегодня ты видишь письмо голым, потом надеваешь на станцию костюм Boot.

6.2.1 Lisp: напечатай «ответ сервера» сам

HTTP — письма. Не облако. Не магия Спринга. Письма.

```
(defun http-ok (body)
  (format nil "HTTP/1.1 200
OK~%Content-Type: text/
plain~%~%~a" body))
```

Давай медленно `format nil` — не пишет в терминал, а **возвращает строку**. Как `render-room` против `print-rooms`. Сначала статусная строка: версия, код, фраза. Потом заголовки. Потом **пустая строка**. Потом тело. `~%` —

перевод строки. Два `~%~%` подряд после заголовка — та самая пустая строка. Забудь одну — клиент будет ждать заголовки вечно или склеить тело с `Content-Type`. Запомни как заклинание: метод, путь, статус, заголовки, тело. Это модель HTTP/1.1 для экрана, не готовая запись в сокет: в настоящих сообщениях нужны окончания строк CRLF и определённое правило длины тела или закрытия соединения. В Java-примерах эти детали берёт HTTP-библиотека.

Вызови:

```
(http-ok "реактор стабилен")
```

Напечатай результат `format t` с `~s` или просто посмотри. Между

`plain` и `реактор` должна быть пустота-строка. JSON — тоже текст, только с фигурными скобками и обидами на запятые.

Запрос с Земли (это ты изображаешь клиент):

```
GET /health HTTP/1.1
Host: localhost:8080
```

Снова пустая строка в конце заголовков. Метод `GET` — «дай посмотреть». `POST` — «вот тело, положи». Не «дай/пошли» в коде и в curl: пиши `GET` и `POST`, как в RFC и в вакансиях.

Коды, которые хватит на месяц:

- 200 — вот, держи.
- 201 — создал.

- 204 — сделал, тела нет (удалил и молчит).
- 400 — ты прислал ерунду.
- 404 — нет такой дырки или нет такой задачи.
- 500 — мы сами дураки.

Пятьсот стыдно. Четыреста — нормальный разговор. Разница как между «реактор взорвался» и «вы забыли закрыть люк».

Сложи письмо целиком, как будто ты труба, а не фреймворк. Запрос:

```
POST /tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Content-Length: 37

{"title":"починить шлюз"}
```

Ответ, который ты сам склеишь в Lisp и который потом выдаст Spring:

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":1,"title":"починить шлюз"}
```

Что только что случилось Одинаковая пустая строка в обоих письмах — граница. Выше — служебные слова. Ниже — груз. `Content-Length` сейчас можно не считать руками: curl и Boot умеют. Но видеть, что он вообще бывает, полезно: это «сколько байтов в теле», не магия. Один символ UTF-8 может занимать несколько байтов. Здесь тело заканчивается сразу после `}`, без перевода строки в конце. `GET`

тела обычно не несёт. `POST` несёт. Поэтому curl без `-d` на POST — письмо с пустым трюмом. Сервер имеет право обидеться.

Три прогона глазами, пока не Spring.

1. `(http-ok "UP")` — есть ли `200` и пустая строка. Напечатай через `format` с `~s`: два перевода подряд должны быть видны.
2. Поменяй `200 OK` на `200OK` без пробела. Это уже не статусная строка, а каша. Верни пробел.
3. Сначала выполни определение `json-task` ниже, затем `(json-task 1 "шлюз")` — считай кавычки. Если сломал экранирование в `format`, получишь кривой JSON. В

Common Lisp кавычка внутри строки пишется с обратным слэшем.

Учебный JSON без библиотеки:

```
(defun json-task (id title)
  (format nil "{\"id\":~a,
\"title\":\"~a\"}" id title))

(json-task 1 "шлюз")
; {"id":1,"title":"шлюз"}
```

Кавычки в `title` сломают жесть. Квест говорит: не экранируй, учебный жестяной JSON, не для боя. Потом Jackson в Spring сделает это за тебя, и ты ему даже спасибо не скажешь.

Ещё один конверт:

```
(defun http-created (body)
  (format nil "HTTP/1.1 201


```

```
Created~%Content-Type:
application/json~%~%~a" body))
```

Склей: `(http-created (json-task 2 "антенна"))`. Прочитай глазами как почтальон.

Так себе идея
Пробел в `HTTP/1.1 200 OK` обязателен. `200OK` без пробела — не письмо, а каша. Код — число, фраза — слова. Три куска первой строки, не два.

6.2.2 Java: Spring Boot, три дырки в корпусе

https://start.spring.io — Maven, Java 21, галка **Spring Web**. Group `com.example`, Artifact `task-manager`. Скачается zip. Распакуй в `task-manager`. Примеры книги используют API Spring Boot 3.x с Java 21. На момент проверки

издания в октябре 2026 года открытая поддержка ветки 3.5 уже завершилась. Не публикуй эти примеры в интернете на зависимостях без поддержки. Для нового сетевого проекта выбирай поддерживаемый выпуск Boot 4.x и сверяйся с официальным руководством по миграции: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide . Координаты зависимостей и имена пакетов могут отличаться. Проверка всего курса на Boot 4.x ещё не завершена; запиши точную версию и выполни все проверки.

Мак, Windows, WSL
Мак. zip в Downloads, распакуй двойным кликом или `unzip`

`task-manager.zip`.
WSL. Скачай в Windows, копируй в Linux, или `unzip` в Ubuntu: `sudo apt install -y unzip` при необходимости.
Windows без WSL. Проводник → извлечь. Запуск потом `mvnw.cmd`.
Открой папку в IntelliJ как Maven-проект. Подожди, пока справа перестанут крутиться зависимости. Первый раз качает интернет. Без интернета Boot не встанет — это не «ты криво распаковал».

Три дырки. Не тридцать.

```
@RestController
public class TaskController {
    private final List<Task>
tasks = new ArrayList<>();
    private int nextId = 1;
```

```
    @GetMapping("/health")
    public Map<String, String>
health() {
        return Map.of("status",
"UP");
    }

    @GetMapping("/tasks")
    public List<Task> all() {
        return tasks;
    }

    @PostMapping("/tasks")
    public Task
create(@RequestBody TaskRequest
req) {
        Task t = new Task(nextId+
+, req.title());
        tasks.add(t);
        return t;
    }
}

record TaskRequest(String title)
{}
```

Давай медленно

`@RestController` — «я отвечаю на HTTP, тело — данные, не HTML-страница». `@GetMapping("/health")` — если письмо `GET /health`, вызови этот метод. Возврат `Map` Спринг превратит в JSON сам. Ты не пишешь `{"status":"UP"}` руками, хотя в Lisp только что писал — и поэтому не боишься. `@PostMapping` — письмо `POST`. `@RequestBody` — тело письма разверни в `TaskRequest`. `record` в Java 21 — короткий класс: поле `title`, конструктор, геттер `title()`. Не копируй десять строк ритуала, если хватает одной. Список пока **в контроллере**. Завтра это стыдно. Сегодня — чтобы вообще дышало. Толстый вахтёр, который ещё и ящики таскает. Занятие 7 его похудеет. Это локальная учебная модель

для последовательных запросов: `ArrayList` и `nextId++` здесь не защищены от одновременного доступа. Отправляй curl по одному; до многопользовательского сервера ещё нужны хранение в базе и корректная работа с конкурентными изменениями.

Класс `Task` — твой, с id, title, done, геттерами. Спринг сериализует геттеры в JSON. Нет геттера — поля в ответе не будет, и ты будешь винить Boot. Геттер есть — `"id":1`. Магия, которая на самом деле соглашение.

Запуск:

```
./mvnw spring-boot:run
```

На Windows без WSL: `mvnw.cmd spring-boot:run`. В IDEA — зелёная стрелка на класс `...Application` с `@SpringBootApplication`. В логе жди что-то про `Tomcat started on port 8080`. Нет этой строки — не «запустилось». Читай ошибку **выше**. Часто: порт занят, Java не 21, опечатка в пакете.

Потом в **другом** окне:

Мак, Windows, WSL
На маке и в WSL перенос длинной команды — обратный слэш `\`. В cmd.exe — `^`. В PowerShell проще одна длинная строка. `curl` в Windows 10+ уже есть (`curl.exe`). Если PowerShell подменяет `curl` своим `Invoke-WebRequest` — зови `curl.exe`. Если ругается странно — зайди в

Ubuntu (WSL) и дергай оттуда, сервер на `localhost` общий, если Java тоже в WSL. Java на Windows, curl в WSL: `localhost` иногда тот, иногда нет. Держи клиента и сервер в одной вселенной.

```
curl -s localhost:8080/health
curl -s localhost:8080/tasks
```

Первый — `{"status":"UP"}`. Второй — `[]`. Пустой список, не ошибка. Станция живая, дел нет.

```
curl -s -X POST localhost:8080/
tasks \
  -H "Content-Type: application/
json" \
  -d '{"title":"починить шлюз"}'
```

Что только что случилось

`-X POST` — метод. Без него `curl` с `-d` часто и так сделает POST, но лучше быть взрослым явно. `-H` — заголовок: «тело — JSON, не форма с сайта». `-d` — тело. Кавычки вокруг JSON в разных шеллах кусаются. В bash одинарные снаружи, двойные внутри JSON. В cmd.exe — цирк с `\"`. В WSL спокойнее. Ответ — задача с `id`. Потом снова `GET /tasks` — массив из одного. Перезапусти Boot — массив снова пуст. Память процесса. Выключил JVM — трюм забыл. Файлы занятия 4 здесь не подключены. Не подключай «на всякий случай». Месяц 3 — диск настоящий, Postgres.

Одна задача по id — квест 6.J1, идея:

```
@GetMapping("/tasks/{id}")
public ResponseEntity<Task>
one(@PathVariable int id) {
    // найти в списке, ок или 404
}
```

`{id}` в пути — дырка. `@PathVariable` — достань число. `ResponseEntity` — «не только тело, ещё статус». Нашёл — 200 и объект. Нет — 404 без выдуманной задачи. Выдуманная задача с id 0 хуже 404: Земля подумает, что шлюз починен.

Так себе идея
Не тащи JPA, Security и Кафку «потому что в гайде было». Сегодня три дырки. Жадность на аннотации — путь к тёмной стороне и к резюме, которое врёт.

6.2.3 Если сломалось: Boot, порт, curl

`Port 8080 was already in use`. Прошлый `spring-boot:run` жив. Найди окно, Ctrl+C. Или в `application.properties` / yml другой порт — тогда curl тоже на другой. Не плоди три сервера «может этот».

`Whitelabel Error Page` в браузере. Сама страница ещё не говорит, какой статус. Отсутствующий путь — 404; GET по пути, где разрешён только POST, — 405. Проверь через `curl -i` и посмотри, нет ли вместо `/tasks` опечатки `taks`.

`Content-Type` и 415. POST без заголовка JSON. Добавь `-H "Content-Type: application/json"`.

`JSON parse error.` Тело `'{"title": "шлюз"}'` с умными кавычками из мессенджера. Перепиши кавычки руками, тупыми. Или файл `@body.json`.

`curl: (7) Failed to connect.` Сервер не встал или не тот порт. Сначала health. Нет health — нет смысла POST.

`mvnw: command not found.` Ты не в корне проекта, где лежит `mvnw`. `ls` должен показывать `pom.xml`. На маке `./mvnw`, не `mvnw` без точки — иначе PATH.

Компиляция Java та же, плюс:

- класс не в пакете, а Spring ищет в подпакетах от `Application` — положи контроллер **ниже** по дереву пакетов, чем класс с

`@SpringBootApplication`, или укажи `@ComponentScan`. Проще дерево: `com.example.taskmanager`, контроллер там же или в `.web`.

- `record` на Java 17+ ок, на 11 — нет. JDK 21.

Три прогона письма.

1. `GET /health` до и после POST — всегда UP. Health не про задачи.
2. Два POST подряд — id 1 и 2, не дважды 1. `nextId++`.
3. `GET /tasks/99` после квеста — 404, не пустой `{}` с нулями.

Квест 6.L1 · Lisp
`http-not-found`: статус 404, тело `missing`. Станция умеет теряться.

Квест 6.L2 · Lisp
`json-task`: id и title → `{"id":1,"title":"..."}`. Кавычки в title не экранируй — учебный жестяной JSON, не для боя.

Квест 6.L3 · Lisp
`http-created` (201) с `Content-Type: application/json` и телом из `json-task`. Склей в REPL одну строку-письмо. Проверь глазами: первая строка с `201`, пустая строка перед `{`.

Квест 6.J1 · Java
`GET /tasks/{id}` — одна задача или 404. `ResponseEntity` тебе в помощь.

Квест 6.J2 · Java
README: пять примеров curl. Без этого занятие не засчитано, даже если «у меня в IDEA открывается».

Квест 6.J3 · Java
Сравни `POST` без тела и с `{}` через `curl -i`. С показанным контроллером отсутствие тела обычно даёт 400, а `{}` может создать задачу с null в title и вернуть 200: проверки ещё нет. Запиши в README настоящий статус и тело для каждого случая. На следующем занятии отсутствующий title тоже должен давать 400. Если изменённый тобой код даёт 500 — зафиксируй и разбери и это.

Воскресная диверсия
Напиши `Hello` на голом `ServerSocket` (один поток, один ответ). Потом Spring перестанет казаться жрецом. Это просто очень толстый `ServerSocket` в костюме.

Земля получила `UP`. Капитан сказал «неплохо» — на «МОДУЛЕ» это орден. Список дел пока живёт в вахтёре. Завтра вахтёра оттащим от ящиков.

6.3 Занятие 7. Кто отвечает на звонок, а кто думает

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: `error`, `handler-case`, вежливый `'bad-request`.
Java: `@Service`, конструктор, 400 не 500, `DELETE`.
Правило «название не пустое» живёт в сервисе. Тогда его

проверит и тест, и будущая консолька, и ты в три ночи.

Вахта седьмая. Контроллер разжирел: список, номера, создание, поиск. Вахтёр у шлюза ещё и варит сталь. Капитан кричит не потому что жестокий — потому что когда Земля пришлёт вторую дверь (консоль, бот, «ещё один контроллер»), правила разъедутся. Один раз вынеси мозг внутрь.

Контроллер — вахта у шлюза. Сервис — механик внутри.

Вчерашний толстый кусок, чтобы было что вычищать глазами:

```
@PostMapping("/tasks")
public Task create(@RequestBody
TaskRequest req) {
    if (req.title() == null ||
req.title().isBlank()) {
        // возврат что? null?
пустую Task? 500?
    }
    Task t = new Task(nextId++,
req.title());
    tasks.add(t);
    return t;
}
```

Давай медленно
Здесь склеены три работы: понять письмо, проверить название, положить в ящик. Вторая и третья не про HTTP. Они живут так же, если задачу создаёт консоль с занятия 4. Поэтому сервис. Контроллер после диеты: достал тело, позвал `service.create`, упаковал

201 или 400. Всё. Вахтёр не варит сталь.

Три проверки, что разъехалось не только в голове.

1. Поиск по проекту: `new ArrayList` в контроллере — ноль. Есть — список ещё там.
2. Юнит-тест `TaskService` с `new TaskService()` — зелёный без `spring-boot:run`. Если тест требует порт 8080, ты тестируешь HTTP, не правила.
3. Пустой title через curl — 400. Тот же пустой title через `service.create("")` в тесте — исключение. Одно правило, два входа.

```
@Service
public class TaskService {
    private final List<Task>
tasks = new ArrayList<>();
    private int nextId = 1;

    public List<Task> findAll() {
        return
List.copyOf(tasks);
    }

    public Task create(String
title) {
        if (title == null ||
title.isBlank()) {
            throw new
IllegalArgumentException("title
required");
        }
        Task t = new Task(nextId+
+, title.strip());
        tasks.add(t);
        return t;
    }
}
```

Давай медленно

`@Service` — бирка для Спринга: «это бин, положи на полку». Не священное слово. Могло бы быть `@Component`. Сервис — смысл: здесь правила и данные. `List.copyOf` даёт **снимок состава списка**: добавлять, удалять и заменять элементы через него нельзя; `clear()` бросит `UnsupportedOperationException`. Но ссылки ведут к тем же изменяемым объектам `Task`: глубокого копирования нет, изменение самой задачи видно и через снимок. Перенос в сервис не сделал `ArrayList` и `nextId++` потокобезопасными. Это всё ещё локальное упражнение с последовательными запросами; хранение в базе и конкурентные изменения

разберём дальше. `isBlank()` — пусто или одни пробелы. `" "` не название задачи. `strip()` — обрежь края у честного названия. Земля любит пробелы. Исключение — сигнал «так нельзя». Не возвращай `null` «наверно поймут». Контроллер поймёт конкретный тип и превратит в 400.

Спринг подсунет сервис в контроллер через конструктор. Не пиши `new TaskService()` руками — иначе зачем мы вообще завели всю эту религию с бинами.

```
@RestController
public class TaskController {
    private final TaskService
service;
```

```
    public
TaskController(TaskService
service) {
        this.service = service;
    }
}
```

Что только что случилось Один конструктор — Спринг сам найдёт `TaskService` на полке и вложит. Поле `final` — повесили навсегда, не подменят в полночь. Списка в контроллере больше нет. Если есть — ты схитрил, вахтёр снова варит сталь. Проверка: поиск по проекту `new ArrayList` в контроллере. Ноль вхождений. Есть — выноси.

Пустой title — 400, не 500. Пятьсот значит «мы сами дураки». Четыреста — «ты прислал ерунду».

```
@PostMapping("/tasks")
public ResponseEntity<?>
create(@RequestBody TaskRequest
req) {
    try {
        return
ResponseEntity.status(201).body(s
ervice.create(req.title()));
    } catch
(IllegalArgumentException e) {
        return
ResponseEntity.badRequest().body(
Map.of("error", e.getMessage()));
    }
}
```

Давай медленно
201 — создал, не просто «ок, 200». Мелочь, по которой на собесе отличают человека, который видел HTTP, от человека, который видел только зелёную стрелку. `?` у `ResponseEntity` — «тело разное: то Task, то карта с ошибкой». Не идеал архитектуры, зато честно

на одной вахте. Потом можно `@ControllerAdvice`. Сегодня — не надо. Работает — не плоди аннотации.

`TaskRequest` — то, что пришло с улицы. `Task` — то, что живёт дома. Даже если поля похожи, не суй домашнее на улицу. Потом у домашнего появятся секреты (кто автор, внутренний флаг), и ты случайно отдашь их в JSON.

Удаление:

```
public boolean delete(int id) {
    return tasks.removeIf(t ->
t.getId() == id);
}
```

`removeIf` — лямбда, ты их только что гладил в Lisp. Вернёт `true`, если кого-то выкинул.

Контроллер: `true` → 204 без тела, `false` → 404.

```
@DeleteMapping("/tasks/{id}")
public ResponseEntity<Void>
delete(@PathVariable int id) {
    if (service.delete(id)) {
        return
ResponseEntity.noContent().build(
);
    }
    return
ResponseEntity.notFound().build()
;
}
```

curl:

```
curl -i -X DELETE localhost:8080/
tasks/1
```

`-i` покажет статус. У 204 тела ответа нет — норма. Второй раз тот же id — 404.
Идемпотентность бедного

механика: повторно удалить нельзя, уже нет.

Тест сервиса **без HTTP**. Не поднимай весь Boot, чтобы узнать, что пустой title плохой.

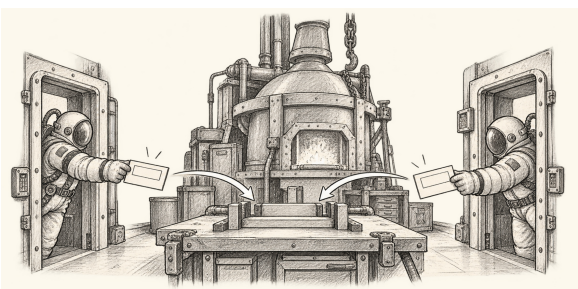
```
class TaskServiceTest {
    @Test
    void emptyTitleRejected() {
        TaskService s = new
TaskService();

assertThrows(IllegalArgumentExcep
tion.class, () -> s.create("
"));
    }

    @Test
    void deleteMissing() {
        TaskService s = new
TaskService();

assertFalse(s.delete(99));
    }
}
```

`new TaskService()` в тесте — ок. Это не контроллер, это чистый класс. Спринг для юнита правил не нужен. `@SpringBootTest` на каждый чих — медленно и хрупко. Прибережём для месяца, когда появится база.



Две двери. Правило одно.
HTTP и консоль могут вызывать один сервис внутри одного приложения.

6.3.1 Lisp: та же вежливость без Спринга

```
(defun create-task (title)
  (if (or (null title) (string=
title ""))
      (error "title required")
      (list :title title :done
nil)))

(defun try-create (title)
  (handler-case (create-task
title)
    (error () 'bad-request)))
```

Что только что случилось `error` бросает. Без ловца SBCL откроет отладчик — ты в нём уже бывал случайно. `handler-case` — «если рвануло, верни символ». `'bad-request` как 400. Успех — список с ключами. Ключи `:title` — ещё один вид бирок, удобно в plist. Не обязательно понимать все

виды. Понимать: правило в `create-task`, перевод в «ответ улице» снаружи. Тот же жир / худой, что Java.

Пробелы: `(string= title "")` не поймает `" "`. Можно `(string= (string-trim '(#\Space) title) "")`. Квест 7.L2, если возьмёшься.

6.3.2 Если сломалось: бины и 400

`required a bean of type TaskService`. Нет `@Service` или класс не сканируется (не тот пакет). Положи сервис рядом с `Application` по дереву.

`NullPointerException` в контроллере. Забыл конструктор, сделал `new` руками пустой сервис? Или `@Autowired`

на поле, а тест создаёт контроллер сам. Учебник просит конструктор. Следуй.

POST пустой title, а статус 500. Исключение не поймал, Boot обернул в 500. Это и есть «мы сами дураки». Поймай `IllegalArgumentException`. Не лови `Exception` — спрячешь настоящие баги.

400, но тело HTML. Вернул строку не через `ResponseEntity` / `@RestController`. Или ошибка до метода. Смотри `curl -i`, не только браузер.

`List.copyOf` и потом `UnsupportedOperationException`. Кто-то снаружи пытается `add` в то, что вернул `findAll`. Хорошо: датчик сработал. Менять список — только методы сервиса.

Закон станции
Правило «название не пустое» живёт в сервисе. Тогда его проверит и тест, и будущая консолька, и ты в три ночи.

Квест 7.L1 · Lisp
`create-task`: пустой title — `(error "title required")`. Поймай `handler-case`, верни `'bad-request`. Вежливость на орбите.

Квест 7.L2 · Lisp
Пустой после trim тоже `'bad-request`. `(try-create " ")` не должен вернуть задачу. Сравни с `(try-create "шлюз")`. Два вызова в REPL — два разных мира.

Квест 7.J1 · Java
Сервис вынесен. В контроллере нет своего `List`. Если есть — ты схитрил, вахтёр снова варит сталь.

Квест 7.J2 · Java
`DELETE /tasks/{id}` → 204 или 404. Тест сервиса без HTTP: удалил и нет.

Квест 7.J3 · Java
POST с `{"title":""}` и POST с пробелами — оба 400, тело с ключом `error`. `curl -i` в README, две реальные простыни ответа, не «ну 400». Если 500 — не засчитано, чини ловлю.

Капитан прошёл по коридору и не нашёл список задач в контроллере. Кивнул. Кивок на «МОДУЛЕ» — редкость, фиксируй в бортовом журнале. То есть в git.

6.4 Занятие 8. Шёпот в логах и сдача вахты

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Сегодня на станцию · 2 ч 40 мин
Lisp: свой `log-info`, зови из создания задачи.
Java: `application.yml`, `Logger`, фильтр `?done=`.
Проверка недели: свежая папка, README, curl. Не «у меня работало».

Вахта восьмая, ночная. `System.out.println` в сервисе — кричать в коридор. Соседний отсек спит. Лог — запись в журнал: кто создал, кто удалил, какой id. Земля потом спросит

«а что в 03:12». Ты не вспомнишь. Журнал вспомнит.

`application.yml` (или `.properties`, если start.spring.io так выдал — не плоди оба):

```
server:
  port: 8080
logging:
  level:
    com.example.taskmanager: INFO
```

Давай медленно
YAML любит отступы. Два пробела, не таб. Сломаешь отступ — Boot не встанет с жалобой на parse. Порт 8080 и так по умолчанию, но явно — честно для README.
`logging.level.пакет: INFO` — что писать из **твоих** классов. `DEBUG` — болтливо, на неделю ок, на сдачу вахты шумно. `ERROR` —

только пожары, пропустишь создание задачи. INFO — «создал id=3», нормальная вахта.

Вместо `System.out` — лог:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger log =
LoggerFactory.getLogger(TaskServi
ce.class);

log.info("created task id={}",
t.getId());
log.info("deleted task id={}",
id);
log.warn("task not found id={}",
id);
```

Что только что случилось `{}` — дырка, подставится аргумент. Не клеить `"id=" + id` как в коридоре: лог умеет лениво, и ты не разведёшь

склейку строк зря, пока лог всё равно выключен. Пока горячего участка нет, но привычка дешёвая.

`getLogger(TaskService.class)` — в журнале будет имя класса. Потом грепаешь `TaskService`, не всю простыню Tomcat. `warn` — не нашли удалить / не нашли отдать. Не `error`: клиент глупый — не реактор. `error` оставь для «не должно было случиться».

Запусти, сделай POST, смотри то же чёрное окно, где `mvnw`. Строка `created task id=1`. Нет строки — логгер не тот пакет, или уровень WARN глобально, или ты логируешь в контроллере, а вызываешь другой класс. Ищи.

Лестница громкости, без таблицы умножения:

- `ERROR` — пожар. Не смогли записать. Не то, что клиент прислал пустой title.
- `WARN` — странно, но живём: удалили id, которого нет.
- `INFO` — вахта: создал, удалил, старт сервера.
- `DEBUG` — для себя ночью. В yml включил, починил, выключил. На сдаче вахты не держи.

Сломай лог специально: поставь уровень `ERROR` на свой пакет, сделай POST, строки `created` не будет. Верни `INFO`. Это тот же ритуал, что красный тест: убедись, что датчик умеет

молчать, когда ты его приглушил.

Так себе идея
Не логируй пароли, токены, тела чужих писем. Названия задач в примерах вымышленные; настоящий `title` тоже может содержать личную или секретную информацию. Привычка: в журнал — id и факт, не вся жизнь запроса. На собесе это слышится как «не свети пароли», даже если пароля ещё нет.

Фильтр к концу недели — квест 8.J2, идея не спрятана:

```
@GetMapping("/tasks")
public List<Task>
all(@RequestParam(required =
```

```
false) Boolean done) {
    if (done == null) {
        return service.findAll();
    }
    return
service.findAll().stream()
        .filter(t -> t.isDone()
== done)
        .toList();
}
```

Давай медленно `@RequestParam(required = false)` — параметра может не быть. Тогда `done` это `null`, не `false`. Важная дырка: без параметра — **все**, и сделанные, и нет. `?done=true` — только сделанные. `?done=false` — только живые. `Boolean`, не `boolean`: примитив не умеет «не прислали». Объект умеет `null`. `stream().filter(...).toList()` — Java 16+, у нас 21. Лямбда снова. Lisp-мозг кивает.

curl:

```
curl -s "localhost:8080/tasks?
done=true"
curl -s "localhost:8080/tasks"
```

Кавычки вокруг URL с `?` — чтобы шелл не съел. На маке и в bash без кавычек `?` иногда глобет файлы. Если вдруг «нет файла done=true» — вот оно, кавычки.

Нужен `complete` по HTTP, иначе фильтр скучный. Минимум: `POST /tasks/{id}/complete` или `PATCH` с телом. Не обязательно REST-идеал. Нужно, чтобы `isDone` умел стать `true` без перезапуска и без рефлексии. Если не успеваешь REST — метод сервиса + временный эндпоинт. Главное — фильтр проверяемый curl.

6.4.1 Lisp: журнал вахты одной функцией

```
(defun log-info (fmt &rest args)
  (format t "INFO ")
  (apply #'format t fmt args)
  (terpri))
```

Что только что случилось `&rest args` — «остальные аргументы списком». `apply` разворачивает список в аргументы `format`. `(log-info "created id=~a" 3)` напечатает `INFO created id=3` и перевод строки. `terpri` — ter-mi-nate print, новая строка, старое короткое слово. Вызови из `create-task` на успешной ветке. На `'bad-request` лучше `WARN`, если заведёшь `log-warn`. Квест 8.L2.

6.4.2 Сдача вахты: чужая машина, твой README

Проверка недели: свежая папка. Не та, где «у меня всё открыто в IDEA три дня».

1. `git clone` себя самого в `/tmp` (мак/WSL) или в другую директорию Windows.
2. JDK 21 есть? `java -version`.
3. `./mvnw test` (или `mvnw.cmd test`) — зелёное.
4. `./mvnw spring-boot:run`.
5. curl из README, **копипаста**, не «я помню наизусть».

Не взлетело — чини README, не говори «ну у меня работало». У тебя работало на **твоей** куче случайностей: порт, JDK из IDEA, рабочая папка, зависимость в кэше Maven.

README — для человека со своей кучей.

Мак, Windows, WSL
На Windows лог в том же чёрном окне, где `mvnw.cmd`. На маке — то же окно Terminal. В IDEA — вкладка Run. Не ищи `log.txt` в корне, пока сам не настроил файл. Сегодня консоль. Когда будет Docker Compose, вывод контейнеров смотри через `docker compose logs`. Файл журнала самого приложения от этого автоматически не появляется. Профессия: знай, куда смотреть.

Чеклист сдачи, вслух:

- Health живой.
- POST создаёт, GET список видит.

- GET по id: есть / 404.
- POST пустого title: 400.
- DELETE: 204 затем 404.
- Лог: create/delete видно.
- `?done=` не падает.
- Тесты без ручного «я потыкал».

Если пункт врёт — это не «мелочь». Это дырка в шлюзе.

6.4.3 Если сломалось: логи и фильтр

Лога нет. Пакет в yml не совпал с реальным (`com.example.taskmanager` vs `com.example.demo`). Скопируй из класса `package ...`.

Слишком много логов Hibernate. Их ещё нет, у тебя

нет JPA. Если добавил JPA «на почитать» — удали. Месяц 3.

`?done=true` пусто всегда. Никто не умеет `complete` по сети, все `done=false`. Сделай эндпоинт или временный `store.complete` в `CommandLineRunner` — нет, не Runner. Эндпоинт. Честный.

`done=да`. 400 от Boot: не `Boolean`. Либо лови, либо в README напиши `true`/`false`, не «да».

Свежий clone, `mvnw` нет. Ты не закоммитил wrapper. Либо закоммить `mvnw`, `mvnw.cmd`, `.mvn/`, либо в README честно `mvn test` и требуй установленный Maven. Wrapper добрее к инспектору.

Квест 8.L1 · Lisp

`log-info`: печатает `INFO` и дальше как `format`. Вызови из `create-task`. Журнал вахты.

Квест 8.L2 · Lisp
`log-warn` тем же жестом, префикс `WARN `. В `try-create` на ветке `'bad-request` — warn, на успехе — info. Два вызова, две строки в REPL разного цвета… ну, разного текста. Цвет в терминале не обязателен, станция и так жёлтая от изоленты.

Квест 8.J1 · Java
Логи на create/delete. README: куда смотреть. На Windows лог в том же чёрном окне, где `mvnw.cmd`.

Квест 8.J2 · Java
`GET /tasks?done=true` — фильтр. Без параметра — все, даже те,

что мы давно обещали починить.

Квест 8.J3 · Java
`WARN` в логе, когда `GET /tasks/{id}` или `DELETE` наткнулись на пусто. Не `ERROR`. В README одна строка-пример из консоли. Если не умеешь вытащить — значит, лога нет, квест 8.J1 ещё живой.

Спрятать на GitHub
Коммит `week8: rest in memory`. Можно тег `v0.1`, как будто вышла игра.

SICP · открывать, только если зудит
Если вдруг понравилось, что HTTP — текст, а JSON — текст, а лог — текст: добро пожаловать. Большая часть бэкенда — договорённости про текст. SICP тут ни при чём, но зуд «а что

под аннотацией» полезный. Воскресный `ServerSocket` его чешет.

Смена сдана. На орбите по-прежнему пахнет изолентой, кофе по-прежнему кто-то выпил, но `curl localhost:8080/health` отвечает `UP`. Данные в памяти — сон. Следующий месяц — Postgres, и сон научится храниться в таблице. Не ставь Hibernate сегодня вечером «забегая». Забегание на «МОДУЛЕ» заканчивается вмятиной на шлюзе.

Чай. Коммит. Спать. Земля подождёт до утра, у неё и так полчаса пинга.

7 Месяц 3. Память, которая не сдувается

К двенадцатой неделе — PostgreSQL, нормальный набор «создать-прочитать-поменять-удалить», миграции Flyway, тесты. Микросервисов нет, и если кто-то шепчет «давай Кафку» — это сирена с соседней станции, не слушай. Застрял на Hibernate три дня? Добро пожаловать в профессию. Остаёшься здесь, не прыгаешь «вперёд».

До сих пор задачи жили в `ArrayList`. Перезапустил Spring — список пустой, как коридор после ночной смены. Файл из месяца 1 уже умел переживать кнопку «закрыть». База — тот

же жест, только с языком, который умеет искать, не читая всё с начала, и с железным правилом: два человека не должны одновременно сломать одну и ту же строку так, чтобы станция раздвоилась.

Сегодня на станцию · 2 ч 40 мин
Пн–Чт: 40 мин Lisp (файлы, вложенные списки, учебные «транзакции») + 120 мин Java (psql, JPA, тесты).
Пятница: добить красный Hibernate. Новых стартеров Spring не открывать.
Суббота: свой CRUD до конца: схема в README, curl на четыре жеста, десяток тестов.
Воскресенье: диверсия — крошечная «база» в файле.

Postgres после этого покажется вежливым.

Закон станции
Сначала руками в `psql`. Потом Java. Кто сразу вешает `@Entity` на пустую таблицу в голове — потом три дня читает `Caused by` и винит Вселенную. Вселенная тут ни при чём. Колонки нет.

7.1 Занятие 9. Сначала руками в базу, потом из Java

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

7.1.1 Lisp: сохранить в файл — уже бессмертие

Процесс умер — память умерла. Файл живёт. В Lisp это выглядит

почти неприлично просто: напечатать значение так, чтобы потом его же прочитать.

```
(defun save-tasks (path tasks)
  (with-open-file (out
path :direction :output :if-
exists :supersede)
    (print tasks out)))

(defun load-tasks (path)
  (with-open-file (in
path :direction :input :if-does-
not-exist nil)
    (if in (read in) nil)))
```

`print`/`read` — Lisp разговаривает сам с собой текстом. Это не Postgres. Это тот же жест: состояние переживает кнопку «закрыть».

Сохрани две формы `defun` выше в `task-files.lisp`, затем загрузи этот код через `(load "task-`

`files.lisp")`. Теперь набери данные в REPL:

```
(defparameter *board*
  '((1 . "починить шлюз")
    (2 . "проверить антенну")
    (3 . "не открывать шлюз 3")))

(save-tasks "module-tasks.lisp-
data" *board*)
```

Выключи SBCL. Открой заново в той же папке и выполни `(load "task-files.lisp")`, чтобы вернуть функции. Данные задач пока только в файле с данными; теперь прочитай их:

```
(load-tasks "module-tasks.lisp-
data")
; ((1 . "починить шлюз") (2 .
"проверить антенну") (3 . "не
открывать шлюз 3"))
```

Что только что случилось

`with-open-file` открывает поток и **закрывает** его, даже если посередине орнули ошибкой. Это тот же жест, что Java `try-with-resources`. Файл, который забыли закрыть, на станции потом не открывается «ну просто так».

Посмотри файл глазами — обычный текстовый редактор, не шаманство. Там скобки и кавычки. Если допишешь рукой ерунду, `read` обидится. Честная обида: «я ждал Lisp-значение, а получил кашу».

Сравни с Java-файлом из месяца 1: там ты сам решал, как класть строку. Здесь Lisp уже умеет сериализовать списки. Postgres умеет сериализовать **таблицы** и отвечать на вопросы вроде

«все несделанные задачи вот этого человека». Файл на это отвечает только если ты сам напишешь цикл. База отвечает одним предложением на своём странном языке.

7.1.2 SQL, этот странный язык про таблицы

SQL — не Java. Не Lisp. Это язык про «покажи мне строки, которые вот такие». Пишешь, **что** хочешь, не **как** бежать по массиву. Postgres сам решит, идти глазами по всей таблице или заглянуть в шпаргалку (индекс — в следующем месяце, не торопись).

Сначала поставь Postgres **локально**. Не «в облаке на бесплатном тарифе». Станция чинится руками.

Мак, Windows, WSL
Мак: `brew install postgresql@16`. Brew сам подскажет, как запустить сервис (`brew services start postgresql@16` или `pg_ctl`). Клиент: `psql postgres`.
WSL (Ubuntu): `sudo apt install -y postgresql postgresql-contrib`, потом `sudo service postgresql start`. Пользователя и базу заведи через `sudo -u postgres psql`.
Windows без WSL: установщик с https://www.postgresql.org/download/windows/ — галка pgAdmin по вкусу, пароль **запиши на бумажку**, правда запиши. Потом либо pgAdmin, либо `psql` из меню.
Пароль суперпользователя — не `1234` и не пустой, даже на локалке. Привычка. Через

месяц этот же палец потянется в `application.yml`.

База `taskdb`. В `psql`:

```
CREATE DATABASE taskdb;
```

Postgres ответит `CREATE DATABASE`. Это не ошибка и не вопрос. Это «сделал». Дальше переключись внутрь:

```
\c taskdb
```

Строка приглашения сменится на `taskdb=#`. Если осталась `postgres=#`, ты всё ещё в служебной базе и сейчас создашь таблицу не там. Потом Java будет искать `tasks` в `taskdb`, не найдёт, и ты полчаса будешь спорить с пустотой.

7.1.3 Сессия, которую надо набрать пальцами

Не читай. Набери. Если скопировал целиком — набери ещё раз, хотя бы `INSERT` и `SELECT`.

```
CREATE TABLE tasks (
    id         BIGSERIAL PRIMARY
KEY,
    title      TEXT NOT NULL,
    done       BOOLEAN NOT NULL
DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT
NULL DEFAULT now()
);
```

Ответ: `CREATE TABLE`. Посмотри, что получилось:

```
\dt
```

```
         List of relations
 Schema | Name  | Type  |  Owner
--------+-------+-------
```

```
+----------
 public | tasks | table | ты
```

`\d tasks` — чертёж таблицы: колонки, типы, default, первичный ключ. Это карта отсека. Без карты ты потом в Hibernate будешь гадать.

Теперь три задачи — священные четыре жеста начинаются с «положить»:

```
INSERT INTO tasks (title) VALUES
('починить шлюз');
INSERT INTO tasks (title) VALUES
('проверить антенну');
INSERT INTO tasks (title) VALUES
('не открывать шлюз 3');
```

Каждый раз: `INSERT 0 1`. Единица — сколько строк приехало. Ноль в середине — устаревшая вежливость про OID, забей.

```
SELECT id, title, done FROM
tasks;
```

```
 id |        title         | done
----+----------------------
+------
  1 | починить шлюз        | f
  2 | проверить антенну    | f
  3 | не открывать шлюз 3  | f
(3 rows)
```

`f` — false. Postgres не рисует галочки. `id` сам вырос: `BIGSERIAL` — счётчик. Ты его не передавал. Не передавай, пока сам не поймёшь, зачем.

Пометить первую сделанной:

```
UPDATE tasks SET done = TRUE
WHERE id = 1;
```

`UPDATE 1` — одна строка. Если написал `WHERE id = 99`, будет `UPDATE 0`. Тишина. Не ошибка.

Просто никого не нашли. В Java ты потом удивишься, что «обновил», а в базе тишина.

```
SELECT id, title, done FROM tasks
WHERE id = 1;
```

```
 id |     title      | done
----+----------------+------
  1 | починить шлюз  | t
```

Удалить:

```
DELETE FROM tasks WHERE id = 1;
SELECT id, title FROM tasks;
```

```
 id |        title
----+---------------------
  2 | проверить антенну
  3 | не открывать шлюз 3
(2 rows)
```

Строка 1 исчезла. Дырки в id — норма. Первичный ключ — не «номер по порядку в отчёте

капитану». Это бирка. Выкинул ящик с биркой 1 — следующие всё равно 4, 5, 6. Не пытайся «поджать». Станция не любит, когда переклеивают бирки задним числом.

Давай медленно
Четыре жеста, запомни телом. `INSERT` — положить. `SELECT` — посмотреть. `UPDATE` — поменять **уже лежащее**. `DELETE` — выкинуть. HTTP из прошлого месяца — те же четыре, только через сеть: POST, GET, PUT/PATCH, DELETE. CRUD — не священное слово из вакансии. Это эти четыре жеста с таблицей. Если можешь сделать их в `psql`, Java — переводчик, не маг.

Попробуй сломать. Это важнее красивого `SELECT`.

```
INSERT INTO tasks (title) VALUES
(NULL);
```

Postgres орнёт что-то вроде:

```
ERROR:  null value in column
"title" of relation "tasks"
violates not-null constraint
```

`NOT NULL` запрещает SQL `NULL`, даже если Java забыла проверить. Но `''` и пробелы он пропускает: непустое название проверяет сервис, а если это должна гарантировать и база — нужен отдельный `CHECK`. База — последний шлюз, но ей надо сказать, какие болты проверять.

```
INSERT INTO tasks (id, title)
VALUES (2, 'дубликат бирки');
```

Если id=2 ещё жив:

```
ERROR:  duplicate key value
violates unique constraint
"tasks_pkey"
```

Первичный ключ — бирка, которая не должна повторяться. Две задачи с одним id — как два отсека с одной табличкой на двери. Пожарная команда приедет не туда.

7.1.4 Карман выживания в psql

Это не «выучи все слэш-команды». Это пять штук, без которых ты слепой:

- `\c имя` — перейти в базу
- `\dt` — какие таблицы
- `\d tasks` — чертёж таблицы
- `\q` — уйти

- Стрелка вверх — предыдущая команда, как в терминале

Точка с запятой в конце SQL **обязательна**. Забудешь — psql будет ждать и смотреть на тебя приглашением `taskdb-#` (минус вместо равно). Допиши `;` и Enter. Все так сидели. Даже те, кто потом пишет про «нативный SQL».

Строки в SQL — одинарные кавычки `'починить шлюз'`. Двойные `"title"` — это имена, и в Postgres они ещё и регистр фиксируют. Пока живи в нижнем регистре без кавычек: `title`, `tasks`, `done`. Java потом будет спорить с `"Title"` vs `title`, и это отдельный цирк.

7.1.5 Из Java: пока без магии, простой JDBC

Когда запрос живой в `psql`, можно звать его из Java. Стартер `spring-boot-starter-jdbc` плюс драйвер Postgres. В `pom.xml` (Spring Boot 3.x, Java 21):

```
<dependency>

<groupId>org.springframework.boot
</groupId>
    <artifactId>spring-boot-
starter-jdbc</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</
groupId>
    <artifactId>postgresql</
artifactId>
    <scope>runtime</scope>
</dependency>
```

Перед Java проверь локальный вход в базу. Команды установки для Mac/WSL выше не задают

единый пароль `postgres`. Этот шаг требует уже настроенной отдельной роли без прав суперпользователя, с паролем и доступом к `taskdb`, её таблицам/последовательностям и изменениям схемы из следующих упражнений. Если роль ещё не готова, сначала закончи её локальную настройку: YAML её не создаёт. Проверь то же TCP-подключение, которое будет использовать Java: `psql -h localhost -p 5432 -U YOUR_APP_ROLE -d taskdb -W`, заменив `YOUR_APP_ROLE` своей настоящей ролью. Введи пароль по запросу, затем выполни `SELECT current_user, current_database();` и `SELECT id, title, done FROM tasks LIMIT 1;`. Успешный вход

через локальный сокет сам по себе не доказывает, что это подключение работает.

Задай `TASK_DB_USER` и `TASK_DB_PASSWORD` в окружении приложения, например в локальной конфигурации запуска IDE вне git. В них должны быть проверенные роль и пароль. `application.yml` читает их; суперпользователя и придуманного пароля по умолчанию здесь нет:

```
spring:
  datasource:
    url: jdbc:postgresql://
localhost:5432/taskdb
    username: ${TASK_DB_USER}
    password: ${TASK_DB_PASSWORD}
```

В README — явно помеченный одноразовый пример пароля,

никогда не настоящий пароль твоей локальной учётки Postgres. Настоящие данные — в локальной конфигурации вне git или в переменных окружения. Пароль, который используется где-то ещё, в git не кладут даже временно. Git помнит дольше, чем ты.

`JdbcTemplate` — тонкий переводчик: вот SQL, вот как строку превратить в объект.

```
record TaskRow(long id, String
title, boolean done) {}

@Repository
public class TaskJdbcRepository {
    private final JdbcTemplate
jdbc;

    public
TaskJdbcRepository(JdbcTemplate
jdbc) {
```

```
        this.jdbc = jdbc;
    }

    public List<TaskRow>
findAll() {
        return jdbc.query(
            "SELECT id, title,
done FROM tasks ORDER BY id",
            (rs, rowNum) -> new
TaskRow(
                rs.getLong("id"),

rs.getString("title"),

rs.getBoolean("done")
            )
        );
    }
}
```

`rs` — текущая строка результата. `rowNum` — номер по порядку, часто не нужен, но сигнатура такая. Один `findAll` с `SELECT`. Контроллер как в прошлом месяце: `GET /tasks` зовёт репозиторий, не держит список

в поле класса. `TaskRow` — record ответа JDBC для этого занятия; `GET /tasks` теперь возвращает `List<TaskRow>`. Старый конструктор консольной `Task` здесь не вызывается.

Когда этот запрос живой — на следующем занятии JPA, эта летающая аннотация. Сегодня важно увидеть: Java не «хранит в базе». Java **спрашивает** базу текстом. Тот же текст, что ты только что набрал в `psql`.

Мак, Windows, WSL
В `application.yml` url будет `localhost` и на маке, и в WSL, и на Windows — если Postgres слушает локально. Если Java в WSL, а Postgres установлен **родным** виндовым MSI, `localhost` иногда капризничает:

либо оба в WSL, либо оба на Windows. Не смешивай без нужды. Станция не любит два капитана.
Порт по умолчанию `5432`. Если установщик предложил другой — запиши. `Connection refused` почти всегда значит: сервис не запущен, или порт не тот, или ты стучишься не в ту машину.

7.1.6 Когда ругается, читай снизу не сразу — но и не сверху

Типичная каша при старте Spring:

```
org.postgresql.util.PSQLException
: FATAL: password authentication
failed
```

Пароль не тот. Не «пересоздать проект». Не «наверно Hibernate». Пароль.

```
org.postgresql.util.PSQLException
: FATAL: database "taskdb" does
not exist
```

Базу не создал, или создал в другом Postgres (виндовый MSI vs WSL — два сервера, две пустоты).

```
Connection to localhost:5432
refused
```

Сервис не запущен. Мак: brew services. WSL: `sudo service postgresql start`. Windows: службы или иконка слона.

```
ERROR: relation "tasks" does not
exist
```

Таблицы нет. Ты в той базе? `\c taskdb`, `\dt`. Часто люди создают таблицу в `postgres`, а Java смотрит в `taskdb`.

Так себе идея
Не ставь `ddl-auto: create` «чтобы само». Сегодня таблицы делает твоя рука в `psql`. Завтра — Flyway. Hibernate, который сам дорисовывает схему в полночь, — как механик, который подкручивает болты без журнала. Наутро никто не знает, какие болты.

Квест 9.L1 · Lisp
Сохрани alist задач в файл и открой **новую** сессию SBCL, загрузи. Бессмертие за пять минут. Если загрузилось то же, что сохранил — ты уже понял базы лучше, чем половина

гайдов с `@Entity` на первой странице.

Квест 9.J1 · Java
В `psql` (или pgAdmin) создай таблицу и три задачи. `SELECT` скопируй в README. Честный вывод, не «ну я сделал». Три строки, заголовок, `(3 rows)` — вот это.

Квест 9.J2 · Java
`GET /tasks` из базы через `JdbcTemplate`. README объясняет настройку с заглушками или явно одноразовыми учебными данными. Настоящие пароли базы остаются вне git, даже на локалке.

Квест 9.J3 · Java
Сломай специально: неверный пароль в yml, запусти, **одну** строку исключения — в

README. Потом верни. Потом несуществующая база. Потом несуществующая таблица. Три разных ора́, три разных смысла. Это не садизм. Это карта аварий.

Спрятать на GitHub
Коммит `week9: psql and jdbc`. В коммите — SQL создания таблицы, не только Java. Схема — тоже код.

7.2 Занятие 10. JPA: таблица притворяется объектом

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Вчера Java спрашивала базу текстом. Сегодня объект притворяется строкой. Это

удобно, пока не врёт. Врёт оно красиво: «у меня же поле есть», а колонки нет.

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;
    private String title;
    private boolean done;

    protected Task() {}

    public Task(String title) {
        this.title = title;
    }

    public Long getId() { return
id; }
    public String getTitle()
{ return title; }
    public boolean isDone()
{ return done; }
    public void setTitle(String
```

```
title) { this.title = title; }
    public void setDone(boolean
done) { this.done = done; }
}
```

Нужен конструктор без аргументов: JPA вызывает его из-под земли, потом расставляет поля. `protected` для JPA достаточно. Он ограничивает обычный прямой вызов `new Task()` из другого пакета, но код в том же пакете по-прежнему может его вызвать; подкласс может вызвать его через `super()`. Это не запрет на все безымянные задачи. Геттеры/сеттеры — да, многословно, мы в Яве. `record` как entity — пока не надо: JPA хочет менять поля у живого объекта, а record — заморозка.

`GenerationType.IDENTITY` — «id выдаст Postgres, у нас `BIGSERIAL`». Не `AUTO` «ну как-нибудь». Не `SEQUENCE`, пока сам не понял, зачем отдельная последовательность. Станция на IDENTITY держится, и нормально держится.

7.2.1 Пустой интерфейс, который врёт что он пустой

```
public interface TaskRepository
extends JpaRepository<Task, Long>
{}
```

Пустой интерфейс, а Спринг дописывает реализацию. Это выглядит как мошенничество. Внутри — не мошенничество, а кодогенерация, но ощущение то же.

`JpaRepository<Task, Long>` значит: сущность `Task`, первичный ключ `Long`. Методы уже есть: `save`, `findById`, `findAll`, `deleteById`. Имена `findByTitle`, `findByDone` Спринг тоже соберёт из названия метода. Пока не собирай десять `findBy`. Один CRUD.

Сервис:

```
@Service
public class TaskService {
    private final TaskRepository
tasks;

    public
TaskService(TaskRepository tasks)
{
        this.tasks = tasks;
    }

    public Task create(String
title) {
```

```
        if (title == null ||
title.isBlank()) {
            throw new
IllegalArgumentException("title
required");
        }
        return tasks.save(new
Task(title.strip()));
    }

    public List<Task> findAll() {
        return tasks.findAll();
    }
}
```

Контроллер как в месяце 2, только список живёт не в поле сервиса, а в Postgres. Перезапустил Spring — задачи на месте. Вот ради чего мы вообще завели слона.

7.2.2 Flyway: схема из файлов, не из полночных догадок

Файл `src/main/resources/db/migration/V1__tasks.sql`:

```
CREATE TABLE tasks (
    id         BIGSERIAL PRIMARY
KEY,
    title      TEXT NOT NULL,
    done       BOOLEAN NOT NULL
DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT
NULL DEFAULT now()
);
```

Два подчёркивания после версии. `V1__tasks.sql` — да. `V1_tasks.sql` — Flyway не съест и обидится загадочно. Имя файла — контракт.

В `application.yml`:

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: true
  flyway:
    enabled: true
```

`validate` — «сверь аннотации с живой схемой, ничего не дорисовывай». Не совпало — не стартуем. Это хорошо. Лучше красный старт, чем тихая колонка-призрак.

Закон станции
Схема живёт в SQL-миграциях. Аннотации описывают то, что **уже есть**. Если поменял поле — напиши `V2__...sql`, не надейся, что Hibernate догадается и все вокруг скажут спасибо.

Стартер: `spring-boot-starter-data-jpa` плюс Flyway (`flyway-core` и для Postgres — `flyway-database-postgresql` в свежих версиях). Версию смотри в документе **твоего** Spring Boot, не в статье 2019 года.

Осторожно, потеря данных: удаление базы уничтожает все её данные, отменить его нельзя. Репетируй только на одноразовой локальной учебной базе, содержимое которой можно потерять. Если в `taskdb` есть что-то ценное, не трогай её: создай новую пустую учебную базу и укажи её в URL datasource. Только для одноразовой базы: удали её, создай пустую, подними приложение. Flyway прогонит V1, в таблице `flyway_schema_history` появится строка. Это журнал шлюза: какие миграции уже были. Не правь руками V1, который уже улетел на другую машину. Только новая версия. Иначе у друга история разъедется, и вы

оба будете правы, и оба красные.

7.2.3 Три дня красный Hibernate: иди по цепочке `Caused by`

Стек Spring — как объявление по станции: сначала «внимание», потом десять уточнений, и только внизу «люк открыт». Иди по цепочке `Caused by`: первое сообщение может быть лишь обёрткой. Найди причину, которая объясняет сбой, и ближайший к ней подходящий кадр стека из твоего кода, если он есть. Не всегда самая верхняя или самая нижняя строка даёт весь ответ.

Вот гипотетический сеанс ужаса: ты добавил в entity поле `createdAt` с `@Column(name =`

`"created_at")`, но подключился к старой таблице без этой колонки. В приведённой выше entity такого поля пока нет, а V1 уже создаёт колонку; сама эта пара не должна давать показанную ошибку. Здесь же старт приложения — полэкрана красного.

```
org.springframework.beans.factory
.BeanCreationException:
Error creating bean with name
'entityManagerFactory' defined in
class path resource
[org/springframework/boot/
autoconfigure/orm/jpa/
HibernateJpaConfiguration.class]:
Failed to initialize dependency
'flywayInitializer' (or similar)
...
Caused by:
org.hibernate.tool.schema.spi.Sch
emaManagementException:
```

```
Schema-validation: missing column
[created_at] in table [tasks]
```

Давай медленно
Первая строка говорит: «не создался бин». Это погода. Конкретная новость в этой цепочке: entity ожидает `created_at`, а в таблице `tasks` колонки нет. Лечение: `\d tasks` в psql, сравни с классом, допиши `V2` или поправь V1 **если ещё никто этим V1 не пользовался**. Не «пересоздать проект». Проект ни при чём. Колонки нет.

Ещё хит:

```
Caused by:
org.hibernate.InstantiationExcept
ion:
No default constructor for
```

```
entity:  :
com.example.taskmanager.Task
```

Конструктора без аргументов нет. JPA не может создать entity из-под земли. Добавь `protected Task() {}`: этого доступа для JPA достаточно. `public` открыл бы его всем; `protected` ограничивает обычные прямые вызовы извне пакета, но оставляет доступ коду в том же пакете и вызову `super()` в подклассе. Безымянные задачи всё ещё возможны.

Ещё:

```
Caused by:
org.springframework.beans.factory
.BeanCreationException:
Error creating bean with name
'taskController'
...
```

```
Caused by: ... No default
constructor for ... TaskService
```

Это уже не Hibernate, а Спринг: сервис без конструктора, который принимает репозиторий, и без пустого. Один конструктор с зависимостями — нормально, Спринг его возьмёт. Два — запутается. Не плоди.

Ещё, любимый цирк с именами:

```
Caused by:
org.hibernate.tool.schema.spi.Sch
emaManagementException:
Schema-validation: missing column
[done] in table [tasks]
```

В SQL ты назвал `is_done`, в Java — `done`. Или наоборот. Или Postgres сложил в `"Done"` с кавычками, а Hibernate ищет `done`. `\d tasks` — правда.

Аннотация `@Column(name = "done")` — если хочешь явно.

Так себе идея
Три дня красный Hibernate — пройди цепочку `Caused by` до причины, которая объясняет сбой. В этих примерах это «нет колонки» или «нет конструктора»; в другом случае причина будет другой. Не «пересоздать проект». Пересоздать проект — как выйти в космос без скафандра, потому что в шлюзе пикнуло.

7.2.4 Проект содержит задачи, JSON не должен есть свой хвост

Связь «проект содержит задачи»:

```
@Entity
@Table(name = "projects")
public class Project {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy =
"project")
    private List<Task> tasks =
new ArrayList<>();
}

@Entity
@Table(name = "tasks")
public class Task {
    // ...
    @ManyToOne(fetch =
FetchType.LAZY)
    @JoinColumn(name =
"project_id")
    private Project project;
}
```

Миграция V2 (раз V1 уже про задачи без проекта — не переписывай V1, допиши):

```
CREATE TABLE projects (
    id   BIGSERIAL PRIMARY KEY,
    name TEXT NOT NULL
);

ALTER TABLE tasks
    ADD COLUMN project_id BIGINT
REFERENCES projects (id);
```

Не отдавай в JSON цикл «проект → задачи → проект → …». Jackson пойдёт по геттерам, как робот-пылесос в зеркальном отсеке. Браузер зависнет, ты будешь винить Вселенную.

Отдавай DTO без обратной ссылки:

```
public record TaskView(Long id,
String title, boolean done) {}
```

```
public record ProjectView(Long
id, String name, List<TaskView>
tasks) {}
```

Сервис собирает `ProjectView` руками. Контроллер не возвращает entity. Entity — домашний халат. JSON — то, что можно показать гостю.

Что только что случилось `mappedBy = "project"` значит: «чувак, колонка связи живёт на стороне Task, не рисуй вторую». Без этого Hibernate иногда заводит лишнюю таблицу-мост, и ты в `\dt` видишь `projects_tasks` как незваного родственника.

`FetchType.LAZY` у `Task.project` просит Hibernate не загружать проект, пока он не понадобится. Коллекция `Project.tasks` тоже по умолчанию ленивая. Обход

этой коллекции для каждого проекта может принести N+1; это следующее занятие. Сегодня не ставь `EAGER` на всякий случай. На всякий случай может притащить всё, когда нужно было только имя.

7.2.5 Починить станцию, когда «ну вчера работало»

- Приложение стартануло, таблиц нет: Flyway выключен или кладёт файлы не в `db/migration`.
- Flyway пишет `checksum mismatch`: ты поправил уже применённый V1. Верни файл как был, сделай V3. Снести базу и прогнать с нуля можно только для **одноразовой локальной учебной базы**, данные

которой не нужны. Сам по себе localhost не делает удаление безопасным: все данные исчезнут. Не чини так базу с ценными данными или прод.

- `failed to connect`: снова пароль, порт, WSL vs Windows.
- Сохранил entity, в psql пусто: смотришь не ту базу / не ту схему `public`. Или транзакция откатилась — об этом завтра.

Квест 10.L1 · Lisp
Проекты вложенными списками. `tasks-of` по id проекта. Карта станции, не таблица Excel. Например `'((1 . (шлюз антенна)) (2 . (кофе)))`.

Квест 10.J1 · Java

Entity, repository, CRUD по HTTP, Flyway V1. Чтобы после удаления базы мир поднимался из SQL, а не из памяти. Проверка только на одноразовой локальной учебной базе: при удалении исчезнут все данные. Если в `taskdb` есть что-то ценное, возьми новую пустую учебную базу и поменяй URL datasource. На этой пустой базе — `spring-boot:run`, таблицы на месте.

Квест 10.J2 · Java
`Project` и `GET /projects/{id}/tasks`. JSON без бесконечной вложенности. Станция не рекурсивна. Ну, почти.

Квест 10.J3 · Java
Намеренно разъедь схему: добавь поле в entity, миграцию

не пиши, `ddl-auto: validate`. Старт красный. Найди в цепочке `Caused by` сообщение о несовпадении схемы и приведи его в README одной цитатой. Потом напиши V2 и почини. Это занятие про чтение ошибок, не про «чтобы зелёное».

7.3 Занятие 11. Всё или ничего, и запах лишних запросов

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

7.3.1 Шлюз и деньги: зачем вообще «сделка»

На станции два люка шлюза. Если открыть оба сразу — воздух улетает, ты тоже.

Правило: либо оба закрыты, либо открыт один, второй держит. Промежуточного «ну я как раз посередине» для воздуха не существует.

Деньги — тот же шлюз. Списать 100 со счёта Алекса, начислить 100 Боре. Если после списания свет моргнул и начисление не случилось — 100 исчезли. Не «временно в пути». Исчезли. Капитан будет недоволен, и это ещё мягко сказано.

```
начало сделки
  Алекс: 500 → 400
  Боря:  100 → 200
конец сделки  → оба изменения
видны
```

Если между двумя UPDATE упало:

```
начало сделки
  Алекс: 500 → 400
  БАБАХ
откат → Алекс снова 500, Боря
100, как будто не было
```

Давай медленно
Транзакция — не «быстро».
Транзакция — **всё или ничего**.
Либо оба люка в правильном положении, либо станция делает вид, что ты к рычагу не прикасался. `@Transactional` на методе сервиса говорит Спрингу: все SQL внутри — одна сделка. Исключение (unchecked, обычный RuntimeException) — откат. Дошёл до конца метода — фиксация, `COMMIT`. Пока нет фиксации, другой сеанс в psql может этих изменений не видеть. Это не баг. Это шлюз ещё не закрылся.

Сначала в учебной базе один раз создай два счёта и дай этим подготовительным командам зафиксироваться:

```
CREATE TABLE accounts (name TEXT
PRIMARY KEY, balance INTEGER NOT
NULL);
INSERT INTO accounts (name,
balance) VALUES ('alex', 500),
('borya', 100);
```

Теперь открой два сеанса psql в той же базе. Сеанс 1:

```
BEGIN;
UPDATE accounts SET balance =
balance - 100 WHERE name =
'alex';
-- ещё не COMMIT
```

Сеанс 2, другое окно psql:

```
SELECT name, balance FROM
accounts WHERE name = 'alex';
```

До фиксации сеанса 1 сеанс 2 не видит его незавершённое изменение. При стандартном для PostgreSQL уровне READ COMMITTED после `COMMIT;` новый `SELECT` в сеансе 2 увидит 400. Если в сеансе 1 вместо этого сказать `ROLLBACK;`, у Алекса останется 500. Более строгая изоляция может дольше держать старый снимок, но не показывает незавершённые записи.

Для задач то же самое: создать проект и первую задачу. Если title задачи пустой — проекта в базе тоже не должно остаться. Иначе в `\dt` живёт проект-сирота, а ты клянёшься, что «метод же упал». Упал. Но без сделки успел закоммитить первую половину.

7.3.2 `@Transactional`: на публичном, не на секретном

```
@Service
public class ProjectService {
    private final
ProjectRepository projects;
    private final TaskRepository
tasks;

    public
ProjectService(ProjectRepository
projects, TaskRepository tasks) {
        this.projects = projects;
        this.tasks = tasks;
    }

    @Transactional
    public Project
createWithFirstTask(String
projectName, String taskTitle) {
        Project p =
projects.save(new
Project(projectName));
        if (taskTitle == null ||
taskTitle.isBlank()) {
            throw new
IllegalArgumentException("title
```

```
required");
        }
        tasks.save(new
Task(taskTitle.strip(), p));
        return p;
    }

    @Transactional
    public void completeAll(Long
projectId) {
        List<Task> found =
tasks.findByProjectId(projectId);
        found.forEach(t ->
t.setDone(true));
    }
}
```

В `completeAll` нет явного `save`. Грязный трюк JPA: объект, которого достали в транзакции, при фиксации сам уедет в UPDATE. Это называется dirty checking, «подглядеть, что испачкали». Удобно. И страшно, когда ты не ждал UPDATE, а он уехал.

На `private` не вешай: Спринг смотрит через прокси и частное не видит. Магия с дыркой.

Давай медленно
Спринг подсовывает вместо твоего сервиса **обёртку**. Вызов снаружи: обёртка открывает транзакцию, зовёт твой метод, закрывает. Вызов `this.createWithFirstTask(...)` из соседнего метода того же класса — обёртку обходит, идёшь напрямую. Транзакции нет. Как постучать в свой же шлюз изнутри: дверь не та. Поэтому `@Transactional` живёт на **публичном** методе, который зовут снаружи — из контроллера, из теста. Не на `private void helper()`.

Проверь глазами. Тест или curl: создай проект с пустым title задачи. В psql:

```
SELECT * FROM projects;
SELECT * FROM tasks;
```

Сравни с тем, что было до неудачного вызова: нового проекта и новой задачи остаться не должно. Обе таблицы пустые только если были пустыми заранее. Новый проект остался без первой задачи — ты изобрёл сироту. Ищи checked-исключение без отката, private-метод, вызов мимо прокси или сохранения вне общей транзакции.

Так себе идея
Не лови `Exception` внутри транзакционного метода и не

проглатывай. Проглотил — Спринг думает, что всё хорошо, делает COMMIT. Пожар записан в журнал как «успешная вахта».

7.3.3 N+1: пятьдесят проектов, пятьдесят один запрос, горелая изолента

Включил `spring.jpa.show-sql=true` (а лучше ещё `logging.level.org.hibernate.SQL: DEBUG`). Сделал `GET /projects`. В логе:

```
Hibernate:
    select p1_0.id, p1_0.name
from projects p1_0
Hibernate:
    select t1_0.id, t1_0.done,
t1_0.project_id, t1_0.title
    from tasks t1_0 where
t1_0.project_id=?
Hibernate:
    select t1_0.id, t1_0.done,
```

```
t1_0.project_id, t1_0.title
    from tasks t1_0 where
t1_0.project_id=?
Hibernate:
    select t1_0.id, t1_0.done,
t1_0.project_id, t1_0.title
    from tasks t1_0 where
t1_0.project_id=?
```

И так дальше. Один `select` проектов. Потом **на каждый** проект — свой `select` задач, потому что список задач ленивый: Jackson (или твой цикл) ткнул в `getTasks()`, Hibernate сходил в базу. 50 проектов — 51 запрос. Это N+1. Пахнет как горелая изолента.

Давай медленно
Почему «плюс один»: N родительских проектов, по запросу на коллекцию задач каждого, плюс один запрос

списка проектов. Не «база медленная». Ты спросил её пятьдесят один раз вместо одного JOIN. На трёх строках не заметишь. На трёх тысячах заметит капитан, пользователь и твой ноутбук, который начнёт выть как насос.

Лечение потом, но сегодня хотя бы запах узнать и **один** раз увидеть лекарство:

```
@Query("select distinct p from
Project p left join fetch
p.tasks")
List<Project> findAllWithTasks();
```

`JOIN FETCH` — «притащи задачи тем же запросом». В логе после этого часто **один** `select ... from projects ... left join tasks`. Не всегда красивый SQL. Зато один. Для того же измеренного `GET /`

projects вызови findAllWithTasks(), затем собери DTO.

Не лечи всё подряд EAGER. EAGER — «всегда тащи». Потом findAll проектов для выпадающего списка будет возить тонны задач, которые никто не просил. Лечи конкретный запрос, который пахнет.

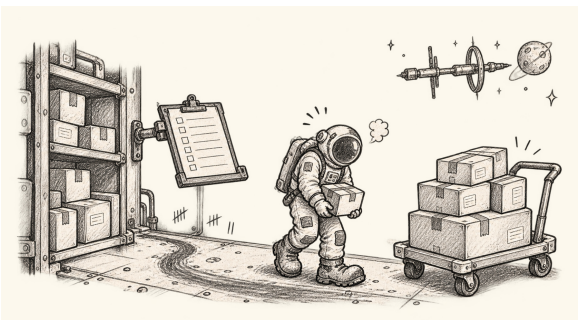
JOIN в psql, чтобы увидеть, чего Hibernate хочет:

```
SELECT p.id, p.name, t.id,
t.title
FROM projects p
LEFT JOIN tasks t ON t.project_id
= p.id
ORDER BY p.id, t.id;
```

Одна таблица слева, задачи справа, пустые задачи — NULL в правых колонках. Это тот же

JOIN FETCH, только руками. Если этот SELECT понятен — аннотация уже не молитва.

Посчитать запросы: один `GET`, глаза в лог, палочками на полях README. Если запросов как отсеков — вот он, N+1, здоровайся.



Запрос один. А рейсов сколько?
Для пятидесяти родителей: один запрос плюс пятьдесят дополнительных — 51. План загрузки выбирают под нужные данные.

7.3.4 Изоляция — слово на собесе, не кнопка на этой неделе

Тебя могут спросить «что такое isolation». Короткий честный ответ: насколько сделка видит чужие незакрытые шлюзы. По умолчанию Postgres `READ COMMITTED` — видишь только зафиксированное. `SERIALIZABLE` — строже и дороже. Не ставь сериализуемость «для надёжности» на учебный CRUD. Надёжность сегодня — `@Transactional` на том методе, где два изменения должны жить или умереть вместе.

Квест 11.L1 · Lisp
Список операций. Если среди них `'fail` — не применять ни одну. Учебная «транзакция»: либо шлюз закрыли, либо не

трогали. Без полуоткрытого люка.

Квест 11.J1 · Java
Один метод: проект + первая задача. Пустой title задачи — проекта в базе тоже не должно остаться. Проверь глазами в `psql`. Не верь только тесту, который смотрит в тот же EntityManager: глянь другим глазом, из другого окна.

Квест 11.J2 · Java
SQL-лог, один `GET` списка, число запросов в README. Если запросов как отсеков — вот он, N+1, здоровайся.

Квест 11.J3 · Java
Тот же `GET`, но через метод с `JOIN FETCH` (или DTO-запрос без ленивой коллекции). Снова посчитай `select` в логе. Два

числа рядом в README: до и после. Если «после» не меньше — fetch не туда прикрутил, или Jackson всё равно тыкает в другую связь.

7.4 Занятие 12. Тесты, которые ходят в настоящую базу

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю. Юнит-тест сервиса с моком репозитория проверяет, что ты вызвал `save`. Он не проверяет, что Postgres проглотил строку, что Flyway прогнался, что `NOT NULL` жив. К концу месяца нужны тесты, которые **стучатся**. Иначе у тебя зелёная полоса и мёртвая база — как

зелёная лампочка у открытого люка.

7.4.1 Идеал — Postgres в коробке на время теста

Testcontainers: Docker поднимает настоящий Postgres, тест гоняет миграции, потом коробка умирает. Для этого примера на Boot 3 / Testcontainers 1.x оставь `spring-boot-starter-test` и добавь `org.testcontainers:postgresql` плюс `org.testcontainers:junit-jupiter`, всё в scope test. Последний модуль даёт `@Testcontainers` и `@Container`; это не одноимённая зависимость самого JUnit. `spring-boot-testcontainers` полезен для дополнительных возможностей Boot, но этому примеру с

`@DynamicPropertySource` не обязателен. Совместимые версии пусть задаёт BOM твоего Boot.

```
@SpringBootTest
@Testcontainers
class TaskServiceIntegrationTest
{

    @Container
    static PostgreSQLContainer<?>
postgres =
        new
PostgreSQLContainer<>("postgres:1
6-alpine");

    @DynamicPropertySource
    static void
props(DynamicPropertyRegistry r)
{

r.add("spring.datasource.url",
postgres::getJdbcUrl);

r.add("spring.datasource.username
", postgres::getUsername);
```

```
r.add("spring.datasource.password
", postgres::getPassword);
    }

    @Autowired TaskService
service;

    @Test
    void createThenFind() {
        Task t =
service.create("шлюз");

assertTrue(service.findById(t.get
Id()).isPresent());
        assertEquals("шлюз",
service.findById(t.getId()).get()
.getTitle());
    }
}
```

Это не «магия Docker». Это честный слон, только временный. Если тест зелёный здесь, велик шанс, что и на твоём `localhost:5432` не соврёт.

Мак, Windows, WSL

Docker Desktop на маке: обычно просто работает. На Windows — ставь с WSL2 backend, не с Hyper-V «потому что галочка». В WSL `docker ps` должен видеть те же контейнеры. Если IDEA на Windows, а Docker только в Ubuntu — снова два капитана. Нет Docker и пока страшно: профиль `test` и H2. В README **честно**: «прод — Postgres, тесты — упрощение». Ложь в README потом всплывёт на собесе быстрее, чем ты скажешь «на самом деле». H2 не Postgres. `TIMESTAMPTZ`, массивы, куча функций — другой диалект. Для джуна на месяц 3 — допустимый костыль, если подписан.

7.4.2 Четыре жеста — четыре теста, не «ну глазами»

```
@Test
void unknownIdIsEmpty() {

assertTrue(service.findById(9_999
_999L).isEmpty());
}

@Test
void updateChangesTitle() {
    Task t =
service.create("старое");
    service.rename(t.getId(),
"новое");
    assertEquals("новое",
service.findById(t.getId()).get()
.getTitle());
}

@Test
void deleteThenGone() {
    Task t =
service.create("временное");
    service.delete(t.getId());

assertTrue(service.findById(t.get
```

```
Id()).isEmpty());
}
```

Красное/зелёное, не «ну глазами потыкал». Глаза врут, когда хочется спать. `assertTrue(true)` — преступление против станции: тест, который не умеет упасть.

Для HTTP через настоящий сервер — `@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)` и `TestRestTemplate`. Для учебной servlet-среды без настоящего сервера — `@SpringBootTest` плюс `@AutoConfigureMockMvc` и `MockMvc`. Хотя бы create и get неизвестного. Четыреста на пустой title, не пятьсот.

Транзакционный тест из занятия 11 тоже сюда. Запомни `projectRepository.count()` до неудачного вызова; после ожидаемого исключения проверь, что число не изменилось. `assertEquals(0, projectRepository.count())` верно только при заведомо пустых исходных данных. Потом глянь в psql один раз в жизни, чтобы поверить, что откат случился.

7.4.3 README этой недели — бортовой журнал, не поэзия

Как поднять Postgres на маке **и** в WSL/Windows. Какой url в yml. Как гонять миграции (они сами на старте — напиши это). Как тесты: `./mvnw test` / `mvnw.cmd test`. Пять curl: health если есть, POST, GET списка, GET одного, DELETE.

Пример JSON. Если Docker для Testcontainers обязателен — одной строкой «без Docker интеграционные тесты не встанут».

Чужая машина (или твоя вторая ОС) — критерий. «У меня в IDEA» не считается сдачей вахты.

7.4.4 Что должно стоять на палубе к концу недели 12

Живой CRUD. Схема в SQL, не в голове. Entity не торчат циклом в JSON. Хотя бы один `@Transactional` сценарий, который откатывается. SQL-лог, который ты видел своими глазами. Десяток тестов, среди них не все моки. Коммит. Это и есть «монолит». Одна программа. Не зоопарк.

Кафку в этот коммит не класть, даже «на почитать». Почитать можно в браузере, не в `pom.xml`.

Квест 12.L1 · Lisp
Учебный «интеграционный тест» без JUnit: сохрани задачи, запусти **новую** SBCL, загрузи, проверь `equal` с тем, что ждал. Если не equal — `(error "станция потеряла журнал")`. То же чувство, что зелёный IT: второй процесс, не та же память.

Квест 12.J1 · Java
create; get неизвестного; update title; delete. Красное/зелёное, не «ну глазами потыкал».

Квест 12.J2 · Java
README: как поднять Postgres на маке **и** в WSL/Windows, миграции, тесты, curl.

Спрятать на GitHub
Коммит `week12: crud postgres`. Кафку в этот коммит не класть, даже «на почитать».

Воскресная диверсия
Своя крошечная «база»: дописываешь файл, в памяти карта id → место в файле. `insert` пишет в конец. `select` по id прыгает. Потом сравни с Postgres и немного его уважай: ты только что набросал кусок того, за что людям платят зарплату и ещё дают отпуска.

SICP · открывать, только если зудит
Состояние, которое переживает процесс — старый вопрос: что такое данные, если программа выключилась. Файл, таблица, лог. Если зудит «а можно ли

базу как список пар» — можно, ты уже делал alist. Postgres — список пар, который научили отвечать на вопросы и не терять хвост при сквозняке.

8 Месяц 4. То, что спрашивают в комнате с вайтбордом

Монолит доводим до состояния «можно показать человеку». Пользователи, пароли (не в открытую, мы не варвары), индексы, страницы по двадцать штук. Параллельно — коллекции и простые задачки, без которых собес превращается в лотерею. Lisp: можно начать крошечный свой язык. Это законно и даже желательно.

К концу шестнадцатой недели ты не «знаешь Spring Security». Ты можешь **вслух** объяснить,

почему пароль не лежит текстом, почему `?userId=` в URL — дырка размером с шлюз, почему `HashMap` потерял ключ, и почему счётчик из двух потоков глючит. Вайтборд это любит. Кафка на вайтборде без этого — свисток без воздуха.

Сегодня на станцию · 2 ч 40 мин
Пн–Чт: 40 мин Lisp (реестр, hash-table, стек, чистота) + 120 мин Java (auth, индекс, коллекции, гонки).
Пятница: добить Security до «чужой id не читается».
Суббота: пагинация, EXPLAIN, три задачки с собеса.
Воскресенье: диктофон, восемь вопросов. Стыдно слушать — хороший знак: уши ещё живые.

Закон станции
Сюда вход с живым CRUD и Postgres. Нет базы — вернись в месяц 3. Пароли на пустой памяти — театр: перезапустил, и все пользователи умерли вместе с декорациями.

8.1 Занятие 13. Кто ты такой и почему пароль не «1234»

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

8.1.1 Сказка про дамп, которую рассказывают слишком поздно

Представь: кто-то слил таблицу `users`. Не «взломал прод в кино». Слил. Бэкап на флешке, логи,

стажёр с `SELECT *`, копия «на посмотреть». Что в файле?

Если так:

```
 id | email              |
password
----+--------------------
+----------
  1 | alex@module.space  | 1234
  2 | borya@module.space | qwerty
```

Поздравляю. У злодея не «хеши». У него **пароли**. Алекс ставит `1234` ещё на почте и на том странном форуме про антенны. Боря — везде `qwerty`. Один дамп — ключи от половины жизни людей, которые тебе доверились. Ключ от шлюза, приклеенный снаружи скотчем, хотя бы видно. Пароль текстом в базе —

скотч, который ты сам наклеил и забыл.

Хеш — односторонняя мясорубка. Исходный кусок мяса из фарша не восстановишь. База хранит фарш. При входе снова прогоняешь пароль через мясорубку и сравниваешь фарш с фаршем. Даже ты, капитан станции, не должен уметь «напомнить пароль». Только сбросить.

```
пароль «комета» → BCrypt →
$2a$10$...длиннаякаша...
```

В дампе каша. Радуга заранее посчитанных «1234» → md5 не помогает, если есть **соль**: кусок случайности, разный у каждого пользователя, замешанный в

хеш. BCrypt соль кладёт внутрь самой строки `$2a$10$...`. Поэтому два человека с паролем `1234` получат **разный** хеш. Красиво и зло.

Давай медленно «Почему нельзя зашифровать пароль, а не хешировать?» Шифр — двусторонний: есть ключ — можно вернуть текст. Значит, ключ где-то лежит, и тот, кто взял базу **и** ключ, снова читает пароли. Хеш не открывается ключом. Открывается только новым «попробуй этот пароль — совпало?». Вот зачем `matches`, а не `decrypt`.

Не пиши свой хеш через `String.hashCode()` или MD5 «для учебника в проде». Учебник —

Lisp `sxhash` ниже, и мы сразу скажем, что это муляж. В Java — BCrypt.

```
@Bean
PasswordEncoder passwordEncoder()
{
    return new
BCryptPasswordEncoder();
}
```

Регистрация — `encode`. Вход — `matches`. Никогда не логируй пароль, даже «на минутку», даже в DEBUG, даже учебный. Привычка заразная. Через полгода DEBUG включишь на проде и отправишь пароли в Grafana. Станция такого не прощает, отдел безопасности тоже.

8.1.2 Таблица людей, не «ну поле в задачах»

Flyway `V3__users.sql` (номер подставь свой, если V2 уже индекс или проекты):

```
CREATE TABLE users (
    id            BIGSERIAL
PRIMARY KEY,
    email         TEXT NOT NULL
UNIQUE,
    password_hash TEXT NOT NULL
);

ALTER TABLE tasks
    ADD COLUMN user_id BIGINT
REFERENCES users (id);
```

Почта уникальная: два Алекса с одним адресом — путаница, кто чей шлюз. `UNIQUE` поймает даже если сервис забыл проверить.

```
@Entity
@Table(name = "users")
```

```
public class UserAccount {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;
    private String email;
    private String passwordHash;
    // пустой конструктор,
геттеры...
}
```

Не называй класс `User` сразу: в Spring Security уже есть `User`. Столкновение имён — отдельный цирк в три часа ночи.

Регистрация:

```
public void register(String
email, String rawPassword) {
    if
(users.existsByEmail(email)) {
        throw new
IllegalArgumentException("email
taken");
```

```
    }
    String hash =
encoder.encode(rawPassword);
    users.save(new
UserAccount(email, hash));
}
```

Вход: нашёл по почте, `encoder.matches(raw, account.getPasswordHash())`. Не совпало — 401, одна и та же формулировка, что «нет такого», что «пароль не тот». Иначе по ответу подбирают, какие почты живые.

Дальше либо JWT, либо сессионная печенька. **Одно**. Не оба, ты не буфет.

- Сессия: сервер помнит «этот браузер — Алекс», в cookie случайный номер. Просто. Масштабировать потом

больнее, сегодня у тебя один сервер.

- JWT: сервер отдаёт подписанный талон. Клиент таскает талон в заголовке `Authorization: Bearer ...`. Сервер не хранит сессию, проверяет подпись. Талон украли — ходят как Алекс, пока не протух.

Так себе идея
Первый попавшийся гайд «JWT за 15 минут» 2018 года может быть уже мёртв. Сверь с документом **твоей** версии Spring Boot. Старые заклинания иногда открывают не ту дверь. `WebSecurityConfigurerAdapter` — музей. Ищи `SecurityFilterChain` и `http.authorizeHttpRequests`.

Минимальный каркас на Boot 3 / Java 21: публичные `POST /auth/register` и `POST /auth/login`, плюс безобидная диагностика `GET /health`. Для маршрутов задач нужна аутентификация. Entry point ниже вернёт гостю 401; проверку JWT ещё надо подключить.

Каркас цепи выглядит примерно так. Это не священный текст: сверь с документом **твоего** Boot.

```
@Bean
SecurityFilterChain
api(HttpSecurity http) throws
Exception {
    http
        .csrf(csrf ->
csrf.disable())
        .sessionManagement(s ->
```

```
s.sessionCreationPolicy(SessionCr
eationPolicy.STATELESS))
        .exceptionHandling(ex ->
ex.authenticationEntryPoint(
            (request, response,
error) ->
response.sendError(401)))
        .authorizeHttpRequests(au
th -> auth
            .requestMatchers(org.
springframework.http.HttpMethod.G
ET, "/health").permitAll()
            .requestMatchers("/
auth/**").permitAll()
            .anyRequest().authent
icated());
    // потом: фильтр JWT перед
UsernamePasswordAuthenticationFil
ter
    return http.build();
}
```

`csrf.disable()` на учебном API с JWT — ок, потому что CSRF про печеньку браузера, а у нас талон в заголовке. На сессиях с cookie так не вырубай. `STATELESS`

— сервер не кладёт HTTP-сессию, каждый запрос несёт талон снова. Entry point выше задаёт 401 для запросов без аутентификации. Это всё ещё каркас: проверку JWT и установку аутентификации надо реализовать, прежде чем правильный талон откроет дверь.

8.1.3 Сессия входа, которую надо увидеть глазами

```
curl -s -X POST localhost:8080/
auth/register \
  -H "Content-Type: application/
json" \
  -d
'{"email":"alex@module.space","pa
ssword":"комета"}'
```

Ответ вроде `201` или тела `{"id":1}`. В psql:

```
SELECT id, email, password_hash
FROM users;
```

```
 id | email              |
password_hash
----+--------------------
+----------------------
  1 | alex@module.space  |
$2a$10$N9qo8uLOickgx2ZMRZoMye...
```

Не комета. Если видишь открытый пароль — ты забыл encode. Стоп. Не коммить.

```
curl -s -X POST localhost:8080/
auth/login \
  -H "Content-Type: application/
json" \
  -d
'{"email":"alex@module.space","pa
ssword":"комета"}'
```

Тело имеет вид `{"token":"header.payload.signature"}`; эти слова — заглушки, не рабочий токен. У подписанного

JWT три части в Base64url через точки. Средняя — payload: декодируешь и видишь `sub` / email. Кодирование не шифрование. Третья часть — подпись. Проверь её, срок действия и ожидаемые claims, прежде чем верить талону. Ключ подписи — в локальной конфигурации или окружении, не в git.

```
curl -s localhost:8080/tasks \
  -H "Authorization: Bearer
вставь-токен"
```

Без заголовка — 401. С талоном — список Алекса, пустой или нет. Чужих задач нет, даже если в базе они есть. Проверь вторым пользователем.

8.1.4 Почему не `?userId=1`

Вот дырка, которую джун проносит в прод чаще, чем забытый `console.log`:

```
GET /tasks?userId=7
```

Сервис верит параметру. Сосед подставляет `?userId=1` и читает список «никому не говорить». На станции это как написать на люке код от шлюза и удивиться, что зашли не те.

Правильно: у `Task` поле пользователя. Сервис берёт текущего из контекста, **не из URL**.

```
public List<Task>
myTasks(Authentication auth) {
    Long me =
currentUserId(auth);
```

```
    return
tasks.findByUserId(me);
}
```

Id в пути `GET /tasks/42` — это id **задачи**, не хозяина. Хозяина достаёшь из талона. Нашёл задачу 42, хозяин не ты — 403 или 404. Выбери одно и будь верен.

Давай медленно
404 «нет такой» чужому — вежливая ложь: не подтверждаешь, что задача существует. 403 «знаю, что есть, не дам» — честнее для своих админок, болтливее для чужих. Учебный монолит: выбери 404 для чужих id и напиши это в README, чтобы через месяц сам не забыл, какое у станции лицо.

Тест с двумя пользователями. Два скафандра, один чужой иллюминатор. Без этого теста Security — декорации: зелёный health и дырявый люк.

```
@Test
void cannotReadForeignTask() {
    String alex =
token("alex@module.space",
"secret1");
    String borya =
token("borya@module.space",
"secret2");
    long id = createTask(alex,
"секрет шлюза");
    getTask(borya,
id).expectStatus().isNotFound();
// или 403
}
```

Не клади userId в JSON тела `POST /tasks` «чтобы было проще». Клиент врёт. Всегда. Даже если клиент — ты.

Тот же грех в красивом виде: `GET /users/7/tasks`. Выглядит как будто настоящий REST. Если 7 берётся из URL, а не сверяется с талоном — дырка та же, только в форме. Станция не обязана иметь «вложенные пользователи» в пути. `GET /tasks` и «кто ты» из SecurityContext — скучно и герметично.

8.1.5 Когда Security ругается, а ты ругаешься в ответ

- `403` на всё, включая login: `requestMatchers("/auth/**")` не тот префикс, или фильтр JWT глотает login раньше времени.
- `401` с правильным талоном: заголовок не `Authorization: Bearer ...`, или лишний перевод строки, или ты

вставил кавычки внутрь. PowerShell любит портить кавычки — зайди в WSL.

- После рестарта все разлогинены: JWT секрет сменился (случайный при каждом старте). Положи секрет в yml на локалке, в git — нет, в `.gitignore` рядом с паролем базы.
- В базе хеш, login всегда false: сравнил строки хеша вручную вместо `matches`. BCrypt при каждом `encode` новый фарш. Сравнивать `equals` двух хешей — путь в стену.
- Два пользователя, оба видят все задачи: `findAll()` вместо `findByUserId`. Security впустила

в дверь, сервис открыл все шкафы.

8.1.6 Lisp: учебный «хеш», который не надо нести в прод

```
(defun hash-dummy (password)
  (sxhash password))
```

Это не криптография. `sxhash` — быстрый номер для таблиц в памяти. Два разных пароля могут столкнуться. Соли нет. Это жест: **секрет ≠ пароль**. Настоящий bcrypt в Lisp на этом курсе не сдаём. Станция и так держится на изоленте.

```
(defparameter *users* (make-hash-
table :test 'equal))

(defun register (email password)
  (when (gethash email *users*)
    (error "exists"))
```

```
  (setf (gethash email *users*)
(hash-dummy password))
  t)

(defun login (email password)
  (eql (gethash email *users*)
(hash-dummy password)))
```

```
(register "alex@module.space"
"комета")
(login "alex@module.space"
"комета") → T
(login "alex@module.space" "нет")
→ NIL
```

В лог пароль не печатай. Даже `(format t "~a~%" password)` «чтобы проверить». Проверил — удали.

Квест 13.L1 · Lisp
Реестр: почта → «хеш». Повтор почты — ошибка. `login` сравнивает хеш. Никаких

паролей в логах, даже учебных, привычка заразная.

Квест 13.J1 · Java
Регистрация и вход. `GET /tasks` — только свои. Чужие не светятся даже «пустым списком с намёком».

Квест 13.J2 · Java
Чужой id — 403 или 404. Выбери одно и будь верен. Тест с двумя пользователями. Два скафандра, один чужой иллюминатор.

Квест 13.J3 · Java
Три curl в README: без талона; с чужим талоном на чужой id; со своим. Три статуса. Если все 200 — люк открыт, не празднуй.

8.2 Занятие 14. Индекс — не ускорить всё, а ускорить вот это

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Книга без оглавления: чтобы найти главу про шлюз, листаешь с первой страницы. Оглавление — шпаргалка «шлюз → стр. 214». Писать книгу чуть дольше (надо обновлять оглавление). Искать — быстрее.

```
CREATE INDEX idx_tasks_user ON
tasks (user_id);
```

Flyway `V4__idx_tasks_user.sql` — одна строка, большой характер. Без подходящего индекса `WHERE user_id = 1` обычно требует полного прохода. С индексом у

PostgreSQL появляется другой путь: Index Scan или Bitmap Index Scan. Но он всё равно выбирает по оценке стоимости; индекс — возможность, не приказ.

Это не ускоряет `SELECT * FROM tasks`. Это ускоряет **вот это** условие. Индекс на все колонки «на всякий случай» — как оглавление, в котором каждое слово: книга толще, писать больнее, пользы кот наплакал.

8.2.1 Сессия EXPLAIN, с тысячами строк, иначе глючит

На трёх строках Postgres может индекс не взять: «да я и глазами вижу». Налей тысячи.

```
INSERT INTO tasks (title,
user_id)
SELECT 'задача ' || g, 1
FROM generate_series(1, 5000) AS
g;

INSERT INTO tasks (title,
user_id)
SELECT 'чужое ' || g, 2
FROM generate_series(1, 5000) AS
g;
```

Десять тысяч строк. Не страшно. Учебный мусор. Пользователи с id 1 и 2 уже должны существовать: проверь `SELECT id, email FROM users` и при необходимости подставь их настоящие id.

До индекса (если ещё не создал — или сними `DROP INDEX idx_tasks_user` на локалке):

```
EXPLAIN ANALYZE SELECT * FROM
tasks WHERE user_id = 1;
```

Кусок правды, похожий на:

```
Seq Scan on tasks
(cost=0.00..188.00 rows=5000
width=...)
  Filter: (user_id = 1)
  Planning Time: 0.1 ms
  Execution Time: 2.3 ms
```

`Seq Scan` — прошли всю таблицу. На десяти тысячах 2 мс. На десяти миллионах уже не шутка, и собес именно про это спрашивает.

Создай индекс, снова:

```
CREATE INDEX idx_tasks_user ON
tasks (user_id);
EXPLAIN ANALYZE SELECT * FROM
tasks WHERE user_id = 1;
```

```
Index Scan using idx_tasks_user
on tasks  (cost=0.29..140.00
rows=5000 width=...)
  Index Cond: (user_id = 1)
  Execution Time: 1.1 ms
```

Цифры и план у тебя могут быть другими. Выполни `ANALYZE tasks;` и сравни настоящие планы. Здесь у пользователя 1 половина строк: даже на десяти тысячах Seq Scan может быть дешевле. Bitmap Scan тоже нормален. Чтобы увидеть влияние избирательности, повтори на данных, где нужному пользователю принадлежит малая доля строк. Записывай выбор PostgreSQL, не слово, которое хотелось увидеть.

Давай медленно

`EXPLAIN` без `ANALYZE` — план, как Postgres **собирается**. С `ANALYZE` — ещё и сколько заняло на самом деле, с живыми цифрами. Для README нужны оба прогона: до и после, на одном и том же объёме. Три строки в таблице — сравнение глючит как вежливый датчик: «всё в норме», а кислород уже свистит.

Не индексируй `title` «потому что вдруг заищем». Вдруг — когда появится `WHERE title = ...` или `LIKE 'foo%'` (префикс). `LIKE '%foo%'` индекс обычный часто не спасёт. Честно скажи на собесе «зависит от запроса», это взрослый ответ, не «индекс всегда быстрее».

8.2.2 Страницы по двадцать, не сто тысяч одним JSON

```
GET /tasks?page=0&size=20
```

Spring Data умеет `Pageable`. Пусть умеет.

```
@GetMapping("/tasks")
public Page<TaskView>
mine(Authentication auth,
@PageableDefault(size = 20, sort
= "id") Pageable pageable) {
    Long me =
currentUserId(auth);
    return tasks.findByUserId(me,
pageable).map(this::toView);
}
```

`Page` в JSON принесёт `content`, `totalElements`, `totalPages`. Браузер и ты заслужили лучшей смерти, чем сто тысяч задач одним ответом. Память сервера тоже. Импортируй `org.springframework.data.web.Pagea`

`bleDefault`. Явный `sort = "id"` задаёт этим примерам устойчивый порядок по умолчанию; пагинация без заданного порядка не обещает, какие строки попадут на каждую страницу.

Мак, Windows, WSL
curl с пагинацией одинаковый на маке и в WSL:

```
curl -s -H "Authorization:
Bearer ..." "localhost:8080/tasks?
page=0&size=20"
```

Кавычки вокруг URL в zsh/bash полезны из-за `?`. В cmd.exe — другие кавычки, проще WSL. PowerShell иногда глотает `?` — тогда тоже кавычки или `curl.exe`.

Нулевой page — первая страница, как у Spring. Если

хочешь «страница 1 для людей» — переводи сам, не злись на фреймворк.

Индекс и пагинация дружат: `WHERE user_id = ? ORDER BY id LIMIT 20 OFFSET 0`. Без индекса — снова Seq, потом сортировка, потом отрезать 20. С индексом по `user_id` уже теплее. Составной `(user_id, id)` — ещё теплее, когда дорастёшь. Сегодня простой `user_id` хватит, если README честный.

Включи `show-sql` и дерни `page=2&size=20`. В логе что-то вроде:

```
select t.id, t.title, t.done,
t.user_id
from tasks t
where t.user_id=?
```

```
order by t.id
limit 20 offset 40
```

`offset 40` — «пропусти две страницы». На огромных таблицах OFFSET дорожает: база всё равно пробегает пропущенные. Для джуна и десяти тысяч строк — нормально. На собесе можно сказать «cursor/keyset потом, сейчас page». Честно.

Так себе идея `@PageableDefault(size = 20)` — размер по умолчанию, не потолок. Задай свойство Boot `spring.data.web.pageable.max-page-size`, например `100`; в `application.yml` внутри `spring.data.web.pageable` это `max-page-size: 100`. Обычный потолок Boot 3 — 2000, поэтому

`size=100000` уже ограничен, но свой предел надо выбрать и проверить. Станция не обязана возить весь трюм.

Квест 14.L1 · Lisp
10000 пар (человек . задача). Поиск проходом. Второй вариант: заранее hash-table. Почувствуй разницу телом, не статьёй. `(get-internal-real-time)` до и после.

Квест 14.J1 · Java
Пагинация + индекс в Flyway `V2`. Если V2 уже занят проектами — следующий свободный номер. Главное файл, не магическая двойка.

Квест 14.J2 · Java
README: `EXPLAIN` до и после на одних данных. Возьми достаточно строк и отметь,

какая доля подходит условию. Seq Scan после добавления индекса — тоже результат для объяснения, не автоматически сломанный опыт.

Квест 14.J3 · Java
`GET /tasks?page=0&size=5` дважды: вторая страница `page=1`. В README два JSON — хотя бы id. Если страницы одинаковые, ты забыл Pageable или всегда page=0.

8.3 Занятие 15. Карманы, равенство и задачки на пальцах

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
На собесе любят одни и те же грабли. Не потому что злодеи.

Потому что на этих граблях ты в проде уже лежал, просто ещё не знал названия.

8.3.1 Два ящика с одним номером, и карта, которая тебя предала

```
class UserId {
    private final long value;
    UserId(long value)
{ this.value = value; }
}
```

Положил в карту:

```
Map<UserId, String> cabin = new
HashMap<>();
cabin.put(new UserId(1),
"Алекс");
System.out.println(cabin.get(new
UserId(1)));
```

Напечатает `null`. Алекс в каюте есть. Ключ «тот же номер» — для карты **другой ящик**. По

умолчанию `equals` — это `==`: тот же объект в памяти, не «похожее внутри». `Object.hashCode()` не обещает адрес памяти или уникальное значение: у разных объектов хеш может совпасть. Два `new UserId(1)` всё равно разные объекты.

Давай медленно
HashMap сначала считает `hashCode`, бежит в корзину, потом в корзине сравнивает `equals`. Если `equals` говорит «мы одинаковые», а `hashCode` разный, поиск может не найти ключ: равные ключи больше не соблюдают контракт хеширования. Контракт: равные объекты обязаны иметь один hashCode. Обратное неверно: один hashCode ещё не

равенство (столкновение). Сломаешь одно, не починив другое — карта глючит тише Hibernate.

Починка руками:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof UserId
other)) return false;
    return value == other.value;
}

@Override
public int hashCode() {
    return Long.hashCode(value);
}
```

Или, Java 21, без стыда:

```
record UserId(long value) {}
```

`record` уже даёт `equals`/`hashCode`. Это не читерство, это 21-й век. На собесе скажи «record», если

спросят «напиши equals» — напиши, чтобы видели, что понимаешь, не только синтаксис.

Так себе идея
Пока ключ лежит в карте, не меняй его поля, участвующие в `equals` или `hashCode`. Сохранённая запись автоматически не переиндексируется при изменении поля; следующий поиск может использовать другой хеш и не найти ключ. `final` поля. Изменяемый ключ — боль. Как бирка на ящике, которую переклеивают, пока ящик в пути.

== для объектов — «тот же ящик?». `equals` — «похожее внутри?». Для строк почти всегда `equals`. == у строк иногда

случайно сработает из пула литералов, и ты поверишь в успех. Не верь. Для `int` == нормальный. Для `Integer` с большим числом == снова ловушка: упаковка показанных констант от −128 до 127 гарантирует общие объекты; вне этого диапазона совместное использование зависит от реализации. На собесе это любят до дрожи.

Набери, не верь на слово:

```
Integer a = 127;
Integer b = 127;
System.out.println(a ==
b);        // true, гарантия для
этой упаковки
Integer c = 128;
Integer d = 128;
System.out.println(c ==
d);        // зависит от
```

```
реализации; часто false
System.out.println(c.equals(d));
 // true, внутри одно число
```

`int` — просто число в регистре, `null` не бывает. `Integer` — ящик, может быть пустым, едет в коллекциях. Одним предложением для кота: «int — значение, Integer — объект-обёртка».

8.3.2 Корзины: как карта теряет ключ по шагам

Представь шкаф на 8 полок. В этой учебной модели `Math.floorMod(hashCode(), 8)` даёт полку от 0 до 7 даже для отрицательного хеша. Настоящий `HashMap` по-своему смешивает хеш и вычисляет

индекс; это модель, не его реализация.

1. `put(new UserId(1), "Алекс")`. Пусть хеш в учебной модели отправил этот ключ на полку 3. Алекс лежит на 3.
2. `get(new UserId(1))`. Новый объект. Пусть его хеш отправил на полку 6: `null`. Даже если хеши совпадут, стандартный `equals` считает объекты разными.
3. Починил только `equals`, оставив унаследованный `hashCode`. Равные ключи всё ещё могут иметь разные хеши, и поиск может не сработать.
4. Починил оба: оба `UserId(1)` дают хеш, скажем, 1, полка

одна, equals говорит «да». Алекс дома.

Что только что случилось Сломай наоборот: `hashCode` всегда `42`, `equals` нормальный. Карта работает, но все ключи на одной полке. Много столкновений может замедлить поиск. В Java 21 заполненная корзина может стать деревом, поэтому «любой плохой хеш превращает HashMap в связный список» — слишком сильное утверждение. Хорошее распределение хеша всё равно важно.



Бирку поменяли. Ящик не переехал. Изменение значимых для хеша полей ключа после вставки может направить поиск в другую корзину.

8.3.3 Список, который почти всегда

`ArrayList` — массив под капотом, растёт. Достать по индексу быстро. Вставить в середину — сдвинуть хвост. `LinkedList` — вагончики. В середину дешево **если ты уже стоишь на вагончике**. С индекса —

сначала иди. На практике джун таскает `LinkedList` «для красоты» и получает медленный `get(i)` в цикле. Редкий гость. Не таскай для красоты.

`HashMap`: ключ, корзина, среднее доставание быстрое. Хуже, когда хеш у всех одинаковый — одна корзина, как очередь в единственный шлюз.

Сложность вслух, грубо, без экзаменационного театра:

- список, найти элемент: идти глазами, чем длиннее тем дольше;
- карта по ключу: в среднем быстро;
- положить в конец ArrayList: обычно дёшево.

Стримы `.filter().map()` — в меру. Стрим ради стрима читается как стихи, которые стыдно. Цикл `for` не стыдно. Стыдно не понимать, что стрим делает.

```
List<String> titles =
tasks.stream()
    .filter(t -> !t.isDone())
    .map(Task::getTitle)
    .toList();
```

Три строки — ок. Двенадцать операций с `collectingAndThen` — нет, не на этой неделе и не на собесе «напишите на бумажке».

8.3.4 Lisp: та же частота слов, тот же карман

```
(defun freq (lst)
  (let ((h (make-hash-table :test
'equal)))
```

```
    (dolist (x lst)
      (incf (gethash x h 0)))
    h))
```

`:test 'equal` сравнивает содержимое строк. По умолчанию проверка — `eql`; отдельно созданные строки с одинаковыми буквами тогда могут оказаться разными ключами. Java `HashMap` для строк уже вызывает `equals`. Lisp заставляет выбрать. Это честно.

```
(freq '("шлюз" "антенна" "шлюз"))
; таблица: "шлюз" → 2, "антенна"
→ 1
```

Стек для скобок — обычный список: `push` в голову, `pop` с головы. Открыли `(` — положили ожидание `)`. Пришла `)` — сняли, совпало? Нет — станцию перекосило. В конце стек

должен быть пуст, иначе люк так и не закрыли.

8.3.5 Алгоритмы: картинки, не марафон медиумов

Grokking Algorithms (там картинки) и 3–4 лёгкие задачи в неделю. Не медиум пачками. Медиум пачками — путь выгореть и возненавидеть скобки уже на Java.

Три штуки, которые реально спрашивают пальцами:

1. Разворот массива на месте — два индекса с концов, менять, пока не встретились.
2. Частота символов — `HashMap<Character, Integer>` или массив на 26, если вход ограничен строчными ASCII-буквами `a–z`.

3. Скобки — стек, см. выше.

Напиши на бумажке. Потом в IDE. Потом сломай вход `"]"` и `""`. Пустой — скобки сбалансированы, кстати. Станция без люков тоже герметична. Спорно философски, удобно в тесте.

Квест 15.L1 · Lisp
Баланс скобок `() []`. Стек — обычный список, `push`/`pop`. Если осталось лишнее — станцию перекосило.

Квест 15.J1 · Java
Ключ карты `UserId`. Два объекта с одним числом внутри — один ключ. `record` уже даёт `equals`/`hashCode`, это не читерство, это 21-й век.

Квест 15.J2 · Java

Три штуки: разворот массива на месте; частота символов; скобки. Без «ещё десять с LeetCode, пока горячо».

Квест 15.J3 · Java
Сломай специально: класс-ключ **без** equals/hashCode, `put` и `get` разными `new`. README: какой `null` увидел. Потом record. Тот же `get` — не null. Две строки вывода. Это история, которую потом расскажешь у доски.

8.4 Занятие 16. Два запроса сразу, и почему счётчик глючит

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Томкат и так многопоточен: два HTTP-запроса могут

одновременно тыкать сервис. Ты этого не заказывал. Оно уже так. В нашей обычной синхронной обработке поток занят на время запроса, а не закреплён навсегда за пользователем. Даже один пользователь может отправить два запроса одновременно. Если оба обращаются к одному `Counter`, его делят потоки обработки этих запросов.

8.4.1 Гонка с цифрами, не с лозунгами

```
class Counter {
    int n;
    void inc() { n++; }
}
```

`n++` выглядит как один шаг. Для железа это примерно:

1. прочитать `n` в регистр;
2. прибавить 1;
3. записать обратно.

Два потока, `n` сейчас 42:

```
поток A прочитал 42
поток B прочитал 42
поток A записал 43
поток B записал 43
```

Два прибавления. Счётчик вырос на **один**. Второе прибавление съели. Это гонка. Не «иногда глючит Java». Это ты разрешил двум механикам крутить один вентиль, не глядя друг на друга.

Ещё раз, медленнее, с номерами шагов. Цель — 2, два потока по одному `inc`, старт `n = 0`:

```
время  | поток A              |
поток B              | n в памяти
-------+----------------------
+----------------------
+-----------
 t1    | прочитал 0
|                      | 0
 t2    |                      |
прочитал 0           | 0
 t3    | 0+1, пишет 1
|                      | 1
 t4    |                      |
0+1, пишет 1         | 1
```

Ожидали 2. Получили 1. Масштаб на 100000 таких пересечений — дыра в десятки тысяч. Не обязательно каждый `inc` пересекается. Достаточно части.

Запусти:

```
Counter c = new Counter();
Thread a = new Thread(() -> { for
(int i = 0; i < 100_000; i++)
c.inc(); });
Thread b = new Thread(() -> { for
(int i = 0; i < 100_000; i++)
c.inc(); });
a.start();
b.start();
a.join();
b.join();
System.out.println(c.n);
```

Ждал 200000. Увидел, например, 143882. Или 178001. Или редкий джекпот 200000 — и решил, что гонки нет. Нет: тебе повезло с таймингом. На Windows потоки те же, сюрприз. На маке тоже. Число будет разным от запуска к запуску. Запиши три запуска в README. Разброс — доказательство, не стыд.

Давай медленно

Почему «иногда 200000»: куски циклов не всегда пересекаются на одном `n`. Планировщик то даст А сто шагов подряд, то перемешает. Гонка — это не «всегда сломано». Это «не обещано, что цело». Станция не может жить на «ну вчера сошлось». Лифт, который иногда падает, — сломанный лифт.

Починка учебная:

```
synchronized void inc() { n++; }
```

В один момент вентиль крутит один поток. Другой ждёт. Снова 200000. Скучно. Правильно.

В настоящей жизни для счётчика — `AtomicInteger` (`incrementAndGet`), для денег и задач — база и транзакция.

Ручной `synchronized` на всём сервисе — очередь в один шлюз: безопасно и медленно. Ручные `new Thread` в проде — как сварка в скафандре: можно, но зачем.

Пул потоков: `ExecutorService`. Не плодить бесконечную армию.

```
ExecutorService pool =
Executors.newFixedThreadPool(4);
pool.submit(() ->
System.out.println("вахта"));
pool.shutdown();
```

Четыре рабочих. Задач сколько угодно в очереди. Тысяча `new Thread` на тысячу задач — тысяча скафандров, кислород кончится.



Квитанций две. Прибавление одно. Один возможный порядок: оба прочитали 42, оба записали 43. Нужна общая синхронизация всей операции.

8.4.2 Откуда гонка в вебе, если я «просто Spring»

Два `POST /tasks` одновременно — обычно ок: две разные строки, id раздаст `BIGSERIAL`. Гонка начинается, когда двое меняют **одно и то же**:

- счётчик «сколько задач у проекта» в поле Java, не в `COUNT`;

- «перевести деньги» двумя UPDATE без транзакции;
- `if (tasks.size() < 10) tasks.add(...)` — оба видят 9, оба добавляют, стало 11.

База с `@Transactional` и правильным `UPDATE accounts SET balance = balance - 100 WHERE id = ? AND balance >= 100` — взрослый вентиль. Проверка «хватает ли денег» в Java, потом UPDATE — детский: между проверкой и записью сосед уже списал.

Цифрами. Алекс 100. Два запроса «списать 100» одновременно:

```
A прочитал balance=100, 100 >=
100? да
B прочитал balance=100, 100 >=
```

```
100? да
A пишет 0
B пишет 0   // оба «успешно»,
денег как не бывало дважды,
повезло что ноль
```

Или хуже: оба пишут `100-100`, но если логика «прибавить на другой счёт», Боря получит 200 из ниоткуда, Алекс ноль. Транзакция + условие в SQL: второй UPDATE заденет 0 строк, сервис скажет «не хватает». Не `Thread.sleep(50)`.

Не чини гонку «ну подождём Thread.sleep». Sleep на собесе — красная лампочка. Sleep в проде — изолента на датчике.

8.4.3 Восемь вопросов коту (да, снова кот)

К концу недели ответь **вслух**:

- чем `int` отличается от `Integer` одним предложением;
- почему строку нельзя поменять «внутри»;
- коллекции и «это быстро / это нет»;
- checked против unchecked;
- `try-with-resources`;
- наследование vs «вложить объект» на своём `Task`;
- что откатывает `@Transactional`;
- зачем `equals` вместе с `hashCode`.

Не можешь без подглядывания — не «ещё главу». Перескажи свой код. Имена полей — шпаргалка. Если стыдно

сказать `mgr` вслух, переименуй в `manager`, рассказ пойдёт.

Lisp на этой неделе — чистота: без глобальных `defparameter`, только аргументы. Гонки в однопоточном SBCL нет, зато привычка таскать состояние аргументами потом помогает не плодить общее поле `int n` на весь сервер.

Можно начать крошечный свой язык: числа и `+`. Если число — верни. Если список с `+` — сложи `ev` хвоста. Это законно. Это даже желательно. Java от этого не отменяется.

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev
```

```
(rest form))))
        (cond
          ((eq op '+) (apply #'+
args))
          ((eq op '-) (apply #'-
args))
          (t (error "unknown ~a"
op))))))

(ev 3)            ; 3
(ev '(+ 1 2 3))   ; 6
(ev '(+ 1 (- 5 2))) ; 4
```

Что только что случилось `mapcar #'ev` — сначала посчитай детей, потом сложи. Как в Lisp вообще: внутренние скобки раньше. Ты только что написал крошечный Lisp внутри Lisp. Станция любит такую рекурсию. Не тащи это в Spring. Тащи в `lisp-experiments`.

Квест 16.L1 · Lisp

Перепиши один этюд станции без глобальных `defparameter`, только аргументы. Чистота. Потом про гонки думать легче.

Квест 16.J1 · Java
Гонка и `synchronized` — отдельный класс в `java-basics`, не в боевом сервере. README: какие цифры увидел. На Windows потоки те же, сюрприз.

Квест 16.J2 · Java
Сорок пять минут, диктофон, восемь вопросов из списка. Слушать себя стыдно. Зато на собесе меньше сюрпризов.

Квест 16.J3 · Java
Тот же счётчик через `AtomicInteger`. Три числа в README: гонка / synchronized / atomic. Два последних должны

совпасть с 200000. Если atomic соврал — ты звал `get+set`, не `incrementAndGet`.

SICP · открывать, только если зудит
Объект как набор функций с секретом внутри. Если вдруг стало интересно, зачем `private`.

Воскресная диверсия
Нарисуй на бумаге два потока и `n++` клеточками: читает / плюс / пишет. Подпиши 42 и 43. Это лучшая шпаргалка к занятию 16, и её не отберёт вайтборд — вайтборд её как раз и ждёт.

Спрятать на GitHub
Коммит `week16: auth indexes races`. В README — EXPLAIN, три цифры гонки, два пользователя. Это портфолио, не «ещё чуть-чуть Кафку».

9 Месяц 5. Слова из вакансий, но на своём железе

Вакансии этого месяца звучат как список запчастей с чужой станции: Redis, очередь, Kafka, Docker, CI. Слова красивые. На собесе их произносят с умным лицом. Твоя задача — не выучить слова, а потрогать каждую штуку **на своём** монолите, пока она глючит, ругается и отказывается стартовать.

Если кэш глючит — ты это увидишь curl’ом. Если очередь не дренируется — увидишь в логе. Если в Docker база «не находится» — почти наверняка ты написал `localhost` внутри

контейнера. Это не теория. Это вечер с чайником.

Закон станции
Сюда вход с живым монолитом: база, вход по паролю, тесты, README. Иначе доделывай месяц 4. Лучше сдвиг, чем Кафка на пустом CRUD. Пустой CRUD с Кафкой — это станция без стен, но с громкоговорителем.

Мак, Windows, WSL
Redis, Postgres и приложение в этом месяце живут в Docker. На маке — Docker Desktop или Colima, как уже стоит. На Windows — Docker Desktop + WSL2. YAML один. Команды `docker compose up` одни. Не ставь Redis «ещё и brew, и MSI, и в WSL apt» сразу: три Redis на

одной машине — это квест, который никто не заказывал.

9.1 Занятие 17. Кэш: быстрый, наглый, иногда глючит

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Кэш — копия поближе. Быстрее базы. Зато может рассказать сказку про задачу, которую ты уже удалил. На станции «МОДУЛЬ» это табличка на стене шлюза: «кислород в норме». Табличку вешают, чтобы не бегать в трюм каждый раз. Если баллон уже пустой, а табличка старая — ты всё равно откроешь люк. Это и есть кэш.

База — трюм. Кэш — табличка. HTTP без кэша каждый раз спускается в трюм. С кэшем — смотрит на стену. Пока табличка свежая, жизнь прекрасна. Потом кто-то поменял баллон, табличку забыл, и собес спрашивает: «обновили, а кэш старый — что будет?»

Если ответишь «ну он сам» — не сам. Сам он только протухнет по TTL, если ты TTL задал. Или будет врать вечно, если задал «навсегда» и забыл сброс.

9.1.1 Зачем он вообще, если база «и так быстрая»

На трёх задачах Postgres отвечает мгновенно, и кэш кажется театром. На трёх

тысячах задач, которые читают сто раз в секунду, трюм начинает пыхтеть. Кэш имеет смысл, когда:

- одни и те же данные читают часто;
- писать можно реже, чем читать;
- ты **согласен** иногда показать чуть устаревшее, либо умеешь сбрасывать ключ на запись.

Список задач пользователя на 30 секунд — учебный потолок. Не кэшируй «все задачи всех людей навечно». Это не ускорение, это музей вранья.

9.1.2 Redis в коробке, не «установлю в систему»

Образ `redis:7`. Не latest. Latest — это «какая попало версия», а ты уже достаточно взрослый, чтобы злиться на внезапные сюрпризы.

В `docker-compose.yml` пока можно **только** Redis, если Postgres у тебя ещё локальный. Потом, на занятии 20, сложишь всех в одну пьесу. Сегодня достаточно:

```
services:
  redis:
    image: redis:7
    ports:
      - "127.0.0.1:6379:6379"
```

`docker compose up -d redis`. Потом:

```
docker compose exec redis redis-
cli ping
```

Должно ответить `PONG`. Это не шутка и не пароль. Это Redis говорит «жив».

Мак, Windows, WSL
Порт 6379 на маке и в Docker Desktop на Windows один и тот же: с хоста `localhost:6379`. Java на хосте (IDEA) ходит в `localhost`. Java **внутри** контейнера приложения на занятии 20 пойдёт на хост `redis`, не на `localhost`. Запиши это карандашом на лбу. Через две недели лоб пригодится.

В Spring Boot 3 / Java 21 обычно стартер кэша плюс Redis. Имена свойств сверяй с **твоей** версией, не с гайдом 2019 года. Грубо:

```
spring:
  cache:
    type: redis
  data:
    redis:
      host: localhost
      port: 6379
```

Закэшируй список задач на 30 секунд. После commit записи — выкинь затронутые ключи списка, включая варианты фильтра и страниц. Не «обнови кэш вручную тем же JSON»: выкинь. Пусть следующий GET сходит в базу и положит свежее. Два действия, не двенадцать.

9.1.3 История первая: табличка глючит (stale cache)

Человек создал задачу «починить антенну». GET `/tasks` — видит. Потом переименовал

в «починить антенну срочно». GET — всё ещё «починить антенну», без «срочно». Прошло десять секунд. Человек ругается, что сервер сломан. Сервер не сломан. Сломан **ты**: закэшировал список и забыл сбросить ключ на update.

Это главный вопрос собеса про кэш. Не «что такое Redis». А: обновили, кэш старый — что будет, и как чинить.

Чинить так:

1. На любой записи (create / update / delete) сбросить все затронутые варианты списка после успешного commit базы. Одна задача меняет фильтры, содержимое страниц и общие счётчики. Для этого маленького

упражнения допустимо очищать весь именованный кэш списков задач; удалить ключ одной страницы недостаточно. Именно кэш списков, не всю базу Redis. Обработчик после commit показан на занятии 18; сама по себе `@CacheEvict` не означает «после commit».

2. TTL, чтобы даже забытый ключ умер сам. 30 секунд для учёбы. В бою TTL подбирают, это не магия числа 30.
3. Не кэшировать то, что каждый раз разное (случайная цитата дня — ладно; «текущий пользователь только что сохранил» — нет).

Давай медленно

Воспроизведи ложь специально. Закомментируй сброс ключа. Создай задачу. GET. Обнови название. GET сразу — старое имя. Подожди 31 секунду. GET — новое. Вот ты **увидел** TTL телом, не статьёй. Верни сброс. Теперь второй GET сразу свежий. README: как повторить оба варианта. Полезно уметь ломать свою станцию намеренно. Иначе на собесе скажешь «ну в теории», и теория улыбнётся тебе жёлтой карточкой.

9.1.4 История вторая: сто слонов по мостику (cache stampede)

Кэш протух. Ключа нет. В эту миллисекунду пришло сто GET `/tasks`. Сто запросов не нашли

табличку, сто полезли в трюм. Postgres получил стадо. Это называют cache stampede или thundering herd — громкое стадо. На станции так: датчик погас, и сразу весь экипаж побежал в один люк.

На трёх пользователях ты этого не увидишь. Для демонстрации оставь Redis включённым, дождись истечения одного горячего ключа и отправь к нему несколько GET параллельно. Если запрос к базе слишком быстрый, временно добавь учебную задержку перед загрузкой. В логе считай одинаковые `SELECT`. Выключенный Redis — другая поломка: без специально настроенного fallback можно

получить ошибку кэша, а не поход в базу.

Лечат не «поставь Redis ещё раз». Redis как раз протух, в этом и шутка. Лечат так:

- **сброс на запись** важнее короткого TTL «чтобы всегда свежо». Свежо и штампед — соседи;
- один пересчёт: кто первый не нашёл ключ, тот идёт в базу, остальные ждут его (в Java это уже lock / `synchronized` на ключ, или библиотеки вроде Caffeine с refresh; в проде ещё singleflight — слово запомни, реализацию не пиши с нуля сегодня);
- иногда честно отдать чуть старое, пока новый ключ считается. Это уже взрослые

игры. Для учебника достаточно понимать **имя беды**.

На собесе хватит: «если много запросов попали в пустой ключ сразу — все пойдут в базу; я бы держал короткий lock на пересчёт или не ставил TTL в одну секунду на горячий ключ». Это лучше, чем «Redis очень быстрый».

9.1.5 Lisp: учебный Redis из палок

Нам не нужен сокет. Нужна hash-table и время. Время в Common Lisp — `(get-universal-time)`, секунды с 1900 года, как бортовые часы, которым всё равно на твой пояс.

```
(defstruct centry value expire)

(defun cset (h key value ttl)
  (setf (gethash key h)
        (make-centry :value value
                     :expire (+
(get-universal-time) ttl))))

(defun cget (h key)
  (let ((e (gethash key h)))
    (when (and e (< (get-
universal-time) (centry-expire
e)))
      (centry-value e))))
```

`cset` кладёт значение и пишет на ящике «годно до». `cget` смотрит на часы. Просрочено — `nil`, как будто ключа не было. Не удаляет обязательно: можно удалять, можно оставлять труп. Для этюда `nil` достаточно.

Что только что случилось Положи `'tasks` на 2 секунды. Сразу `cget` — список. Подожди

три секунды в REPL: `(sleep 3)`. Снова `cget` — `nil`. Ты только что потрогал TTL без Docker. Redis в проде делает примерно этот жест, только с сетью, персистенцией и кучей кнопок, которые тебе пока не нужны.

Сброс на запись — просто `(remhash key h)`. Забыл `remhash` — получишь сказку, пока не выйдет срок. Та же ложь, что в Java, только без YAML.

Квест 17.L1 · Lisp
Hash-table с протуханием по времени. `get` после срока — `nil`. Учебный Redis из палок.

Квест 17.J1 · Java
Кэш списка, 30 секунд, сброс вариантов после commit записи. Прогрей две страницы и варианты `done=true` / `done=false`;

проверь, что ответы не подменяют друг друга, затем измени задачу и снова проверь каждый затронутый вариант. README: как увидеть ложь кэша, **если забыть** сброс. Полезно уметь ломать свою станцию намеренно.

Квест 17.J2 · Java
Короткий TTL (секунда-две). Дождись истечения ключа и отправь двадцать `curl` параллельно (`xargs -P` или несколько окон). В логе посчитай обращения к базе. Последовательный цикл обычно даст один промах, затем попадания — это нормально. Если и с параллелью один SELECT, проверь, перекрылись ли запросы и не включено ли

объединение загрузок. Это наблюдение, не бенчмарк.

Так себе идея
Не кэшируй ответы с персональными данными «на всех» одним ключом `tasks`. Ключ должен знать **чьё** это и **какой список**: ID вошедшего пользователя плюс все параметры, влияющие на ответ (`done`, `page`, `size`, поля сортировки и их направления в заданном порядке), после применения значений по умолчанию и ограничений. Например: `tasks:user:42:done:all:page:0:size:20:sort:id,asc`. Здесь `all` означает отсутствие фильтра `done`; это не `false`. Пользователя бери из аутентификации, не из произвольного параметра запроса. Иначе сосед по

станции прочитает твой список «никому не говорить», либо вторая страница превратится в первую. Кэш — не место, где внезапно пропадают права.

Спрятать на GitHub
Коммит вроде `week17: redis cache ttl 30s`. В README — два сценария: ложь без сброса и правда со сбросом. Скрин лога не обязателен. Слова «я видел stale» обязательны.

9.2 Занятие 18. «Сделай потом»: очередь, не второй сервер

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю. Письмо «задачу создали» не должно валить HTTP, если

почта лежит. Представь: ты на вахте, человек просит «запиши задачу и скажи на землю». Если ты сначала звонишь на землю, а антенна молчит три минуты, человек у шлюза стоит и ненавидит тебя. Правильно: записать задачу, ответить «принял», а записку «сказать человеку» положить в лоток. Лоток — очередь.

Сначала событие Спринга или `BlockingQueue` в том же процессе. Потом RabbitMQ — **один** сценарий. Кафка сегодня не нужна, сколько бы её ни было в вакансии мечты. Кафка — журнал на следующем занятии. Очередь — лоток. Это разные предметы, даже если в резюме их пишут через запятую как братьев.

9.2.1 Сначала лоток в кают: `BlockingQueue`

В Java есть очередь, которая умеет **ждать**. Не крутиться в `while (true)` и жечь процессор, а спать, пока кто-то не положит записку.

```
import
java.util.concurrent.ArrayBlockin
gQueue;
import
java.util.concurrent.BlockingQueu
e;

BlockingQueue<String> q = new
ArrayBlockingQueue<>(16);
```

Ёмкость 16 — на станции лоток не резиновый. Если положить 17-ю, пока никто не забрал, `put` **встанет и будет ждать**. Это не баг. Это «не теряй записки и не раздувай память».

Давай медленно
Два персонажа. Продюсер — вахтёр. Консьюмер — посыльный.

```
Thread producer = new Thread(()
-> {
    try {
        q.put("сказать человеку:
задача 7");

System.out.println("положил");
    } catch (InterruptedException
e) {

Thread.currentThread().interrupt(
);
    }
}, "producer");

Thread consumer = new Thread(()
-> {
    try {
        String note = q.take();
        System.out.println("взял:
" + note);
    } catch (InterruptedException
```

```
e) {

Thread.currentThread().interrupt(
);
    }
}, "consumer");

consumer.start();
producer.start();
```

`take` без записки — ждёт. `put` без места — ждёт. `InterruptedException` — «меня попросили остановиться»; мы ставим флаг обратно и не делаем вид, что ничего не было. Запусти в `java-basics`, не в боевом сервере. Посмотри порядок строк: иногда «положил» раньше «взял», иногда наоборот — это два потока, не детектив.

`offer` не ждёт: не влезло — `false`. `poll` не ждёт: пусто — `null`. Для

«сделай потом» в том же процессе обычно `put`/`take`: лучше постоять, чем потерять «сказать человеку».

Почему не `ArrayList` плюс свой поток с `sleep(100)`? Потому что `sleep` — гадание. `take` — контракт. На собесе «я крутил sleep в цикле» звучит как «я чинил шлюз молотком». Молоток иногда работает. Потом отваливается люк.

9.2.2 Спринг: событие, не второй микросервис

В монолите можно развязать код событием: создал задачу — опубликовал событие, слушатель пишет в лог после коммита. В показанном варианте слушатель синхронный: HTTP ещё ждёт

его завершения. Это первый шаг к уведомлениям, но пока не фоновая очередь.

Грубый каркас:

```
public record
TaskCreatedEvent(long id, String
title) {}
```

В публичном методе сервиса с `@Transactional`, после `save`, но до выхода из метода, опубликуй событие через внедрённый `ApplicationEventPublisher publisher`. Публикация должна попасть в ту же транзакцию: без активной транзакции этот слушатель по умолчанию не вызывается.

```
publisher.publishEvent(new
TaskCreatedEvent(saved.getId(),
saved.getTitle()));
```

Слушатель:

```
@Component
public class NotifyListener {
    private static final Logger
log =
LoggerFactory.getLogger(NotifyLis
tener.class);


@TransactionalEventListener(phase
= TransactionPhase.AFTER_COMMIT)
    public void
on(TaskCreatedEvent e) {
        log.info("сказать
человеку: задача {} «{}»",
e.id(), e.title());
    }
}
```

`AFTER_COMMIT` — не кричи «создали», если транзакция откатилась. Это обработчик после коммита в том же процессе, без хранения и гарантии доставки. Для фоновой работы нужен

отдельно настроенный executor / `@Async` или очередь; для переживания сбоя — ещё и надёжное хранение события. Сегодня оставь короткую строку лога.

Настоящую почту не подключай. Лог — достаточно. SMTP на учебной станции — это приглашение потратить пятницу на пароль от Gmail.

9.2.3 Когда уже RabbitMQ, и когда ещё нет

RabbitMQ — отдельный брокер: положил сообщение, забрал другой процесс, даже если твой сервер умер и встал заново. Это

уже «лоток в соседней комнате». Имеет смысл, когда:

- «сказать человеку» должно пережить рестарт JVM;
- или этим займётся другой сервис (ты его **не** выкусываешь сегодня).

Один сценарий: после create кладёшь JSON в очередь `task-created`. Отдельный `@RabbitListener` пишет тот же лог. Docker-образ `rabbitmq:3-management`. Если брокер не встаёт за вечер — откатись на `BlockingQueue` / событие Спринга и честно напиши в README. Честность снова красивее галочки.

Так себе идея

Не поднимай «микросервис уведомлений» отдельным репозиторием, пока compose монолита не встаёт. Выкусить сервис — легко. Потом чинить «а задача в базе есть, а письмо ушло про откат» — нет. Боль транзакций между двумя базами мы не заказывали.

9.2.4 Lisp: очередь как список с плохим характером

```
(defparameter *q* nil)

(defun enqueue (x)
  (setf *q* (append *q* (list
x))))

(defun dequeue ()
  (let ((x (first *q*)))
    (setf *q* (rest *q*))
    x))
```

`append` каждый раз гоняет весь список. Для этюда можно. Для станции в бою — как носить болты по одному из трюма в нос, каждый раз начиная от люка. В Java для очереди — `ArrayDeque` или `ArrayBlockingQueue`, не изобретай `append`.

`drain` — пока не пусто, печатай:

```
(defun drain ()
  (loop while *q*
        do (format t "~a~%"
(dequeue))))
```

Что только что случилось Три `(enqueue ...)`, потом `(drain)`. Очередь пустая. Ещё `(drain)` — тишина. FIFO: кто первый лёг в лоток, того первого и прочитают. Это не приоритет «антенна важнее мытья иллюминатора». Приоритет —

другая история, сегодня не про неё.

Квест 18.L1 · Lisp
Очередь уведомлений. `drain` печатает все. Как вахтёр, который наконец дошёл до стопки записок.

Квест 18.J1 · Java
После create — событие, слушатель пишет в лог «сказать человеку …». Тест слушателя не обязателен. Рабочий код обязателен.

Квест 18.J2 · Java
Отдельный класс в `java-basics`: `BlockingQueue` на 4 места, продюсер кладёт 8 записок, консьюмер забирает. Лог: когда ждал `put`, когда ждал `take`. README — пять строк «что

такое блокирующая очередь». Без Спринга. Руками.

Воскресная диверсия
Нарисуй на бумажке: HTTP-запрос → сервис → база → событие → лоток → «письмо». Где человек уже получил 201? Если после базы — хорошо. Если после письма — антенна снова держит шлюз. Бумага дешевле Kafka.

9.3 Занятие 19. Кафка: журнал, а не телефон

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Кафка — не «вызвать соседний сервис». Это журнал: писатель дописал в тему, читатель ползёт со смещения. Как

бортовой лог, который нельзя стереть ластиком. На «МОДУЛЬ» такой журнал ведут у реактора: смена дописала «давление 1.2», следующая смена читает **с закладки**, а не звонит предыдущей и не просит «повтори, я не записал».

Если Docker уже друг: один брокер, одна тема `task-events`, продюсер из сервиса, консьюмер, который логирует. Если тяжело — честный конспект в `docs/kafka.md` и **не ломай** монолит. В резюме Кафка только если открывается **этот** код. Слово без репозитория — свисток без воздуха.

9.3.1 Чем журнал не телефон (Kafka vs HTTP)

HTTP — телефон. Ты звонишь, ждёшь гудок, слышишь «201» или «500», кладёшь трубку. Если абонент спит, ты висишь или получаешь ошибку **сейчас**. Два сервиса по HTTP знают друг друга по имени и по контракту «я жду ответ».

Кафка — журнал на столе вахты.

- Писатель не знает, читает ли кто-то. Дописал строку — пошёл чинить антенну.
- Читатель приходит когда хочет. Спал час — догонит. Закладка (offset) помнит, до куда дочитал.
- Строки не «переписать». Обычно дописывают. Старое

остаётся. Это бесит, пока не поймёшь зачем: можно переиграть день, можно отладить «кто что положил в 03:12».

- Если читателей двое с разными закладками — оба прочитают одно и то же, каждый в своём темпе. Телефон так не умеет, если ты не конференция и не ад.

Поэтому фраза из вакансии «интеграция через Kafka» не значит «вместо REST». Часто REST остаётся для человека и для «сделай сейчас». Журнал — для «случилось, разберётесь». Создали задачу — HTTP 201 человеку. В журнал — событие `TaskCreated`, чтобы потом

письмо, поиск, аналитика, кто угодно, **не держа трубку**.

Давай медленно
Сравни вслух, хоть коту:

1. Человек POST `/tasks`. Сервис пишет в Postgres. Ответ 201. Это HTTP.
2. Тот же сервис дополнительно кладёт в тему `task-events` JSON `{ "id": 7, "title": "шлюз" }`. Это продюсер. Он не ждёт, пока письмо уйдёт.
3. Другой процесс (или поток) читает тему с offset 0, потом 1, потом 2. Это консьюмер. Упал — встал, спросил брокер «я на 2», читает дальше.
4. Прямой HTTP-вызов лежащего «сервиса писем» может заставить POST задачи

ждать или завершиться ошибкой; точный статус выбирает приложение. Если брокер уже подтвердил запись события и ещё хранит её, потребитель писем сможет догнать позже. Но запись в Postgres и отправка в Kafka здесь не одна атомарная операция: сбой между ними может потерять уведомление.

Если путаешь — вернись к бумажке. Телефон vs бортовой журнал. Не «ещё одна очередь, только модная».

Очередь (занятие 18) отдаёт записку одному обработчику. Чтение Kafka не удаляет запись: двигается offset. Но бумага не вечна — срок и объём хранения

задаёт retention, а compaction может убирать старые значения ключа. Offset относится к одной партиции; независимые consumer groups читают журнал каждая со своей закладкой. Наш вектор ниже показывает только append/read, без этих правил хранения.

9.3.2 Один брокер, одна тема, никаких кластеров на кухне

В Docker для учёбы часто берут образ, где Kafka уже не требует отдельного ZooKeeper (версии меняются, README образа читай сегодня, не «как в статье 2020»). Один брокер. Одна тема `task-events`. Репликация «три брокера, rack awareness» — не твоя смена.

Продюсер из Spring (имена классов снова сверяй с версией):

```
kafkaTemplate.send("task-events",
String.valueOf(id), json);
```

`send` асинхронный: возврат из вызова ещё не подтверждает запись брокером. Проверяй результат future или callback и обрабатывай ошибку. Учебный фрагмент не реализует надёжную доставку при сбоях.

Консьюмер:

```
@KafkaListener(topics = "task-
events")
public void on(String payload) {
    log.info("из журнала: {}",
payload);
}
```

Если это встаёт — в README: как `compose up`, как увидеть строку в логе после POST. Если не встаёт за вечер — **стоп**. Монолит не виноват. Пиши `docs/kafka.md`.

9.3.3 Честный конспект, если брокер тебя съел

В `docs/kafka.md` своими словами, не копипаста из Википедии:

- зачем лог, если уже есть HTTP;
- что такое topic, partition (хотя бы: «папка журнала» и «несколько тетрадей, чтобы писать параллельно»);
- что такое offset;
- чем consumer group отличается от «просто два читателя»;

- что ты **не** поднимал (кластер, exactly-once, Schema Registry).

Это сильнее на собесе, чем «я проходил курс». Курс не открывается на GitHub. Твой файл — открывается.

9.3.4 Lisp: крошечная Кафка в REPL

Вектор, только дописывать, читать с индекса. Старое не удаляй.

```
(defparameter *log* (make-array
0 :adjustable t :fill-pointer 0))

(defun produce (x)
  (vector-push-extend x *log*))

(defun consume (offset)
  (when (< offset (length *log*))
    (aref *log* offset)))
```

`produce` всегда в конец. `consume` с нуля, с единицы, с семи — как закладка. Нет ключа «удали событие 3». Хочешь «не читать» — подвинь offset.

Что только что случилось `(produce 'airlock) (produce 'alarm) (consume 0)` → `AIRLOCK`. `(consume 1)` → `ALARM`. `(consume 2)` → `nil`. Журнал всё ещё длины 2. Ты не стёр воздух. Ты только посмотрел.

Мы сделаем журнал. Игра подождёт воскресенье.

Квест 19.L1 · Lisp
Вектор-лог: только дописывать, читать с индекса. Старое не удаляй. Крошечная Кафка в REPL. Мы сделаем журнал.

Квест 19.J1 · Java

Либо живые producer/consumer, либо страница своими словами: зачем лог, чем не HTTP. Честность важнее галочки. Правда красивее на собесе, чем «ну я проходил курс».

Квест 19.J2 · текст
В `docs/kafka-vs-http.md` таблица на пять строк: действие человека, HTTP, журнал. Пример: «сервис писем лежит». Что видит POST `/tasks` в каждом мире. Без английской простыни. Своими словами, как бортмеханик, не как лендинг.

SICP · открывать, только если зудит
Лог как последовательность — родственник «поток событий» из глав про streams. Если вдруг зудит, не вместо Docker. После.

Станция сначала должна уезжать в коробке.

9.4 Занятие 20. Коробка, в которой станция уезжает к другому

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
До сих пор станция жила в кают-е: IDEA, локальный Postgres, «у меня работает». Сегодня она должна уехать к другому человеку (или к тебе на другой ОС) командой из README. Коробка — Docker. Расписание коробок — Compose. На Windows Docker Desktop — тот же compose, файлы те же. YAML допускает LF и CRLF. А вот скрипт `mvnw`, который запускает

Linux, сохрани с LF: там CRLF может сломать запуск. Ещё проверь, не забыл ли `.dockerignore`.

Проверка: другой человек делает `docker compose up` и проходит curl из README. Если только «у меня в IDEA» — это ещё не станция, это макет в каюте.

9.4.1 Dockerfile, строка за строкой

Файл без расширения, имя `Dockerfile`, в корне `task-manager`. Инструкции задают шаги сборки. `RUN` и `COPY` могут добавлять слои файловой системы, а, например, `CMD` задаёт метаданные запуска. Не каждая строка — новый файловый слой. Результаты

шагов кэшируются: поменял Java-код — не надо заново качать JDK, если `FROM` и зависимости выше не трогал. Поэтому **сначала** манифест, потом исходники. Не наоборот.

Учебный, честный, Java 21:

```
FROM maven:3.9-eclipse-temurin-21
AS build
WORKDIR /src
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/
*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/
app/app.jar"]
```

Давай медленно

Читаем как вахтёрский журнал, не как заклинание.

`FROM maven:3.9-eclipse-temurin-21 AS build` — образ с Maven и JDK 21: нужны и сборщик, и компилятор. Имя стадии `build`. Не `latest`.

`WORKDIR /src` — «дальше все команды из этой папки». Как `cd`, только в слое.

`COPY pom.xml .` — сначала описание проекта, потом исходники. В этом простом варианте Maven запускается уже после `COPY src`, поэтому изменение `.java` заново запускает сборку и разрешение зависимостей. Отдельный `RUN mvn dependency:go-offline` перед `COPY src` позволит кэшировать зависимости отдельно. Сегодня

главное: jar собирается в Linux, не «я принёс jar с мака».

`RUN ... package` — собери jar внутри Linux. `skipTests` в образе спорный: тесты должны жить в CI, не обязательно внутри сборки образа. Если тесты лезут в localhost-Postgres, образ как раз и не соберётся. Поэтому тесты — снаружи, в GitHub Actions.

Второй `FROM eclipse-temurin:21-jre` — **другой** образ, без компилятора. В прод (и в учебный прод) JRE легче. Это multi-stage: из первой стадии забираем только jar.

`COPY --from=build ... app.jar` — переложи артефакт. Не исходники. Не `.git`. Не `target/` с хоста целиком.

`EXPOSE 8080` — табличка «я слушаю 8080». Сама по себе дырку в хост не дырявит. Дырявит `ports:` в compose.

`ENTRYPOINT ["java", "-jar", ...]` — **exec-форма** (JSON-массив). Не `java -jar` строкой через shell, если не надо. Так сигналы «остановись» доходят до JVM, а не до `/bin/sh`.

Рядом `.dockerignore`:

```
.git
target
.idea
*.md
.env
.env.*
```

Иначе в контекст уедет пол-диска и твои секреты из `.env`, если они вдруг лежат рядом. `.env` в образ не копировать.

Никогда. Даже учебный пароль `postgres/postgres` лучше скормить переменной, чем запечь.

Мак, Windows, WSL
Сборка образа: `docker build -t task-manager:dev .` в папке проекта. На маке и в WSL одинаково. В PowerShell тоже, если Docker Desktop запущен. В показанной стадии Maven уже установлен. Если вместо неё берёшь чистый JDK-образ и Maven Wrapper, скопируй и `mvnw`, и каталог `.mvn`, сохрани для скрипта LF и право execute. Одного JDK для команды `mvn` мало.

9.4.2 Compose: три жильца, одна сеть, и ловушка `localhost`

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: taskdb
      POSTGRES_USER: task
      POSTGRES_PASSWORD: task
    volumes:
      - pgdata:/var/lib/
postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
"pg_isready -U task -d taskdb"]
      interval: 5s
      timeout: 3s
      retries: 10
    ports:
      - "127.0.0.1:5432:5432"

  redis:
    image: redis:7
    ports:
      - "127.0.0.1:6379:6379"

  app:
```

```
    build: .
    depends_on:
      db:
        condition:
service_healthy
    environment:
      SPRING_DATASOURCE_URL:
jdbc:postgresql://db:5432/taskdb
      SPRING_DATASOURCE_USERNAME:
task
      SPRING_DATASOURCE_PASSWORD:
task
      SPRING_DATA_REDIS_HOST:
redis
    ports:
      - "8080:8080"

volumes:
  pgdata:
```

Имена сервисов `db`, `redis`, `app` — это DNS внутри сети Compose. Контейнер `app` стучится в хост **`db`**, не в `localhost`.

Давай медленно
Почему все падают в одну яму.

Внутри контейнера `localhost` — это **он сам**. Приложение спрашивает Postgres на `localhost:5432` — стучится в свой собственный пустой карман. Там нет Postgres. Postgres в соседнем контейнере по имени `db`.

С хоста (твой curl, твоя IDEA) `localhost:5432` — это проброс `ports:`. Поэтому **с мака** `psql -h localhost -U task -d taskdb` работает, а **из приложения в Docker** — нет.

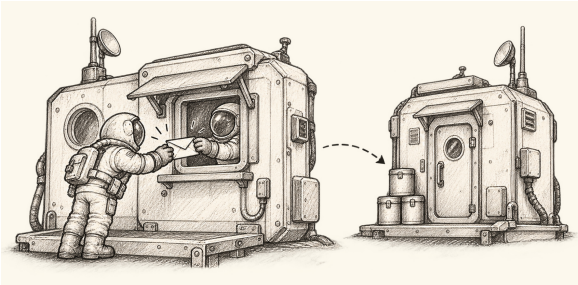
Два мира, два имени:

- Java в IDEA на хосте → `localhost`
- Java в контейнере `app` → `db` и `redis`

URL базы — из окружения, не зашитый. Один `application.yml` с дефолтом для каюты, в compose — переменные, которые перекрывают. Внутри образа без пометки `localhost` — ловушка. Почти все в неё падают. Ты теперь знаешь табличку «мокро».

`depends_on` без healthcheck — «контейнер стартовал», не «Postgres готов принимать соединения». `pg_isready` проверяет готовность сервера, но не подтверждает правильность пароля приложения. Поэтому оставь healthcheck, а URL, логин и пароль отдельно проверь реальным подключением приложения.

Пароль `task/task` допустим только для одноразовой учебной базы. Поэтому порты базы и Redis привязаны к `127.0.0.1`: без адреса Docker обычно публикует порт на всех интерфейсах хоста. В песочнице тоже не храни настоящие данные. Боевой пароль — не в публичный GitHub, даже в `docker-compose.override.yml`, который ты «случайно» закоммитил.



«Здесь» зависит от того, где ты стоишь.
В этой Compose-сети app обращается к базе по db:5432. localhost ведёт в сам вызывающий контейнер.

9.4.3 CI: робот, которому всё равно, какая у тебя IDEA

Файл `.github/workflows/ci.yml`. Не «когда-нибудь». На каждый push. Зелёная галка на GitHub приятнее, чем самооценка.

```
name: ci

on:
  push:
```

```
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-
java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: tests
        run: mvn -B test
```

Давай медленно `name: ci` — как подписать вахту. Вкладка Actions покажет это имя.

`on: push` — любой толчок в репозиторий. `pull_request` — ещё и на PR, чтобы ты видел ошибку **до** слияния. Можно сузить

ветки позже. Сейчас пусть падает всегда.

`jobs.build` — одна работа. Можно несколько, не надо.

`runs-on: ubuntu-latest` — чужая машина, Linux. Поэтому «у меня на маке» робота не волнует. Тесты должны быть зелёными без кликов мышью.

`actions/checkout@v4` — забери код. Без этого робот сидит в пустой каюте.

`actions/setup-java@v4` + `temurin` + `21` — тот же JDK, что в книге. `cache: maven` — не качай полинтернета каждый раз.

`mvn -B test` — `-B` это batch: меньше интерактива, меньше «нажми Enter». Тесты. Не `package` с пропуском тестов. Если тесту

нужен Postgres, либо Testcontainers, либо H2 **только** в `src/test`, и в README это написано. Врать роботу «зелёное, но базу я не поднимал» можно ровно до первого человека, который поверит галке.

Секреты в YAML не клади. Роботу не нужен твой пароль от Земли. Если когда-нибудь понадобится токен — GitHub Secrets, не строка в файле.

Мак, Windows, WSL
Первый push с Windows: для единообразия сохраняй `ci.yml` в UTF-8 с LF, хотя сам YAML допускает и CRLF. Если Actions ругается на символ, смотри точное место, отступы и кодировку; BOM в начале YAML

сам по себе тоже допустим. А Linux-скрипты вроде `mvnw` держи с LF. Не каждый красный запуск — война операционных систем.

9.4.4 Проверка, что станция уехала

Свежая папка и отдельный учебный Compose-проект на чужом ноутбуке или второй ОС. Для обычного перезапуска хватит `docker compose down` и снова `up`. Осторожно: `docker compose down -v` удаляет и том `pgdata`, а вместе с ним все данные этой базы. Это сброс одноразовой песочницы, не обычный перезапуск.

```
docker compose up --build
```

Потом с хоста:

```
curl -s localhost:8080/health
```

Логин, список задач — как в README. Если health живой, а база нет — снова `localhost` внутри. Если порт занят — на хосте уже сидит старый Postgres из месяца 3. Выключи локальный или поменяй проброс на `5433:5432` и **напиши это в README**. Молчаливый порт — враг.

Квест 20.J1 · Java
Compose поднимает базу и приложение. URL базы — из окружения, не зашитый `localhost` внутри образа без пометки. Внутри сети Docker хост базы часто называется `db`, не `localhost`. Да, это ловушка. Почти все в неё падают.

Квест 20.J2 · Java
`.github/workflows/ci.yml` — сборка и тесты. Зелёная галка на GitHub приятнее, чем самооценка.

Квест 20.J3 · текст
В README раздел «Почему не localhost»: пять-семь предложений. Хост vs контейнер. Имена `db` и `redis`. Куда ходит curl с мака. Куда ходит JVM внутри `app`. Если однокурсник после этого всё равно напишет `localhost` в образе — это уже его вахта, не твоя.

Так себе идея
Не выкусывай «микросервис уведомлений», пока compose монолита не встаёт. Если выкусил — в README напиши,

что стало больнее: данные, транзакции, отладка. Боль — учебный материал, не стыд.

Спрятать на GitHub
Коммит `week20: compose postgres redis app`. Тег, если хочется. README начинается с `docker compose up`, не с «открой IDEA». Иначе тег — пустышка.

Воскресная диверсия
Сломай одну строку Dockerfile специально (`FROM` на несуществующий тег или опечатка в `ENTRYPOINT`). Прочитай ошибку вслух. Почини. Страх «докер слишком большой» проходит, когда ошибка стала конкретной, а не космической.

Закон станции
К концу месяца в вакансии можно честно подчеркнуть:

кэш, очередь **или** журнал, коробка. Подчеркнуть — значит открыть файл. Не подчеркнуть слово, которого нет в `git log`.

10 Месяц 6. Выход в открытый космос, то есть на рынок

Новых священных технологий почти нет. Есть продукт и письма незнакомым людям. Сорок минут Lisp остаются: когда откажут (откажут), скобки всё ещё слушаются, в отличие от HR.

Это странный месяц. Ты уже умеешь больше, чем кажется в 2 часа ночи. Рынок об этом не знает, пока ты не напишешь README так, чтобы незнакомец за полторы минуты понял, **что открывать**, и пока ты не отправишь письмо, в котором есть правда, а не «стрессоустойчивый». Станция

«МОДУЛЬ» тут ни при чём. При чём — календарь и голос.

Закон станции
Мерило месяца — слоты в календаре и отправленные письма, не новая глава про Kubernetes. Kubernetes подождёт. HR не подождёт «ещё чуть-чуть Кафку»: он просто возьмёт другого, у которого README открывается.

10.1 Занятие 21. README как иллюминатор, не как чердак

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.
Человек с той стороны смотрит полторы минуты. Иногда сорок секунд. Он не читает твой

внутренний монолог про «я ещё допилю». Он смотрит верх экрана. Если там пусто, или «task manager на спринг бут» без глагола, или стена эмодзи — он закрывает вкладку. Как иллюминатор, заляпанный изнутри: снаружи кажется, что станции нет.

Порядок, который работает:

1. README `task-manager`: что умеет, curl или картинка, как запустить на маке **и** на Windows/WSL, из чего сделано.
2. В репозитории нет `.env` с секретами и нет папки `target/`.
3. Коммиты не `asdf`.

4. `java-basics` пусть живёт — это путь, не позор. Позор — притворяться, что Hello не было.

Резюме: одна страница. Стек, два проекта со ссылками, как с тобой связаться. Курсы внизу или никак. Lisp — в «ещё вот».

10.1.1 Чёрный ящик, который надо вынести на солнце

Открой свой README сейчас, до правок. Засеки минуту. Представь, что ты однокурсник с Windows, Docker Desktop только вчера поставили, IDEA другая. Что сломается первым?

Обычно:

- нет команды запуска, есть «открой проект»;

- Postgres «ну вы знаете»;
- curl только на GET, а про логин молчок;
- Java 17 в тексте, в `pom` уже 21, или наоборот;
- картинка из 2024, эндпоинты с тех пор переехали.

Это не стыд. Это чердак. Чердак у всех. Иллюминатор протирают тряпкой, не самооценкой.

10.1.2 Образец README (укради структуру, не биографию)

Ниже — **учебный** текст. Подставь свои URL, свои эндпоинты, свой пароль учебной базы. Если у тебя JWT — покажи два curl: регистрация и «список со своим токеном».

Если сессия — покажи cookie. Не пиши «и т.д.». «И т.д.» на собесе значит «я не проверял».

```
# task-manager

Личные задачи по HTTP. Не трекер
на миллион человек — учебный
монолит, на котором можно ткнуть
CRUD, вход и тесты.

## Что умеет

- регистрация и вход (JWT)
- свои задачи: создать / список /
поменять / удалить
- список с пагинацией
- кэш списка в Redis на 30
секунд, сброс на запись

## Запуск (Docker)

Нужны Docker Desktop (мак или
Windows+WSL2) и свободные
порты 8080, 5432, 6379.

    docker compose up --build
```

````
Проверка:

    curl -s localhost:8080/health

Дальше — регистрация, логин,
POST /tasks. Примеры ниже.

## Запуск без Docker (каюта)

JDK 21, PostgreSQL 16, Redis 7.
Создай базу `taskdb`.
Скопируй `src/main/resources/
application-example.yml`
в `src/main/resources/
application.yml`. Шаблон в
репозитории содержит
только настройки и подстановки
окружения, без секретов.
Задай проверенные `TASK_DB_USER`
и `TASK_DB_PASSWORD` в окружении
запуска;
локальные файлы с настоящими
паролями в git не клади.

    ./mvnw spring-
boot:run          # мак, WSL
    mvnw.cmd spring-
````

```
boot:run        # Windows

## Стек

Java 21, Spring Boot, PostgreSQL,
Flyway, Redis, JUnit.

## Чего нет (чтобы не врать)

Kafka в бою не крутится.
Уведомления — синхронный
обработчик
после коммита внутри монолита и
строка лога. Фоновой очереди нет.
```

Картинка Postman или скрин curl — плюс, не обязаловка. Обязаловка — чтобы однокурсник **без тебя в мессенджере** увидел health `UP`.

Мак, Windows, WSL
В README две колонки команд — не эстетика, выживание. Мак и WSL: `./mvnw`, обратный слэш в многострочном curl. Windows

cmd: `mvnw.cmd`, `^`. В PowerShell проще одной строкой через `curl.exe`. Не обещай общий `localhost` во всех режимах WSL2: при обычном NAT обращение из WSL к серверу Windows требует IP хоста; mirrored networking позволяет `localhost`. Проще сначала запускать сервер и curl в одной среде.

10.1.3 Репозиторий, который не пахнет общежитием

`.gitignore` должен есть: `target/`, `.idea/`, `.env`, `*.iml`, `.DS_Store`. Если `target/classes` в GitHub — рекрутер не всегда поймёт, но разработчик на собесе поймёт и вздохнёт. Вздох на собесе — плохая валюта.

Секреты: учебный `task/task` в compose — ок, это песочница. Токен от почты, пароль «как на проде», ключ AWS, который ты «на секунду» положил — не ок. Даже в истории git. Если уже запушил — смени ключ, старый отзови, не делай вид, что «никто не видел». Видели боты.

Коммиты: `week12: flyway tasks table` читается. `fix`, `asdf`, `не работает` — бортовой журнал пьяной смены. Не переписывай всю историю ради красоты. Со следующего понедельника пиши нормально.

`java-basics` и `lisp-experiments` пусть остаются публичными. Это путь от Hello. Человек, который прячет первую

программу, часто прячет и вторую. Ты не прячешь.

10.1.4 Резюме на одну страницу, не на роман

Имя, город (или «удалённо»), почта, GitHub, телефон если готов слышать незнакомые номера.

Дальше **не** «цель: развиваться в дружном коллективе». Цель и так ясна: работу. Дальше стек одной строкой: Java 21, Spring, PostgreSQL, Docker, Git.

Два проекта:

- task-manager — что делает, ссылка, одна деталь («JWT, кэш Redis, compose»).
- второй, если есть: shop, android-calc, хоть консольный список. Лучше

живой маленький, чем «в планах микросервисы».

Опыт: учёба, подработка, «писал то-то». Не ври даты. Курсы — внизу, одной строкой, или никак. Lisp: «для себя, Common Lisp, учебные этюды» — в «ещё вот», не в шапку вместо Java.

Квест 21.J1 · Java
Перепиши README так, чтобы однокурсник поднял проект без тебя в мессенджере. Мак и Windows. Если всплывёт «ну это же очевидно» — не очевидно.

Квест 21.J2 · текст
Восемь–двенадцать строк про task-manager: зачем, что сделал, чем пользовался, ссылка. Без «коммуникабельный, стрессоустойчивый».

Квест 21.J3 · текст
Резюме на одну страницу. Сфотографируй или PDF. Прочитай вслух за сорок секунд. Если за сорок секунд не ясно, **чем ты полезен** — режь прилагательные, не стек.

Спрятать на GitHub
Проверь глазами, не «git status зелёный»: открыл GitHub в инкогнито, README, нет `target/`, есть compose, есть CI-галка. Инкогнито — чтобы не путать «я залогинен и всё знаю» с «незнакомец».

10.2 Занятие 22. Репетиция в каюте

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

Три куска по пятнадцать–двадцать минут. Не «когда будет настроение». Настроение на собесе не придёт. Придёт человек с ноутбуком и вопросом «расскажи про проект».

1. Java и SQL (соединение таблиц, индекс, транзакция, чем 401 не 403).
2. «Расскажи task-manager» без экрана, три минуты, потом «а если база умрёт».
3. Задачка на двадцать минут вслух. Молчание на собесе читается как «я умер».

Запиши голос. Слушать себя стыдно. Стыд дешевле отказа, в котором ты не понял, **где** поплыл.

10.2.1 Плохой питч, три минуты (так делать не надо)

«Ну это как бы таск-менеджер на Спринге. Ну там Бут сам поднимает, ну и Джава, ну база Постгрес. Я добавил эндпоинты. Ну и Redis, потому что в вакансиях Redis. Ещё хотел Кафку, но не успел. В общем, обычный CRUD. Ну и тесты есть, ну как бы.»

Что слышит человек напротив:

- ты не автор, а свидетель Спринга;
- Redis «потому что вакансии» — красный флаг;
- Kafka, которой нет, заняла место в рассказе у того, что **есть**;

- «обычный CRUD» — сам себя уменьшил, хотя у тебя JWT и чужие задачи не светятся.

Спринг **сам** ничего не расскажет на собесе. Расскажешь ты.

10.2.2 Хороший питч, те же три минуты (укради каркас)

«Я сделал сервис личных задач — REST, Java 21, Spring Boot, PostgreSQL.

Проблема учебная, но настоящая: список дел, который переживает перезапуск, и чужие дела нельзя читать. Поэтому у задачи есть хозяин: id пользователя из логина, не из параметра URL.

Показать можно вот что: регистрация и вход, JWT, CRUD,

пагинация. Миграции Flyway — схема в git, не «у меня в голове». На чтение списка стоит Redis на 30 секунд; ключ учитывает пользователя, фильтры, страницу и сортировку, а после commit записи сбрасываю затронутые варианты; без сброса кэш глючит, я это специально ломал и писал в README.

Уведомления — событие после коммита транзакции и короткий синхронный обработчик, который пишет в лог «сказать человеку». Фоновую очередь пока не делал. Kafka в проде не внедрял: понимаю, чем журнал не HTTP, в репозитории либо маленький брокер, либо конспект — как получилось честно.

Если база недоступна, запросам, которым она нужна, может вернуться 5xx. Попадания в кэш могут ещё отдавать старое до истечения TTL; это не восстановление базы, а ограниченная отсрочка. Compose поднимает приложение, Postgres и Redis, в README curl с мака и с Windows.

Testcontainers уже запускает проверки базы из месяца 3 без «базы у меня на столе». Следующим шагом расширил бы проверки отката и аккуратнее вынес уведомления.»

Используй только те утверждения каркаса, которые подтверждает твой

репозиторий. Если проверки из месяца 3 ещё не готовы, скажи об этом прямо. Засеки. Если больше четырёх минут — режь. Если меньше полутора и нет ни одного **твоего** решения — добавляй не технологии, а «зачем».

Давай медленно
Каркас на бумажке, четыре блока, по одному предложению каждый, потом нарастить:

1. Что это, кому.
2. Одно правило, которым гордишься (свои задачи, транзакция, сброс кэша).
3. Чем это **не** является (не микросервисы, не Kafka в бою).

4. Как поднять и что сломается, если Postgres лёг.

Без экрана. Руки можно. Ноутбук — нет. На собесе экран могут не дать «потыкать», а дать «расскажи». Рассказ должен стоять сам.

10.2.3 Кусок про Java и SQL, чтобы не мычать

Говори короткими правдами. Не лекцией.

- **Соединение таблиц:** `tasks.user_id` ссылается на `users.id`. JOIN — «положи рядом». Без JOIN получишь либо сырой id, либо N+1, который ты уже нюхал в месяце 3.
- **Индекс:** шпаргалка «где у этого пользователя задачи».

Писать чуть медленнее, искать быстрее. На трёх строках Postgres индекс может не взять — это ты тоже уже писал в README.

- **Транзакция:** несколько записей как один жест. Упал — ничего. `@Transactional` на публичном методе сервиса, не на `private`, не «на всякий контроллер».
- **401 и 403:** 401 — «я не знаю кто ты» (нет/протух токен). 403 — «знаю кто, сюда нельзя». Путать их — как путать «шлюз заперт, потому что ты не назвался» и «назваться назвался, но в реактор нельзя».
- **equals и hashCode:** == это тот же ящик. Для ключа карты —

содержимое. `record` уже даёт оба.

Если не помнишь — скажи «сейчас точно не сформулирую, вот как я это делал в коде» и опиши свой файл. «Не знаю» плюс карта лучше, чем сказка про MVCC из середины.

10.2.4 Lisp: крошечный `eval`, ради которого скобки вообще стоят в расписании

Не весь язык. Один вечер радости. Уметь посчитать `(+ 1 2)` и `(- 5 3)` и вложенность. Это не «написать SBCL». Это понять, что скобки — дерево, а дерево можно пройти.

Давай медленно
Форма — либо число, либо список, у которого голова —

операция, хвост — аргументы. Аргументы тоже формы: поэтому вложенность.

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev
(rest form))))
        (cond
          ((eq op '+) (apply #'+
args))
          ((eq op '-) (apply #'-
args))
          (t (error "unknown ~a"
op))))))
```

`(ev 3)` — число, вернуть как есть. `(ev '(+ 1 2))` — не число, голова `+`, хвост `(1 2)`, каждый хвост прогнать через `ev` (это числа), сложить. `(ev '(+ 1 (- 5 3)))` — внутреннее `(- 5 3)` станет `2`, потом `(+ 1 2)`.

Сломай: `(ev '(+ 1 foo))`. Ошибка. Почини либо не принимай символы, либо научи `foo` быть переменной — это уже следующий вечер, не жадничай.

Хочешь чуть больше языка — добавь `*` тем же `cond`. Хочешь унарный минус `(- 3)` — помни, что `apply #'-` в Common Lisp уже умеет. Не тащи `lambda` и `defun` внутрь `ev` в тот же вечер: получится учебник компиляторов, а у тебя завтра письмо. Напёрсток, не станция внутри станции.

Нарисуй дерево на бумажке. Без бумажки половина людей решает, что `eval` — магия REPL. Это не магия. Это `first`/`rest` и

вера, что меньший кусок считается.

Что только что случилось `(ev '(+ 1 2))` → 3. `(ev '(- 5 3))` → 2. `(ev '(+ 1 (- 5 3)))` → 3. Ты только что написал язык размером с напёрсток. Напёрсток важнее, чем «я читал про компиляторы». На собесе можно улыбнуться: «для себя считал скобки». Кто-то улыбнётся в ответ. Этого достаточно.

Квест 22.J1 · практика
Запиши рассказ о проекте. Прослушай. Если каждые пять секунд «ну это Спринг сам» — перескажи, пока не появишься ты.

Квест 22.L1 · Lisp

Крошечный `eval`: `(+ 1 2)`, `(- 5 3)`, вложенность. Не весь язык. Один вечер радости. Это та самая игра, ради которой Lisp вообще стоит в расписании.

Квест 22.J2 · текст
На одной странице: плохой питч (свой, честно, как сейчас говоришь) и хороший (по каркасу из главы). Не выдумывай технологии, которых нет в репозитории. Если Kafka нет — в хорошем питче её нет: есть обработчик события после коммита и слова «фоновую очередь пока не делал».

Воскресная диверсия
Позови живого человека на пятнадцать минут. Пусть задаст «а если база умрёт» и «чем 401

не 403». Живой хуже кота: кот не морщится. Морщины полезны.

10.3 Занятие 23. Письма в неизвестность

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю. Пять–десять откликов в неделю. Не пятьдесят копипастой. Не один «когда будет идеальное письмо». Сопроводительные — про **свой** сервер, не «быстро обучаюсь». Вакансия с Кафкой и кубером: можно писать, если готов сказать правду: «в бою не внедрял, вот монолит и вот очередь». Ложь сгорает на втором вопросе. Иногда на первом.

HR не читает душу. HR читает, есть ли Java, есть ли ссылка, не кричит ли письмо «ГОТОВ К ВЫЗОВАМ!!!». Дальше письмо попадает к человеку, который **может** открыть GitHub. Ему нужна зацепка, не роман.

10.3.1 Образец письма (короткое, с одной правдой)

Тема: Java / junior, task-manager — отклик на «Backend Java, команда X»

```
Здравствуйте.

Видел вакансию «Java-разработчик»
в N: у вас PostgreSQL,
REST и очередь сообщений.

Я полгода собирал учебный
монолит: Java 21, Spring Boot,
PostgreSQL, JWT, тесты, Docker
Compose.
```

```
Задачи только свои, кэш Redis со
сбросом на запись.
Уведомления — синхронный
обработчик события после коммита,
не фоновая очередь.

Kafka в проде не внедрял. Журнал
vs HTTP понимаю, в репозитории
честный конспект / учебный брокер
(как есть в README).
Готов обсудить, чего не хватает
под вашу вакансию.

GitHub: https://github.com/<ник>/
task-manager
Почта / телеграм: ...

Имя
```

Что здесь работает: название конторы, **одна** деталь вакансии, которая у тебя правда есть (Postgres, REST), одна деталь, которой нет — сразу честно. Ссылка. Как связаться.

Что не работает: «являюсь целеустремлённым». «Рассмотрю стажировку и junior и middle». «Kafka, Kubernetes, AWS, Grafana» списком, если из этого у тебя только слово Grafana в заголовке курса.

10.3.2 Плохое письмо (чтобы было с чем сравнить)

```
Здравствуйте!!! Меня зовут Алекс.
Я очень хочу у вас работать,
быстро обучаюсь,
стрессоустойчив, коммуникабелен,
готов к вызовам.
Изучал Java, Spring, Kafka,
Docker, k8s, React.
Прилагаю резюме.
```

Три восклицательных знака — датчик паники. Стек стеной — датчик вранья: React рядом с k8s без ссылки значит «я читал

слова». Нет названия конторы. Нет GitHub. Нет **одной** фразы про то, что ты **сделал**. Такое письмо можно не посылать: экономия чужого времени и своего стыда.

Второй плохой жанр — простыня на три экрана: детство, курсы, «в школе любил математику». Человек с той стороны не нанимает школу. Он нанимает, откроется ли ссылка и не врёшь ли ты про Кафку.

10.3.3 Честные ответы, которые можно произнести вслух

На собесе (и в письме, если спросят) лучше короткая правда, чем длинная сказка. Вот заготовки. Говори своими

словами, не заучивай как клятву.

«Какой у вас опыт с Kafka?»

«В бою не внедрял. Поднимал учебный брокер / читал модель лога: topic, offset, чем это не HTTP. В проекте события идут через Spring-событие после коммита. Если у вас Kafka — мне нужно время въехать в **ваш** кластер и конвенции, не в википедию.»

Если брокер у тебя реально в compose и в логе видны сообщения — скажи так. Не прибедняйся. Не прибедняйся **и** не говори «ну в принципе прод».

«Почему Redis?»

«Список задач читают часто, пишут реже. Закэшировал на 30 секунд, в ключе пользователь, фильтры, пагинация и сортировка. После commit записи сбрасываю затронутые варианты списка. Специально ломал сброс — видел stale. Stampede на трёх пользователях не ловил, но знаю, что короткий TTL и толпа одновременных GET ударят в базу.»

«Микросервисы?»

«Один процесс, одна база. Выкусывать уведомления не стал: иначе письмо уезжает про откат. Когда составной жест переживёт одну транзакцию — тогда и разговор.»

«Почему Lisp в резюме?»

«Сорок минут в день, чтобы не соскучиться и видеть списки. На работу не претендую как лиспер. Могу показать крошечный eval. Если неинтересно — ок, давайте Java.»

«Где вы видите себя через пять лет?»

Не «СТО вашей компании». «Пишу бэкенд, который не стыдно чинить ночью, и понимаю, что делаю». Скучно. Скучное лучше фантастики.

«Есть вопросы к нам?»

Да. «Как выглядит первый месяц джуна?» «Кто ревьюит?» «Прод на чём, и пустят ли к логам?» Вопрос «а у вас Kafka?» имеет смысл, если ты готов к

ответу. Вопрос «сколько платите» — лучше после, когда они сами, или очень спокойно, если это уже второй разговор.

«Расскажи про HashMap»

«Ключ, корзина, в среднем достать быстро. Два объекта с одним и тем же смыслом должны быть равны через equals и иметь один hashCode, иначе карта их разведёт по разным полкам. Изменяемый ключ — боль: положил, поменял поле, не находишь. У себя для id делаю record.»

«Что такое транзакция?»

«Несколько записей в базу как один жест. Либо все, либо ничего. В Spring — `@Transactional` на методе сервиса. Если создать

проект и задачу, а title пустой — хочу, чтобы проекта тоже не было. Это у меня в коде месяца 3, могу открыть.»

Если не можешь открыть — не ссылайся. Скажи идею. Ссылка на файл, которого нет, сгорает быстрее Kafka-лжи.

10.3.4 Куда слать, чтобы не орать в вакуум

Сайты вакансий, телеграм-чаты стажировок, люди с курса, **тёплые** письма («ты же говорил, у вас ищут»). Холодных пять–десять в неделю. Если отправил пятьдесят копий за ночь — ты не герой, ты спам. Спам помечают.

Вакансия «опыт от трёх лет, Kafka, k8s, highload» — можно одно письмо с правдой, если остальное (Java, SQL) совпадает. Нельзя десять часов дописывать Кафку ради этой одной вакансии. Календарь важнее их стека мечты.

Стажировка без зарплаты: решай сам, не геройствуй из стыда. Три месяца «за опыт» без ментора и без кода в git — часто дороже, чем ещё десять писем. Если ментор есть, код есть, Java живая — можно. Если «заваривать кофе и смотреть Jira» — нельзя, даже если на двери написано Kafka.

Квест 23.J1 · текст
Шаблон письма: название конторы + одна деталь из

вакансии, которая у тебя **правда** есть. Не «готов к вызовам».

Квест 23.J2 · практика
Вслух, на диктофон: ответ про Kafka (не внедрял / вот что внедрял), про Redis, про микросервисы. Три минуты на все три. Прослушай. Если звучит извинение за то, что ты жив — перепиши тон. Правда без ежа в голосе.

Так себе идея
Не подставляй в письмо чужой README и чужие curl. На втором собесе попросят «открой свой репозиторий и поменяй вот это». Чужой репозиторий не откроется твоими руками.

10.4 Занятие 24. После провала — в тот же день, не через месяц стыда

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю. Откажут. Иногда тишиной, что хуже слов. Иногда «решили двигаться с другим кандидатом». Иногда на вопросе про `HashMap`, где ты начал сказку. Это не приговор станции. Это датчик.

Список вопросов, где поплыл. Обычно SQL, `HashMap`, транзакции, коды HTTP. Закрой дыру точечно. Не «перечитаю весь учебник с занятия 0». Станцию тоже чинят по датчику, не перестраивают от киля каждую смену.

В тот же день, пока память горячая:

1. Что спросили (буквально, хоть криво).
2. Что ответил (честно, «я мычал»).
3. Какой ответ был бы не стыдный — пять–восемь предложений.
4. Ссылка на свой файл, если тема у тебя в коде есть. Если нет — маленький этюд в `java-basics`, не новый фреймворк.

Карточка живёт в папке `interview-log/` или в бумажной тетради. Бумага не стыдится. GitHub можешь не пушить, если внутри «я тупил». Можешь пушить без тупня: только

вопрос и аккуратный ответ. Как бортовой журнал.

10.4.1 Типичные дыры и чем их не лечить

Поплыл на JOIN — не покупай курс «SQL за 48 часов». Напиши три запроса на **своих** таблицах: задачи пользователя, задачи с именем владельца, количество задач на человека. Вслух.

Поплыл на «что будет, если два запроса сразу» — вернись к счётчику и `@Transactional`, не к книге про JMM целиком.

Поплыл на «расскажи проект» — это занятие 22, не Кафка. Кафка тут ни при чём, сколько бы ни хотелось спрятаться в новую тему.

Тишина после десяти писем — тоже датчик. Проверь: есть ли ссылка, открывается ли compose, не ругается ли README на Windows. Иногда рынок просто медленный. Иногда иллюминатор грязный. Грязь ты чинишь. Медленный рынок — письмами дальше, не «пережду в стыде».

10.4.2 Образец карточки (можно почти скопировать форму)

```
Дата: 2026-08-13
Где: контора N, скрининг
Вопрос: чем 401 отличается от 403?
Что сказал: «ну оба ошибка»
Как надо:
  401 — не представился (нет токена, протух).
  403 — представился, сюда нельзя (чужая задача).
```

```
  В task-manager чужой id даёт
403 или 404 —
  смотри TaskService.java, метод
getOwned.
Этюд: нет, код уже есть.
```

Три таких листа за месяц — уже не «я вечно туплю», а карта дыр. Карта чинится. Вечное тупление — нет.

Квест 24.J1 · Java
Карточка на дыру: вопрос, ответ на пять–восемь предложений, ссылка на свой файл, если есть. Это твой новый бортовой журнал.

Квест 24.J2 · текст
Три карточки заранее, до первого провала: 401 vs 403; зачем индекс; что делает транзакция на create проекта с задачей. Пиши как говоришь,

не как Википедия. Потом, когда всплывёт на собесе, будет не «надо было готовиться», а «я это уже произносил».

Закон станции
Запрещено после отказа: удалять репозиторий, молчать месяц, начинать «полное переписывание на микросервисы». Разрешено: карточка, один этюд, следующее письмо на этой неделе.

10.5 Занятие 25. Код руками, без волшебной кнопки

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю.

Сорок пять минут, задача лёгкая, стандартная библиотека, хоть в блокноте. На Windows — Notepad++ или IDEA без автодополнения, если хватит духа. Вслух: что инвариант, какой пример, насколько это быстро.

Вайтборд (или Miro, или лист A4) — не место показывать, что ты помнишь стримы наизусть. Место показать, что ты **думаешь**: пример, край, сложность «пойду по списку / положу в карту».

Три учебных, злых по таймеру:

- анаграммы: одни и те же буквы, порядок другой;

- слить два уже отсортированных массива в один отсортированный;
- уникальные почты (как скажет задача: точки, плюс — читай условие, не угадывай Gmail).

Давай медленно
Пока таймер не тикает, отрепетируй ритуал:

1. Повтори условие своими словами. Если не повторил — уже ошибка, решил не ту задачу.
2. Пример: `"ab"`, `"ba"` — да; `"ab"`, `"abc"` — нет.
3. Край: пустая строка, один символ, разный регистр — спроси, или выбери и скажи вслух.

4. Идея: сортировка символов или частотный массив. Сложность.
5. Код. Не молчи. «Сейчас положу частоты в HashMap» — слышно, что живой.
6. Прогони пример пальцем. Найди дыру сам, раньше интервьюера.

Если застрял на пять минут — скажи «вижу два пути, беру более простой, вот этот». Молчание читается как «я умер». Ошибка в коде читается как «человек, с которым можно чинить».

На Java 21 можно `record`, можно массивы `char[]`. Не надо `Stream` ради Stream. На вайтборде стрим часто кривой и его не прогонишь.

Lisp-версия анаграмм — тот же жест, другие скобки. На собесе это шутка в конце, не основной номер. Основной номер — Java, которую они нанимают.

10.5.1 Анаграммы пальцем, пока таймер ещё не злой

Условие: две строки состоят из одних и тех же букв.

```
static boolean anagram(String a,
String b) {
    if (a.length() != b.length())
{
        return false;
    }
    char[] x = a.toCharArray();
    char[] y = b.toCharArray();
    Arrays.sort(x);
    Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

Вслух: «разная длина — сразу нет. Иначе сортирую символы,

сравниваю массивы». Пример: `listen` / `silent` — да. Сложность: сортировка, не «я пройду все перестановки». Перестановки — путь геройствовать и умереть на `abcd`.

Второй путь — частоты в `HashMap<Character,Integer>`: плюс по первой строке, минус по второй, все нули. На вайтборде сортировка короче. На собесе скажи оба, возьми один, напиши один. Два недописанных пути хуже одного рабочего.

Слить массивы: два индекса `i` и `j`, третий массив, кто меньше — того клади, двигай. Хвост допиши. Не `sort` всего после `concat`: тебя как раз проверяют,

умеешь ли пользоваться тем, что **уже** отсортировано.

Мак, Windows, WSL
Тренируйся в той раскладке, в которой пойдёшь. Если собес на Windows в их офисе, а ты всю жизнь на маке — один вечер в Notepad не убьёт. Скобки и точка с запятой те же. Кнопки другие. Лучше выругаться дома, чем в комнате с маркером.

Квест 25.J1 · Java
Три с таймером: анаграммы; слить два отсортированных массива; уникальные почты. Таймер злой. Так и надо.

Квест 25.L1 · Lisp
Те же анаграммы в Lisp. Потом можно на собесе сказать: «я это ещё и скобками делал». Кто-то улыбнётся. Этого достаточно.

10.6 Занятие 26. Ходить, а не «ещё чуть-чуть Кафку»

Ежедневная практика: Lisp 40 мин · Java 120 мин. Тема и задания — на неделю. Мерило месяца — календарь слотов, не новая глава. Lisp сорок минут. Воскресенье — прогулка. Серьёзно, прогулка.

«Ходить» значит: отклик ушёл, слот стоит, ты выспался, README открывается, питч произносился вслух хотя бы дважды. Не значит: выучил ещё один брокер за ночь перед звонком. Ночной брокер на собесе пахнет страхом. Страх слышно.

10.6.1 Как выглядит неделя, в которой ты уже на рынке

Понедельник: два отклика по шаблону, вакансии разные, в каждом письме одна правда из **их** текста.

Вторник: карточка из прошлого отказа или тренировочный вопрос. Сорок пять минут вайтборда.

Среда: слот, если дали. Если не дали — ещё два письма. Не «жду судьбу».

Четверг: Lisp, мелкий баг в task-manager, который ты обещал себе после питча («проверка отката в Testcontainers», «понятнее ошибка 401»). Один баг, не рефакторинг вселенной.

Пятница: добить сломанное. Новых учебников не открывать. Особенно про Kubernetes.

Суббота: проект как проект, не как жертва собесу.

Воскресенье: ноги, воздух, станция «МОДУЛЬ» без ноутбука. Мозг без воздуха врёт, что «ещё одна глава спасёт». Не спасёт.

10.6.2 В сам день собеса

Ноутбук заряжен. IDE открыта на проекте, если просят «показать». Zoom/Meet проверен вчера, не за пять минут. Наушники. Стакан воды. README сам знаешь наизусть — не потому что заучил, потому что писал.

Опоздал на три минуты — напиши. Молчание хуже трёх минут.

Спросили то, чего не знаешь: «не знаю в бою, вот как я это понимаю, вот где вранье в моей модели». Потом можно спросить «как у вас». Диалог. Не экзамен на отчисление, хотя ощущение то же.

После звонка — карточка в тот же день. Даже если кажется, что «всё хорошо». Хорошо тоже надо уметь повторить.

10.6.3 Звонок, офис, белая доска в чужой комнате

Zoom: камера на уровне глаз, не в ноздри. Фон спокойный. Телефон в режиме «Не беспокоить»; если он нужен как

запасная раздача интернета, оставь мобильные данные включёнными и заранее проверь подключение. Ссылка на Meet открыта за десять минут, не за десять секунд: обновление браузера любит совпасть с твоим входом.

Офис: паспорт, если пропуск. Блокнот. Своя ручка. Вода. Наушники не нужны, если они зовут в переговорку. Ноут — если сказали «приноси», и зарядка к **их** розеткам. В России розетки обычно те же, в коворкинге иногда нет.

На вайтборде пиши крупно. Мелкий почерк в углу — «я стесняюсь своего кода». Сверху условие своими словами. Слева пример. Справа код. Сотри не из

паники, из «этот кусок врёт, вот новый». Попроси минуту подумать — нормально. Попроси подсказку после семи минут честной борьбы — тоже нормально. Попроси подсказку через тридцать секунд молчания — рано.

Если экран шарят и просят открыть IDEA: отключи автодополнение, если хватит духа, или оставь и **не** принимай первую глупость, которую IDE тебе подсказывает. Интервьюер видит, когда ты жмёшь Tab не глядя.

10.6.4 Когда книга кончилась

Ты закончил книгу, когда:

- на GitHub виден путь от Hello до сервера с базой;

- свой монолит чинишь без театра;
- честно говоришь, чего не делал.

Оффер — не кнопка в конце главы. Это вероятность. План её поднимает. Дату не обещает. Станция «МОДУЛЬ» тоже никто не обещал, что не развалится — а она всё ещё там. Как-то.

Вечер перед слотом: не Кафка. README вслух. Один curl. Питч один раз на диктофон. Сон. Если не спится — бумажка с четырьмя блоками питча под подушку, не статья на Хабре «как устроен Raft». Raft подождёт вторника после отказа. Или после оффера. Ему всё равно.

Если письма молчат — не обязательно ещё полгода Кафки. Дальше в книге короткий запасной люк: Android, калькулятор, студия с developer.android.com. Java та же. Экран другой.

Квест 26.J1 · практика
Календарь на две недели: слоты откликов (даты), один слот «репетиция питча», один слот «три задачи с таймером». Повесь на стену или в календарь, который ты **видишь**. Невидимый план — это мечта, не вахта.

Квест 26.L1 · Lisp
Сорок минут в тот день, когда отказали. Любой этюд станции, хоть `ev`, хоть датчик кислорода.

Скобки не знают про HR. Это и есть смысл сорока минут.

Спрятать на GitHub
В профиле GitHub: закрепи `task-manager`. Описание профиля — одна строка, не цитата про страсть к коду. Ссылка в резюме открывается. Проверь с телефона.

Профиль без закреплённого репозитория — чердак. Человек с той стороны не будет искать `task-manager` в списке из двадцати форков чужих туториалов. Закрепи. Убери из закреплённого форк `spring-petclinic`, если ты его не чинил. Форк без коммитов — не проект.

Проверь ссылки с телефона: GitHub, резюме-PDF, почта в профиле. Рекрутер часто сидит

в метро, не за большим монитором. Если README на узком экране — стена, режь. Если почта с опечаткой — письма «ушли в космос», станция ни при чём.

И на этом книга как учебник кончается. Дальше — чужие комнаты, чужие вопросы, свои файлы. Станция «МОДУЛЬ» никуда не денется. Скобки тоже. Иди.

Если страшно — нормально. Страшно чинить реактор в первый месяц, страшно жать Send. Жать всё равно. Письмо, которое не ушло, не открывается. Станция, которую не включили, не пищит датчиками — и это не

спокойствие, это тишина пустого коридора.

Чай. Send. Карточка. Скобки.

Так себе идея
Не исчезай в «перепишу всё на Kotlin + Kafka + k8s за август». Август кончится, писем не будет, монолит будет мёртв в ветке `rewrite`. Рынок берёт живое. Живое — то, что поднимается сегодня.

11 Если на бэкенд не берут

Иногда рынок бэкенда смотрит на джуна как шлюз на метеорит: вежливо, но не открывается. Kafka в вакансии, «опыт от года», тишина после десяти писем. Это не значит, что полгода выкинул. Java никуда не делась. Просто дверь другая: **карман**. Телефон в руке — тот же компьютер, только без серверной комнаты и с кнопкой «назад».

Android-приложения давно пишут на Kotlin, но Java там жива, и студия её ест. Ты уже знаешь классы, `if` и «почему подчёркнуто ошибкой». Этого хватит, чтобы собрать

калькулятор и не умереть. Если пойдёт — Kotlin догонишь за пару недель: он с Java из одной семьи, только меньше воды.

Lisp на телефон ставить не обязательно. Сорок минут скобок можно оставить. Станции «МОДУЛЬ» всё равно, с какого экрана ты чинишь датчики.

Это не «бросай бэкенд». Это запасной люк. Калькулятор за вечер дешевле, чем ещё три месяца «подкручу Kafka и точно возьмут». Телефоны людям нужны каждый день. Серверам — тоже, но HR серверов капризнее.

11.1 Где взять студию (и почему первый вечер — это боль)

Сайт: developer.android.com/studio — официально, для мака, Windows и Linux. Не качай «android studio cracked» с четырнадцатой страницы поиска. Это не экономия, это приключение с вирусом. Станция и так на изоленте, вирус ей не нужен.

Ставится долго: сама студия, Android SDK, эмулятор, образы системы. Места на диске — гигабайты, не «ещё один VS Code». Двадцать гигабайт — нормальный счёт, не баг. Если диск на сто, а занято девяносто восемь — либо чисти, либо USB-телефон без эмулятора.

Эмулятор на забитом диске умирает стыдливо, ошибками про android-30, которые не про android-30.

Мак, Windows, WSL
На маке студия родная, Apple Silicon (M1/M2/M3) хочет образ эмулятора **arm**, не старый x86 «потому что в гайде 2019». Студия обычно сама подсказывает. Если скачал x86 на M-чип — будет либо медленно через трансляцию, либо никак. Не геройствуй.

На Windows это **родное** окно, не WSL: эмулятору нужен Hyper-V или собственный движок, в Ubuntu внутри Windows он часто обижается. WSL отлично гоняет твой Spring. Телефонный эмулятор — нет, не путай двери.

Linux: можно, но драйверы GPU и KVM — отдельная вахта. Если Linux у тебя «чтобы было», а основной ноут Windows — ставь студию на Windows. Одна боль лучше двух.

Первый запуск: мастер, логин в Google необязателен для калькулятора (можно пропустить). Потом SDK Manager докачает ещё кусок интернета. Чай. Не два чая и «я, наверное, сломал» — прогресс-бар Android так дышит.

New Project → **Empty Views Activity** (не Compose, не пугайся слова Jetpack пока). Language: **Java**. Minimum SDK — как студия предлагает, не самый древний и не самый свежий «только для пикселей из

будущего». Имя пакета вроде `com.example.calc` — потом заменишь, если стыдно `example`. Для учёбы `example` не стыдно.

Проект впервые индексируется минутами. Нижняя полоска Gradle шевелится. Красный `R` в первые две минуты — часто «ещё не проснулась», не «ты идиот». Build → Make Project. Подожди. Если после Make всё ещё красный `R.id` — Build → Clean Project, потом снова Make. Иногда студия просто не проснулась. Как датчик на «МОДУЛЬ».

11.1.1 Эмулятор против провода: два способа увидеть кнопку

Эмулятор: Device Manager → Create → какой-нибудь Pixel без

16K экрана и без складных крыльев. System image — свежий стабильный, с Google APIs, не «Android Canary с Марса». Первый старт — чай. Иногда два. Холодный старт эмулятора на слабом ноуте — это маленький грех индустрии, не твой.

Если ноутбук слабый, вентилятор воет, эмулятор рисует слайд-шоу — **не надо геройствовать**. Включи USB-debugging на своём телефоне и гоняй на железе:

1. Settings → About phone → семь раз по Build number. Появится «вы разработчик». Да, семь. Это не шутка 2012 года, это всё ещё так.

2. Developer options → USB debugging.
3. Кабель, которым **ходит файл**, не «только зарядка» из ящика.
4. Разрешить отладку на экране телефона, когда спросит. Галка «всегда с этого компьютера» — по вкусу.
5. В студии сверху устройство должно сменить имя с эмулятора на модель телефона.

Не стыдно. На настоящей станции тоже сначала смотрят датчик в коридоре, а не строят симулятор коридора.

Так себе идея
На работе телефон —
отдельный мир:

корпоративные профили, «нельзя». Для учёбы свой аппарат ок. Не ставь debug на телефон бабушки «на минутку». Бабушка не подписывала этот квест.

Если студия не видит телефон: другой кабель, другой порт, на Windows — драйвер производителя, режим File transfer / PTP, не Charge only. Это самая скучная боль главы. Скучная боль лечится перебором, не новым учебником.

11.1.2 Лицензии, гипервизор и Gradle, который «ещё качает»

Первый билд может полчаса качать интернет в `~/.gradle`. Это не зависло, если полоска шевелится. Если не шевелится

десять минут — смотри сеть, прокси конторы, VPN, который режет dl.google.com. На Windows антивирус иногда долго проверяет jar в кэше Gradle. Сначала проверь его журнал и официальные рекомендации для доверенной установки Studio. Не отключай защиту и не исключай весь кэш «на всякий случай»; на рабочем компьютере разберись с администратором.

SDK licenses: студия обычно спрашивает Accept. Если в логе `You have not accepted the license agreements` — SDK Manager, галки, Accept. В командной строке когда-нибудь встретишь `sdkmanager --licenses`. Сегодня хватит кнопки в GUI.

Эмулятор на Windows без виртуализации — слайд-шоу или отказ. В BIOS/UEFI включают VT-x / AMD-V. На домашнем ноуте это иногда «я не знаю, что такое BIOS». Документация Microsoft про Hyper-V и WSL2 уже есть на машине, если ты ставил Docker Desktop: часто виртуализация уже включена. Если Docker жив, а эмулятор нет — не обязательно BIOS; смотри, не занял ли Hyper-V всё, и какой образ скачал.

На маке «эмулятор не стартует» часто лечится: докачать system image, Cold Boot Now в Device Manager, не паниковать на первом `Waiting for target device`. Cold boot — как перезапуск станции, не как «удалить всё».

11.1.3 Ошибки, которые не значат «ты идиот»

`SDK location not found` — студия не знает, где SDK. File → Settings → Appearance & Behavior → System Settings → Android SDK (на маке это Android Studio → Settings). Путь должен существовать. Если переехал на новый диск — путь старый, студия ругается. Укажи новый, не переустанавливай всё с нуля.

`Failed to install the following Android SDK packages` — сеть, лицензия, или диск кончился. Докачай через SDK Manager. Не качай zip «sdk-tools» с четырнадцатой страницы.

`INSTALL_FAILED_INSUFFICIENT_STORAGE` на эмуляторе — можно создать новый AVD побольше.

Wipe Data в Device Manager сбрасывает существующий эмулятор: установленные в нём приложения и их данные удалятся. Сначала сохрани всё нужное. Телефон: почисти кэш, это уже быт, не Java.

`Duplicate class` / Gradle conflict — ты добавил библиотеку, которую студия уже тащит. Для калькулятора библиотек почти не нужно. Если полез в Firebase «на пять минут» — вот оно. Выкинь, пока калькулятор не считает.

`R` красный после переименования id в XML — Make Project. Имя в XML и `R.id.имя` должны совпасть буква в букву. `plus` и `Plus` — разные. Android не прощает регистр,

как Java классы, только злее, потому что ошибка всплывает не сразу.

Invalidate Caches — последняя кнопка, не первая. Сначала Make, Clean, «тот ли модуль запускаешь». Invalidate как перезапуск станции целиком: помогает, когда уже ничего не понятно, и иногда не помогает.

Ещё: запускаешь не тот Run Configuration — зелёная стрелка на библиотеке, не на app. Слева выбери app. Или устройство «No devices»: эмулятор не дождался, телефон отвалился. Это не Java. Это вилка не в той розетке.

11.2 Калькулятор на коленке

Два числа, плюс и минус, потом умножить и разделить, ответ на экране. Не банк и не Telegram. Зато ты увидишь: кнопка → твой Java-код → цифры. Как `Scanner` в консоли, только пальцем.

В `activity_main.xml` (вкладка Code, не только Preview):

```
<?xml version="1.0"
encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://
schemas.android.com/apk/res/
android"

android:layout_width="match_paren
t"

android:layout_height="match_pare
```

```
nt"

android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/a"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:hint="первое
число"

android:inputType="numberDecimal|
numberSigned" />

    <EditText
        android:id="@+id/b"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:hint="второе
```

```
число"

android:inputType="numberDecimal|
numberSigned" />

    <Button
        android:id="@+id/plus"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:text="+" />

    <Button
        android:id="@+id/minus"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:text="-" />

    <Button
        android:id="@+id/mul"
```

```
android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:text="×" />

    <Button
        android:id="@+id/div"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:text="÷" />

    <TextView
        android:id="@+id/out"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt"
        android:textSize="24sp"
        android:text="?" />
</LinearLayout>
```

`LinearLayout` — коробки столбиком. `orientation="vertical"` — как список в коридоре. `EditText` — поле, куда тыкают. `inputType` просит числовую клавиатуру со знаком и дробной частью, не весь роман. Десятичный разделитель на клавиатуре зависит от языка и настроек. `TextView` — только показывает, пальцем туда не печатают.

Вкладка Preview ошибается реже, чем раньше, но Код — источник правды. Если Preview красивый, а на телефоне нет кнопки — ты смотрел не тот XML.

`dp` — density-independent pixels, пиксели, независимые от плотности экрана: чтобы

кнопка не была микроскопической на одном телефоне и огромной на другом. `sp` — scale-independent pixels, единицы размера текста, которые учитывают и плотность, и выбранный пользователем масштаб шрифта. Размер текста в `dp` лучше не ставить без нужды: человек с плохим зрением имеет право на крупные буквы. `match_parent` — забери ширину/высоту родителя. `wrap_content` — по содержимому. Если все `wrap_content` в столбике — поля нормальные. Если `ListView` тоже `wrap`, он может сжаться и ты решишь, что список «не работает».

В MainActivity.java — тот же Java, только вход не Scanner, а поля на экране:

```
package com.example.calc;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import
androidx.appcompat.app.AppCompatA
ctivity;

public class MainActivity extends
AppCompatActivity {
    private EditText a;
    private EditText b;
    @Override
    protected void
onCreate(Bundle
savedInstanceState) {

super.onCreate(savedInstanceState
);

setContentView(R.layout.activity_
```

```
main);

        a = findViewById(R.id.a);
        b = findViewById(R.id.b);
        TextView out =
findViewById(R.id.out);
        Button plus =
findViewById(R.id.plus);
        Button minus =
findViewById(R.id.minus);
        Button mul =
findViewById(R.id.mul);
        Button div =
findViewById(R.id.div);

        plus.setOnClickListener(v
-> show(out, num(a) + num(b)));

minus.setOnClickListener(v ->
show(out, num(a) - num(b)));
        mul.setOnClickListener(v
-> show(out, num(a) * num(b)));
        div.setOnClickListener(v
-> {
            double y = num(b);
            if (y == 0) {

out.setText("нельзя");
```

```
                return;
            }
            show(out, num(a) /
y);
        });
    }

    private void show(TextView
out, double value) {

out.setText(String.valueOf(value)
);
    }

    private double num(EditText
field) {
        String s =
field.getText().toString().trim()
;
        if (s.isEmpty()) {
            return 0;
        }
        try {
            return
Double.parseDouble(s);
        } catch
(NumberFormatException e) {
            return Double.NaN;
```

```
        }
    }
}
```

Пакет `com.example.calc` замени на тот, который студия выбрала сама. Зелёная стрелка. Если красным `R.id` — Clean, иногда студия просто не проснулась.

11.2.1 Пустое поле, буквы и деление на ноль — это не «потом»

Пустой ввод: человек открыл приложение и сразу ткнул `+`. Без проверки `parseDouble("")` бросает `NumberFormatException`, активность падает, Android рисует «приложение остановлено». На станции так датчик кислорода не чинят: не взрывают отсек, если кто-то забыл цифру.

В коде выше пустое → `0`. Это учебный жест, как в консоли «нуль если молчит». Можно иначе: писать в `out` «введи два числа» и не считать. Выбери одно и будь верен. Главное — **не падать**.

`NumberFormatException`: `inputType` не железный занавес. С клавиатуры всё равно иногда пролезает странное, с копипаста — тем более. `parseDouble` имеет право обидеться. Поймай. `NaN` в ответе лучше, чем краш; ещё лучше — слово «ерунда» на экране. `Double.NaN` в `+` заражает сумму, пользователь увидит `NaN`. Можно проверить `Double.isNaN` и написать «не число». Сделай, когда калькулятор уже складывает.

В этом упражнении `Double.parseDouble` ждёт десятичную точку: вводи `1.5`, не `1,5`. Локальную запятую он сам не переводит, даже если клавиатура её предлагает. Для локализованного приложения нужен разбор чисел с учётом локали; это отдельный шаг.

Деление на ноль с `double` не бросает исключение: ненулевое конечное число, делённое на ноль, даёт `Infinity` или `-Infinity`, а `0.0 / 0.0` даёт `NaN`. Проверка `y == 0` выше останавливает оба случая. Пользователю лучше «нельзя», чем такой ответ без объяснения. Целочисленное деление мы не используем: поля десятичные.

Давай медленно

Пальцем, не глазами.

1. Введи `2` и `3`, нажми `+`. Должно быть `5.0` или `5`.
2. Сотри оба поля, нажми `+`. Не должно вылететь. Либо `0.0`, либо твоя надпись.
3. Введи `8` и `0`, нажми `÷`. «нельзя», не краш, не `Infinity`.
4. Умножь `1.5` и `2`. Если получилось `2` — ты где-то отрезал дробь до `int`. Вернись к `double`.

Квест A.J1 · Java
Кнопки `×` и `÷` живые. Деление на ноль — текст «нельзя», приложение живо. Скрин в README репозитория `android-calc`.

Квест A.J2 · Java

Пустое поле не роняет активность. Поймай `NumberFormatException`. Напиши в `out`, что ввод плохой, или аккуратно считай пустое за ноль — но тогда **напиши в README**, какое правило. Скрытое «пустое это ноль» без слов — ловушка для будущего себя.

Можно явно, без NaN:

```
private Double
parseOrNull(EditText field) {
    String s =
field.getText().toString().trim()
;
    if (s.isEmpty()) {
        return null;
    }
    try {
        return
Double.parseDouble(s);
    } catch
```

```
(NumberFormatException e) {
        return null;
    }
}
```

В слушателе: если `null` — `out.setText("введи числа")` и `return`. Пустое и «асдф» ведут себя одинаково честно. Нуль как «человек молчит» иногда врёт: `2+` пустое стало `2+0`. Для калькулятора станции лучше показать ошибку и выйти из слушателя через `return`, чем додумать число или уронить приложение.

11.3 Жизненный цикл: зачем активность не «просто main»

В консоли был `main`. Вызвали — живёт, пока не кончится. На

телефоне экран живучее и капризнее: человек свернул, позвонили, повернул телефон, системе мало памяти. Активность — не вечный процесс. Это смена на вахте с журналом «сейчас я на экране / уже нет».

Грубо, без диссертации:

- `onCreate` — родилась. Здесь `setContentView`, здесь находишь кнопки. При создании каждого нового экземпляра, в том числе после пересоздания экрана.
- `onStart` — сейчас меня **могут** увидеть.
- `onResume` — я на переднем плане, тыкают в меня.

- `onPause` — активность выходит из состояния Resumed, например при переходе к другой активности; часть экрана ещё может быть видна. Здесь не надо считать большие суммы, здесь «на паузу».
- `onStop` — меня не видно (ушли на другой экран, нажали Home).
- `onDestroy` — экземпляр активности завершается или пересоздаётся, например при повороте. Если система убивает весь процесс ради памяти, этот callback не гарантирован.

Поворот экрана по умолчанию часто **убивает** активность и создаёт заново. Поэтому поле,

которое ты держал в обычной переменной, может забыть цифру. Для калькулятора на пять минут это терпимо. Для списка задач — уже обидно. Потом будут `ViewModel` и «состояние переживает поворот». Сегодня достаточно знать имя беды: «я повернул телефон, и счётчик обнулился — это не магия: создали новую Activity и вызвали `onCreate`».

Давай медленно
Поставь в каждый метод лог:

```
Log.d("CALC", "onCreate");
```

То же в `onStart`, `onResume`, `onPause`, `onStop`. Logcat внизу студии, фильтр `CALC`. Нажми Home. Вернись. Поверни телефон. Прочитай ленту. Это не ритуал

ради ритуала: ты увидишь, что Home не равен «процесс умер», а поворот часто равен «умерла активность, родилась новая».

Если Logcat пустой: справа вкладка Logcat, устройство выбрано то же, что зелёная стрелка, уровень Debug, не Error. Пакет приложения в фильтре, если логов слишком много: эмулятор болтливый, как датчики «МОДУЛЬ» в полнолуние. `Log.d` не виден на уровне Error — ты смотришь не ту полку.

Официально и без наших шуток: гид по lifecycle в документации Android. Английский. Мы предупреждали ещё в предисловии.

Поворот и смерть активности — не единственная засада. Система может убить процесс, когда ты давно в фоне, и потом вернуть тебя в `onCreate` с `savedInstanceState`. Текст `EditText` с уникальными id система обычно восстанавливает сама. Собственное временное состояние, например результат расчёта, клади в `Bundle` в `onSaveInstanceState` и читай в `onCreate`, если `savedInstanceState != null`. Это не замена сохранению важных данных на диск. Это уже «состояние переживает вахту». Не обязательно в первый вечер. Обязательно знать, что переменная в классе — не сейф.

Квест A.J3 · Java

Логи на `onCreate` / `onStart` / `onResume` / `onPause` / `onStop`. README: что печатается, если нажать Home и вернуться, и что — если повернуть телефон. Своими словами, пять предложений. Не копипаста из гида.

11.4 Список задач на телефоне — набросок, не третий учебник

Калькулятор считает. Список дел ты уже писал в консоли и по HTTP. На телефоне это те же `ArrayList<String>`, только список **рисуется**. Не тащи сразу Firebase, карты и «как в Instagram». Сначала список, который **показывается**.

Набросок на один-два вечера, не на архитектуру Google:

1. Новый экран или тот же: `EditText` «название», кнопка «добавить», `ListView` или `RecyclerView`. Для первого раза `ListView` + `ArrayAdapter` проще. `RecyclerView` правильнее и злее. Если `ListView` заработал — ты уже победил. Потом `RecyclerView`.
2. В памяти `ArrayList<String> tasks`. Добавил — `adapter.notifyDataSetChanged()`. Само сворачивание список обычно не стирает. Но пересоздание активности или смерть процесса потеряет состояние, если его не сохранить.

3. Хочешь бессмертие — `SharedPreferences` (маленькая кучка ключ-значение) или файл, как `tasks.txt`. SQLite / Room — когда список живой и ты злишься на файл. Не в первый вечер.
4. Потом можно тыкать в элемент и помечать done. Потом — ходить в **твой** `task-manager` по HTTP. Это уже «телефон как клиент к твоему серверу». Красиво для портфолио. Сначала пусть список живёт без сети: в метро сервер «МОДУЛЬ» не отвечает.

XML для наброска (не копируй слепо id, если уже заняты):

```
<EditText
    android:id="@+id/title"
    android:hint="задача"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt" />

<Button
    android:id="@+id/add"
    android:text="добавить"

android:layout_width="match_paren
t"

android:layout_height="wrap_conte
nt" />

<ListView
    android:id="@+id/list"

android:layout_width="match_paren
t"
    android:layout_height="0dp"
    android:layout_weight="1" />
```

`layout_weight` в вертикальном `LinearLayout` — «забери остаток экрана». Без него список иногда сжимается в полоску и ты думаешь, что Android сломан. Сжат layout, не Android.

В Java — набросок, не священный код:

```
List<String> tasks = new
ArrayList<>();
ArrayAdapter<String> adapter =
new ArrayAdapter<>(
        this,
android.R.layout.simple_list_item
_1, tasks);
list.setAdapter(adapter);
add.setOnClickListener(v -> {
    String t =
title.getText().toString().trim()
;
    if (t.isEmpty()) {
        return;
    }
    tasks.add(t);
```

```
adapter.notifyDataSetChanged();
    title.setText("");
});
```

Пустую задачу не клади. Ты это правило уже ставил в сервисе на сервере. Здесь то же: вахтёр не принимает пустую записку. Здесь используем `trim()` и `TextUtils.join`: наличие метода в JDK на компьютере ещё не обещает его наличие на старом Android. `String.strip()` требует API 33, `String.join()` — API 26, если не настроен подходящий desugaring.

Хочешь, чтобы список пережил кнопку «назад» — набросок на `SharedPreferences`:

```
prefs.edit().putString("tasks",
android.text.TextUtils.join("\n",
tasks)).apply();
```

Прочитать при старте: `getString`, `split`, в список. Это не база. Это `tasks.txt` в кармане. Для десяти дел хватит. Для тысячи — Room, когда доживёшь. Не пиши свой SQL «потому что в бэкенде Postgres»: на телефоне другой быт, другой диск, другой «процесс убили».

Сеть к своему `task-manager`: на эмуляторе `localhost` — это **сам эмулятор**, не твой Мак и не Docker. Часто пишут `10.0.2.2` как «хост машины с эмулятором». На настоящем телефоне в одной Wi-Fi — IP компьютера в локалке, и сервер должен слушать не только `localhost`. Это

та же ловушка, что `db` и `localhost` в Compose, только в кармане. Пока список без сети — эту яму себе ещё не ставишь. Когда полезешь — README, не «ну оно же локально».

Cleartext HTTP: Android по умолчанию не любит `http://` без TLS. Учебный сервер на `http://10.0.2.2:8080` может получить `Cleartext HTTP traffic not permitted`. Для учёбы в `usesCleartextTraffic` или network-security-config — да, с пометкой в README «только локалка». В прод это не несут. Как учебный пароль `task/task`: можно дома, нельзя на Землю.

Квест A.J4 · Java
Набросок списка: поле, кнопка, `ListView`, три задачи пальцем.

Скрин. Не обязательно переживать поворот и диск. В README честно: «пока только память». Честность снова красивее галочки «почти как Todoist».

Спрятать на GitHub
Репозиторий `android-calc`. README: как открыть в Android Studio, какой API, скрин. Это тоже портфолио — «я не только JSON в Postman тыкал». Если дошёл до списка — либо второй репозиторий `android-tasks`, либо папка в том же. Не смешивай без нужды калькулятор и список в одном хаосе без слов.

Так себе идея
Не тащи сразу Firebase, карты и «как в Instagram». Сначала калькулятор, который **считает**.

Потом список задач на телефоне — ты его уже писал в консоли.

11.5 Манифест, строки и вторая дверь

`AndroidManifest.xml` — паспорт приложения. Без него система не знает, как зовут активность и с какой начать. Студия его пишет сама. Ты туда заходишь, когда:

- добавляешь **вторую** активность — её надо прописать, иначе «activity not found»;
- нужен интернет: `<uses-permission android:name="android.permission.INTERNET" />`;

- хочешь, чтобы иконка и имя на рабочем столе были не `com.example.calc`.

Имя на рабочем столе лучше не хардкодить в манифесте строкой. Клади в `res/values/strings.xml`:

```
<resources>
    <string
name="app_name">калькулятор
МОДУЛЬ</string>
    <string name="hint_a">первое
число</string>
</resources>
```

В XML экрана: `android:hint="@string/hint_a"`. Зачем: завтра перевод, послезавтра крупный шрифт, и ты не ползаешь по всем layout. Для учёбы две строки — уже жест взрослого. Для Instagram —

обязательно. Мы не Instagram. Две строки всё равно стоит.

Вторая активность — вторая вахта. New → Activity → Empty Views Activity, имя `AboutActivity`. В манифесте студия обычно допишет сама. Переход:

```
startActivity(new Intent(this,
AboutActivity.class));
```

`Intent` — записка: «открой вот этот класс». Можно положить лишние данные:

```
Intent i = new Intent(this,
AboutActivity.class);
i.putExtra("last",
out.getText().toString());
startActivity(i);
```

На той стороне: `getIntent().getStringExtra("last")`. Это параметры запуска экрана,

не HTTP и не база данных. Не используй Intent как хранилище постоянно меняющегося списка. Для состояния экрана — `onSaveInstanceState` или ViewModel; для долговременного хранения — файл или база.

Давай медленно
Кнопка «о станции» на калькуляторе. Второй экран: две фразы и последнее посчитанное число. Назад — системная стрелка, ты её не программируешь в первый вечер. Если число на втором экране пустое — `putExtra` забыл или ключ опечатал. Ключи — строки, Java их не проверит. Опечатка `last` / `Last` — классика, как `R.id.plus` и `Plus`.

Квест A.J5 · Java
Вторая активность, Intent, `putExtra` с последним ответом калькулятора. Скрин двух экранов. Если системная «назад» не возвращает на калькулятор — ты, скорее всего, ушёл в какую-то странную launchMode в манифесте. Верни как студия поставила.

11.6 Gradle: почему зелёная стрелка иногда полчаса думает

Проект Android — не один `javac Hello.java`. Это Gradle: скрипт, который качает SDK, зависимости, ресурсы, клеит

APK. Два файла, которые ты трогаешь:

- `build.gradle.kts` (или `.gradle`) **модуля** `app` — `minSdk`, `targetSdk`, зависимости;
- иногда корневой — репозитории, откуда качать.

`minSdk` — самый старый Android, на который ты согласен. Чем ниже, тем больше бабушкиных телефонов, тем больше «этого API нет». Для учёбы оставь то, что студия предложила. Не ставь 14 «чтобы всех покрыть»: покрыть ты не покроешь, зато `HttpURLConnection` внезапно из 2009.

Зависимости для калькулятора почти не нужны. `implementation(...)` появляется,

когда полезешь в сеть или в Room. Каждая строка — обещание качать интернет и однажды словить конфликт версий. Пустой калькулятор с двадцатью библиотеками — станция без стен, но с каталогом запчастей.

Первый билд качает мир. Второй — быстрее. `Build → Clean` — когда студия сходит с ума, не каждый раз «на всякий случай». Clean каждый раз — как перестраивать реактор перед каждой проверкой датчика.

Мак, Windows, WSL
Кэш Gradle живёт в домашней папке: `~/.gradle` на маке и в WSL, `C:\Users\<ты>\.gradle` на родном Windows. Студия Android — родное окно

Windows, не WSL: не ищи кэш внутри Ubuntu, его там нет. Диск кончился — чаще всего раздулся именно `.gradle` плюс эмуляторы в `Android/Sdk`.

11.7 Память поворота и SharedPreferences без мистики

При обычных настройках поворот часто пересоздаёт активность. Обычные поля нового экземпляра начинаются заново; несохранённый `ArrayList` в поле `MainActivity` теряется. Поэтому список «три задачи пальцем» после поворота пустой — не баг Android, а ты хранил быт в существе, которое система имеет право убить.

Минимальный сейф на один экран. `a` и `b` — поля `MainActivity` из примера выше, инициализированные после `setContentView`; локальные переменные из `onCreate` здесь недоступны. У `EditText` с уникальным id текст обычно сохраняется системой и без нашего кода. Ниже — явная учебная запись в `Bundle`; для заметного собственного состояния можно так же сохранить вычисленный результат.

```
@Override
protected void
onSaveInstanceState(Bundle
outState) {

super.onSaveInstanceState(outStat
e);
```

```
    outState.putString("a",
a.getText().toString());
    outState.putString("b",
b.getText().toString());
}
```

В `onCreate`, после `setContentView` и присваивания `a` и `b` через `findViewById`:

```
if (savedInstanceState != null) {

a.setText(savedInstanceState.getS
tring("a", ""));

b.setText(savedInstanceState.getS
tring("b", ""));
}
```

`Bundle` — мешок ключ-значение, который система **может** вернуть при пересоздании. Не база. Не файл. Поворот и «не хватило памяти в фоне» — да.

Переустановил приложение — нет.

Пережить закрытие — `SharedPreferences`:

```
SharedPreferences prefs =
getSharedPreferences("station",
MODE_PRIVATE);
prefs.edit().putString("tasks",
android.text.TextUtils.join("\n",
tasks)).apply();
```

Прочитать: `getString("tasks", "")`, `split`, в список. `apply()` планирует запись на диск асинхронно; это не обещание, что частые большие записи никогда не задержат интерфейс. `commit()` — ждёт диск, редко нужно на учёбе. Не клади туда пароли «как в Spring Security». Это записка на холодильнике, не сейф.

ViewModel — карман, который переживает поворот, но не переживает смерть процесса. Для списка на вечер можно без него. Для собеса Android-джуна слово знать полезно: «состояние экрана — во ViewModel, диск — в репозитории, активность только рисует». Ты это уже слышал как контроллер / сервис. Тот же жест, другой люк.

Квест A.J6 · Java
Калькулятор: два поля переживают поворот через `Bundle`. README: что будет, если убрать задачу приложения из Recent и открыть его снова (поля **не** обязаны выжить — это честно). Потом по желанию — список задач в

SharedPreferences, чтобы Recent уже не стирал.



Экран тот же. Вахтенный новый. Activity может быть создана заново. Состояние интерфейса и постоянные данные живут по-разному.

11.8 Телефон как клиент к твоему серверу

Когда список в памяти наскучил, можно сходить в **свой** `task-manager`. Это красиво для портфолио: «вот бэкенд, вот карман». Сначала прочитай

ловушки, потом пиши `HttpURLConnection`.

- Эмулятор: `localhost` — сам эмулятор, не Мак и не Docker. Хост машины часто `10.0.2.2`. Настоящий телефон в Wi-Fi — IP компьютера в локалке, сервер слушает `0.0.0.0`, не только `127.0.0.1`.
- HTTP без TLS Android режет. Учебный флаг cleartext — в README «только локалка».
- Сеть нельзя из UI-потока: `NetworkOnMainThreadException`. Либо `new Thread`, либо (лучше) executor. Иначе калькулятор «завис» на медленном Wi-Fi, а это ты залез в сеть из `onClick`.
- JSON: тот же текст, что curl печатал в месяце 2.

`org.json.JSONObject` встроен. Jackson на телефон тащить ради двух полей — жадность.

Эскиз, не священный клиент:

```
new Thread(() -> {
    HttpURLConnection c = null;
    try {
        URL url = new
URL("http://10.0.2.2:8080/
health");
        c = (HttpURLConnection)
url.openConnection();

c.setConnectTimeout(3000);
        c.setReadTimeout(3000);
        int code =
c.getResponseCode();
        runOnUiThread(() ->
out.setText("health " + code));
    } catch (Exception e) {
        runOnUiThread(() ->
out.setText(e.getMessage()));
    } finally {
        if (c != null) {
            c.disconnect();
```

```
        }
    }
}).start();
```

`runOnUiThread` — обратно на экран: трогать `TextView` можно только с вахты UI. Иначе редкий краш, который на эмуляторе «иногда». Станция такие «иногда» не любит.

Таймаут соединения ограничивает ожидание подключения, таймаут чтения — ожидание данных после него. `finally` освобождает соединение и при ошибке. Этот маленький опыт предполагает, что экран остаётся открытым: обычный поток не связан с жизненным циклом Activity и после поворота может отправить результат старому экрану. Для

настоящего клиента отдельно организуй отмену работы и передачу результата текущему экрану с учётом lifecycle; одного `runOnUiThread` для этого мало.

Квест A.J7 · Java
Кнопка «связь»: GET `/health` твоего сервера с эмулятора **или** с телефона. В README — какой URL и почему не `localhost`. Если сервер не запущен — на экране ошибка, не краш. Это и есть клиент.

11.9 Kotlin с той же кнопки, не с нового учебника

Студия умеет смешивать Java и Kotlin в одном модуле. Не надо. Но один раз увидеть, как выглядит слушатель, полезно —

чтобы «Kotlin обязателен» в вакансии не звучало как другой язык с Марса.

```
plus.setOnClickListener {
    out.text = (num(a) +
num(b)).toString()
}
```

Тот же `setOnClickListener`, меньше букв, `text` вместо `setText`. Идеи старые. Синтаксис новый. Учи, когда калькулятор на Java **считает**, список **показывается**, поворот **не стирает** поля. Раньше — три магии, ноль починок.

Официальные курсы: developer.android.com/courses — часто сразу Kotlin. Можно смотреть как субтитры: «ага, это мой `onCreate`». Не как приказ выкинуть Java.

11.10 Собес с телефона, если люк стал дверью

Спросят не Kafka. Спросят:

- чем активность не `main`, что будет при повороте;
- зачем `strings.xml`, зачем манифест;
- почему сеть не из UI-потока;
- чем `ListView` отличается от `RecyclerView`: оба переиспользуют строки; у RecyclerView явный `ViewHolder`, гибкие layout managers и обновления отдельных элементов;
- куда девать состояние: экран / ViewModel / диск.

Ответы у тебя уже из этой главы плюс из месяца 1 (список)

и месяца 2 (HTTP). Не учи «Android Architecture Components» пачкой до первого экрана. Экран, который открывается, плюс честный README — снова сильнее сертификата.

APK на свой телефон без магазина: Build → Build APK(s). Файл в `app/build/outputs/apk/`. На телефоне разрешить установку из этого источника. Это не публикация в Google Play. Play — кабинет, подпись, комиссии, модерация. Для джуна «вот APK и вот скрин» достаточно. Магазин подождёт оффера или упрямства.

11.10.1 `RecyclerView` в двух словах, когда ListView уже не стыдно

`ListView` на учёбе честный и тоже переиспользует строки через `convertView`. Короткий ответ «почему RecyclerView»: явный `ViewHolder`, гибкие схемы раскладки и удобные обновления отдельных элементов. Ни один из них не обязан создавать тысячу строк одновременно. Для трёх задач хватит простого варианта; дальше выбирай по задаче.

Пишется злее: свой `Adapter`, свой `ViewHolder`, `onCreateViewHolder` / `onBindViewHolder`. Студия умеет заготовить. Не копируй первый гайд 2016 года с `android.support` — это мёртвый пакет. Сейчас `androidx.recyclerview`.

Правило люка: ListView живой → потом RecyclerView. Не наоборот. Иначе неделю будешь дружить с холдером, а кнопка «плюс» так и не сложится.

11.10.2 Когда приложение «остановилось»

Красный диалог на телефоне — не мистика. В Logcat ищи `FATAL EXCEPTION` и первую строку **твоего** пакета, не три экрана `android.view`. Дальше обычная Java: NPE, не тот id, сеть с UI-потока, `findViewById` вернул null потому что layout другой.

`NullPointerException` на `plus.setOnClickListener` — кнопки нет в **этом** XML. Ты смотришь `activity_main`, а `setContentView` дергает другой layout. Или id `plus` в Preview, а в Code

опечатка. Сверь `R.id` и XML букву в букву.

`CalledFromWrongThreadException` — тронул `TextView` из `new Thread`. Верни через `runOnUiThread`. Это не «Android сложный». Это две вахты: сеть и экран, и они не одни.

Стек читай сверху вниз, как в Spring: первая **твоя** строка важнее, чем `Handler.dispatchMessage`. Скопируй в поиск без имени пакета `com.example.calc`, если стыдно. Ошибка от этого не меняется.

11.11 Дальше — документация, не третий учебник

Официально и по делу:

- developer.android.com/docs — как устроена активность, layout, жизненный цикл. Английский. Мы предупреждали.
- developer.android.com/courses — курсы Google, часто с Kotlin. Не пугайся: синтаксис новый, идеи старые.
- Поиск по студии: двойной Shift, имя `findViewById` — прыгнешь в исходники. Это не магия, это Java.

Если бэкенд всё-таки взял — Android можно не трогать. Если не взял — калькулятор за вечер

дешевле, чем ещё три месяца «подкручу Kafka и точно возьмут». Телефоны людям нужны каждый день. Серверам — тоже, но HR серверов капризнее.

Kotlin, когда калькулятор надоест: тот же `onCreate`, те же id, меньше слов вокруг. Не учи Kotlin **вместо** первого экрана. Экран важнее диалекта.

View Binding и Jetpack Compose — слова, которые студия шепчет с порога. Binding — чтобы не писать `findViewById` вручную. Compose — другой способ рисовать экран, без XML. Оба подождут, пока `findViewById` и XML тебе понятны. Иначе ты выучишь три магии сразу и не

починишь ни одну. Станция любит один датчик за вечер.

Тёмная тема: `Theme` в `themes.xml`, или просто тёмный фон у `LinearLayout` и светлый текст. Для портфолио скрин «светлая / тёмная» выглядит взрослее, чем ещё одна зависимость. Не ставь библиотеку тем ради двух цветов.

Воскресная диверсия
Добавь кнопку × и ÷, если вдруг ещё нет. Потом тему потемнее. Потом «а давай это будет калькулятор станции: ватты и кислород». Сорвёшься в игру — нормально. Калькулятор станции всё равно честнее ещё одной библиотеки.

Закон станции

Запасной люк — не позор. Позор — год делать вид, что «ещё чуть-чуть бэкенд», и не иметь ни одного экрана, который открывается без тебя. Зелёная стрелка на телефоне считается. Даже если считает только 2+2.

Если через неделю калькулятор наскучил, а бэкенд всё ещё молчит — не стыдно остаться на телефоне дольше. Java та же. Собес Android-джуна тоже спрашивает списки, жизненный цикл и «почему упало». Ты уже знаешь, что ответить про пустой ввод. Это больше, чем «я скачал студию».

Сорок минут Lisp в этот люк тоже входят. Скобки не конкурируют с XML. Вечер: XML

и кнопки. Утро: `ev` или датчик кислорода. Станция «МОДУЛЬ» не ревнует к Pixel.

Скрин в README — не тщеславие. Это доказательство, что зелёная стрелка у тебя **была**, не «ну вроде запускалось». Телефон на столе, калькулятор с `2+2`, дата. Как `curl` в месяце 2. Тот же жест, другой иллюминатор.

Если студия всё ещё качает SDK, пока ты это читаешь — пусть качает. Чай. Потом Empty Views Activity, Java, две кнопки. Не третий учебник. Не Firebase. Две кнопки. Станция «МОДУЛЬ» когда-то тоже началась с одного датчика, который врал, и с человека, который это заметил.

Чеклист люка, если забыл:

- студия с developer.android.com, не «cracked»;
- Empty Views Activity, Java;
- эмулятор или USB-debugging;
- плюс, минус, умножить, делить, пустое не роняет;
- скрин в `android-calc`.
- вторая дверь (`AboutActivity`) — если уже не страшно;
- поля переживают поворот — если уже совсем не страшно.

Дальше документация Google, не третий учебник и не Firebase «на пять минут». Станция «МОДУЛЬ» когда-то тоже началась с одного датчика.

12 Отгадки

Сначала своя попытка. Правда. Если сразу сюда — мозг ничего не получит, только тёплое чувство «я как будто умею». Ниже — один из рабочих вариантов, не священное писание.

12.0.1 Занятие 0

Отгадка 0.L1

```
(+ 1 2 3 4 5)        ; 15
(* 2 3 4)            ; 24
(* (- 10 3) 2)       ; 14
```

Отгадка 0.J1

```
public class Hello {
    public static void
main(String[] args) {
        System.out.println("Имя:
```

```
Алекс");
        System.out.println("Дата:
2026-08-13");
    }
}
```

В терминале: `javac Hello.java && java Hello`.

Отгадка 0.G1
`git add -A && git commit -m "week0: hello name" && git push`. В браузере на GitHub видны файлы ветки `main`.

Отгадка 0.L2
`(+ 2 "станция")` — type error: `+` ест числа. `(concatenate 'string "МОДУЛЬ-" "1")` → `"МОДУЛЬ-1"`. Журнал: ошибка и удачный результат рядом, чтобы через месяц не искать в памяти.

Отгадка 0.J2

Вторая `println` с числом отсеков. Без повторного `javac` на экране старое число: JVM запускает `.class`, не `.java`. После `javac` — новое. Вот зачем два шага.

12.0.2 Занятие 1

Отгадка 1.L1

```
(defun dock-ok (speed)
  (if (< speed 5) "мягко"
"слишком быстро"))
```

Отгадка 1.L2

```
(defun airlock (inside outside)
  (if (= inside outside) 'open
'sealed))
```

Отгадка 1.J1

```
Scanner in = new
Scanner(System.in);
int a = in.nextInt(), b =
in.nextInt(), c = in.nextInt();
```

```
double avg = (a + b + c) / 3.0;
System.out.println(avg);
if (a < 0 || b < 0 || c < 0)
System.out.println("ALARM");
```

Целочисленное деление `(a+b+c)/3` отбросит дробь — для среднего используй `3.0`.

Отгадка 1.J2

```
int secret = 1 + (int)
(Math.random() * 10);
int g = new
Scanner(System.in).nextInt();
if (g == secret)
System.out.println("верно");
else if (g < secret)
System.out.println("мало");
else System.out.println("много");
```

Отгадка 1.L3

```
(defun lamp (energy)
  (cond
    ((>= energy 80) 'green)
```

```
    ((>= energy 40) 'yellow)
    (t 'red)))
; (lamp 100) GREEN, (lamp 80)
GREEN, (lamp 79) YELLOW,
; (lamp 40) YELLOW, (lamp 0) RED
```

Отгадка 1.J3

```
while (true) {
    System.out.print("Энергия
0..100: ");
    String raw =
in.nextLine().trim();
    try {
        int n =
Integer.parseInt(raw);
        if (n >= 0 && n <= 100) {

System.out.println(n);
            break;
        }
    } catch
(NumberFormatException e) {
        // снова приглашение
    }
}
```

12.0.3 Занятие 2

Отгадка 2.L1

```
(defun rooms-report (lst)
  (loop for room in lst
        for i from 1
        do (format t "~a. ~a~%" i
room)))
```

Отгадка 2.L2

```
(defun has-reactor-p (lst)
  (not (null (member 'reactor
lst))))
```

Отгадка 2.J1
Цикл `while (true)`, `String line = in.nextLine()`, `split(" ", 2)`. `add` — `tasks.add(parts[1])`. `list` — нумерованный `for`. `del` — `remove(Integer.parseInt(n) - 1)` с проверкой границ. `quit` — `break`.

Отгадка 2.J2

```
Map<String, Integer> ages =
Map.of("Ann", 20, "Bob", 15,
"Cyd", 33);
ages.forEach((name, age) -> {
    if (age > 18)
System.out.println(name);
});
```

`Map.of` — неизменяемая карта; для учебного вывода достаточно.

Отгадка 2.L3

```
(defun prepend-airlock (rooms)
  (cons 'airlock rooms))
; (defparameter *r* '(corridor))
; (prepend-airlock *r*) =>
(AIRLOCK CORRIDOR), *r* ещё
(CORRIDOR)
```

Отгадка 2.J3

```
Map<String, Integer> energy = new
HashMap<>();
energy.put("reactor", 80);
```

```
energy.put("antenna", 20);
energy.put("garden", 0);
energy.forEach((room, n) -> {
    if (n < 30)
System.out.println(room);
});
```

12.0.4 Занятие 3

Отгадка 3.L1

```
(defun clamp (n lo hi)
  (min hi (max lo n)))
(defun spend (energy cost) (clamp
(- energy cost) 0 100))
(defun recharge (energy delta)
(clamp (+ energy delta) 0 100))
(defun simulate (costs)
  (let ((e 100))
    (dolist (c costs) (setf e
(spend e c)))
    e))
```

Отгадка 3.L2

```
(defun render-room (room) (format
nil "[~a]" room))
```

```
(defun print-rooms (lst)
  (dolist (r lst) (format t
"~a~%" (render-room r))))
```

Отгадка 3.J1

`done 2` → `store.complete(2)` ищет `Task` с `id == 2` и вызывает `complete()`. Печать: id, title, флаг done.

Отгадка 3.J2

Поле `private int priority` в конструкторе по умолчанию `0`. Сеттер или параметр `add(title, priority)`. `list` печатает `priority + " " + id + " " + title`.

Отгадка 3.L3

```
(defun docking-score (speed fuel
angle)
  (let ((speed-part (if (< speed
5) 10 0))
        (fuel-part (if (> fuel
```

```
20) 5 0))
        (angle-part (if (< (abs
angle) 10) 3 0)))
    (+ speed-part fuel-part
angle-part)))
```

Отгадка 3.J3
В `TaskStore`: цикл по `tasks`, `t.getId() == id`, иначе `null`. В `main`: `show 2` → `findById(2)`, при `null` печать `missing`.

12.0.5 Занятие 4

Отгадка 4.L1

```
(defun max-list (lst)
  (if (null (rest lst))
      (first lst)
      (let ((m (max-list (rest
lst))))
        (if (> (first lst) m)
(first lst) m))))
```

Отгадка 4.L2

```
(defun filter-positive (lst)
  (cond
    ((null lst) nil)
    ((> (first lst) 0)
     (cons (first lst) (filter-
positive (rest lst))))
    (t (filter-positive (rest
lst)))))
```

Отгадка 4.J1
При старте `titles = TaskFile.load(Path.of("tasks.txt"))`, восстановить `Task` с новыми id по порядку. При `quit` — `titles` из `store.all()` через `getTitle()`, `save`. Пустой файл / нет файла → пустой список.

Отгадка 4.J2
README: JDK 21, `javac`/`java` или IDEA, таблица команд, пример диалога. Без этого неделя 4 не закрыта.

Отгадка 4.L3

```
(defun rooms-length (lst)
  (if (null lst) 0 (+ 1 (rooms-
length (rest lst)))))
(defun find-room (room lst)
  (cond
    ((null lst) nil)
    ((eq (first lst) room) lst)
    (t (find-room room (rest
lst)))))
```

Отгадка 4.J3
В README четыре беды: нет `javac` → ставь JDK 21 и PATH; `cannot find symbol` → импорт или компиль все `.java`; нет `tasks.txt` → пустой список, не падение; буквы вместо числа → повторить ввод.

12.0.6 Занятие 5

Отгадка 5.L1

```
(mapcar (lambda (n) (min 100 (* n
2))) '(10 60 40))
; (20 100 80)
```

Отгадка 5.L2

```
(defun offline-modules (alist)
  (mapcan (lambda (pair)
            (if (eq (cdr pair)
'down) (list (car pair)) nil))
          alist))
```

Отгадка 5.J1

`addIncreasesSize`; `completeMarksDone`; `completeMissing` — `assertThrows(NoSuchElementException.class, () -> store.complete(99))` или `assertFalse(store.complete(99))`, если выбрал `boolean`.

Отгадка 5.J2

Временно закомментируй `task.setDone(true)`. Красный тест.

Верни. README: `mvn test` или зелёная стрелка IDEA.

Отгадка 5.L3

```
(defun energy-of (module alist)
  (cdr (assoc module alist)))
; (energy-of 'reactor *energy*)
=> 80, 'observatory => NIL
```

Отгадка 5.J3

`git checkout -b feat/tests`, коммит, `git switch main`, `git merge feat/tests`. В README три строки `git log --oneline`.

12.0.7 Занятие 6

Отгадка 6.L1

```
(defun http-not-found ()
  (format nil "HTTP/1.1 404 Not
Found~%Content-Type: text/
plain~%~%missing"))
```

Отгадка 6.L2

```
(defun json-task (id title)
  (format nil "{\"id\":~a,
\"title\":\"~a\"}" id title))
```

Отгадка 6.J1

```
@GetMapping("/tasks/{id}")
public ResponseEntity<Task>
one(@PathVariable int id) {
    return tasks.stream()
        .filter(t -> t.getId() ==
id)
        .findFirst()
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.no
tFound().build());
}
```

Отгадка 6.J2
Пять curl: health, list empty, POST, list one, GET by id. Ожидаемый JSON коротко.

Отгадка 6.L3

```
(defun http-created (body)
  (format nil "HTTP/1.1 201
Created~%Content-Type:
application/json~%~%~a" body))
; (http-created (json-task 2
"антенна"))
```

Отгадка 6.J3

`curl -i -X POST ... -d '{}'` — скопируй строку `HTTP/1.1 ...` в README. До занятия 7 часто 500; после ловли — 400. Честно зафиксируй то, что видишь.

12.0.8 Занятие 7

Отгадка 7.L1

```
(defun create-task (title)
  (if (or (null title) (string=
title ""))
      (error "title required")
      (list :title title :done
nil)))

(defun try-create (title)
```

```
  (handler-case (create-task
title)
    (error () 'bad-request)))
```

Отгадка 7.J1
Контроллер с `private final TaskService service` и конструктором. Аннотации `@RestController`, `@Service`. Списка в контроллере нет.

Отгадка 7.J2
Сервис `boolean delete(int id)` / `void` + исключение. Контроллер 204 `noContent` или 404. Юнит-тест сервиса без `@SpringBootTest`.

Отгадка 7.L2

```
(defun blank-p (s)
  (or (null s) (string= (string-
trim '(#\Space) s) "")))
(defun create-task (title)
  (if (blank-p title)
      (error "title required")
```

```
      (list :title (string-trim
'(#\Space) title) :done nil)))
```

Отгадка 7.J3

`create` ловит `IllegalArgumentException` → 400 и `{"error":"title required"}`. Два `curl -i` в README: `""` и `" "`.

12.0.9 Занятие 8

Отгадка 8.L1

```
(defun log-info (fmt &rest args)
  (format t "INFO ")
  (apply #'format t fmt args)
  (terpri))
```

Отгадка 8.J1

`LoggerFactory.getLogger`. `log.info("created id={}", id)`.

Отгадка 8.J2

```
@GetMapping("/tasks")
public List<Task>
all(@RequestParam(required =
false) Boolean done) {
    if (done == null) return
service.findAll();
    return
service.findAll().stream()
        .filter(t -> t.isDone()
== done)
        .toList();
}
```

Отгадка 8.L2

```
(defun log-warn (fmt &rest args)
  (format t "WARN ")
  (apply #'format t fmt args)
  (terpri))
; в try-create: успех — log-info,
error — log-warn и 'bad-request
```

Отгадка 8.J3

`log.warn("task not found id={}", id)` в сервисе или контроллере на 404. В README строка из консоли Run / `mvnw`.

12.0.10 Занятие 9

Отгадка 9.L1
См. `save-tasks` / `load-tasks` в тексте главы. В новой сессии `(load-tasks "module-tasks.lisp-data")`.

Отгадка 9.J1
Три `INSERT`, `SELECT id, title FROM tasks;`, вывод скопировать в README в блок кода.

Отгадка 9.J2
`spring.datasource.url=jdbc:postgresql://localhost:5432/taskdb`. `JdbcTemplate.query` с `RowMapper`. `GET /tasks` читает БД.

Отгадка 9.J3
Неверный пароль: `password authentication failed`. Нет базы: `database "taskdb" does not exist`. Нет таблицы: `relation "tasks"`

`does not exist`. Три цитаты в README, потом рабочий yml.

12.0.11 Занятие 10

Отгадка 10.L1

```
(defun tasks-of (pid projects)
  (cdr (assoc pid projects)))
; пример: '((1 . (a b)) (2 .
(c)))
```

Отгадка 10.J1
`JpaRepository<Task, Long>`, `POST` → `save`, Flyway `V1__tasks.sql` совпадает с `@Table`. `ddl-auto: validate`.

Отгадка 10.J2
`GET` собирает DTO: id проекта, имя, список `{id,title}` без вложенного project в каждой задаче.

Отгадка 10.J3

Поле в entity без колонки → `Schema-validation: missing column [...]`. В цепочке `Caused by` найди причину с сообщением о несовпадении схемы и процитируй её в README. Потом `V2__...sql` с `ALTER TABLE ... ADD COLUMN`.

12.0.12 Занятие 11

Отгадка 11.L1

```
(defun apply-ops (state ops)
  (if (find 'fail ops)
      state
      (append state ops)))
```

Учебная модель: «всё или ничего».

Отгадка 11.J1
Один `@Transactional` метод: `projectRepository.save` затем `taskRepository.save`. Пустой title

— `IllegalArgumentException` до второго save или после проверки. Тест: count проектов не вырос.

Отгадка 11.J2
В логе посчитайте `select`. Вставьте число в README. Если по запросу на элемент — ты видел N+1.

Отгадка 11.J3
`JOIN FETCH` или DTO-запрос. В логе один (или два: count + data) `select` с `join`, не пачка одинаковых `where project_id=?`. Два числа: до / после.

12.0.13 Занятие 12

Отгадка 12.L1

```
(save-tasks "t.dat" '((1 .
"шлюз")))
```

```
; новая сессия SBCL:
(equal (load-tasks "t.dat")
'((1 . "шлюз")))
; T, иначе (error "станция
потеряла журнал")
```

Отгадка 12.J1
Четыре теста: create возвращает id; get неизвестного — empty/404; update меняет title; delete затем get пусто.

Отгадка 12.J2
README: Postgres version, `flyway`, `mvn test`, curl CRUD, ограничение H2 если есть.

12.0.14 Занятие 13

Отгадка 13.L1
Только учебная модель: `sxhash` не защищает пароли. Для настоящих аккаунтов нужна специальная схема

хеширования с солью, как в Java-занятии.

```
(defparameter *users* (make-hash-
table :test 'equal))
(defun register (email password)
  (when (gethash email *users*)
(error "exists"))
  (setf (gethash email *users*)
(sxhash password))
  t)
(defun login (email password)
  (eql (gethash email *users*)
(sxhash password)))
```

Отгадка 13.J1
`Task.userId` из `Authentication`. Репозиторий `findByUserId`. Security: authenticated для `/tasks/**`.

Отгадка 13.J2
Сервис: если `!java.util.Objects.equals(task.getUserId(), current)` → 404 (скрытие

факта) или 403. Тест с двумя пользователями.

Отгадка 13.J3
Без `Authorization` — 401. Чужой id — 404 или 403 (как выбрал в 13.J2). Свой — 200 и тело. Три curl в README.

12.0.15 Занятие 14

Отгадка 14.L1
Проход: `(loop for (u . tid) in pairs if (eql u user) collect tid)`. Индекс: `gethash user table`. Для 10000 это уже ощутимо в REPL, если крутить тысячи раз.

Отгадка 14.J1
`V2__idx_tasks_user.sql`: `CREATE INDEX ...`. Контроллер `Page<Task>` + `Pageable`.

Отгадка 14.J2

Два блока `EXPLAIN ANALYZE` в README: Seq Scan vs Index Scan (зависит от объёма; на трёх строках оптимизатор может не взять индекс — налей тысячи строк).

Отгадка 14.J3 `page=0&size=5&sort=id,asc` и `page=1&size=5&sort=id,asc` на одних и тех же неизменных данных. Id в `content` не должны пересекаться. Если пересекаются, проверь устойчивую сортировку по уникальному ключу, параллельные изменения данных и передачу Pageable в репозиторий.

12.0.16 Занятие 15

Отгадка 15.L1

```
(defun balanced-p (s)
  (let ((st nil))
    (loop for ch across s do
      (cond
        ((char= ch #\() (push #\)
st))
        ((char= ch #\[) (push #\]
st))
        ((member ch '(#\)
#\]) :test #'char=)
         (unless (and st (char=
ch (pop st)))
           (return-from balanced-
p nil)))))
    (null st)))
```

Отгадка 15.J1

```
record UserId(long value) {}
// record уже даёт equals/
hashCode.
// Если свой класс —
Objects.equals /
Objects.hash(value).
```

Отгадка 15.J2
Reverse: два индекса `i`, `j`.
Частота:

`HashMap<Character,Integer>`.
Скобки: `ArrayDeque<Character>` как стек.

Отгадка 15.J3
Класс без equals: `get(new UserId(1))` → `null` после `put(new UserId(1), ...)`. `record UserId(long v)` — находит. Два `println` в README.

12.0.17 Занятие 16

Отгадка 16.L1
Вместо `*energy*` — `(defun tick (energy cost) ...)`. Цепочка вызовов передаёт новое значение. Глобаль не читается.

Отгадка 16.J1
Два `Thread` по 100000 `inc`. Печать `n`. Затем `synchronized void inc()`. Сравни.

Отгадка 16.J2

Не код: файл `core-answers.md` или голос. Чеклист из текста занятия 16.

Отгадка 16.J3

`AtomicInteger n = new AtomicInteger();` в `inc` — `n.incrementAndGet()`. Три запуска: дыра / 200000 / 200000.

12.0.18 Занятие 17–20

Отгадка 17.L1

```
(defstruct centry value expire)
(defun cget (h key)
  (let ((e (gethash key h)))
    (when (and e (< (get-
universal-time) (centry-expire
e)))
      (centry-value e))))
(defun cset (h key value ttl)
  (setf (gethash key h)
        (make-centry :value value
                     :expire (+
(get-universal-time) ttl))))
```

Отгадка 17.J1
`@Cacheable` / ручной `RedisTemplate`. В ключе вошедший пользователь и фактические `done`, страница, размер, поля и направления сортировки в заданном порядке; отсутствие фильтра `done` отличается от `false`. После commit create / update / delete сбрасывай все затронутые варианты списка. Для упражнения очищай весь именованный кэш списков задач, не всю базу Redis. `convertAndSend` не удаляет записи кэша. `@CacheEvict(allEntries = true)` очищает свой именованный кэш при вызове через прокси кэша; выполнение после commit организуй явно, как на занятии 18. README: разные ответы для страниц/

фильтров и воспроизведение stale без сброса.

Отгадка 17.J2
TTL 1–2 с, ключ протух, пачка GET. В логе несколько одинаковых SELECT — stampede. README: число и как запускал (`xargs -P` или два окна).

Отгадка 18.L1
`enqueue` через `append` или хвост. `drain`: `dolist` пока очередь не пуста, `format t`.

Отгадка 18.J1
Опубликуй объект `TaskCreatedEvent` через `ApplicationEventPublisher.publishEvent(...)` внутри публичного метода сервиса с `@Transactional`. Обработчик — `@TransactionalEventListener`, внутри лог. По умолчанию ему

нужна активная транзакция; после успешного коммита он выполняется синхронно, если асинхронность отдельно не настроена.

Отгадка 18.J2 `ArrayBlockingQueue<>(4)`, продюсер 8 раз `put`, консьюмер `take`. Лог «жду put» / «взял». Без Spring.

Отгадка 19.L1

```
(defparameter *log* (make-array
0 :adjustable t :fill-pointer 0))
(defun produce (x) (vector-push-
extend x *log*))
(defun consume (offset)
  (when (< offset (length *log*))
(aref *log* offset)))
```

Отгадка 19.J1

Либо `KafkaTemplate.send("task-events", json)`, либо честный `docs/kafka.md`.

Отгадка 19.J2
Таблица: POST задач при мёртвой почте — HTTP 201 vs журнал догонит. Пять строк своими словами, не Википедия.

Отгадка 20.J1
Compose: сервисы `db`, `app`, `depends_on`, healthcheck Postgres. App: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/taskdb`.

Отгадка 20.J2

```
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-
java@v4
        with: { distribution:
temurin, java-version: 21 }
      - run: mvn -B test
```

Отгадка 20.J3
С хоста curl → `localhost:8080`. JVM в контейнере `app` → хост `db`, не `localhost`. `localhost` внутри контейнера — он сам.

12.0.19 Занятие 21–26

Отгадка 21.J1
Проверка: новая директория, только README. Секундомер. Если застрял — дыра в README, не в «очевидности».

Отгадка 21.J2
Шаблон: «REST для личных задач. Java 21, Spring Boot, PostgreSQL, JWT. Регистрация,

CRUD, пагинация. Ссылка: github.com/…/task-manager».

Отгадка 21.J3
Одна страница: имя, связь, стек строкой, два проекта со ссылками. Курсы внизу. Lisp в «ещё вот». Сорок секунд вслух.

Отгадка 22.J1
3 минуты: проблема → что показать (API, БД, auth) → одно техническое решение, которым гордишься → что бы переписали.

Отгадка 22.L1

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev
(rest form))))
        (cond
```

```
          ((eq op '+) (apply #'+
args))
          ((eq op '-) (apply #'-
args))
          (t (error "unknown ~a"
op))))))
```

Отгадка 22.J2
Плохой: «Спринг сам, CRUD, хотел Кафку». Хороший: проблема → свои задачи/JWT → кэш со сбросом → чего нет (Kafka в бою) → compose. Только то, что в git.

Отгадка 23.J1
«Видел вакансию N. Делал CRUD с PostgreSQL и тестами (ссылка). Kafka в проде не внедрял; очередь/события — в монолите. Готов обсудить».

Отгадка 23.J2

Вслух: «в бою не внедрял, вот лог vs HTTP; Redis — TTL и сброс, ломал stale; микросервисы не выкусывал из-за транзакции». Без извинения за жизнь.

Отгадка 24.J1
Одна карточка = вопрос + ответ + ссылка на файл проекта (`TaskService.java:40`).

Отгадка 24.J2
Три карточки до собеса: 401 vs 403; индекс; транзакция create проекта+задачи. Своими словами, не Википедия.

Отгадка 25.J1
Anagram: сортировка массива символов или частоты. Merge: два индекса. Emails: нормализация + и точки локальной части по условию задачи.

Отгадка 25.L1

```
(defun anagram-p (a b)
  (string= (sort (copy-seq a)
#'char<)
           (sort (copy-seq b)
#'char<)))
```

Живой coding важнее идеала: компилируемость, примеры, сложность.

12.0.20 Глава «как устроен компьютер»

Отгадка 0b.1
Мак / WSL:

```
mkdir module-cabin
cd module-cabin
echo "станция МОДУЛЬ" > note.txt
pwd
ls
cat note.txt
```

Вторая строка в `note.txt` — ровно то, что напечатал `pwd`

(например `/Users/ты/dev/module-cabin` или `/home/ты/module-cabin`). PowerShell: `New-Item`, `Get-Location`, `Get-Content`. Если путь «не там» — ты создал папку не оттуда, где стоял. Это и есть отгадка.

Отгадка 0b.2
Кладовая: файл `Hello.java` (и после `javac` — `Hello.class`). Стойка: байткод и переменные запущенного процесса `java`. Повар: CPU выполняет инструкции JVM. Иконка Run только просит шефа (ОС) запустить этот рецепт.

Отгадка 0b.3
Байты те же. Сломался договор, как их читать. UTF-8 кладёт русскую букву в два байта, Windows-1251 ждёт один байт на букву — поэтому глаз видит

крякозябры. Вернул UTF-8 — слово снова `МОДУЛЬ`. Не перепечатывай крякозябры «как есть» в новый файл: тогда портить уже сами числа.

12.0.21 Бортовой журнал «МОДУЛЬ»

Отгадка S.L1

```
(defun ticks (n)
  (loop repeat n do (tick))
  *energy*)
```

Рекурсия: `(defun ticks (n) (if (<= n 0) *energy* (progn (tick) (ticks (- n 1)))))`. `clamp-energy` уже в `tick` — ниже нуля не уйдёшь.

Отгадка S.L2

```
(defun exits-report ()
  (dolist (pair *doors*)
```

```
    (format t "~a -> ~a~%" (car
pair) (cdr pair))))
```

Отгадка S.L3

```
(defun drop (item)
  (if (member item *pocket*)
      (progn
        (setf *pocket* (remove
item *pocket*))
        (setf (cdr (assoc *here*
*at*))
              (cons item (stuff-
here)))
        (format t "бросил ~a~%"
item))
      (format t "в кармане нет
~a~%" item)))
```

Проверка: взять в реакторе, `walk` в коридор, `drop`, вернуться — в реакторе пусто.

Отгадка S.L4

```
(defparameter *lamp* 'dead)
(defun whack ()
  (cond
    ((not (eq *here* 'galley))
'wrong-room)
    ((not (member 'wrench
*pocket*)) 'need-wrench)
    (t (setf *lamp* 'ok)
'fixed)))
```

В `look` коридора: если `*lamp*` равно `'ok` — другая строка про балласт. Иначе честный комментарий: бутафория.

Отгадка S.L5

```
(defun save ()
  (save-world "module-save.lisp-
data"))
(defun load-game ()
  (if (probe-file "module-
save.lisp-data")
      (load-world "module-
save.lisp-data")
      'no-save))
```

`(load-game)` читает сохранение. `load` остаётся загрузчиком файлов Common Lisp; текстовая команда `load` может вызывать `load-game`.

`probe-file` — «есть ли банка». После починки щели `*fixed*` должен быть в plist, иначе загрузка забудет изоленту.

Отгадка S.L6
Одна строка ручной таблицы, старт из `airlock`: airlock 0, corridor 1, galley 2, reactor 2, cupola 3. Полное табличное решение требует строки для каждого возможного `*here*`; недостижимому отсеку соответствует `nil`. Или обойди граф от `*here*` со списком посещённых и счётчиком шагов. Кратчайший путь не

обязателен. `airlock` → `cupola` — 3 люка через коридор и камбуз, не 1.

Отгадка S.L7

```
(defun coffee ()
  (cond
    ((not (eq *here* 'galley))
'wrong-room)
    ((< *energy* 4)
     (format t "пар. энергии
~a~%" *energy*)
     'steam)
    (t
     (setf *energy* (- *energy*
4))
     (if (member 'mug *pocket*)
         (format t "в кружку Щ.
энергия ~a~%" *energy*)
         (format t "на пол.
энергия ~a~%" *energy*))
     'sludge)))
```

12.0.22 Лаборатория Java

Отгадка J.L1

Сначала яма: `nextInt` + `nextLine` → пустые скобки. Потом:

```
int energy =
Integer.parseInt(in.nextLine().tr
im());
String cmd = in.nextLine();
```

Комментарий: `nextInt` оставляет `конец строки` в stdin, его съест следующий `nextLine`.

Отгадка J.L2
`split(" ")`, длина 3, `parseInt` в `try`. `energy`/`oxygen` 0..100, `temp` −20..40. Не число — сообщение, `continue`. Процесс не умирает.

Отгадка J.L3
`Files.exists` — нет файла → дефолты и строка `no station.txt, using defaults`. На каждую строку свой `try` вокруг `parseInt`, битое — `System.err`, остальные

ключи живут. Не один `catch (Exception e)` на весь load.

Отгадка J.L4

```
return "{\"energy\":" + e + ",
\"oxygen\":" + o + ",\"temp\":" +
t + "}";
```

Валидатор зелёный. Имя комнаты с `кавычкой` без экрана — красный. README: Jackson экранирует строки и вложенность, чтобы не собирать JSON конкатенацией.

Отгадка J.L5

`HttpClient.send` GET `https://httpbin.org/get` — статус и `body.substring(0, Math.min(200, body.length()))`. Второй URL `/status/404` — напечатать 404, это не поломка клиента. Третий —

`catch`, имя класса исключения + `getMessage()`.

Отгадка J.L6
Три файла, состояние только в `Station`.

```
javac Station.java
StationFile.java StationDash.java
java StationDash
```

README: эти команды + диалог `status` / `set energy 55` / `save` / `quit`.

Отгадка 26.J1
Две недели в календаре: даты писем, слот питча, слот трёх задач. Видно глазами, не «в голове».

Отгадка 26.L1
В день отказа — любой этюд станции, хоть `(ev '(+ 1 2))`. Скобки не знают HR.

12.0.23 Android

Отгадка A.J1
Слушатели на `mul` и `div`. Если делитель 0 — `out.setText("нельзя")`, не `Infinity` и не краш.

Отгадка A.J2
Пустое: `trim().isEmpty()` до parse. `NumberFormatException` поймать. README: пустое это ноль **или** надпись «введи числа».

Отгадка A.J3
`Log.d` в `onCreate`/`onStart`/`onResume`/`onPause`/`onStop`. Home: pause+stop, возврат start+resume. Поворот: часто destroy+новый onCreate.

Отгадка A.J4
`ArrayList` + `ArrayAdapter` + `ListView`. Три пальцевых задачи. README: «пока память». Скрин.

Отгадка A.J5
`AboutActivity`, `Intent` + `putExtra("last", ...)`. На той стороне `getStringExtra`. Системная «назад» без своей магии.

Отгадка A.J6
`onSaveInstanceState` / чтение в `onCreate` для двух полей. Recent убивает процесс — поля могут исчезнуть, это в README. Список — по желанию `SharedPreferences`.

Отгадка A.J7
Поток не UI, GET health, URL `10.0.2.2` с эмулятора. Ошибка на экране, не краш. README: почему не localhost.

12.0.24 Макросы, CLOS, живой образ

Отгадка M.L1
`macroexpand-1` на `(module-when test a b)` даёт `if` с `(progn a b)`. Без `progn` у `if` одна форма на ветку: `b` потеряется или станет «иначе».

Отгадка M.L2
`with-energy`: `gensym` на стоимость, сравнение с `*energy*`, при нехватке `'too-low` без `decf`. При хватке — `decf`, потом тело. Два прогона: 10 против 15 и 20 против 15.

Отгадка M.L3
`twice-wrong` подставляет аргумент дважды: два `tick`. `twice-right` — `let` + `gensym`, один побочный эффект. Смотри `macroexpand-1` обоих, не «ну вроде одинаково».

Отгадка C.L1
`defclass module`, `make-instance`, `setf` через accessor, `module-status` — одна строка с именем и энергией.

Отгадка C.L2
`antenna` наследник `module`. Свой `defmethod module-status`. `(mapcar #'module-status (list reactor dish))` — две разные строки.

Отгадка C.L3
`:around` + `call-next-method`, при энергии 0 дописать `ALARM`. Один обходчик на всех, не два одинаковых `if`.

Отгадка C.L4
Класс `station`, слот список модулей, `station-report` = `mapcar #'module-status`. Два разных класса в списке.

Отгадка C.L5
`(defclass comm-antenna (module powered radio) ())`, `typep` на всех троих — `t`. Слоты с обеих сторон читаются. В Java двух классов-родителей со слотами нет, только один `extends`.

Отгадка C.L6
`sb-mop:finalize-inheritance`, затем `sb-mop:class-precedence-list`, до и после смены порядка в `defclass`. Имена в CPL едут. Это не баг, это рычаг.

Отгадка I.L1
Без `(quit)`: старый `defmethod`, вызов, новый `defmethod`, вызов на том же экземпляре. Данные те же, строка статуса другая. Если помог только новый `make-instance` — образ ты убил.

13 Приложения

13.1 А. Неделя на одной салфетке

- **Пн–Чт:** 40 мин Lisp (станция МОДУЛЬ) + 120 мин Java (чуть посмотреть, потом пока не заработает).
- **Пт:** Lisp + добить обломки.
- **Сб:** два-три часа только проект.
- **Вс:** гулять или маленькая диверсия. Не третья глава.

SICP — когда сам спросил. Микросервисы — после одной нормальной программы. GitHub — с занятия 0. Английский — каждый день, но не вместо Java: в расписание книги не входит.

Если вечер съели — не «навёрстывай три главы». Один кусок Lisp **или** один кусок Java, и коммит. Салфетка не судья.

13.2 B. Шпаргалка в коридор перед собесом

Читать вслух, не глазами. Глаза врут: «ну я это знаю». Рот проверяет.

Java: `==` и `equals`, строки, списки и «это быстро / нет», `ArrayList` против `LinkedList`, `HashMap`, дженерики без фанатизма, checked/unchecked, `try-with-resources`, объекты на своём `Task`, `final`. `int` против `Integer` одним предложением. Почему строку нельзя поменять внутри.

Spring: коробка с зависимостями, бин, тонкий контроллер, сервис, что откатывает `@Transactional`, контроллер не ходит в репозиторий напрямую. Откуда берётся объект в конструкторе, если ты не писал `new`.

HTTP и SQL: GET можно повторять, POST дважды — осторожно. 401 не 403, 400 не 500. Ключи, индекс, транзакция, JOIN против двух запросов. `EXPLAIN`, хотя бы идея.

Железо: как стартует приложение, куда пишутся логи, зачем Docker, чем образ не контейнер, почему внутри compose хост базы `db`, а не `localhost`.

Если на вопрос «что такое транзакция» лезет «ну это когда» и тишина — вернись к занятию 11, не читай про Kafka.

13.3 C. Что ещё читать, если не надоело

- Conrad Barski, **Land of Lisp** — орки, игры, тот же Common Lisp. Если хочется второй вселенной на те же сорок минут.
- Один Java-справочник, не три курса сразу. Horstmann или что угодно, лишь бы **один**.
- Spring Guides — по одному в тему недели, не «все гайды за выходные».
- Postgres на своей таблице `tasks`. Документация

postgresql.org, раздел SQL. Скучно, полезно.

- Aditya Bhargava, **Grokking Algorithms** — картинки, совесть чиста.
- SICP — точечно, когда сам спросил про рекурсию или «данные как код».
- Официальные материалы по Java: https://dev.java/learn/ . Старые https://docs.oracle.com/javase/tutorial/ рассчитаны на JDK 8; основы полезны, современные возможности сверяй с документацией Java 21.

Не читай пять книг параллельно. Читай одну, пиши код. Книга без кода — сериал.

13.4 D. Сосед-ИИ

Можно: «почему не компилируется», «что за портянка в ошибке», «сравни два моих варианта», «объясни эту строку как человеку, который только что узнал про скобки».

Нельзя: «напиши task-manager с нуля» и выдать чужое за портфолио. Спросят строку, которой ты не касался. Будет обидно и справедливо.

Серая зона: «набросай скелет теста, я сам допишу ассерты». Можно, если допишешь. Нельзя, если скелет уже содержит всю логику, а ты только переименовал переменные.

Закон станции
Правило соседа: после подсказки ты должен суметь **выключить чат** и повторить шаг руками. Если не можешь — это был не сосед, это был джинн, и ты в долгу, который всплывёт в комнате с вайтбордом.

13.5 Е. Про копирование

Код из учебника — в свои учебные репозитории, пожалуйста. Решения квестов — не коммерческий продукт без своей работы. Станция МОДУЛЬ и так на изоленте, давай без судебных исков.

Чужой код с GitHub в портфолио без пометки — враньё. Форк с подписью «я это

менял вот тут» — нормально. Разница в одной фразе в README.

13.6 F. Мак, Windows, WSL — шпаргалка команд

Дальше «как в книге» = Мак или Ubuntu внутри WSL2. Родной Windows — правая колонка, когда очень надо.

Зачем	Мак / WSL (Ubuntu)	Windows без WSL
Пакеты	`brew` / `sudo apt`	Установщики, `winget`, Chocolatey
SBCL	`brew/apt install sbcl`	sbcl.org → Windows
JDK 21	`openjdk@21` / `openjdk-21-jdk`	Adoptium MSI, PATH

Зачем	Мак / WSL (Ubuntu)	Windows без WSL
Git	`brew/apt install git`	git-scm.com
Postgres	`brew/apt` + service	Установщик postgresql.org
Maven wrapper	`./mvnw …`	`mvnw.cmd …`
Перенос в curl	обратный слэш `\`	cmd: `^`; лучше PowerShell одной строкой
Выход из SBCL	`(quit)` или Ctrl+D	`(quit)`
Docker	Docker Desktop / engine	Docker Desktop + WSL2
Проект в IDEA	открыть папку	или `\\wsl$\Ubuntu\home\…`

Зачем	Мак / WSL (Ubuntu)	Windows без WSL
Где я	`pwd`	`cd` без аргументов в cmd; `pwd` в Git Bash
Список файлов	`ls -la`	`dir` / `Get-ChildItem`
Копировать файл	`cp a b`	`copy a b`
Права на mvnw	`chmod +x mvnw`	не нужно: `mvnw.cmd`

Правило, которое экономит месяц жизни: на Windows **сразу WSL2**, Java-команды из учебника копировать в Ubuntu. IntelliJ может остаться «оконной» и открыть папку внутри WSL. Postgres и Docker — тоже лучше с одной стороны стены: либо всё в WSL, либо всё

родное. Два капитана, помнишь.

Если `localhost` из WSL не видит базу, поставленную MSI на Windows — это не ты глупый. Это два мира. Поставь Postgres в Ubuntu (`apt`) и живи спокойно.

PATH сломался: закрой все окна терминала. Не одно. Все. Открой новое. Если не помогло — перезагрузка. Это не шаманство, это кэш окружения, который держит мёртвый путь как драгоценность.

13.7 G. Если совсем застрял

Порядок, не паника:

1. Прочитай **первую** ошибку целиком. Не скрин верхней половины.
2. Проверь папку: `pwd` / `ls`. Файл на месте?
3. Проверь, ту ли программу вызвал: `which java`, `java -version`.
4. Минимальный пример: одна функция, не весь сервер.
5. Тогда сосед-ИИ или поиск. В запрос — ошибка, не «у меня не работает спринг».

Если застрял на три дня на Hibernate — это профессия, не знак уйти. Если застрял на три

дня на установке JDK — вернись в мастерскую, не прыгай в месяц 3.

13.8 Н. Английский на салфетке

Слова, без которых больно:

`build`, `run`, `debug`, `commit`, `push`, `pull`, `merge`, `branch`, `issue`, `stack trace`, `null`, `exception`, `timeout`, `port already in use`, `permission denied`, `not found`, `bad request`, `unauthorized`, `forbidden`.

Не учи как словарь к ЕГЭ. Встретил в ошибке — переведи **эту** фразу, запиши в блокнот станции. Через месяц блокнот сам станет словарём.

13.9 I. Что считать «я закончил книгу»

Не последняя страница. А это:

- на GitHub виден путь от Hello до сервера с базой;
- README поднимает проект на чужой машине (или на твоей второй ОС);
- ты честно говоришь, чего не делал;
- ходишь на собеседования, а не копишь главы.

Оффер — не кнопка. Страницы после месяца 6 — про письма и отказ, не про ещё один фреймворк. Если хочется ещё фреймворк вместо письма — это страх, не учёба. Страх понятный. Письмо всё равно надо.

14 Словарь станции

Не Википедия. Википедия объясняет так, будто ты уже знаешь соседние тридцать слов. Здесь — как бортмеханику в коридоре: одна шутка, чтобы не уснуть, и одно объяснение, которым можно **пользоваться**.

Слова английские, потому что они английские. Переводить `null` как «ничто» можно один раз в душе. В коде и на собесе всё равно `null`.

Листал по нужде. Заучивать с понедельника — плохой квест.

14.1 REPL

Read-Eval-Print Loop: прочитал, посчитал, напечатал, снова

ждёт. Как попугай с калькулятором.

На практике: чёрное окно SBCL со звёздочкой `*`. Пишешь форму, жмёшь Enter, видишь ответ. Не «запустить проект». Разговор. Если не потрогал руками — ты читал про Lisp, а не занимался им.

14.2 JDK

Java Development Kit. Коробка, в которой лежит плита **и** переводчик.

`java` запускает уже сваренное. `javac` компилирует исходник. Без JDK у тебя иногда есть только `java` (JRE / какой-нибудь runtime) — запустить чужое можно, своё не собрать. Для

этой книги нужна 21-я. Не «какая нашлась в магазине приложений».

14.3 JVM

Java Virtual Machine. Плита, на которой бежит байткод. Не твоя операционка, а ещё один слой: «мы притворимся одинаковой машиной на маке, Windows и сервере».

Поэтому `.class` не «программа Windows». Поэтому ошибка OutOfMemory — про память **этой** плиты, не всегда про диск. Поэтому «у меня в IDEA одно, в терминале другое» часто значит: две JVM, разные.

14.4 class

Чертёж вещи. Не сама вещь.

`class Task` говорит: у задачи будут id, название, флажок. `new Task(...)` — уже железяка в памяти. Файл обычно называется так же, как чертёж: `Task.java`. Java это обожает. Станция тоже любит подписи на ящиках.

14.5 method

Глагол у вещи. «Что умеет делать».

`complete()` у задачи, `add(...)` у склада. `main` — особый метод: сюда JVM входит, как в главный люк. Функция в Lisp — родственник по духу, только без анкеты `public static`.

14.6 HTTP

Язык писем между браузером (или твоим `curl`) и сервером. Не «интернет целиком». Интернет — коридор. HTTP — бланк: метод, путь, заголовки, тело.

GET — «покажи». POST — «прими новое». 200 — ок. 404 — нет такой двери. 500 — у них на кухне пожар. Пока не выучишь бланк, Spring будет казаться магией кнопок.

14.7 JSON

Текст, который притворяется данными: `{"energy": 80, "room": "reactor"}`. Фигурные — объект, квадратные — список. Кавычки у ключей обязательны, в отличие от твоего настроения.

Это не база. Это конверт. База живёт у них, JSON ездит по HTTP. Сломаешь запятую — «их машина» скажет «не прожевала», и будет права.

14.8 Git

Не GitHub. Git — дневник папок у тебя на диске: что изменилось, когда, зачем. GitHub — шкафчик в интернете, куда этот дневник ещё кладут.

Без Git у тебя есть «финальная версия» и «финальная2_точно». Со станцией так не летают. `git status` — посмотреть под ноги. Делай это чаще, чем `git commit`.

14.9 commit

Снимок: «вот в таком виде папка была, вот сообщение зачем». Не сохранение файла. Файл сохраняет редактор. Коммит — метка, к которой можно вернуться.

Пиши сообщение как человеку через полгода. `week4: save tasks` лучше, чем `asdf`. Коммит не обязан быть красивым. Обязан быть.

14.10 SQL

Язык разговора с таблицами. `SELECT` — дай строки. `INSERT` — положи. `UPDATE` / `DELETE` — как звучит, только без «ну почти все».

Это не Java. Это отдельный диалект кладовщика. Spring его прячет, пока не спрячет плохо — тогда ты всё равно откроешь консоль Postgres и напишешь `SELECT` руками. Лучше уметь до этого дня.

14.11 JOIN

Склеить строки двух таблиц по ключу. Задачи плюс имена людей, отсеки плюс датчики.

Без JOIN новичок делает два запроса и склеивает в уме. С JOIN склеивает база, она для этого стоит. Забудешь условие склейки — получишь декартово произведение: каждая задача с каждым человеком, праздник, потом дисковое место плачет.

14.12 index

Оглавление для столбца. «Где все задачи пользователя 42» без чтения всей таблицы с фонариком.

Индекс занимает место и чуть замедляет запись: надо обновить и таблицу, и оглавление. На трёх строках не нужен. На трёх миллионах — очень даже. `EXPLAIN` покажет, пользуются ли им. Гадание по ощущению — нет.

14.13 transaction

«Всё или ничего». Снять энергию с реактора **и** включить антенну. Упало второе — первое не должно остаться наполовину.

`@Transactional` на сервисе — «эта работа одна каша». Не магия скорости. Магия честности. Две кнопки «сохранить» без транзакции — способ получить станцию, которой нет в природе: деньги списались, билет не выдали.

14.14 bean

Объект, которым управляет Spring: создал, отдал другим в конструктор, один на всех (часто).

Не зерно. Не шутка про кофе, хотя все её делают, вот и я. Если ты написал `new` у сервиса сам — это не бин, это ты. Контейнер тогда пожимает плечами.

14.15 controller

Тонкий вход с улицы. HTTP пришёл сюда. Дальше — не сюда.

Контроллер переводит письмо в вызов сервиса и обратно в JSON. Если в контроллере SQL — станция смотрит на тебя. Если бизнес-правило «нельзя пустое имя» только в контроллере — кто-то вызовет сервис в обход, и имя будет пустое.

14.16 service

Правила станции. Можно ли стыковаться, хватает ли энергии, не дублируется ли задача.

Сюда кладут смысл. Сюда кладут транзакцию. Тесты без

веба живут здесь. Если сервиса нет, и всё в контроллере — у тебя не архитектура, у тебя один большой `main` с аннотациями.

14.17 repository

Дверь в базу. «Положи задачу», «дай задачу по id». Не «реши, имеет ли право пользователь».

JPA-репозиторий часто выглядит как интерфейс без тела — Spring дописывает реализацию. Это удобно, пока не удобно слишком: тогда пишешь запрос сам и вспоминаешь SQL без стыда.

14.18 Docker

Способ сказать: «запусти эту же кухню у тебя, не только у меня». Не виртуальная машина с виндой внутри (хотя рядом стоит). Коробка с процессом и его банками.

«У меня работает» лечится не мемом, а Dockerfile и `compose`. Первый раз будет больно. Второй — меньше. На Windows сразу с WSL2, иначе боль вечная.

14.19 image

Снимок кухни: какие банки положить, какой рецепт запускать. Файл (почти), который можно скачать.

Образ не бежит. Он лежит. `postgres:16` — чужой образ базы. Свой образ приложения — «JDK + мой jar + как стартовать». Версии образов фиксируй. `latest` — сюрприз в три часа ночи.

14.20 container

Запущенный образ. Каша на плите из того снимка.

Остановил контейнер — процесс остановился, но записываемый слой остался. Удалил и заменил — данные только в этом слое могут исчезнуть. Данные, которые жалко, клади в том (volume). Два контейнера из одного образа — две каши, не одна.

14.21 cache

Шпаргалка рядом со стойкой: «это мы недавно уже считали». Быстро. Врёт, если забыть выкинуть.

Кэш без инвалидации — как датчик, который показывает вчерашний воздух. `@Cacheable` красиво. `delete` при обновлении обязательно. Иначе демо: «я поменял, а сайт нет». Сайт честный. Шпаргалка старая.

14.22 queue

Очередь работ: положили сообщение, кто-то потом возьмёт. Не «сделай сейчас в этом же запросе».

Письмо «задача создана» может подождать. Пользователь уже

получил 200. Письмо обработает другой повар. Если этот повар умер — письмо должно **остаться**, не раствориться. Иначе очередь — театр.

14.23 Kafka

Очередь, которая ещё и журнал: сообщения помнятся, их можно перечитать с места.

Не первый инструмент джуна. Не доказательство взрослости. В резюме без своего кода — красная тряпка. В этой книге появляется поздно и по делу: «много событий, хотим не потерять». Если дел на один монолит — живи без Кафки, станция выдержит.

14.24 stack trace

Портянка «кто кого вызвал, когда упало». Читать **снизу вверх** или сверху — как удобно, но **первая понятная твоя строка** важнее чужих внутренностей Spring.

Это не шифр. Это карта коридоров в момент пожара. Номер строки — подарок. Если его нет, ты смотришь не тот билд. Не бойся длины. Бойся пустого `catch`.

14.25 null

«Коробки нет». Не ноль. Не пустая строка. Нет ящика.

В Java `null` кусает: `NullPointerException` — ты попросил метод у отсутствия. В

Lisp родственник по духу — `nil`, но с другим характером. На собесе «что такое null» — не шутка. Скажи: ссылка никуда. Потом скажи, как не принести его в сервис без нужды.

14.26 exception

Исключение. Способ крикнуть «так нельзя» вместо тихого вранья.

`throw` — крик. `try/catch` — услышал. Лови конкретное. Пустой `catch` — выключить сирену изолентой. Checked в Java (`IOException`) — компилятор-кляузник: признайся, что диск бывает против. Бесит. Полезен.

14.27 recursive

Функция вызывает себя на **меньшем** куске. Коридор чистят: этот отсек, потом остальные.

Без стопа (`null` список, `n == 0`) — `Control stack exhausted` / `StackOverflowError`. Это не мистика. Это матрёшка без последней куклы. Рекурсия не «для умных». Рекурсия для списков и деревьев, пока не щёлкнет. Потом можно `loop`.

14.28 cons

Склеить голову и хвост. Главный конструктор списков в Lisp. `(cons 'airlock '(corridor))` → `(AIRLOCK CORRIDOR)`.

Выглядит как опечатка в учебниках дедушек. Это не опечатка. Список — цепочка cons. Поймёшь cons — поймёшь, почему `rest` дешёвый, а «добавь в конец» в лоб — нет.

14.29 symbol

Бирка. Имя. В Lisp `'reactor` — не строка `"reactor"`. Строку сравнивают иначе, бирку иначе.

Символ — слово языка. Он может именовать переменную или функцию, быть меткой отсека. Апостроф значит «не вычисляй, положи как есть». Голый `reactor` в выражении ищет значение переменной; если она не связана, будет ошибка. `(reactor)` вызывает

функцию. Бирка одна, работа разная.

14.30 t

Да. Истина. «Во всех остальных случаях» в `cond`.

Не число 1. Не строка `"true"`. В Lisp просто `t`. Писать `истина` в коде не надо. Писать `true` тоже не обязательно — это из соседнего языка. Здесь `t`.

14.31 nil

Нет. Пустой список. «Ничего». Один актёр на две роли, и всем нормально.

`(if nil ...)` не зайдёт. `(rest '())` даст `nil`. Ложь и пустота встретились в шлюзе и не стали спорить. В Java так

нельзя, там `false`, `null` и пустой `List` — трое разных граждан.

14.32 Maven

Сборщик Java-проекта: зависимости, тесты, упаковка. Файл-ритуал `pom.xml`.

`./mvnw test` — «поставь, что написано, прогони тесты». Wrapper (`mvnw`) кладут в репозиторий, чтобы у соседа была та же версия Maven. Не надо «скачай Maven отдельно и настрой как жрец». На Windows без WSL — `mvnw.cmd`.

14.33 Flyway

Миграции базы: файлы `V1__tasks.sql`, `V2__idx.sql`.

История схемы, не «подкрутил руками в проде».

Порядок важен. Один раз применилось — не редактируй старый файл как будто никто не видел. Видели. База видела. Новый файл — новый номер. Это скучно и спасает станции.

14.34 JPA

Договор «класс ↔️ таблица». Поля — столбцы. `save` — почти INSERT/UPDATE.

Hibernate часто под капотом. `@Entity` — «это строка таблицы, не просто POJO в вакууме». Удобно, пока N+1 не придёт. Тогда вспоминаешь, что под одеялом SQL, и это нормально, не измена.

14.35 equals

«Это та же задача по смыслу?», не «это тот же ящик в памяти?».

== для объектов в Java — «тот же адрес на стойке». `equals` — «считаем равными». Для строк почти всегда `equals`. Забудешь — баг на собесе и в проде. Record часто даёт `equals` сам. Свой класс — напиши, или живи с болью.

14.36 thread

Нитка выполнения. Несколько поваров в одной программе.

По умолчанию твой `main` — одна нитка. Веб-сервер — много: по запросу. Две нитки портят один счётчик без `synchronized` / нормальной очереди — гонка.

Не начинай с «параллельности ради резюме». Начни, когда один повар не успевает, и измерь.

14.37 port

Дверь на машине. Число. 443 — https, 8080 — учебный сервер, 5432 — Postgres.

Два процесса одну дверь не делят: «port already in use». Значит, старый сервер жив или Slack занял. `localhost:8080` — эта машина, эта дверь. Не «сайт в облаке». Дверь.

14.38 localhost

Сам себе. `127.0.0.1`. Кухня, не улица.

Браузер на `localhost` не ходит «в интернет», он стучит в процесс на этой же коробке. В WSL иногда «localhost Windows» и «localhost Ubuntu» — две кухни с дыркой. Если база «не видна», проверь, **чья** это дверь.

14.39 API

Договор: как звать чужую программу. Часто HTTP+JSON, не всегда.

«Есть API» значит: можно письмом, не только мышкой. Твой Spring-контроллер **и есть** API. Чужой httpbin — тоже. Документация API — какие двери, какие письма, какие ответы. Без неё ты археолог.

14.40 REST

Стиль HTTP-API: ресурсы, глаголы, статусы. `/tasks/3` — задача 3. GET — читай. PUT/PATCH — меняй.

Не религия. Не «обязательно микросервисы». Слово на вакансиях значит: «умеешь сделать CRUD без изобретения своего Skype». Если спор «это настоящий REST» длится час — иди пиши работающий GET.

14.41 bytecode

То, во что `javac` пережёвывает `.java`. Файл `.class`. Не машинный код твоего процессора, а диалект JVM.

Поэтому одна Java-программа бежит на разных ОС — плита

разная снаружи, внутри байткод один. Смотреть байткод каждый день не надо. Знать, что зелёная кнопка его производит — надо.

14.42 annotation

Ярлык над кодом: `@Override`, `@RestController`, `@Test`. Компилятор или фреймворк читает ярлык и ведёт себя иначе.

Это не комментарий. В комментариях врут люди. Аннотацию ест инструмент. Лишняя `@Transactional` на `private` методе — ярлык, который **не сработает**, как табличка «мокро» на сухом полу. Читай, куда клеишь.

14.43 DTO

Data Transfer Object. Коробка «вот что отдаём на улицу», не обязательно вся сущность из базы.

Чтобы в JSON не утекло поле `passwordHash`. Чтобы не тащить ленивую связь и не словить ленивый пожар. Скучное слово. Взрослая привычка. На первых порах можно отдать сущность — потом обожжёшься, и словарь вдруг станет родным.

14.44 IDE

Редактор, который ещё компилирует, отлаживает, подсказывает. IntelliJ — наш. VS Code — тоже, только Java-плагины собери.

Не компилятор. Не Git. Не замена голове. Кнопка Run внутри вызывает те же `javac`/`java`, только без твоего `pwd`. Раз в неделю запускай из терминала, чтобы помнить.

14.45 PATH

Список папок, где система ищет программы, когда ты пишешь `javac` без полного пути.

«Команда не найдена» часто значит: банка есть, в PATH коридора нет. Поставил JDK — пропиши PATH или закрой-открой терминал. В WSL и на маке это обычная боль первого дня, не приговор.

14.46 stdin / stdout

Стандартный вход и выход. Клавиатура и экран по умолчанию.

`Scanner(System.in)` читает stdin. `System.out.println` пишет stdout. Можно подменить файлом: `java Energy < input.txt`. Ошибка часто отдельно: stderr. Поэтому «пропал вывод» иногда значит: смотришь не в тот поток.

14.47 hash

Отпечаток данных. Один и тот же вход у одной хеш-функции — тот же отпечаток. У разных данных отпечаток может совпасть: это коллизия. Хеш — подсказка, не доказательство равенства.

`HashMap` ищет по отпечатку ключа, поэтому ключ должен нормально равняться (`equals` + `hashCode`). Для настоящих паролей нужна специальная адаптивная схема хеширования с солью, например BCrypt из курса; обычного хеша недостаточно. `sxhash` в учебном Lisp — игрушка, не сейф.

14.48 N+1

Сначала один запрос «дай список», потом по запросу **на каждый** элемент. Сто задач — сто один поход в базу.

На трёх строках не видно. На проде видно. Лечится JOIN / `join fetch` / не лазить в связи в цикле. Если лог SQL похож на

пулемёт — ты видел N+1. Поздравляю. Теперь имя есть.

14.49 garbage collector

Сборщик мусора. JVM (и SBCL тоже) сам выкидывает банки, на которые никто больше не указывает.

Ты не `free` каждый объект. Это не значит «память бесконечная». Утечка в Java — держать список ссылок на всё, что когда-либо видел. GC честный: не трогает то, до чего ты до сих пор дотягиваешься.

14.50 endpoint

Дверь API: метод + путь. `GET /tasks`. `POST /tasks`.

Не «весь сервер». Одна ручка. В Postman / curl ты дергаешь endpoint. В контроллере один метод часто = один endpoint. Если ручка делает пять разных дел в зависимости от настроения параметра — это уже не дверь, это чердак.

14.51 entity

Объект, который **живёт в базе**. Строка таблицы в виде класса.

Не каждый класс — entity. `Task` с `@Entity` — да. `EnergyReport`, который ты собрал из трёх таблиц только чтобы отдать JSON — скорее DTO. Путаешь — получишь странный `save` и странный JSON.

14.52 primary key

Главный ключ строки. Обычно `id`. Уникальный. По нему JOIN, по нему «дай задачу 3».

Не название задачи: названия повторяются. Не «номер в списке на экране»: список врёт после сортировки. Настоящий id. Станция не спорит с этим уже лет шестьдесят.

14.53 foreign key

Ссылка на чужой primary key. У задачи — `project_id`. «Эта строка про тот проект».

База может следить, чтобы проект существовал. Это ограничение, не каприз. Без него появляются задачи призрачных проектов. С ним —

ошибка при вставке, и это подарок.

14.54 log

Дневник процесса: строки во время жизни. Не `print` для себя в `main`, а нормальные уровни: info, warn, error.

Лог читают, когда уже больно. Пиши так, чтобы будущий ты понял **какой id** и **что хотели**. Не пиши пароли. Не пиши персональные данные «для отладки». Станция записывает энергию, не переписку экипажа.

14.55 pom.xml

Паспорт Maven-проекта. Координаты, версия Java, зависимости, плагины.

XML злой на пробелы в душе, на самом деле просто многословный. Не копируй чужой `pom` целиком «на всякий случай». Каждая зависимость — чужой код в твоей станции. Иногда нужный. Иногда транзитивный цирк.

14.56 Spring

Рама, на которую вешают веб, бин, доступ к базе. Boot — «рама уже с батареями, жми Run».

Не язык. Не замена Java. Слой удобства и сюрпризов. Пока не умеешь без Spring сложить

класс, список и файл — рано. Когда умеешь — Spring экономит недели. Когда не умеешь — экономит понимание.

14.57 constructor

Особое объявление, которое инициализирует новый экземпляр при `new`; имя совпадает с именем класса. Это не обычный метод. Обычному классу без объявленных конструкторов Java добавляет конструктор без аргументов, если у родителя есть доступный конструктор без аргументов. Поля сначала получают значения по умолчанию, вроде нуля или `null`; явные

инициализаторы полей тоже выполняются.

Шутка станции: конструктор — единственный момент, когда можно честно сказать «без id не родишься». Потом сеттеры начинают врать.

14.58 static

«Принадлежит чертежу, не одной железяке». `main` статический, потому что JVM ещё не сделала `new` твоего класса, а войти надо.

Злоупотребляешь `static` всем подряд — у тебя снова глобали со звёздочками, только без звёздочек. Иногда надо. Часто лень.

14.59 package

Коридор имён. `com.module.tasks`. Чтобы два класса `Task` не подрались. Первая строка файла. Папка на диске должна совпасть, иначе Java обижается как кладовщик.

Не религия «как в большой компании». Просто не клади сто классов в корень без таблички.

14.60 import

«Возьми чертёж из того коридора». Не копипаста кода. Не «скачать из интернета». Если без import — пиши полное имя, как адрес с индексом.

Звёздочка `import java.util.*` работает. Потом в IDE не видно,

откуда `List`. Для учёбы лучше явный импорт.

14.61 boolean

Да/нет. `true` / `false`. Не `t` и не `nil`. Не 1 и 0, хотя в других языках врут, что одно и то же.

`if (energy)` в Java не прокатит, если `energy` это `int`. Кладовщик требует явный вопрос. Иногда бесит. Реже врёт.

14.62 String

Текст. Объект, не «массив букв, который ты сам паяешь», хотя внутри почти так.

Склеивание `+` удобно и иногда стыдно по скорости. Для дашборда хватит. `equals` для сравнения, мы это уже орали.

Кавычки только двойные: `'a'` в Java — символ, не строка.

14.63 int

Целое в коробке фиксированного размера. Деление `7 / 2` даст `3`. Дробь выкинули в шлюз без сирены.

Для среднего пиши `3.0` или `double`. Для денег на работе — не `double`, но это уже не первый месяц, слава богу.

14.64 array / список

Массив — ящик на N гнёзд, N решил при рождении. `ArrayList` — ящик, который умеет докладывать.

Индексы с нуля. Люди с единицы. Переводчик — ты.

`IndexOutOfBounds` — подарок: гнезда нет, не «элемент null».

14.65 404 / 500

404 — двери нет. 500 — дверь есть, за ней пожар.

Путать их стыдно на собесе и в логах. Клиенту иногда врут 404 вместо 403, чтобы не светить, что секрет существует. Это политика, не HTTP-романтика.

14.66 cookie / session / JWT

Способы помнить, кто ты, между письмами. HTTP сам по себе забывчивый, как CPU без RAM.

Cookie — записка в браузере. Сессия — записка на сервере плюс номер. JWT — записка,

которую сервер подписал и отдал с собой. Для словаря: **не логин в каждом клике**. Детали — когда дойдёшь до Spring Security. Не раньше, пожалуйста.

14.67 curl

Терминальный браузер без картинок. `curl URL` — пошли GET, напечатай тело.

Флаги потом: `-i` заголовки, `-X POST`, `-d` тело. Если умеешь curl, умеешь проверить API без Postman. Postman красивее. curl всплывает на сервере, где нет мыши.

14.68 WSL

Windows Subsystem for Linux. Ubuntu часто живёт **внутри** Windows. Не вторая Windows в окошке и не «поставь Linux вместо». Терминал и файлы Linux рядом, с дыркой наружу.

Эта книга на Windows живёт здесь: `sbcl`, `javac`, `./mvnw` — в WSL, не в CMD. Диск Windows торчит как `/mnt/c/`. Можно. Права файлов там клоуны. Клади учебный код в домашний каталог Ubuntu, `~/dev/`. . . . Если «в проводнике вижу, в WSL нет» — смотришь **две** кухни. PATH тоже два: тот, что в PowerShell, станцию не кормит.

14.69 SDK

Software Development Kit. Коробка, чтобы **собрать** софт под платформу: компилятор, библиотеки, документация, иногда эмулятор.

JDK — SDK для Java. Android SDK — для телефона. Runtime (`java` без `javac`) — плита без сварочника: чужое запустишь, своё не сваришь. IDE не SDK. IDE — мастерская, которая **зовёт** SDK. На вакансии «опыт с SDK» часто значит: ты ставил коробку и собирал хоть что-то, не только открыл редактор.

14.70 JAR

Java ARchive. Почти zip: `.class`, ресурсы, манифест. Расширение `.jar`.

`java -jar app.jar` — запусти сваренное как программу. Maven `package` кладёт такой файл в `target/`. На сервер едет jar, не папка с `.java`. Разорвать jar как zip можно, чтобы посмотреть. Править `.class` хекс-редактором — цирк. Исходник правят, потом снова `package`.

14.71 classpath

Список мест, где JVM ищет классы и банки, **когда программа уже бежит**.

`ClassNotFoundException` часто значит: файл на диске есть, в classpath коридора нет. В IDE зелёная кнопка собирает classpath сама. В терминале `java -cp "lib/*:classes" Main` — ты. Если настроен Exec Maven Plugin, Maven подставляет зависимости в этот список, поэтому `./mvnw compile exec:java -Dexec.mainClass=Main` и «голый» `java` ведут себя разно. Пока это не щёлкнет, фраза «у меня в IDEA работает» будет приходить раз в неделю, как течь в реакторе.

14.72 new

Оператор «сделай железяку по чертежу». `new Task()` — объект в памяти, хозяин ты.

Рядом bean. Spring создаёт сервис **сам** и отдаёт в конструктор. Если в контроллере написал `new TaskService()` — это не бин, это вторая параллельная станция в кармане. Тесты врут, транзакция не та, база другая. Правило: `new` для своих данных (`Task`, `ArrayList`, DTO). Для сервиса и репозитория — контейнер. Вот вся разница bean vs `new`, не философия зёрен.

14.73 migration

Скрипт «схема базы стала другой». Файл с номером. Flyway или Liquibase применят его **один раз** и запоминают в служебной таблице.

Это не UPDATE руками на проде. Прод уже прожевал V3, а ты подкрутил V1 — у тебя локально «и так сойдёт», у команды ад. Новый файл — новый номер. Откат часто ещё одна миграция, которая чинит, а не машина времени. Replica читает ту же историю. Иначе узлы разъедутся, и это уже не шутка про изоленту.

14.74 replica

Копия базы (или сервиса): чтобы читать с нескольких стоек и не умереть в одиночку.

Пишешь обычно в primary / leader. Читаешь иногда с replica. Шутка физики: копия отстаёт на секунду, и GET «не видит» только что сохранённую строку.

Это replication lag, не баг твоего `save`. На учебной станции одна Postgres — replica не нужна. На собесе слово значит «копия», не «второй Docker ради резюме».

14.75 offset

Закладка в журнале Kafka: «я прочитал досюда». Число. Не id задачи и не номер строки в SQL.

Консьюмер двигает offset, когда обработал. Сдвинул раньше — сообщение для себя потерял. Не сдвинул — после рестарта съест снова. Отсюда «at least once» и идемпотентность приходят парой. Пока Kafka нет — представь закладку в бортжурнале станции: страница 47, не «где-то про изоленту».

14.76 consumer group

Несколько едоков Kafka, которые делят **один** журнал так, чтобы одно сообщение внутри группы съел **один**.

Две группы — два независимых чтения: и дашборд, и биллинг могут съесть одно и то же событие. Два едока **в одной** группе — делят партиции, не работу. Имя группы — часть договора: сменил строку — offsets другие, «как будто ничего не читал». Отладка начинается с вопроса «какая группа?», не с перезапуска брокера из паники.

14.77 CI

Continuous Integration. Робот гоняет тесты на каждый push

или PR, не «у меня на ноуте зелёное».

GitHub Actions, GitLab CI, Jenkins — кухни разные, идея одна: свежий клон, `./mvnw test`, галочка или крест. CI красный, у тебя зелёный — врёт **чья-то** кухня. Часто: файл не в git, другая версия JDK, тест, который смотрит часы. Лечится не кнопкой «ещё раз Run». Лечится логом робота. CD (delivery/deploy) — сосед: ещё и **выкатить**. Сначала научись, чтобы робот умел сказать «тесты упали».

14.78 linter

Программа, которая ворчит на стиль и глупые ошибки **до** прогона.

Не компилятор. Компилятор говорит «это не Java». Линтер говорит «это Java, но == у строк, и переменная висит». Checkstyle, SpotBugs, ESLint в соседнем мире. Правила живут в репозитории, не в «у меня в IDE жёлтое». Выключить линтер целиком — сирена изолентой. Одно правило глупое — выключи **его**. Formatter (пробелы, переносы) — родственник, не враг. Споры «табы vs пробелы» лечат файлом в репо, не чатом до утра.

14.79 PR

Pull Request. В GitLab чаще Merge Request. Просьба: «возьмите мою ветку в общую».

Не коммит. В PR коммитов может быть пачка. Там дифф, описание, бот CI, комментарии людей. Мелкий PR легче смотреть. PR на три тысячи строк «я всё сделал» — никто не читает, вливают из усталости, потом ищут пожар. Для учёбы: PR сам себе тоже учит писать «что и зачем», не только код. Review — человек смотрит дифф. Не суд. Не «найди десять ниток».

14.80 rebase

Переложить свои коммиты поверх свежего `main`, как будто ты начал сегодня.

История становится прямой. `merge` делает узел: «здесь встретились две линии». Rebase

врёт изящно. Не rebase-ь ветку, которую уже тянут другие люди: перепишешь историю им под ноги. Свою учебную до PR — можно. Если страшно — `merge`. Станция не снимет зачёт за отсутствие дзена. `git rebase -i` на первой неделе — способ сломать вахту и научиться `reflog` раньше, чем хотелось.

14.81 conflict

Два изменения в **одном** месте. Git не знает, чья строка святая.

Маркеры `<<<<<<<`, `=======`, `>>>>>>>` — не порча файла, а бланк. Убери бланк, оставь смысл, `git add`. Конфликт не «Git сломался». Конфликт — два механика крутили один вентиль. Лечится чтением **обоих** кусков, не

кнопкой «всё слева». После — собери и прогони тесты. Иначе вольёшь станцию, которая не компилируется, зато история «чистая».

14.82 8080

Учебный порт веб-сервера. Не 80: тот часто занят и просит прав админа. Не 443: это https в проде.

`localhost:8080` — браузер стучит в **эту** машину, **эту** дверь. «Connection refused» — процесса за дверью нет. «Already in use» — процесс есть, второй не влезет. Убей старый или смени порт. 8081 в чужом гайде — не священное число, а другая дверь, когда 8080 занята.

localhost — см. выше: кухня, не орбита.

14.83 CORS

Cross-Origin Resource Sharing. Браузер не даст сайту с одного адреса дёргать API с другого, пока сервер не скажет «можно».

Это защита **пользователя в браузере**, не шифр и не «бэк сломался». `curl` CORS не соблюдает: curl не браузер. Поэтому «в curl 200, на фронте ошибка» — часто CORS. Пока нет отдельного фронта на другом порту, можно не лечить. Когда появится — заголовок на **сервере**. Расширение «выключить безопасность браузера» — не профессия, а изолента на сирену.

14.84 lazy

«Пока не тронь». В JPA связь подгружается в момент обращения к полю, не в момент первого `SELECT`.

Удобно, пока сессия к базе жива. Сессия закрылась, ты ткнул `task.getProject()` — `LazyInitializationException`. Классика станции. Лечится не «поставь EAGER на всё» (привет, N+1 и JOIN на полэкрана). Лечится: достань нужное, пока репозиторий ещё в транзакции; либо DTO; либо явный fetch. Ленивость — не лень программиста. Это **когда** ехать в базу.

14.85 eager

«Тащи сразу». Связь грузится вместе с сущностью.

На одной задаче с одним проектом — ок. На списке ста задач, у каждой комментарии, у комментариев авторы — дерево SQL. Eager как изолента на все трубы: держит, пока нельзя дышать. У многих `@ManyToOne` дефолт как раз eager, и ты уже платишь, не видя счёта. Смотри лог SQL. Пулемёт — у явления есть имена: N+1, lazy, eager. Primary key и foreign key тут ни при чём: связь **есть**. Спор только **когда** её тащить.

14.86 NULL

В SQL: «значения нет». Не ноль. Не пустая строка. Отдельный зверь.

`WHERE energy = NULL` строки с пустой энергией **не найдёт**. Надо `IS NULL`. Сравнение с NULL даёт UNKNOWN, не true — поэтому `AND`/`OR` там плывут иначе, чем в Java. В Java `null` — «ссылки нет». Похожие буквы, разный ритуал. `Integer energy` может быть `null` (коробка пустая). `int energy` не может быть `null`. Поле или элемент массива типа `int` по умолчанию равен нулю. Локальному `int` надо присвоить значение до использования, иначе компилятор ругается. Принять ноль по умолчанию за «датчик

показал ноль» — уже можно соврать. JDBC: `wasNull()`, иначе ноль и NULL склеются в одну кашу. Выключить реактор нулём, думая, что выключил отсутствием — сюжет Щ.

14.87 schema

Чертёж таблиц: столбцы, типы, primary key, foreign key, ограничения.

Данные — строки. Схема — правила полок. Миграция меняет схему. `SELECT` без схемы — археология. В Postgres слово ещё и пространство имён внутри базы (`public`). На первом месяце: «как устроены таблицы», не три значения из документации. Нарисуй

таблицы на бумаге. Бумага врёт меньше, чем «ну там entity».

14.88 environment variable

Переменная окружения. Подпись на двери **процесса**: `DATABASE_URL`, `JAVA_HOME`.

Не поле класса. Не глобаль со звёздочками. ОС (или Docker, или CI) кладёт строку, программа читает. Пароли в `application.properties` в git — плохая шутка. Пароли в env — обычная взрослая. `echo $PATH` — ты уже смотрел env. «У соседа работает» — сравни env, не только код. Код одинаковый. Двери разные.

14.89 working directory

Папка, **от которой** считаются относительные пути. `station.txt` без пути — «там, где процесс встал».

Не папка исходника. Не корень репо, если ты его не выбрал. `pwd` в терминале. Working directory в IDEA. WSL vs проводник Windows. Три разных «здесь». Сейв «пропал» — почти всегда это. Напечатай абсолютный путь один раз. Потом спорь с конфигурацией, не с призраками.

14.90 branch

Ветка. Параллельная линия коммитов. `main` — общая. `week4-save` — твоя песочница.

`git switch` не копирует папку магией: он переставляет файлы под эту линию. Незакоммиченное может приехать с тобой или помешать. Ветка дешёвая. Бояться её — как бояться второй кружки. Бояться незакоммиченной каши на двух ветках сразу — здраво. `origin` — чужой шкафчик (часто GitHub), не святой синоним ветки.

14.91 0.0.0.0

«Слушай на всех интерфейсах этой машины», не только на localhost.

Сервер на `127.0.0.1` виден себе. На `0.0.0.0:8080` — ещё и с телефона в той же Wi-Fi, если

файрвол пустит. Новичок открывает «на всех» и думает, что это интернет. Это кухня с форточкой в подъезд, не орбита. Для учёбы localhost хватит. `0.0.0.0` — когда правда надо с другой коробки.

14.92 override

`@Override` — «я подменяю метод предка», не новый метод с похожим именем.

Без аннотации опечатка в имени даст **новый** метод, а тот, что ты думал переопределить, останется старым. Компилятор молчит. Аннотация ругается. Клей её всегда. Это сирена на опечатке `equals` / `hashCode` / `toString`, не бюрократия. Overload — сосед: то же имя,

другие аргументы. Путать overload и override на собесе — классика, как == у строк.

14.93 interface

Договор: «умеешь вот эти методы», без рассказа **как**.

`List` — интерфейс. `ArrayList` — одна реализация, `LinkedList` — другая. В типе переменной пиши `List`, если нет причины прилипнуть. Spring любит интерфейсы репозиториев: контейнер подсунет тело. На первом месяце интерфейс — розетка, не «гексагональная архитектура». Розетку понял — гексагон подождёт.

14.94 401 / 403

401 — не представился (или билет не приняли). 403 — представился, сюда нельзя.

Путать с 404 стыдно иначе: 404 двери нет, 403 дверь есть, охрана против. Свой API: не мажь всё 500. 400 — клиент криво написал. 500 — ты. Клиент должен понять, **чья** это проблема, без чтения твоей души.

14.95 volume

Том Docker: папка, которая переживает смерть контейнера.

Остановка и перезапуск того же контейнера не стирают его записываемый слой. А удаление и замена контейнера

могут потерять данные, хранившиеся только в нём. Volume — кладовая **рядом**, не внутри одноразовой каши. На учёбе: один named volume для базы. `compose down -v` снесёт том вместе с контейнером — флаг `-v` здесь не «verbose», а «убери кладовую». Прочти дважды, потом жми.

14.96 healthcheck

Дверь «я живой?». Часто `GET /health` или `/actuator/health`.

Оркестратор стучит сюда, не на главную страницу. Health зелёный, а ответы 500 — ты проверял не то. Если health ходит в базу, падение Postgres станет «весь сервис мёртв» в глазах робота. Иногда это

честно. Для дашборда станции health — команда `status` с энергией. Тот же инстинкт, только без оркестратора.

14.97 timeout

Сколько ждать, прежде чем сказать «дверь молчит». Сеть, база, `HttpClient`.

Без timeout запрос висит, пока не надоест ОС. Слишком короткий — живая база кажется мёртвой в час пик. На учёбе десять секунд на httpbin — нормально. В проде число берут из измерения, не из «ну тысяча, чтобы наверняка». Retry без timeout — способ ударить лежачего ещё раз.

14.98 idempotent

Один и тот же запрос дважды даёт тот же **смысл**, не обязательно тот же байт в логе.

GET обычно идемпотентен: «покажи задачу 3» хоть десять раз. POST «создай задачу» дважды — две задачи, если не стеречься. Kafka at least once + неидемпотентный обработчик — дубли в базе. На станции: `tape-leak` второй раз не должен тратить вторую изоленту, которой уже нет. Вот и слово.

14.99 prepared statement

SQL с дырками под значения, которые драйвер подставит **как данные**, не как текст запроса.

`WHERE name = ?` плюс параметр — имя с кавычкой не станет вторым запросом. Склейка `'...' + userInput` — SQL injection, классика, станция открывает шлюз незнакомцу. На JPA `save` это прячет. Ручной JDBC — не прячет. Даже учебный `SELECT` лучше с `?`, чем с конкатенацией «ну это же своё число».

14.100 record

Короткий класс «только данные»: поля, конструктор, `equals`, `hashCode`, `toString` — Java пишет сама.

Удобно для DTO. Не entity: JPA любит пустой конструктор и сеттеры, record с этим спорит. Не замена всего. Если ловишь себя на восьмистрочном классе

с тремя полями и руками написанным `equals` — record. Если ловишь себя на «а добавлю-ка поведение» — обычный класс, не геройствуй.

14.101 CLOS

Объектная система Common Lisp: классы со слотами, глаголы снаружи (`defgeneric` / `defmethod`), несколько родителей сразу. Java до сих пор даёт один `extends`. Не «Java в скобках». Другой центр: сначала действие, потом паспорт железяки.

14.102 макрос

Код, который пишет код. Получает формы, возвращает форму. `macroexpand-1` — рентген.

Функция, которой «просто лень дать имя», макросом не является. Если аргумент с побочным эффектом подставился дважды — ты написал `twice-wrong`.

14.103 живой образ

Процесс Lisp, в котором можно переопределить метод, не убивая данные. Так чинили Remote Agent на Deep Space 1. Gradle так не умеет, и это не его вина: его не звали на зонд.

Если слова нет в этом словаре — не беда. Часто это либо имя чужой библиотеки (гугли «что это за зверь»), либо маркетинг вакансии. Сначала спроси: **какой датчик на станции это**

чинит? Если никакой — можно не учить на этой неделе.

15 Бортовой журнал «МОДУЛЬ»

Это не расписание. Расписание — в месяцах 1–6. Здесь — лишние сорок минут, когда квест недели уже зелёный, а пальцы ещё хотят скобок.

Станция «МОДУЛЬ» в основном курсе вечно чинится. В журнале она ещё и **играется**. Сюжет свой: не орки, не подземелья из чужой книги. Орбита, изолента, записки предыдущего механика. Его звали… в журнале вахты стёрлось. Осталась буква «Щ» и пятно от кофе.

Закон станции

Эти квесты не вместо Java. Не вместо занятия. Если сервер лежит — поднимай сервер. Журнал — для воскресенья и для зудящих скобок.

Говорим с SBCL. Файлы клади в `lisp-experiments/station-log/`. Коммить, когда не стыдно **или** особенно когда стыдно.

15.1 Вахта 1. Энергия, которая убегает

Предыдущий механик оставил на столе клочок:

Реактор врёт. На панели 100. В коридоре холодно. Не верь панели. Верь функции.
Мы не будем делать игру с картинками. Мы сделаем мир из скобок. Мир честнее панели.

Запусти REPL, `sbcl`, звёздочка. Сначала глобаль — да, звёздочки, да, кричим:

```
(defparameter *energy* 100)
(defparameter *leak* 7)
```

`*leak*` — сколько энергии съедает каждый тик просто так. Щель в обшивке. Или чайник, который забыли. Для модели всё равно.

```
(defun clamp-energy (n)
  (cond
    ((< n 0) 0)
    ((> n 100) 100)
    (t n)))

(defun tick ()
  (setf *energy* (clamp-energy (-
*energy* *leak*)))
  *energy*)
```

Что только что случилось

`tick` меняет глобаль и **возвращает** новое значение. В REPL ты увидишь число. Это не «процедура, которая молча портит мир» — ты ещё и ответ получил. Удобно смотреть, не оборачивая в `print`.

```
(tick)  ; 93
(tick)  ; 86
```

Два тика — и уже не сто. Панель врала бы, если бы мы её нарисовали один раз и забыли вызывать `tick`.

Подзарядка от солнечной панели, пока станция на светлой стороне:

```
(defun recharge (delta)
  (setf *energy* (clamp-energy (+
*energy* delta)))
  *energy*)
```

```
(recharge 20)  ; зависит, сколько
уже утекло
```

Аварийный расход — открыть антенну, моргнуть Земле:

```
(defun pulse-antenna ()
  (if (< *energy* 15)
      'too-dark
      (progn
        (setf *energy* (clamp-
energy (- *energy* 15)))
        'ping)))
```

Давай медленно `progn` — «сделай несколько форм, верни последнюю». Без него `if` имеет только одно «тогда». Хочешь и отнять, и вернуть символ — пакуй. Иначе вернётся число от `setf`, а `'ping` даже не поедет.

Сыграй в REPL пять тиков, один `pulse-antenna`, ещё тик. Смотри, когда появится `'too-dark`. Это не баг. Это сюжет.

Квест S.L1 · Lisp
`ticks`: число n, повтори `tick` n раз, верни финальную энергию. Без цикла тоже можно — рекурсией. С циклом — `(loop repeat n do (tick))`. Не уходи ниже нуля: `clamp-energy` уже стоит на страже, не обходи его.

15.2 Вахта 2. Отсеки — это граф, не коридор из рекламы

На схеме у входа нарисованы прямоугольники и стрелки. Прямоугольники — комнаты. Стрелки — люки. Это **граф**:

узлы и рёбра. Слово взрослое. Штука — список соседей.

Пять отсеков журнала. Запомни их, они пригодятся в приключении:

- `airlock` — шлюз. Пахнет металлом и чужими перчатками.
- `corridor` — коридор. Лампочка мигает в морзянку, но это просто плохой балласт.
- `galley` — камбуз. Кружка с буквой «Щ». Кофе кончился в прошлом квартале.
- `reactor` — реактор. Тепло. Гудит. Врать панели здесь неприлично.
- `cupola` — купол. Земля в иллюминаторе. Для сюжета: здесь сильный приёмник.

Соседи — alist. Ключ — комната, значение — список, куда есть люк.

```
(defparameter *doors*
  '((airlock  . (corridor))
    (corridor . (airlock galley
reactor))
    (galley   . (corridor
cupola))
    (reactor  . (corridor))
    (cupola   . (galley))))
```

Из шлюза только в коридор. Из коридора — почти всюду. Из реактора назад в коридор, без потайной трубы. Мы могли бы добавить трубу. Тогда граф станет веселее и опаснее. Пока без трубы: иначе первая игра будет про то, как ты застрял в вентиляции, а не про скобки.

```
(defun neighbors (room)
  (cdr (assoc room *doors*)))

(neighbors 'corridor)
; (AIRLOCK GALLEY REACTOR)
```

Можно ли шагнуть?

```
(defun connected-p (from to)
  (member to (neighbors from)))
```

`member` вернёт хвост от находки или `nil`. Для `if` этого хватит: не-`nil` — да.

Что только что случилось `(connected-p 'airlock 'reactor)` → `nil`. Люка нет. Не «почти рядом на бумажке». Рядом на бумажке — это геометрия. В графе есть только рёбра. Станция не обязана быть честной планировкой офиса.

Где ты стоишь:

```
(defparameter *here* 'airlock)

(defun look ()
  (format t "ты в ~a~%" *here*)
  (format t "люки: ~a~%"
(neighbors *here*))
  *here*)
```

```
(defun walk (to)
  (if (connected-p *here* to)
      (progn
        (setf *here* to)
        (look))
      (format t "люка в ~a нет~%"
to)))
```

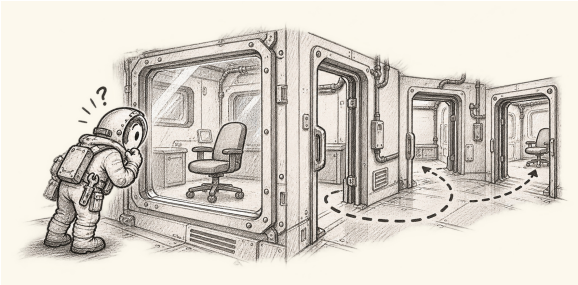
В REPL:

```
(look)
(walk 'corridor)
(walk 'cupola)   ; нет люка,
ругается
(walk 'galley)
(walk 'cupola)
```

Ты только что написал движок приключения. Картинки нет.

Картинка врала бы: будто мир больше, чем пять символов. Мир — пять символов и люки. Честно.

Квест S.L2 · Lisp
`connected-p`: из комнаты A в B **за один шаг** уже есть. Сделай `exits-report`: для **каждой** комнаты напечатай имя и соседей. Данные бери из `*doors*`, не копируй пять `format` руками. `dolist` по `*doors*`.



Вижу отсек. Не вижу двери.
Близость на рисунке — ещё не ребро. Маршрут должен идти по настоящим связям.

15.3 Вахта 3. Записки Щ. и кружка

Теперь предметы. Не инвентарь на сорок слотов. Карман: список символов.

```
(defparameter *pocket* nil)

(defparameter *at*
  (copy-tree
   '((airlock  . nil)
```

```
    (corridor . (tape))
    (galley   . (mug note))
    (reactor  . (wrench))
    (cupola   . (photo)))))
```

`copy-tree` создаёт для инвентаря новые cons-ячейки. `take` и `drop` меняют эти ячейки, поэтому сам цитированный шаблон трогать нельзя. В `reset-world` тоже нужна новая копия.

`tape` — изолента. Ей будут чинить утечку, иначе зачем сюжет. `mug` — кружка Щ. `note` — записка. `wrench` — ключ на семнадцать, единственный. `photo` — Земля, снятая на плёнку, потому что цифровой модуль опять «обновляется».

```
(defun stuff-here ()
  (cdr (assoc *here* *at*)))
```

```
(defun take (item)
  (let ((here-stuff (stuff-
here)))
    (if (member item here-stuff)
        (progn
          (setf *pocket* (cons
item *pocket*))
          (setf (cdr (assoc
*here* *at*))
                (remove item
here-stuff))
          (format t "взял ~a~%"
item))
        (format t "здесь нет
~a~%" item))))
```

Давай медленно `setf` по `(cdr (assoc …))` меняет ячейку в **том же** cons, который живёт в `*at*`. Поэтому мир меняется. Если бы мы собрали новый список «снаружи» и забыли его присвоить — комната вечно держала бы кружку. Это не магия. Это указатель на пару.

```
(defun read-note ()
  (if (member 'note *pocket*)
      (format t "Щ.: утечка в
реакторе. изолента в коридоре. не
пей из кружки.~%")
      (format t "записки нет в
кармане. может, она ещё в
камбузе.~%")))
```

Сыграй сценарно, как репетицию:

```
(defparameter *here* 'airlock)
(walk 'corridor)
(take 'tape)
(walk 'galley)
(take 'note)
(read-note)
(take 'mug)
```

Кружку можно взять. Пить не будем: предыдущий механик предупредил, и учебник не страховая.

Квест S.L3 · Lisp

`drop`: выложить предмет из кармана в текущую комнату. Если предмета нет — честно скажи. `remove` из `*pocket*`, `cons` в список комнаты. Проверь: взял ключ в реакторе, ушёл в коридор, бросил, вернулся — ключа в реакторе нет. Иначе ты менял не тот список.

15.4 Вахта 4. Пять комнат, одна утечка

Игра целиком. Тик энергии на каждый `walk`. Починить реактор можно, только если ты **в** `reactor` и в кармане `tape`.

Собери в файл `module-adventure.lisp` (в REPL потом `(load "module-adventure.lisp")` из той папки, `pwd` помни).

Мир сбрасывается функцией, не «закрой SBCL и молись»:

```
(defparameter *fixed* nil)

(defun reset-world ()
  (setf *energy* 100)
  (setf *leak* 7)
  (setf *here* 'airlock)
  (setf *pocket* nil)
  (setf *at*
        (copy-tree
         '((airlock  . nil)
           (corridor . (tape))
           (galley   . (mug
note))
           (reactor  . (wrench))
           (cupola   .
(photo)))))
  (setf *doors*
        '((airlock  . (corridor))
          (corridor . (airlock
galley reactor))
          (galley   . (corridor
cupola))
          (reactor  . (corridor))
          (cupola   . (galley))))
```

```
  (setf *fixed* nil)
  'ready)
```

`*fixed*` — починили ли щель. Пока `nil`, каждый ход жрёт `*leak*`.

Ход — не отдельная кнопка. Ход — когда пошёл:

```
(defun walk (to)
  (cond
    ((not (connected-p *here*
to))
     (format t "люка в ~a нет~%"
to))
    ((<= *energy* 0)
     (format t "темно. энергия 0.
станция не в настроении.~%"))
    (t
     (setf *here* to)
     (unless *fixed*
       (tick))
     (look)
     (when (stuff-here)
       (format t "на полу: ~a~%"
(stuff-here))))))
```

Починка:

```
(defun tape-leak ()
  (cond
    ((not (eq *here* 'reactor))
     (format t "утечка не здесь.
щит врёт, но трубы в
реакторе.~%"))
    ((not (member 'tape
*pocket*))
     (format t "изоленты нет. Щ.
писал: коридор.~%"))
    (t
     (setf *fixed* t)
     (setf *leak* 0)
     (setf *pocket* (remove 'tape
*pocket*))
     (format t "замотал. гул стал
скучнее. это комплимент.~%"))))
```

Победа — не отдельный экран. Спроси сам:

```
(defun status ()
  (format t "энергия ~a, щель ~a,
карман ~a~%"
          *energy*
```

```
          (if *fixed* 'замотана
'сипит)
          *pocket*))
```

Что только что случилось Партия: `(reset-world)`, `(walk 'corridor)`, `(take 'tape)`, `(walk 'reactor)`, `(tape-leak)`, `(status)`. Энергия чуть утекла по дороге. Дальше тики не жрут. Можно идти смотреть Землю в купол, как человек, у которого смена внезапно стала скучной.

Проигрыш: бродить без изоленты, пока `*energy*` не станет 0. Тогда `walk` отказывается. Станция не взрывается в ASCII-графике. Она просто не открывает люк. Это злее.

Текст «приключения» живёт в `look` и в записке. Можно

добавить описания комнат — alist строк.

```
(defparameter *flavor*
  '((airlock  . "холодный пол. перчатки кого-то, размер не твой.")
    (corridor . "лампочка морзянкой врёт. это просто балласт.")
    (galley   . "кружка Щ. и запах того, что когда-то было кофе.")
    (reactor  . "тепло. если сипит — не поэзия, а щель.")
    (cupola   . "земля большая. ты маленький. изолента важнее лирики.")))

(defun look ()
  (format t "~a~%" (cdr (assoc *here* *flavor*)))
  (format t "отсек ~a | люки: ~a~%" *here* (neighbors *here*))
  *here*)
```

Не превращай это в роман. Две строки на комнату. Игра в REPL живёт ходами, не абзацами.

Квест S.L4 · Lisp
Предмет `wrench` в реакторе. Команда `whack`: если ключ в кармане и ты в `galley` — «починил» лампочку коридора, поставь `*lamp* 'ok`. Если ключа нет — `'need-wrench`. Придумай сам, влияет ли лампа на игру. Хоть одной строкой в `look`. Без влияния — бутафория, тоже сойдёт, но честно напиши в комментарии, что бутафория.

15.5 Вахта 5. Сохранить мир, иначе сон сожрёт кашу

RAM — стойка. Выключил SBCL — каша смахнута. Банка на диске — `with-open-file`.

Мы сохраним не «красивый формат для людей», а то, что Lisp умеет прочитать обратно: `print` / `read`.

```
(defun world-plist ()
  (list :energy *energy*
        :leak *leak*
        :here *here*
        :pocket *pocket*
        :at *at*
        :doors *doors*
        :fixed *fixed*))

(defun save-world (path)
  (with-open-file (out
path :direction :output :if-
```

```
exists :supersede)
    (print (world-plist) out))
  path)
```

Давай медленно `print` пишет так, чтобы `read` понял. `format` пишет так, чтобы понял ты. Для сейва нужен `print`. Для «на полу: кружка» — `format`. Не путай плиты.

```
(defun apply-world (plist)
  (setf *energy* (getf
plist :energy)
        *leak*   (getf
plist :leak)
        *here*   (getf
plist :here)
        *pocket* (getf
plist :pocket)
        *at*     (getf plist :at)
        *doors*  (getf
plist :doors)
        *fixed*  (getf
plist :fixed))
  'loaded)
```

```
(defun load-world (path)
  (with-open-file (in
path :direction :input)
    (apply-world (read in))))
```

Партия:

```
(reset-world)
(walk 'corridor)
(take 'tape)
(save-world "module-save.lisp-
data")
```

Закрой SBCL. Открой снова. Загрузи файл с определениями, потом:

```
(load-world "module-save.lisp-
data")
(status)
(look)
```

Карман должен помнить изоленту. Если не помнит — либо сохранил не тот `pwd`, либо

загрузил определения **после** сейва и перезатёр `reset-world`. Порядок: сначала код игры, потом `load-world`.

Так себе идея
`read` ест lisp-данные. Не корми его файлом из интернета «ну это же текст». Это не JSON-парсер с диетой. Это открытый рот языка. Учебный сейв — свой, с диска рядом.

Квест S.L5 · Lisp
Команда `save` без аргумента пишет `module-save.lisp-data` в текущую папку. Текстовая команда `load` читает; назови её функцию `load-game`, потому что `load` уже занят загрузчиком файлов Common Lisp. Если файла нет — `'no-save`, не падение. Проверь: починил

щель, сохранил, `reset-world`, загрузил — щель снова замотана. Если нет, в plist забыл `*fixed*`.

15.6 Вахта 6. Карта на салфетке и чуть-чуть «умного» хода

Граф уже есть. Можно спросить: есть ли путь **длиннее** одного люка.

Обход в глубину учебный, без очередей из учебника алгоритмов: рекурсия с карманом «уже был».

```
(defun reachable (from)
  (labels ((walk-rooms (room
seen)
             (if (member room
seen)
                 seen
```

```
                (let ((seen2
(cons room seen)))
                  (dolist (n
(neighbors-from (from-table)
room) seen2)
                    (setf seen2
(walk-rooms n seen2)))
                  seen2)))
          (neighbors-from (table
room)
            (cdr (assoc room
table)))
          (from-table ()
*doors*))
    (walk-rooms from nil)))
```

Стоп. Это уже хочется упростить, потому что вложенность путает. Проще так:

```
(defun reachable (from)
  (let ((seen nil))
    (labels ((visit (room)
               (unless (member
room seen)
                 (push room seen)
```

```
                  (dolist (n
(neighbors room))
                    (visit n)))))
      (visit from)
      seen)))
```

Что только что случилось `(reachable 'airlock)` должен вернуть все пять отсеков: граф связный. Отрежь люк коридор↔️камбуз в голове: тогда купол станет островом, если стартовать из шлюза. Попробуй временно `(setf *doors* …)` без `galley` у коридора — и посмотри `reachable`. Потом `(reset-world)`.

Это не курсовая по графам. Это ответ на «почему комната есть на схеме, а `walk` не пускает»: потому что путь — цепочка рёбер, а не соседство по картинке.

Квест S.L6 · Lisp

`steps-to`: из `*here*` в комнату `to`, верни число люков в **одном** известном пути (не обязан кратчайший, но не бесконечный). Нет пути — `nil`. Можно вручную для пяти комнат таблицей, можно обходом. Таблица на пяти узлах — не стыдно. Стыдно вернуть 1 для `airlock → cupola`.

15.7 Вахта 7. Мини-цикл «игра сама спрашивает»

Надоело писать `(walk 'x)` — сделай петлю. Всё ещё REPL-дух: ты не компилируешь движок. Ты читаешь строку.

```
(defun play ()
  (reset-world)
```

```
  (look)
  (loop
    (when (<= *energy* 0)
      (format t "энергия
кончилась. вахта закрыта.~%")
      (return))
    (format t "> ")
    (finish-output)
    (let* ((line (string-downcase
(read-line)))
           (parts (split-line
line))
           (cmd (first parts))
           (arg (second parts)))
      (cond
        ((string= cmd "quit")
(return))
        ((string= cmd "look")
(look))
        ((string= cmd "status")
(status))
        ((string= cmd "take")
(take (intern (string-upcase
arg))))
        ((string= cmd "walk")
(walk (intern (string-upcase
arg))))
        ((string= cmd "tape")
```

```
(tape-leak))
        ((string= cmd "save")
(save-world "module-save.lisp-
data"))
        ((string= cmd "load")
         (if (probe-file "module-
save.lisp-data")
             (load-world "module-
save.lisp-data")
             (format t "нет
сохранения~%")))
        (t (format t "команды:
look walk take tape status save
load quit~%"))))))
```

`split-line` напиши сам: разрезать по пробелу. Учебно:

```
(defun split-line (s)
  (let ((p (position #\Space s)))
    (if p
        (list (subseq s 0 p)
(subseq s (1+ p)))
        (list s))))
```

Двух слов хватит: `walk corridor`. Третье слово станция проглотит

вместе со вторым. Живи с этим или пиши нормальный split позже.

`intern` + `string-upcase` — мост из «текста человека» в символ Lisp. Без этого `walk` сравнивает символ с строкой и никогда не ходит. Классика. Щ. наверняка об это споткнулся, оттого и кружка.

Так себе идея `(play)` заберёт REPL в себя. Выход — `quit`. Не Ctrl+C сразу, дай шанс. Если всё же нажал Ctrl+C, SBCL обычно откроет отладчик. Выбери перезапуск с подписью ABORT, чтобы вернуться к `*`; мир мог остаться посреди хода. Потом `(reset-world)`.

15.8 Вахта 8. Кофейный автомат, который врёт

В камбузе стоит автомат. На кнопке написано «кофе». Внутри — функция. Предыдущий механик Щ. подключил его к `*energy*` станции. Каждый стакан — 4 единицы. Если энергии мало, автомат печатает пар и ничего не наливает.

```
(defun coffee ()
  (cond
    ((< *energy* 4)
     (format t "пар. вранье на
кнопке. энергии ~a~%" *energy*)
     'steam)
    (t
     (setf *energy* (- *energy*
4))
     (format t "жижа. не кофе.
зато горячо. энергия ~a~%"
```

```
*energy*)
     'sludge)))
```

Что только что случилось
Это не новая игра. Это **тот же мир**. `coffee` меняет ту же глобаль, что `tick`. Кофе тратит энергию и после починки щели; в конце концов на следующую кружку уже не хватит. Сюжет честный: изолента не бесконечный реактор.

Добавь в `play` команду `coffee` без аргументов. В `world-plist` ничего нового: энергия уже там. Сейв сам помнит, сколько ты «пил».

Квест S.L7 · Lisp
Автомат наливает, только если ты в `galley`. Иначе `'wrong-room`. Если кружка `mug` в кармане —

текст другой: «хотя бы в кружку Щ., не на пол». Без кружки — на пол, энергия всё равно списывается. Станция не нянька.

Карта на салфетке после автомата не меняется. Меняется только смысл камбуза: теперь это не декорация с запиской, а комната с побочным эффектом. Так оживают графы. Не картинкой. Глаголом.

15.9 Вахта 9. Антенна и трюм — граф растёт не картинкой

На салфетке пять кружков. Станция врёт: есть ещё два люка, которые Щ. не дорисовал,

потому что кончился карандаш. Мы дорисуем скобками.

- `cargo` — трюм. Пахнет пылью и коробками, на которых написано «не кантовать» на трёх языках, все врут.
- `antenna` — антенный отсек. Холодный. Иллюминатор маленький. Отсюда моргали Земле, пока не замёрз разъём.

Люки. Трюм из шлюза: логично, груз не тащат через камбуз. Антенна из купола: тоже логично, кабель уже идёт туда. В графе логика — ребро, не «ну рядом же».

```
(defparameter *doors*
  '((airlock  . (corridor cargo))
    (corridor . (airlock galley
```

```
reactor))
    (galley   . (corridor
cupola))
    (reactor  . (corridor))
    (cupola   . (galley antenna))
    (cargo    . (airlock))
    (antenna  . (cupola))))
```

Давай медленно
Ребро пишется **дважды**, если люк двусторонний. Добавил `(airlock . (corridor cargo))` и забыл `(cargo . (airlock))` — из трюма не выйдешь. Это не баг Lisp. Это ты нарисовал одностороннюю дверь и удивился. Проверь `(neighbors 'cargo)` сразу, не после сюжетной арки.

Комната без строки в `*flavor*` — `assoc` вернёт `nil`, `cdr` от `nil` в `look` даст `nil`, `format` напечатает `NIL`. Честно и некрасиво. Добавь

описание в тот же момент, что и ребро. В `*at*` тоже нужна запись, хотя бы `(cargo . nil)`. Без неё `stuff-here` вернёт `nil`, а вот `drop` не сможет изменить `cdr` отсутствующей пары.

```
(setf *flavor*
      (append *flavor*
              '((cargo   . "пыль. коробка с надписью не кантовать. уже кантовали.")
                (antenna . "холодно. разъём в инее. земля далеко и не торопится."))))

(setf *at*
      (append *at*
              (copy-tree
               '((cargo   . (solder))
                 (antenna . (frost-note))))))
```

`solder` — припой. Пригодится, когда иней на антенне не

возьмётся изолентой. `frost-note` — записка Щ.: «не дуй феном. энергия. терпение.»

`append` к глобали не меняет старый список на месте — он **строит новый**. Поэтому `setf`. Если напишешь голый `append` и не присвоишь, мир останется пятикомнатным, а ты будешь орать на REPL. Классика.

`reset-world` тоже обнови. Иначе после сброса новые отсеки исчезают, как будто карандаш Щ. снова сломался. Сброс — договор с собой: **весь** мир в одном месте.

В REPL после правки дверей:

```
(neighbors 'airlock)  ; (CORRIDOR CARGO)
(walk 'cargo)
```

```
(take 'solder)
(walk 'airlock)
(walk 'corridor)
; ... до купола, потом:
(walk 'antenna)
```

Если `walk` ругается «люка нет», ты либо не в той комнате, либо ребро одностороннее, либо `*here*` не то, что думаешь. `(look)` перед паникой. Панель врёт меньше, чем память.

Квест S.L8 · Lisp
Самопроверка: отдельного ответа в главе «Оттадки» нет. Впиши `cargo` и `antenna` в `*doors*`, `*at*`, `*flavor*` и в `reset-world`. Команда `read-frost-note`: если `frost-note` в кармане — напечатай предупреждение Щ. про фен. Нет в кармане — честно скажи, где лежит (антенна). Проверка: из шлюза

в трюм и обратно; из купола в антенну; из реактора в антенну за один `walk` — нельзя.

15.10 Вахта 10. Два бага, которые выглядят как «Lisp сломался»

Не сломался. Ты забыл кавычку. Или построил матрёшку без последней куклы. Разберём оба, медленно, в этом же мире.

15.10.1 Баг 1. Забыл апостроф

Хочется пойти в реактор. Пишешь как человек:

```
(walk reactor)
```

SBCL ругается примерно: переменная `REACTOR` не связана. Здесь `reactor` вычисляется как

переменная, а не вызывается как функция. Ты не пошёл.

Давай медленно
Lisp сначала **вычисляет** аргументы, потом вызывает функцию. `walk` хочет символ комнаты. `'reactor` — «положи бирку, не вычисляй». Без апострофа `reactor` — «найди значение этой переменной». Функция с таким именем живёт в другом пространстве имён: `defun` не задаёт значение переменной. Переменной нет. Пожар.

Сравни в REPL, не в голове:

```
'reactor     ; REACTOR  — бирка
reactor      ; ошибка, если
переменная не связана
'corridor
(walk 'corridor)   ; ход
```

```
(walk corridor)    ; снова
ошибка, если *не* сделал
(defparameter corridor 'corridor)
ради шутки. не делай.
```

Та же яма у `take`:

```
(take tape)     ; ищет переменную
TAPE
(take 'tape)    ; берёт предмет
```

И у `defparameter` мира:

```
(defparameter *here* airlock)   ;
вычисляет airlock — нет такого
(defparameter *here* 'airlock)  ;
стоит в шлюзе
```

Что только что случилось Апостроф — не украшение. Это приказ «не готовь, положи сырым». Строка `"reactor"` — другой тип. `eq` символа и строки не сработает. `walk` сравнивает символы. Поэтому в `play` мы

делали `intern` и `string-upcase`: мост из текста человека в бирку.

Если ошибка говорит `undefined variable` / `unbound` и в форме нет апострофа у имени комнаты или предмета — сначала поставь `'`, не переписывай движок.

15.10.2 Баг 2. Бесконечная рекурсия

Граф станции **циклический**: шлюз ↔️ коридор. Обход без кармана «уже был» ходит по кругу, пока стек не кончится.

Вот «очевидная» функция «все комнаты, куда достану»:

```
(defun reachable-broken (from)
  (cons from
```

```
        (mapcan (lambda (n)
(reachable-broken n))
                (neighbors
from))))
```

Вызови `(reachable-broken 'airlock)`. Не надо ждать час. Через мгновение: `Control stack exhausted` / stack overflow. Это не «компьютер слабый». Это матрёшка: airlock → corridor → airlock → corridor → …

Давай медленно
Рекурсия без стопа на **этом же** узле — бесконечность. Стоп «соседей нет» здесь не спасает: соседи всегда есть, просто свои. Нужен список `seen`. Вахта 6 его уже писала. Сломай нарочно, посмотри портянку, **потом** верни `seen`. Иначе слово «стек» останется звуком.

Вторая классика — функция вызывает себя **с тем же аргументом**:

```
(defun look ()
  (look)
  (format t "ты в ~a~%" *here*))
```

Первая строка тела — снова `look`. До `format` очередь не дойдёт никогда. Стек вырастет. Ошибка та же, причина ещё глупее: опечатка «позвать себя вместо `look-at-doors`», или `walk` в конце вызывает `look`, а `look` из хороших чувств вызывает `walk` в ту же комнату.

```
; кольцо walk → look → walk
(defun walk (to)
  (setf *here* to)
  (look))
(defun look ()
```

```
  (walk *here*))   ; «обновить»,
ха
```

Лечится: нарисуй стрелки **кто кого зовёт**. Если кольцо без изменения аргумента — ты не обошёл граф, ты сел на вентилятор.

Как читать ошибку: сверху (или снизу, как удобно) ищи **свою** функцию, повторённую много раз подряд. `LOOK LOOK LOOK LOOK` — вот оно. Не `SBCL internals` на два экрана.

Починка `reachable` — та, что уже была: `seen`, `unless (member room seen)`, `push`, потом соседи. Проверка: `(reachable 'airlock)` вернёт семь отсеков, не умрёт. Отрежь люк `cupola → antenna` и

стартуй из шлюза — `antenna` не в списке. Это граф, не картинка.

Квест S.L9 · Lisp
Самопроверка: отдельного ответа в главе «Отгадки» нет. Воспроизведи оба бага **нарочно**: (1) `(walk reactor)` без апострофа — запиши в журнал точный текст ошибки; (2) `reachable-broken` — увидь stack overflow, не оставляй крутиться. Потом почини (2) через `seen`. Сравни длину списка с семью комнатами. Если комнат меньше — забыл ребро в одну сторону.

15.11 Вахта 11. Иней на антенне — ремонт как бой

Не орки. Иней. Толщина в условных сантиметрах. Ты бьёшь нагревом, иней бьёт энергию станции. Кто кончится первым.

Мир уже умеет `*energy*`. Добавим противника:

```
(defparameter *frost* 12)
```

12 — толщина. 0 — разъём сухой, антенна может `ping`. Каждый `blast` — вспышка нагрева: −3 к инею, −5 к энергии. Только в `antenna`. Если энергии меньше 5 — вспышка не вспыхнет, напечатай `'too-dark`.

```
(defun blast ()
  (cond
    ((not (eq *here* 'antenna))
     (format t "иней не здесь.
тащи ноги в antenna.~%")
     'wrong-room)
    ((< *energy* 5)
     (format t "темно. нагревать
нечем. энергия ~a~%" *energy*)
     'too-dark)
    ((<= *frost* 0)
     (format t "уже сухо. не
геройствуй, разъём не любит.~%")
     'already-clear)
    (t
     (setf *energy* (clamp-energy
(- *energy* 5)))
     (setf *frost* (max 0 (-
*frost* 3)))
     (format t "шипит. иней ~a,
энергия ~a~%" *frost* *energy*)
     (if (<= *frost* 0) 'clear
'frosted))))
```

Что только что случилось `max 0` — чтобы иней не ушёл в минус и не стал «ещё суше». Смысла нет, сюжет портится.

`clamp-energy` на энергии уже стоит. Два стража, два ресурса. Это вся «боевая система»: два числа и правило, когда ход законен.

Припой из трюма — сильный ход. Тратит 8 энергии, снимает 8 инея. Без припоя в кармане — `'need-solder`. С припоем — снимает предмет (один раз, это не бесконечная катушка).

```
(defun solder-joint ()
  (cond
    ((not (eq *here* 'antenna))
'wrong-room)
    ((not (member 'solder
*pocket*)) 'need-solder)
    ((< *energy* 8) 'too-dark)
    (t
     (setf *energy* (clamp-energy
(- *energy* 8)))
     (setf *frost* (max 0 (-
*frost* 8)))
```

```
    (setf *pocket* (remove
'solder *pocket*))
    (format t "припой сел. иней
~a~%" *frost*)
    (if (<= *frost* 0) 'clear
'frosted))))
```

Иней не джентльмен. Если остаёшься в антенне вместо ремонта, пусть команда `wait` наращивает иней. Для ходьбы пока действуют прежние правила `walk`:

```
(defun wait ()
  (unless *fixed*
    (tick))
  (when (and (eq *here* 'antenna)
(> *frost* 0))
    (setf *frost* (+ *frost* 1))
    (format t "иней подрос.
теперь ~a~%" *frost*))
  (status))
```

Проигрыш: энергия 0, иней ещё > 0. Для победы: иней 0, ты в

антенне, и осталось хотя бы 15 энергии на сигнал:

```
(defun ping-earth ()
  (cond
    ((not (eq *here* 'antenna))
'wrong-room)
    ((> *frost* 0) 'iced)
    ((< *energy* 15) 'too-dark)
    (t
     (setf *energy* (clamp-energy
(- *energy* 15)))
     (format t "пинг ушёл. земля
не обязана ответить. ты сделал,
что мог.~%")
     'ping)))
```

Партия-репетиция, не роман:

```
(reset-world)
(walk 'cargo)
(take 'solder)
; дойти до antenna через corridor
galley cupola, энергия капает
(solder-joint)
(blast)      ; иней 4 -> 1
```

```
(blast)      ; иней 1 -> 0
(ping-earth)
```

Добавь `*frost*` в `reset-world` (снова 12) и в `world-plist` / `apply-world`. Иначе сейв забудет войну с инеем, и это обидно после трёх вспышек.

В `play` команды: `blast`, `solder`, `wait`, `ping`. Без аргументов. Третье слово станция по-прежнему проглотит. Живи.

Так себе идея
Не добавляй «случайный критический удар» на первой вахте. Сначала правило должно быть повторяемым: 12 инея, blast по 3, посчитай на бумаге, сколько вспышек. Если бумага и REPL расходятся — глючит

код. Потом уже `(random 3)`, если зудит.

Квест S.L10 · Lisp
Самопроверка: отдельного ответа в главе «Оттадки» нет. `blast` / `solder-joint` / `ping-earth` как выше. Победа — `'ping` после сухого разъёма. Проигрыш — `walk` или `blast` при энергии 0. В `status` печатай иней. Сейв помнит `*frost*`. Проверка: без припоя чистая вспышками; с припоем — меньше ходов. Запиши в комментарии, сколько энергии ушло в каждом сценарии.

15.12 Вахта 12. Что лежит в сейве, по слогам

Файл `module-save.lisp-data` — не роман и не JSON. Это то, что

`print` написал, чтобы `read` прожевал. Открой его **глазами** после `(save-world "module-save.lisp-data")`.

Типичная каша (у тебя числа другие):

```
(:ENERGY 86 :LEAK 7 :HERE
CORRIDOR :POCKET (TAPE) :AT
((AIRLOCK) (CORRIDOR) ...) :DOORS
((AIRLOCK CORRIDOR
CARGO) ...) :FIXED NIL :FROST 12)
```

Идём слева направо, как кладовщик.

Список. Внешние скобки — одна форма. `read` съест её целиком. Потеряешь закрывающую — `read` будет ждать до конца файла и обидится. Добавишь лишнюю — следующая форма, `load-world` её не ждёт, мир прожуёт только

первую, хвост проигнорирует или упадёт. Не «почти JSON». Скобки считаются.

Ключи с двоеточием. `:ENERGY` — keyword. Сам себе значение, не надо кавычки. `getf` ищет именно его. Напишешь `ENERGY` без двоеточия — другой объект, `getf` вернёт `nil`, энергия станет `nil`, `tick` взорвётся на вычитании. Двоеточие не украшение.

Числа. `86` — число. Не `"86"`. Если руками впишешь кавычки, `*energy*` станет строкой, `(- *energy* 7)` скажет type error. Правка сейва руками — ок для учёбы, тип не меняй.

Символы. `CORRIDOR`, `TAPE`, `NIL`. Не строки. `NIL` — и ложь, и пустой список, мы это уже ели. `:FIXED`

NIL значит «щель ещё сипит», не «строка nil».

Вложенные скобки. Карман (TAPE) — список из одного символа. Пустой карман — NIL или (). *at* и *doors* — alist, списки пар. (AIRLOCK . (CORRIDOR CARGO)) и (AIRLOCK CORRIDOR CARGO) — две записи одного списка: cdr в обоих случаях вернёт (CORRIDOR CARGO). print обычно выбирает короткую запись. А вот (AIRLOCK (CORRIDOR CARGO)) добавляет уровень вложенности: хвост станет ((CORRIDOR CARGO)). Эти лишние скобки действительно запутают neighbors.

Давай медленно
print пишет для read. format пишет для тебя. Сейв — print. Если «чтобы красиво» вставишь

запятые, как в JSON, `read` подавится. Запятая в Lisp — другой синтаксис (не твой друг здесь). Пробелы можно. Запятые — нет.

Ручная диверсия, полезная:

```
(reset-world)
(save-world "module-save.lisp-
data")
```

Открой файл в редакторе. Смени `:ENERGY 100` на `:ENERGY 3`. Сохрани. В REPL:

```
(load-world "module-save.lisp-
data")
(status)
```

Энергия должна быть 3. Если 100 — загрузил не тот путь, или после `load-world` вызвал `reset-world`. Порядок: код игры → `load-world` → **не** сброс.

Вторая диверсия: сотри одну закрывающую скобку. `load-world` должен упасть. Прочти ошибку. Верни скобку. Это не «файл битый мистически». Это форма не закрыта.

Третья: напиши `:ENERGY "много"`. Загрузка может пройти, мир станет ядовитым. Первый `tick` покажет правду. Валидации в `apply-world` нет — это учебный сейв, не банк. Поэтому `read` с диска **своего** — ок, с диска из интернета — нет. Уже орали, орём ещё.

Квест S.L11 · Lisp
Самопроверка: отдельного ответа в главе «Отгадки» нет. Сохрани мир, руками поставь `:FROST 1` и `:HERE ANTENNA`. Загрузи. Один `blast` должен сделать иней

0 (если энергии хватает). Второй файл: сломай скобку, поймай ошибку `read`, почини. В журнал вахты — одна фраза: чем `print`-сейв отличается от `energy=80` в Java-дашборде.

15.13 Зачем это было

Ты сделал: состояние, граф, предметы, побочный эффект, сейв, крошечный язык команд, два новых отсека, два разобранных бага, войну с инеем. Это вся профессия, только без зарплаты и без Jira.

В основном курсе те же идеи придут как `TaskStore`, HTTP и таблица. Здесь они пахнут металлом шлюза. Можно вернуться на вахту 1 и добавить вторую щель. Можно не

возвращаться. Журнал не ставит зачёт.

Если увлёкся и пишешь текстовую станцию третью ночь — поставь будильник на Java. Станция «МОДУЛЬ» в резюме не кормит. Скобки кормят мозг. Мозг потом кормит Java.

Воскресная диверсия
Нарисуй семь кружков и люки. Походи фишкой. Потом походи в REPL. Если расходятся — врёт либо бумага, либо `*doors*`. Обычно `*doors*`.

16 Макросы и CLOS: станция учится новым словам

Это не месяц из расписания. Это воскресная дыра для тех, кому скобок мало: сначала макросы — код, который пишет код, потом CLOS — объекты по-лисповски, не «как Java, только в скобках».

Если task-manager лежит — поднимай task-manager. Если Hibernate красный третий день — читай `Caused by`, не эту главу. Сюда заходят, когда основной курс зелёный или когда воскресенье зудит и Java уже сказала «достаточно».

Закон станции

Макрос — не «продвинутый defun». Функция считает значения. Макрос получает **формы** и возвращает другую форму, которую Lisp потом посчитает. Перепутаешь — получишь либо магию, которой нет, либо кашу, которую стыдно читать через неделю.

CLOS — Common Lisp Object System. Не «классы как в Java». Там методы живут **внутри** класса. Здесь методы живут снаружи: обобщённая функция, а классы только говорят, **какой** метод выбрать. На станции это удобно: один `status` для реактора, антенны и чайника, без иерархии на десять этажей.

Файлы: `lisp-experiments/macros-clos/`. Коммить. Даже кривой макрос.

16.1 Зачем макрос, если есть функция

Функция видит значения. Макрос видит код **до** того, как его посчитали.

```
(+ 1 2)
```

`+` получает `1` и `2`. Ему всё равно, написал ты `(+ 1 2)` или `(+ x y)`, где `x` уже 1.

А вот `if` не может быть обычной функцией в наивном смысле: если бы считали оба аргумента заранее, ветка «иначе» выполнялась бы всегда. В Common Lisp `if` —

специальный оператор. Макросы — способ делать себе такие штуки, не прося комитет языка.

Самый честный учебный пример — «сделай сам `when`». `when` уже есть. Мы напишем свой, чтобы увидеть кишки, и сразу его забудем в пользу встроенного. Как разобрать датчик, который и так работает.

Сначала — что код в Lisp **уже данные**.

```
'(+ 1 2)
; (+ 1 2)

(first '(+ 1 2))
; +

(second '(+ 1 2))
; 1
```

Список `(+ 1 2)` можно собрать руками:

```
(list '+ 1 2)
; (+ 1 2)
```

И **выполнить**:

```
(eval (list '+ 1 2))
; 3
```

`eval` в обычном коде — запах горелой изоленты. Макрос как раз для того, чтобы **не** вызывать `eval` руками: ты возвращаешь форму, а компилятор/интерпретатор сам её вставит туда, где макрос вызвали.

Давай медленно
Функция: на вход числа и списки-значения, на выход значение.
Макрос: на вход куски кода

(ещё не посчитанные), на выход кусок кода.
Потом Lisp считает этот кусок **на месте вызова**. Как если бы ты дописал скобки сам, только станция дописала за тебя.

16.1.1 Обратная кавычка: шаблон с дырками

Писать `(list 'if test (list 'progn ...))` руками — боль. Есть шаблон: обратная кавычка `(backquote) и запятая ,.`

lisp (let ((n 3)) `(+ 1 ,n))
`; (+ 1 3)`

Всё под — `как quote, *кроме* того, что после запятой: это подставить значение.`

`Запятая-собака ,@ — подставить *элементы* списка, не список`

целиком:

lisp (let ((body „(a b c)))
```
(progn ,@body))
; (PROGN A B C)
```

Без `,@` было бы `(PROGN (A B C))` — один аргумент, который сам список. Собака распаковывает. На станции: изолента **внутрь** ящика vs ящик с изолентой в другом ящике.

Что только что случилось Обычный апостроф цитирует форму. Обратная кавычка — шаблон, запятая — вычисляемая дырка. Если забыл запятую, в шаблоне останется символ `n`, а не число 3. При выполнении полученной формы Lisp будет искать значение переменной `n` на

месте вызова: её может не быть, или значение окажется чужим. Читай, что макрос **вернул**, прежде чем винить SBCL.

16.1.2 Первый макрос: свой `when`

Встроенный `when`: если условие, сделай тело (может быть несколько форм). Иначе `nil`.

```
(defmacro module-when (test &body
body)
  `(if ,test
       (progn ,@body)
       nil))
```

`&body` — то же, что `&rest`, только для макросов вежливо: «тут тело». Редакторы благодарят отступом.

Проверка не «вызови и поверь», а **раскрой**:

```
(macroexpand-1 '(module-when (>
*energy* 10)
                   (format t "ещё
живы~%")
                   (tick)))
```

Должно вылезти что-то вроде:

```
(IF (> *ENERGY* 10)
    (PROGN (FORMAT T "ещё
живы~%") (TICK))
    NIL)
```

`macroexpand-1` раскрывает внешний макровызов один раз. `macroexpand` повторяет это, пока внешняя форма остаётся макровызовом; внутрь всех подформ он не обходит. В нашем примере оба остановятся на `if`, а `format` в теле останется как был. На учёбе начинай с `-1`, чтобы видеть каждый внешний шаг.

Давай медленно

1. Набери `defmacro` как выше.
2. Вызови `macroexpand-1` на форме с двумя строками тела.
3. Убедись, что обе строки внутри `progn`.
4. Потом уже вызывай `module-when` вживую.

Если сразу «вживую» — ты тестируешь удачу, не макрос.

Зачем `progn`? У `if` одна форма на ветку. Тело макроса — несколько. Склеить в одну — `progn`. Функция так не сделает: к моменту вызова функции тело уже посчитали бы.

Так себе идея
Не пиши макрос, если хватает функции. Макрос оправдан,

когда нужно **не считать** какой-то аргумент, или ввести новое имя (`let` внутри), или чтобы вызов выглядел как новая грамматика. «Чтобы было круто» — плохая причина. Круто через неделю читать свой `defun`.

16.1.3 Макрос, который бережёт реактор

Хочется писать:

```
(with-energy 15
  (print 'warming-up)
  (print 'ready))
```

Смысл: если энергии меньше 15 — не выполнять тело, вернуть `'too-low`. Если хватает — выполнить, энергию списать **один раз** в начале. Функцией

неудобно: тело опять посчитается до входа. Макрос:

```
(defmacro with-energy (cost &body
body)
  (let ((cost-name (gensym
"COST")))
    `(let ((,cost-name ,cost))
       (if (< *energy* ,cost-
name)
           'too-low
           (progn
             (decf
*energy* ,cost-name)
             ,@body)))))
```

`gensym` — свежее имя, которого нет в твоём коде. Опасность не в инициализаторе `(let ((cost ,cost)) ...)`: там виден внешний `cost`. Захват случается в подставленном теле. При достаточной энергии `(let ((cost 99)) (with-energy 15 cost))` должен вернуть 99; макрос с

внутренним именем `cost` вернул бы 15. Это **захват**. На четвёртой строке он уже ждёт. Привычка: внутренние имена макроса — через `gensym`.

Что только что случилось `(gensym "COST")` вернёт что-то вроде `#:COST1234`. Некрасиво в expand. Зато не совпадёт с твоим `cost`. Некрасиво лучше, чем тихо неверно.

Проверка:

```
(defparameter *energy* 10)
(macroexpand-1 '(with-energy 15
(print 'boom)))
(with-energy 15 (print 'boom))
; TOO-LOW
*energy*
; 10
```

Энергия не списалась — тело не жило. Теперь `*energy*` = 20,

снова `with-energy` 15 — тело живёт, энергия 5.

Квест M.L1 · Lisp `module-when` и `macroexpand-1` на форме с **двумя** выражениями в теле. В комментарии — что было бы, если забыть `progn`.

Квест M.L2 · Lisp `with-energy`: мало энергии — `'too-low` и `*energy*` не меняется; хватает — тело и списание. Проверь оба. Счётчик `cost` через `gensym`, не «ну у меня имя редкое».

16.1.4 Когда макрос глючит: дважды посчитал

Классика:

```
(defmacro twice-wrong (x)
  `(+ ,x ,x))
```

Вызов `(twice-wrong (tick))` подставит `(tick)` **дважды**: два тика, не один. Если `tick` пишет в `*energy*`, станция потеряет вдвое. Раскрыл — увидел двух близнецов.

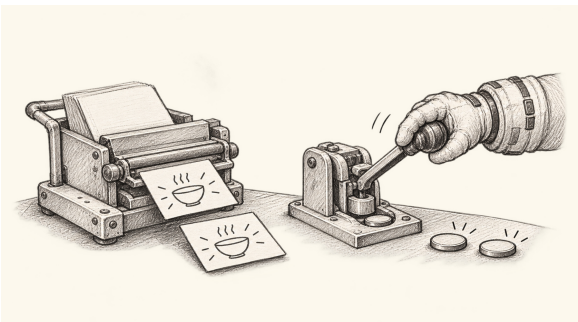
Правильно: сначала в `let` одно имя, потом два раза имя.

```
(defmacro twice-right (x)
  (let ((g (gensym "X")))
    `(let ((,g ,x))
       (+ ,g ,g))))
```

`(twice-right (tick))` → один `tick`, сложить значение с собой. Не два тика.

Давай медленно
После каждого макроса спрашивай: **если аргумент — форма с побочным эффектом, сколько раз она выполнится?**

Если не один — почти всегда нужен `let` + `gensym`. Это не паранойя. Это датчик утечки.



Две копии команды — две работы. Раскрытие копирует формы; выполнение запускает их позже. Привязка let позволяет повторно использовать вычисленное значение.

16.1.5 Макросы, которые ты уже ел, не зная

`defun` — макрос (над более низким). `loop` — макрос, целый язык. `with-open-file` — макрос: открой, сделай тело, закрой

даже если упало. `setf` — макрос. Ты ими пользовался с недели 1.

Java так не умеет. У неё аннотации, кодогенерация, lombok, процессоры — соседние галактики. Поэтому в Java пишут больше букв, а в Lisp иногда пишут новое слово языка. Не тащи макросы в Java-голову как «надо на работе». На работе у тебя Spring. Макрос — чтобы мозг знал, что грамматика не священная корова.

SICP · открывать, только если зудит

«Код как данные» — сердцевина SICP и Lisp. Если зудит, зачем `eval` и цитата — туда, куски про quotation. Не вместо этой главы:

сначала свой `module-when`, потом Абельсон.

16.2 CLOS: объекты, у которых методы снаружи

В Java: класс `Task`, внутри методы `complete()`, `getTitle()`. Объект несёт и данные, и глаголы.

В CLOS: класс несёт **слоты** (поля). Глаголы живут в **обобщённых функциях**. Одна функция `status`, несколько методов: для реактора, для антенны, для «чего угодно». Когда вызываешь `(status x)`, Lisp смотрит класс `x` и выбирает метод.

Это не «хуже Java» и не «лучше». Это другой центр: глагол важнее паспорта объекта. На станции удобно: ты всё время зовёшь `status`, а железяки разные.

16.2.1 Класс — чертёж слотов

```
(defclass module ()
  ((name :initarg :name
         :accessor module-name)
   (energy :initarg :energy
           :initform 0
           :accessor module-
energy)))
```

`defclass` — объявить класс. Имя `module`. `()` — нет родителей (пока). Дальше слоты:

- `:initarg :name` — при создании можно сказать `:name "reactor"`.

- `:accessor module-name` — Lisp напишет читалку и писалку. Не геттер на три строки.
- `:initform 0` — если не сказал энергию, будет 0.

Создать:

```
(defparameter *reactor*
  (make-instance 'module :name
"reactor" :energy 80))

(module-name *reactor*)
; "reactor"

(module-energy *reactor*)
; 80

(setf (module-energy *reactor*)
75)
```

`make-instance` — как `new`, только слово честнее: экземпляр. `setf` с accessor — как сеттер, только без отдельного `setEnergy`.

Что только что случилось `*reactor*` теперь не число и не alist. Это объект. `describe` в REPL покажет слоты, если забыл, что внутри:

```
(describe *reactor*)
```

Полезно, когда слотов станет пять и память уже глючит.

Без accessor можно `(slot-value obj 'energy)` — грубо, как открыть панель отвёрткой. Accessor — нормальная дверь. `slot-value` — когда пишешь инфраструктуру. В квестах этой главы — accessor.

16.2.2 Обобщённая функция и метод

```
(defgeneric module-status (m)
  (:documentation "Строка для
панели. Один глагол, разные
железяки."))

(defmethod module-status ((m
module))
  (format nil "~a energy=~a"
          (module-name m)
          (module-energy m)))
```

`defgeneric` — «будет функция `module-status` от одного аргумента». Можно не писать: первый `defmethod` иногда заведёт generic сам. Явно — честнее, как README.

`((m module))` — параметр `m`, и он **специализирован** на класс `module`. Не тип Java в смысле компилятора. Выбор метода в

runtime: какой класс пришёл, такой метод.

```
(module-status *reactor*)
; "reactor energy=75"
```

Теперь наследник — антенна, у которой ещё и уровень сигнала:

```
(defclass antenna (module)
  ((signal :initarg :signal
           :initform 0
           :accessor antenna-
signal)))

(defmethod module-status ((m
antenna))
  (format nil "~a energy=~a
signal=~a"
          (module-name m)
          (module-energy m)
          (antenna-signal m)))

(defparameter *dish*
  (make-instance 'antenna :name
"dish" :energy 20 :signal 3))
```

```
(module-status *dish*)
; "dish energy=20 signal=3"

(module-status *reactor*)
; по-прежнему метод для module
```

`antenna (module)` — антенна **есть** модуль. Слоты имени и энергии наследует. Метод для `antenna` **конкретнее**, его и возьмут. Для голого `module` — общий.

Давай медленно
В Java ты бы написал `Antenna extends Module` и `@Override status()`. В CLOS override живёт не в классе, а в отдельном `defmethod`. Класс можно вообще не трогать, добавляя методы в другом файле. Это странно, если пришёл из Java. Это нормально, если думаешь глаголами: «ещё один способ показать status».

16.2.3 Несколько аргументов — не только «этот объект»

В Java метод принадлежит одному классу. В CLOS можно специализировать **несколько** аргументов. Учебный жест, не диссертация:

```
(defgeneric dock (ship port))

(defmethod dock ((ship module)
(port module))
  (format nil "~a -> ~a" (module-
name ship) (module-name port)))
```

Потом заведёшь метод `(dock (ship shuttle) (port airlock))` — другой текст. Выбор по **паре** классов. В Java для этого танцы с visitor. Здесь — второй параметр в `defmethod`. Не обязательно в квесте. Обязательно один раз увидеть,

чтобы «объектная модель» не казалась копией Java.

16.2.4 `call-next-method`: не копируй родителя руками

```
(defmethod module-status :around
((m module))
  (let ((s (call-next-method)))
    (if (<= (module-energy m) 0)
        (concatenate 'string s "
ALARM")
        s)))
```

`:around` — обёртка вокруг остальных методов. `call-next-method` — «сделай то, что и так сделали бы». Потом мы дописали `ALARM`, если энергия ноль.

Есть ещё `:before` и `:after` — до и после основного. На учёбе хватит `:around` или простого метода у наследника. Не строй

комбинатор на пять этажей в первый вечер. Станция держится на изоленте, не на МОР.

16.2.5 Несколько родителей сразу. Java до сих пор так не умеет

В Java у класса один `extends`. Потом сколько угодно `implements` — но это **интерфейсы**: обещания без (почти) состояния. Настоящих двух пап со слотами нет. Пишешь `extends Module implements Powered, Radio` и дальше руками растаскиваешь, что куда.

CLOS разрешает несколько родителей **как классов**:

```
(defclass powered ()
  ((watts :initarg :watts
```

```
          :initform 0
          :accessor watts)))

(defclass radio ()
  ((freq :initarg :freq
         :initform 0
         :accessor radio-freq)))

(defclass comm-antenna (module
powered radio)
  ())
```

`comm-antenna` — и модуль, и питание, и радио. Слоты `energy`, `watts`, `freq` все на месте. Не «притворись интерфейсом». Предки настоящие.

```
(defparameter *comm*
  (make-instance 'comm-antenna
                 :name "comm"
                 :energy 40
                 :watts 12
                 :freq 437))

(list (module-name *comm*) (watts
```

```
*comm*) (radio-freq *comm*))
; ("comm" 12 437)

(typep *comm* 'module)   ; T
(typep *comm* 'powered)  ; T
(typep *comm* 'radio)    ; T
```

Метод `module-status` для `module` уже работает: антенна **есть** модуль. Можно дописать метод специально для `comm-antenna` или для `powered` — generic выберет самый подходящий.

Порядок родителей важен. Lisp строит **список предшествования классов** (CPL): кто конкретнее, чей слот `:initform` взять, чей метод ближе. Если два деда с одним именем слота — будет правило, не лотерея. В Common Lisp порядок CPL задаётся правилами стандарта.

Запоминать алгоритм сейчас не обязательно. Обязательно: `(sb-mop:class-precedence-list (find-class 'comm-antenna))` в REPL, если вдруг непонятно, **чей** метод взялся.

```
(sb-mop:finalize-inheritance
(find-class 'comm-antenna))
(mapcar #'class-name
        (sb-mop:class-precedence-
list (find-class 'comm-antenna)))
; (COMM-ANTENNA MODULE POWERED
RADIO ... STANDARD-OBJECT ... T)
```

`sb-mop` — пакет метаобъектного протокола SBCL; это не функция стандарта ANSI Common Lisp. В других реализациях доступ к MOP может отличаться. `finalize-inheritance` подготавливает сведения о наследовании класса перед запросом.

`STANDARD-OBJECT` и `T` — предки этих классов. Их не ты писал. Многоточия выше обозначают возможные внутренние классы реализации; полный список может зависеть от реализации Lisp. Если в списке `POWERED` раньше `RADIO` — так ты их поставил в `defclass`. Поменяй местами — поменяются. Это не баг. Это рычаг.

Давай медленно
Java: один папа, много табличек «я умею вот это».\ CLOS: несколько пап, слоты и методы со всех.\ Ромб «оба деда → один внук» в Java запрещён для классов как раз потому, что страшно. В CLOS не запрещён: порядок в `defclass` и CPL решают спор. Это не «лучше для Spring». Это **есть**, и во многих языках до

сих пор нет. Python чуть умеет. Java — нет. C# — нет. Запомни как датчик: «объектная модель» не равна «как в Java».

Так себе идея
Не строй на учёбе родословную на шесть этажей «потому что можно». Множественное наследование легко превратить в трюм, где никто не помнит, чей это `status`. Два предка с разным смыслом (`powered` и `radio`) — ок. Два предка, которые оба хотят быть «главным модулем», — запах. Как микросервисы: возможность не равна необходимости.

Квест C.L5 · Lisp
Класс хотя бы с **двумя прямыми родительскими классами**: например `powered` +

`radio`, или `module` + `powered`. Создай экземпляр, проверь `typep` на каждого предка, прочитай слоты с обеих сторон. В комментарии одна фраза: чего в Java для этого нет (не «интерфейс», а именно **два класса-родителя со слотами**).

Квест C.L6 · Lisp

`(sb-mop:class-precedence-list (find-class 'comm-antenna))` — или твоего класса. Запиши порядок имён. Поменяй местами независимых прямых родителей в `defclass` и снова. После каждого определения вызови `(sb-mop:finalize-inheritance (find-class 'comm-antenna))` перед чтением CPL. Даже у пустых независимых родителей порядок изменится. Если нет — проверь, что новое

определение выполнено и ты смотришь нужный класс в том же пакете.

Так себе идея
Не учи «весь CLOS» как таблицу умножения. `defclass`, `make-instance`, accessor, `defgeneric`/`defmethod`, один наследник — уже больше, чем нужно, чтобы на собесе по Lisp (редкость) не молчать, и чтобы Java-классы казались **одним** способом, не единственным.

16.2.6 Станция из объектов, не из глобалей

В журнале «МОДУЛЬ» мир жил в `*energy*` и `*here*`. Это нормально для игры на три экрана. Когда сущностей пять, глобали начинают врать: чья энергия, какой отсек.

Эскиз, не весь движок:

```
(defclass station ()
  ((modules :initarg :modules
            :initform nil
            :accessor station-
modules)))

(defun station-report (st)
  (mapcar #'module-status
(station-modules st)))

(defparameter *module*
  (make-instance 'station
                 :modules (list
*reactor* *dish*)))

(station-report *module*)
```

Список объектов. `mapcar` по глаголу `module-status`. Каждый объект сам знает, как ответить, потому что метод выбран по классу. В Java это `for (Module m : list) m.status()` — тот же жест, другой адрес глагола.

Квест C.L1 · Lisp
Класс `module` со слотами name и energy. `make-instance`, поменять energy через accessor, `module-status` печатает оба. `describe` в комментарий не надо — в REPL живьём.

Квест C.L2 · Lisp
Класс `antenna` наследник `module`, слот `signal`. Свой `module-status`. Два объекта в списке, `mapcar #'module-status` — две разные строки, один глагол.

Квест C.L3 · Lisp
Метод `:around` или отдельная функция `alarm-p`: энергия 0 — в статусе есть `ALARM`. Реактор на 0 и антенна на 0 оба орут. Не копипаста двух `if` в двух методах, если уже умеешь `call-next-method`. Если не умеешь —

два `if`, но напиши в комментарии, чем это хуже.

16.3 Макрос + CLOS: не смешивать в блендере сразу

Можно написать макрос `define-module`, который разворачивается в `defclass` + метод. Во второй вечер. В первый — **не надо**. Сначала каждый инструмент отдельно. Иначе баг: не понятно, виноват макрос, CLOS или ты.

Порядок, который не взрывается:

1. Функции и списки (ты уже).
2. Макрос, который раскрыл и **прочитал** expand.
3. Один класс, один метод.

4. Наследник.
5. Только потом «макрос пишет defclass».

Закон станции
Если `macroexpand-1` ты не запускал — макроса у тебя нет, есть надежда. Если `describe` на экземпляр не вызывал — класса у тебя нет, есть `defparameter` с объектом, который ты боишься открыть.

16.4 Как это рифмуется с Java, без проповеди

- Java-класс ≈ CLOS-класс + методы, **прибитые** к нему. CLOS-методы можно дописать в другом файле.
- Java `interface` ≈ «набор generic», но Java требует,

чтобы класс **заявил**, что умеет. CLOS не требует заявления: метод для класса можно определить рядом.

- В Java для открытия и гарантированного закрытия ресурса уже есть `try-with-resources`. Lisp использует `with-open-file`. Отличие макросов — возможность самим добавлять другие управляющие конструкции, а не отсутствие автоматического закрытия файлов в Java.
- `record` в Java 21 — слоты без церемонии. Ближе к `defclass` с accessor, чем к старому JavaBean на сто строк.
- Spring `@Service` — не макрос, хотя **ощущение** «язык

дописали аннотацией» похоже. Не ври на собесе, что аннотация = макрос. Соседние галактики.

На собесе по Java это не спросят. В голове осядет: объект — не единственная религия. Когда в Java полезешь в visitor или в «утиную» сериализацию, вспомни `defgeneric`. Когда копируешь один и тот же try/catch в десятый раз — вспомни, что в Lisp это был бы макрос, а в Java — метод с лямбдой. И напиши метод с лямбдой, не жди макросов от Oracle.

16.5 Типичные поломки

Макрос не раскрывается, вызывается как функция. Забыл `defmacro`, написал `defun`.

Тогда аргументы сначала посчитаются. `when` превратится в ловушку. Смотри `macro-function`:

```
(macro-function 'module-when)  ; функция или NIL
```

`NIL` — это не макрос.

`comma not inside backquote`. Запятая в обычном коде. Обратная кавычка потерялась.

Не тот ключ инициализации. `:initarg :name`, а создаёшь `:title`. Неизвестный initarg обычно вызывает ошибку, а не молча записывает `nil` в слот. В нашем `module` пропуск `:name` оставляет слот имени несвязанным, потому что у него нет `:initform`. Чтение такого слота вызывает `unbound-`

`slot`. Имена — бирки, не «ну понял же».

`there is no applicable method`. Вызвал generic на числе или на `nil`. Метод есть для `module`, пришло что-то другое. `(class-of x)` в REPL.

Наследник не видит слот. Опечатка в имени родителя. В SBCL `(defclass antenna (modlue) ...)` может оставить `modlue` как ещё не определённый родительский класс. Завершение определения наследования или создание экземпляра затем вызывает ошибку, пока этот родитель не определён; здесь нужен `module`. Скобки вокруг родителя обязательны: `(module)`, не `module` в одиночку в некоторых

раскладках мозга. Пиши как в примере.

Квест M.L3 · Lisp
Сломай `twice-wrong` через `(twice-wrong (tick))` (или счётчик `incf`). Покажи в комментарии: два побочных эффекта. Почини через `twice-right`. `macroexpand-1` обоих.

Квест C.L4 · Lisp
Станция: класс `station`, слот список модулей, функция `station-report`. Минимум два модуля разных классов. Печать отчёта. Это не журнал «МОДУЛЬ» целиком — это панель из объектов.

Спрятать на GitHub
Папка `macros-clos/`: `when.lisp`, `energy-mac.lisp`, `modules.lisp`. README: как загрузить в SBCL

(`(load ...)` по порядку). Один `macroexpand-1` вставить в README как доказательство, что ты раскрывал, не только вызывал.

Воскресная диверсия
Напиши макрос `dountil-energy` (пока энергия выше N, делай тело). Раскрой. Не запускай без expand: легко словить бесконечный цикл, если условие не то. `Ctrl+C` в SBCL — твой скафандр.

16.6 Программа, которая не просит перезапуск

В Java цикл такой: написал → `javac` или Maven → собрал jar → остановил сервер → залил → запустил → молишься, что `main` тот. На телефоне ещё хуже: APK,

установка, зелёная стрелка, активность родилась заново. Горячая замена в IDE иногда глючит. «Переписать метод у живого объекта» штатно язык не обещает.

В Common Lisp штатно обещает. REPL — не игрушка первой недели. Это **тот же** мир, в котором крутится программа. `defun` и `defmethod` не «компилируют проект». Они подменяют функцию **в уже живом образе**.

```
(defmethod module-status ((m
module))
  (format nil "~a energy=~a"
          (module-name m)
          (module-energy m)))

(module-status *reactor*)
; "reactor energy=75"
```

Не выходи из SBCL. Не грузи файл заново «с нуля», если не хочешь. Просто набери **другой** метод:

```
(defmethod module-status ((m
module))
  (format nil "[~a] ~a%"
          (module-name m)
          (module-energy m)))

(module-status *reactor*)
; "[reactor] 75%"
```

Тот же объект `*reactor*`. Тот же процесс. Новый глагол. Никакого `mvnw`, никакого «подожди, Gradle качает». Станция не перезагружалась. Датчик сменил прошивку на лету.

То же с обычной функцией: переопределил `tick` — следующий вызов уже новый,

глобали на месте. Сломал — переопредели ещё раз. Образ помнит данные. Ты меняешь поведение.

Что только что случилось
Это и есть интерактивное программирование, не «консоль, чтобы потыкать плюсики». Ты не пересобираешь мир. Ты чинишь его, пока он включён. Emacs так живёт десятилетиями: переопределил функцию — редактор уже другой, сессия та же. Не случайно эту книгу писали внутри Emacs. Рекурсия, да. Снова.

Почти нигде так больше нет. Smalltalk умел. Некоторые образы Erlang/Elixir горячо

подменяют модуль, но это другая религия. Python «перезапусти ячейку в ноутбуке» — двоюродный племянник, плюс грабли с уже созданными объектами. Java HotSwap в отладчике — иногда, если сигнатура метода не менялась, и чтобы ты не расслаблялся. Lisp не делает из этого трюк отладчика. Это способ **писать**.

Давай медленно

1. Создай `*reactor*`, вызови `module-status`. \
2. Не `(quit)`. Набери новый `defmethod`. \
3. Снова `module-status` на **том же** объекте. \

4. Если увидел старую строку — проверь образ, пакет, специализатор метода и более конкретные методы. Обычное переопределение `defclass` обновляет существующие экземпляры; оно не оставляет их автоматически в отдельном «старом» классе. `(class-of *reactor*)` и ещё раз метод.

Квест I.L1 · Lisp
Живой объект, два `defmethod` подряд без выхода из SBCL. В комментарии: что осталось тем же (данные) и что сменилось (глагол). Если сменилось только после `make-instance` заново — ты не то упражнение сделал, ты перезапустил станцию.

16.7 Сто миллионов миль и один REPL

Иногда эту историю рассказывают как «спутник сходил с орбиты, подключились, починили». Ближе к правде — ещё страннее.

1999 год. Зонд NASA **Deep Space 1**, далеко от Земли (десятки миллионов миль, задержка туда-обратно — десятки минут, не «пинг из соседней комнаты»). На борту эксперимент Remote Agent: кусок автономии, написанный в основном на Common Lisp (Harlequin Lisp на VxWorks). Не учебный REPL. Настоящий. На железе за сто с лишним миллионов долларов.

Софт должен был управлять зондом несколько дней сам. На земле его месяцами гоняли, кусок даже «доказали» модель-чекером. На второй день ожидаемое событие не случилось. Гонка, которая **ни разу** не вылезла в тестах: дедлок. Перезалить прошивку как телефонное приложение нельзя: почта до Марса не ходит, церемония «скомпилируй — положи в jar — залей — перезапусти `main`» здесь стоит света и риска потерять машину.

На борту был REPL. С Земли, через антенны Deep Space Network, запросили backtrace: кто кого ждёт. Нашли зависание. Не «пересобрали проект». Вручную дернули то

событие, которого ждал код, — и агент поехал дальше. Рон Гаррет, который это рассказывает, говорит прямо: без цикла «прочитал — посчитал — напечатал» на корабле эту штуку не починили бы.

Закон станции
Вот зачем в первой главе чёрное окно не было игрушкой. Тот же жест, что `(+ 2 3)`: живой язык рядом с живыми данными. На DS1 данными была станция ценой с небольшой город. У тебя — `*reactor*`. Ритуал тот же. Масштаб разный.

По личному рассказу Рона Гаррета, ещё до полёта управленческие решения

сократили роль Remote Agent до короткого эксперимента. Его технический успех не вернул Lisp прежних позиций в JPL. Это оценка участника, не официальная позиция NASA. Это не мораль «Lisp победил». Это мораль «язык, в котором программа — не священный бинарник, а разговор, иногда спасает железо, до которого не дотянуться руками». Станция «МОДУЛЬ» выдумана. DS1 — нет. Исходник байки: текст Гаррета **Lisping at JPL** и его доклад про отладку с шестидесяти миллионов миль.

Не ври на собесе, что «я чинил спутник». Ври меньше: «в Lisp можно переопределить метод, не убивая процесс; так однажды отлаживали зонд». Этого

хватит, чтобы человек с той стороны стола либо улыбнулся, либо спросил про `HotSwap`. Оба исхода хорошие.

16.8 Lisp и искусственный интеллект, тот самый первый

Джон Маккарти придумал Lisp в 1958-м не чтобы кормить джунов Java. Чтобы говорить с машиной про **символы**: не только числа, а списки, правила, «если вот это — то вот то». На двадцать-тридцать лет Lisp стал родным языком искусственного интеллекта. MIT, Стэнфорд, экспертные системы, «машины Lisp» фирмы Symbolics — отдельные

компьютеры, потому что обычные тогда задыхались.

Идея была красивая: интеллект как манипуляция символами. Код = данные, макрос дописывает язык под задачу, REPL позволяет точить мысль, не пересобирая мир. На бумаге и в лаборатории получалось. На практике упирались и в железо, и в ограничения самих методов. Памяти мало, сборщик мусора на тогдашних машинах — роскошь, символьный поиск взрывается комбинаторикой быстрее, чем успеваешь купить ещё килобайт. Обещания нередко обгоняли возможности. Дополнительный килобайт решал не каждую задачу.

Потом наступила зима: деньги ушли, Lisp-машины разорились, индустрию перекормили обещаниями. Не потому что скобки глупые. Сошлись технические ограничения, завышенные ожидания, стоимость внедрения и сокращение поддержки. Это короткий набросок истории, а не объяснение одной причиной.

Про ящик отдельно, потому что это не метафора. Lisp-машина Symbolics — отдельный компьютер, чтобы сборщик мусора и списки не стыдились бытового железа. Стоила как хороший автомобиль. Обычный офисный ПК того же года Lisp ел медленно и с обидой. Когда обещали «искусственный

интеллект к понедельнику», в понедельник обнаружилось, что памяти на правила про весь мир не хватает, а комбинаторика «если-то» растёт быстрее счёта. Можно было купить ещё одну Lisp-машину. Нельзя было купить ещё один мир в килобайтах. Но беды индустрии не сводились к нехватке памяти: ожидания рынка и стоимость полезного внедрения тоже были частью истории. Скобки счетов не подписывали.

Сейчас «ИИ» снова на плакатах, только ящик другой: видеокарты, тензоры, статистика, не экспертная система на правилах.

Железо стало гораздо мощнее. Подход не тот — и ладно. Lisp из этого не обязан снова стать языком вакансий. Он уже сделал главное: показал, что программа может быть **пластичной**. Макрос, CLOS с несколькими родителями, метод, который меняешь, пока объект жив, зонд, который не перезагружают ради одной гонки, — ветки одного дерева. Дерево посадил Маккарти, поливали лаборатории ИИ — через подъёмы и зимы.

Давай медленно
Если спросят «Lisp же мёртвый?» — не надо защищать вакансии. Скажи: на нём росли ранние исследования ИИ, коммерческий ажиотаж потом спал по нескольким причинам,

а идеи (REPL, код как данные, живой образ) расползлись по другим языкам кусками. Java взяла объекты и сборщик мусора, макросы не взяла, несколько родителей не взяла, горячую подмену метода не взяла. Куски, не дерево. Поэтому сорок минут скобок в этой книге — не ностальгия ради резюме. Это посмотреть на дерево, а не только на мебель из IKEA.

Эта книга писалась в Emacs. Значительная часть возможностей редактора написана на Emacs Lisp; новую команду можно добавить, не перезапуская редактор. Книга про живой язык — в редакторе с живым языком. Станция любит такие петли.

16.9 Зачем тогда Lisp. Итог, не проповедь

Собери на одну панель, пока не разлетелось по главам:

- **Код — данные.** Список можно собрать, разобрать, скормить макросу. Язык не закрыт комитетом. Ты дописываешь грамматику, когда жест повторяется. В Java для этого аннотации, кодогенерация, lombok — соседние галактики, плюс церемония сборки.
- **Макрос считает формы, не значения.** Поэтому `when` и `with-open-file` существуют. Поэтому `(twice-wrong (tick))` опасен. Поэтому `macroexpand-1` — датчик, не учёность.

- **CLOS: глагол снаружи.** Метод не прибит к классу навечно. Несколько родителей со слотами — да, в 2026-м Java всё ещё нет. Несколько аргументов у generic — да. `call-next-method` — не копипаста папы.
- **Живой образ.** Переопределил `defmethod` — станция уже другая, объекты те же. Интерактивное программирование, не «консоль для Hello». Почти нигде так больше не кормят с ложки.
- **ИИ.** Lisp был одним из главных языков раннего ИИ. У зим были технические и экономические причины, а не одна виноватая скобка.

Скобки переживают зимы лучше, чем гранты.

- **DS1.** REPL на зонде. Гонка. Backtrace с задержкой света. Починка без «залей APK». Не сказка про падающий спутник — рассказ участника проекта.

Зачем это джуну Java? Не чтобы писать Spring на CLOS. Чтобы видеть **край** языка: что в Java выбор, а что привычка индустрии. Когда повторяется свой управляющий шаблон, которого нет в готовых конструкциях языка, — вспомни макрос. Когда `extends` одного папы мало — вспомни, что это ограничение Java, не физика. Когда Gradle пять минут собирает, чтобы сменить

строку статуса — вспомни `defmethod` в том же REPL. Когда на собесе скажут «настоящий язык — Java» — можно кивнуть: кормит. И иметь в кармане сорок минут, которые кормят любопытство.

Язык может быть странным **и** полезным для головы. Мы это показываем дырявой станцией и чужим зондом.

Спрятать на GitHub
В README папки `macros-clos/` три доказательства, не лозунги: вывод `macroexpand-1`; `typep` на объект с двумя родителями; два разных `module-status` на одном и том же экземпляре без `(quit)`.

16.10 Что не надо уносить на работу завтра

Не надо переписывать Spring на CLOS. Не надо макрос на каждый чих в учебном Lisp. Надо:

- уметь прочитать `macroexpand-1`;
- отличить «не считать аргумент» от «посчитать и передать»;
- создать класс со слотами и два метода одного глагола;
- знать, что несколько родителей в CLOS — норма, в Java — нет;
- один раз переопределить метод, **не** убив процесс;

- не врать, что Java-класс и CLOS-класс — одно и то же слово;
- если зайдёт про ИИ — не путать 1958-й со скобками и 2026-й с видеокартами, но помнить, чья это была первая мастерская.

Этого хватит, чтобы макросы не казались космосом, а Java-объекты — религией. Станция «МОДУЛЬ» после этого всё ещё дырявая. Просто у дыр появились **типы железяк**, а у языка — **новые слова**. Изолента никуда не делась. Зонд, на всякий случай, тоже когда-то держался не только на изоленте. Ещё на REPL.

16.11 Если за ужином спросят, зачем тебе скобки

Короткий ответ, без лекции и без стыда за Java.

Lisp — язык, в котором программа не прибита гвоздями к файлу на диске. Макрос дописывает грамматику. CLOS разрешает несколько родителей, чего Java до сих пор не делает классами. Метод можно сменить, пока объект жив. На этом росли ранние исследования ИИ — через подъёмы и зимы. На этом однажды работал зонд, чей backtrace читали на Земле.

Тебе это не нужно, чтобы пройти Spring. Тебе это нужно,

чтобы не принять Spring за физику. Физика — компьютер слушается точных инструкций. Spring — один из способов эти инструкции упаковать. Скобки — другой. Оба честные. Один кормит. Второй не даёт соскучиться. Книга с самого титула так и обещала.

Если после этой главы хочется выкинуть Java — не надо. Хаски всё равно разбудят в семь, а вакансия всё равно на Java. Если после этой главы хочется выкинуть Lisp — тоже не надо. Сорок минут никуда не делись. Просто теперь ты знаешь, **куда** они деваются: в дерево, а не в ещё одну аннотацию.

17 Лаборатория: ещё Java руками

Спрингу сюда нельзя. Кто пролез — выгнать. Здесь консоль, файлы, текст, который притворяется данными, и одно письмо в сеть без фреймворка.

Лаборатория не заменяет месяц 1–2. Она для вечера, когда `TaskStore` уже зелёный, а хочется ещё раз потрогать плиту голыми руками. Папка: `java-basics/lab/`. Команды — как в книге: мак или Ubuntu в WSL. `javac`, `java`, JDK 21.

Закон станции
Каждую программу сначала сломай сам. Потом почини по тексту ошибки. Лаборатория,

где всё сработало с первого копипаста — экскурсия, не лаборатория.

17.1 Лаба 1. Дашборд станции, с ошибками как у людей

Хотим консоль:

```
МОДУЛЬ dashboard. energy=80
oxygen=40
> status
energy 80
oxygen 40
> set energy 55
ok
> save
wrote station.txt
> quit
```

После нового запуска — `load` поднимает числа с диска. Пока звучит как занятие 3. Сейчас мы дойдём **через ямы**.

17.1.1 Яма 0. Пустой main, чтобы было что запускать

`StationDash.java`:

```
public class StationDash {
    public static void
main(String[] args) {

System.out.println("МОДУЛЬ
dashboard");
    }
}
```

```
javac StationDash.java
java StationDash
```

Если `javac` «не является командой» — окно не то или PATH. Глава про компьютер, отсек PATH. Не эта лаборатория. Сначала плита.

17.1.2 Яма 1. Scanner и «оно съело строку»

Добавляем ввод. Типичный первый грех:

```
import java.util.Scanner;

public class StationDash {
    public static void
main(String[] args) {
        Scanner in = new
Scanner(System.in);
        System.out.print("energy:
");
        int energy =
in.nextInt();

System.out.print("команда: ");
        String cmd =
in.nextLine();
        System.out.println("ты
сказал [" + cmd + "]");

System.out.println("energy=" +
energy);
    }
}
```

Запусти. Введи `80`, Enter. Курсор **не** ждёт команду. Печатает `ты сказал []`. Магия? Нет.

Давай медленно `nextInt` съел число, **оставил** конец строки в stdin. `nextLine` увидел этот конец и обрадовался: пустая команда. Это не баг Java «у меня». Это два способа есть поток: токенами и строками. Смешал — получи пустоту.

Починка, учебная, без религии:

```
int energy =
Integer.parseInt(in.nextLine().tr
im());
```

Всё через строки. Потом парси. Тогда Enter принадлежит тебе, не призраку.

Вторая починка: после `nextInt` вызвать лишний `nextLine()`, чтобы выбросить хвост. Работает. Легко забыть. Для дашборда бери `nextLine` всегда.

Квест J.L1 · Java
Воспроизведи яму: `nextInt` + `nextLine`, посмотри пустые скобки. Потом перепиши на `parseInt(nextLine())`. В комментарии над `parseInt` одна строка: **зачем**. Не «так надо». Зачем.

17.1.3 Яма 2. Цикл, который умеет quit

```
int energy = 80;
int oxygen = 40;
Scanner in = new
Scanner(System.in);

System.out.println("МОДУЛЬ
```

```
dashboard. energy=" + energy + "
oxygen=" + oxygen);

while (true) {
    System.out.print("> ");
    String line =
in.nextLine().trim();
    if (line.isEmpty()) {
        continue;
    }
    if (line.equals("quit")) {
        break;
    }
    if (line.equals("status")) {

System.out.println("energy " +
energy);

System.out.println("oxygen " +
oxygen);
        continue;
    }
    System.out.println("не знаю:
" + line);
}
```

Пустая строка — не ошибка. Человек жмёт Enter от скуки.

`continue`. `quit` — `break`, не `System.exit(0)` как из пушки по чайке.

Сравнивай строки через `equals`, не `==`. Иначе в одних запусках «сработает» (пул строк), в других — нет, и ты напишешь в чат «Java сломалась». Java не сломалась. Ты сравнил адреса мисок.

17.1.4 Яма 3. set energy 55 — пилим строку

Хочется `set energy 55`. Три куска.

```
String[] parts = line.split(" ");
```

Яма: два пробела подряд. `set energy  55` даст пустой кусок в массиве. Яма: `set energy`. Нет числа. Яма: `set energy много`.

Пишем честно:

```
if (parts.length == 3 &&
parts[0].equals("set")) {
    String what = parts[1];
    int value;
    try {
        value =
Integer.parseInt(parts[2]);
    } catch
(NumberFormatException e) {
        System.out.println("это
не число: " + parts[2]);
        continue;
    }
    if (value < 0 || value > 100)
{

System.out.println("диапазон
0..100");
        continue;
    }
    if (what.equals("energy")) {
        energy = value;
        System.out.println("ok");
    } else if
(what.equals("oxygen")) {
        oxygen = value;
        System.out.println("ok");
    } else {
```

```
System.out.println("датчик
неизвестен: " + what);
    }
    continue;
}
```

Что только что случилось
Это не REPL Lisp, но жест тот же: прочитал строку, решил, ответил, снова `>`. Дашборд — REPL для бедных. Бедные мы сегодня.

Не глотай `NumberFormatException` молча. Напечатай кусок, который не число. Будущий ты скажет спасибо, когда вставишь неразрывный пробел из мессенджера.

Квест J.L2 · Java
Команда `set` для `energy` и `oxygen`, границы 0..100, плохие числа не

роняют процесс. Добавь датчик `temp` (температура коридора, пусть будет −20..40). Три проверки руками: ок, не число, вне диапазона.

17.1.5 Яма 4. Файл. Диск говорит нет

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import java.util.List;
```

Сохранение в две строки — человеческое, не JSON пока:

```
energy=80
oxygen=40
```

```
static void save(int energy, int
oxygen, Path path) throws
IOException {
    String text = "energy=" +
energy + "\n" + "oxygen=" +
```

```
oxygen + "\n";
    Files.writeString(path,
text);
}
```

`throws IOException` — честно. Диск бывает занят, путь бывает в фантазии.

Загрузка:

```
static int[] load(Path path)
throws IOException {
    int energy = 80;
    int oxygen = 40;
    if (!Files.exists(path)) {
        return new int[] {energy,
oxygen};
    }
    List<String> lines =
Files.readAllLines(path);
    for (String raw : lines) {
        String s = raw.trim();
        if
(s.startsWith("energy=")) {
            energy =
Integer.parseInt(s.substring("ene
```

```
rgy=".length()));
        } else if
(s.startsWith("oxygen=")) {
            oxygen =
Integer.parseInt(s.substring("oxy
gen=".length()));
        }
    }
    return new int[] {energy,
oxygen};
}
```

Яма: файла нет — **не падаем**. Дефолты. Яма: файл есть, внутри `energy=abc` — упадём на `parseInt`. Поймай:

```
try {
    energy =
Integer.parseInt(...);
} catch (NumberFormatException e)
{
    System.err.println("битая
строка: " + s);
}
```

В `main` команда `save` / `load`, путь `Path.of("station.txt")`. Это **текущая** папка процесса. Не «рядом с исходником в голове». `pwd` перед `java StationDash`. Если файл появился «не там» — ты стоял не там.

Так себе идея
Java умеет работать с путями, в которых есть пробелы или кириллица. Здесь для первого сейва берём `station.txt` в текущей папке. Произвольные пути оставим на потом. Хватит.

Типичная ошибка: сохранил, открыл файл в редакторе, видишь всё, программа грузит дефолты. Причина: запустил из IDEA с working directory в другом месте. Либо печатай `Path.of("station.txt").toAbsoluteP`

`ath()` при сейве — и перестань гадать.

Квест J.L3 · Java
`save` / `load` для трёх датчиков (с температурой, если делал). Нет файла — живые дефолты, сообщение `no station.txt, using defaults`. Битое число — строка в stderr, остальные датчики пусть живут. Не «весь load в одном catch Exception».

17.1.6 Яма 5. Разрезать main, пока не стыдно

Когда `main` длиннее экрана, вынеси:

- поля состояния — маленький класс `Station` с `energy`, `oxygen`, методами `statusText()`, `set(String, int)`;

- файл — `StationFile.save(Station, Path)`;
- цикл команд — `main` или `run()`.

Это не Spring. Это «чтобы не потеряться». Тот же инстинкт, что сервис и репозиторий, только без аннотаций и без резюме.

Сломай нарочно `set`: забудь верхнюю границу. Введи 10000. Посмотри, как глючит дашборд. Верни границу. Вот лабораторная работа. Отчёт — git diff, не эссе.

17.2 Лаба 2. JSON руками и упоминание Jackson

Сервер в основном курсе будет плеваться JSON. До Спринга полезно один раз **собрать строку самим** и один раз **испугаться**.

Дано: энергия 80, кислород 40. Хотим:

```
{"energy":80,"oxygen":40}
```

```
static String toJson(int energy,
int oxygen) {
    return "{\"energy\":" +
energy + ",\"oxygen\":" + oxygen
+ "}";
}
```

В Java кавычка внутри строки — `\"`. Красиво не будет. Зато видно: JSON — текст.

Яма: название отсека.

```
static String roomJson(String
room, int energy) {
    return "{\"room\":\"" + room
+ "\",\"energy\":" + energy +
"}";
}
```

Введи комнату `corridor`. Ок. Введи комнату `cor"idor` с кавычкой. JSON сломается: строка кончится рано, едок на той стороне подавится. Введи обратный слэш. То же.

Давай медленно
Руками JSON можно, пока данные — числа и слова без кавычек. Как только строка с улицы — ты пишешь экранирование или берёшь библиотеку. Иначе это не JSON, а костюм на Хэллоуин.

Мини-парсер для **своего** же формата, без претензий. Только плоский объект с двумя int, который **мы** только что написали:

```
static int[]
fromJsonEnergyOxygen(String json)
{
    int e = grabInt(json,
"energy");
    int o = grabInt(json,
"oxygen");
    return new int[] {e, o};
}

static int grabInt(String json,
String key) {
    String needle = "\"" + key +
"\":";
    int i = json.indexOf(needle);
    if (i < 0) {
        throw new
IllegalArgumentException("нет
ключа " + key);
    }
    int start = i +
```

```
needle.length();
    int end = start;
    while (end < json.length() &&
Character.isDigit(json.charAt(end
))) {
        end++;
    }
    return
Integer.parseInt(json.substring(s
tart, end));
}
```

Это учебный нож. Он умрёт на пробеле после `:`, на отрицательном числе, на вложенности. **Пусть умрёт**. Ты увидишь границу ножа.

Так себе идея
Не тащи этот `grabInt` в task-manager. Там Jackson / Gson / что даст Spring. Здесь нож, чтобы понять, **зачем** они.

Jackson — библиотека: объект ↔️ JSON, экранирование,

списки, даты. В Maven это зависимость. В лаборатории достаточно **знать имя** и причину: кавычки, юникод, вложенность, не изобретать. Когда дойдёшь до Spring, `@RestController` часто вообще спрячет Jackson под одеяло. Одеяло не отменяет, что под ним текст.

Попробуй руками в дашборде команду `json` — печатает `toJson`. Команду `json` после `load` — те же числа, другой костюм. Это не база. Это конверт.

Квест J.L4 · Java
`toJson` для трёх датчиков. Проверь глазами: вставь вывод в https://jsonlint.com или в любой валидатор. Потом сломай: добавь кавычку в **имя**

выдуманного поля комнаты без экрана — валидатор должен обидеться. Напиши в README одну фразу, почему Jackson существует.

17.3 Лаба 3. HTTP без Спринга: HttpClient и чужая кухня

Сеть — письма. Письмо можно послать **из** Java, не только из браузера.

Пакет `java.net.http` в JDK 11+ есть сам. Не Maven. Не Spring. Плита уже умеет.

Сервис httpbin.org (или `https://httpbin.org/get`) отвечает на GET JSON-ом: какой URL пришёл, какие заголовки, откуда стучали. Учебная эхо-стенка.

Если httpbin лежит (бывает) — любой простой GET на `https://example.com`: там HTML, не JSON, но письмо всё равно письмо.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import
java.net.http.HttpResponse;
import java.io.IOException;

public class StationPing {
    public static void
main(String[] args) throws
IOException, InterruptedException
{
        HttpClient client =
HttpClient.newHttpClient();
        HttpRequest req =
HttpRequest.newBuilder()
                .uri(URI.create("
https://httpbin.org/get"))
                .GET()
                .build();
        HttpResponse<String> res
= client.send(req,
```

```
HttpResponse.BodyHandlers.ofStrin
g());

System.out.println("status " +
res.statusCode());

System.out.println(res.body());
    }
}
```

Давай медленно
`send` — синхронно: стоим у двери, ждём конверт. Поток Java на это время занят. Для дашборда ок. Для тысячи писем в секунду — уже другая вахта.

`InterruptedException` — «нас ткнули, пока ждали». `IOException` — конверты, DNS, обрыв Wi-Fi. Не лови их пустым catch. Напечатай `e.getMessage()`.

Яма: нет сети. Будет исключение, не JSON. Это

правильное поведение. Поймай, напечатай «сеть не взяла трубку», не падай без текста.

Яма: опечатка в URL. `httpbin.org/get` с русской «е» — шедевр. Копируй латиницу.

Яма: http vs https. httpbin часто хочет https. `301` / редирект. HttpClient умеет ходить за редиректом, но статус надо смотреть. Если 301 и пустое тело — ты не «не получил JSON», ты получил указатель «ищи там».

Разбор крошечный, без полного JSON-парсера: найди в теле `"url"` глазами. Потом `indexOf`. Потом пойми, что руками стыдно, и остановись. Лаборатория про **письмо**, не про второй Jackson.

Добавь в дашборд команду `ping`: дергает httpbin, печатает только `statusCode`. Если не 200 — всё равно напечатай код. 503 чужой кухни — не твой баг. 404 — не та дверь. 0 у тебя не будет: статус всегда пришёл **если** `send` не кинул. Кинул — запрос не завершился: смотри тип и сообщение исключения. Сеть при этом может работать.

Мак, Windows, WSL

Мак / WSL. Обычно просто работает.

Корпоративный прокси. Возможны отказ подключения, ошибка TLS или ожидание до настроенного таймаута. Не лечи это обходами ограничений: проверь разрешённые настройки сети

или сделай лабораторную на домашней сети.

Оффлайн. Нормальный исход — пойманный IOException. Засчитано, если сообщение человеческое.

Квест J.L5 · Java
Класс `StationPing` или команда `ping` в дашборде. Напечатай статус и **первые 200 символов** тела, не роман. Второй запрос: `https://httpbin.org/status/404` — увидь 404 и не называй это «у меня сломалось». Третий: неверный хост `https://no-such-module-xxxx.example` — поймай исключение, напечатай тип (имя класса) и сообщение.

17.4 Лаба 4. Склеить: дашборд умеет сейв и письмо

Не обязательно. Если остались силы: команда `report` собирает JSON руками и… никуда не постит, если не хочешь регистрировать чужие API. Напечатай JSON. Рядом команда `ping`. Два мира: диск и сеть. Оба не магия.

Если хочется POST (не обязательно для зачёта журнала):

```
HttpRequest req =
HttpRequest.newBuilder()
        .uri(URI.create("https://
httpbin.org/post"))
        .header("Content-Type",
"application/json")
        .POST(HttpRequest.BodyPub
```

```
lishers.ofString(toJson(energy,
oxygen)))
        .build();
```

httpbin вернёт тебе твой JSON внутри своего JSON. Зеркало. Увидишь, **что** улетело, включая кавычки. Если улетело криво — виноват `toJson`, не «HTTP».

Так себе идея
Не пости свой дашборд на случайные чужие URL «для проверки». httpbin — для этого. Чужой прод — нет. Станция «МОДУЛЬ» и так на изоленте, давай без инцидентов.

17.5 Сборка дашборда целиком, без геройства

Ниже — скелет, который можно набрать **после** ям, не вместо

них. Если скопируешь сразу — ямы не случатся, пальцы ничего не получат.

`Station.java`:

```
public class Station {
    int energy = 80;
    int oxygen = 40;
    int temp = 21;

    String statusText() {
        return "energy " + energy
+ "\n"
             + "oxygen " + oxygen
+ "\n"
             + "temp " + temp;
    }

    String set(String what, int
value) {
        if (what.equals("energy")
|| what.equals("oxygen")) {
            if (value < 0 ||
value > 100) {
                return "диапазон
0..100";
```

```
            }
        } else if
(what.equals("temp")) {
            if (value < -20 ||
value > 40) {
                return "диапазон
-20..40";
            }
        } else {
            return "датчик
неизвестен: " + what;
        }
        if
(what.equals("energy")) energy =
value;
        else if
(what.equals("oxygen")) oxygen =
value;
        else temp = value;
        return "ok";
    }

    String toJson() {
        return "{\"energy\":" +
energy
             + ",\"oxygen\":" +
oxygen
             + ",\"temp\":" +
```

```
temp + "}";
    }
}
```

`StationFile.java` — `save`/`load` как в яме 4, только три ключа. Имена полей совпадают с файлом: меньше фантазии, меньше багов.

`StationDash.java` — цикл `while (true)`, `split`, `quit` / `status` / `set` / `save` / `load` / `json` / `ping`.

Компиляция двух-трёх файлов в одной папке:

```
javac Station.java
StationFile.java StationDash.java
java StationDash
```

IDEA делает это кнопкой. Терминал делает это явно. Явно полезнее один вечер.

Ошибка `cannot find symbol Station` — ты компилируешь не из той папки или забыл файл в списке. `pwd`. `ls *.java`.

Давай медленно
Класс `Station` — состояние на стойке. `StationFile` — кладовая. `HttpClient` в `ping` — письма. Если через месяц кто-то скажет «контроллер, сервис, репозиторий», ты уже это нюхал. Только без аннотаций и без зарплаты.

17.5.1 Ещё одна яма: рабочая папка IDEA

Run → Edit Configurations → Working directory. Если там корень проекта, а ты думал «рядом с java-файлом», `station.txt` родится в корне. Git увидит лишний файл. Ты не

увидишь его в `ls` учебной папки. Оба правы. Пути разные.

Печатай абсолютный путь при первом `save`. Один раз. Потом либо смирись, либо поправь конфигурацию. Не чини это копированием файла руками «ну вот же он».

17.5.2 Ещё одна яма: разделитель строк

`Files.writeString` пишет те символы перевода строки, которые есть в переданной строке: наш `\n` остаётся LF и на Windows. Сам по себе метод не превращает его в `\r\n`. `readAllLines` распознаёт LF, CRLF и CR. Платформенный разделитель при необходимости выбирай явно через `System.lineSeparator()`.

17.5.3 HttpClient чуть аккуратнее

Таймаут, чтобы висеть не вечно:

```
HttpRequest req =
HttpRequest.newBuilder()
        .uri(URI.create("https://
httpbin.org/get"))
        .timeout(java.time.Durati
on.ofSeconds(10))
        .GET()
        .build();
```

Заголовок `User-Agent` иногда просят вежливые серверы. httpbin не капризничает. Чужой прод капризничает. Для лаборатории не надо притворяться браузером.

Тело может быть огромным. `substring(0, Math.min(200, body.length()))` — первые 200. Без `Math.min` поймаешь

`StringIndexOutOfBounds`, и это будет смешно ровно один раз.

Квест J.L6 · Java
Вынеси состояние в `Station`, файл в `StationFile`, цикл оставь в `StationDash`. Три файла компилируются из терминала. README: команды компиляции и запуска и пример диалога на пять строк. Без README лаба не закрыта — как в неделе 4, только короче.

17.6 Что должно остаться в пальцах

- Ввод: не мешай `nextInt` и `nextLine`.
- Команды: пили строку, проверяй длину массива.

- Файл: нет файла — дефолт; битая строка — сообщение, не немой вылет.
- JSON: текст; кавычки опасны; библиотека существует не из снобизма.
- HTTP: клиент, запрос, статус, тело, исключение на обрыве.

Это тот же камбуз, что в главе про компьютер: повар, стойка, кладовая, письма. Только теперь ты ещё и рецепты пишешь.

Воскресная диверсия
Запусти дашборд из терминала, не из стрелки. Сохрани. Найди `station.txt` через `ls` в **той** папке, где `pwd`. Если файла нет в Finder «рядом с исходником» — поздравь себя: ты только что почувствовал процесс.

Лаборатория закрыта, когда: (1) яма с `Scanner` воспроизведена своими руками; (2) `station.txt` находится через `pwd`, не через молитву; (3) JSON хотя бы один раз проверили валидатором; (4) HTTP хотя бы один раз вернул не-200 и ты не запаниковал. Спринг подождёт. Он никуда не денется, к сожалению.

Авторские права и лицензия

Иллюстрированное издание · октябрь 2026. Текст: Евгений Тюркин и Грок. Карандашные рисунки и реплики на полях созданы для обучения; это не воспроизведение пометок из настоящего библиотечного экземпляра. Иллюстрации созданы с помощью ИИ.

Учебные примеры рассчитаны на версии и условия, указанные в тексте. Редакторские чтения и отдельные тесты примеров не заменяют проверку зависимостей, настроек безопасности и развёртывания в собственной среде.

MIT License

Copyright (c) 2026 Evgeniy Tyurkin

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the «Software»), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or

substantial portions of the Software.

THE SOFTWARE IS PROVIDED «AS IS», WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.